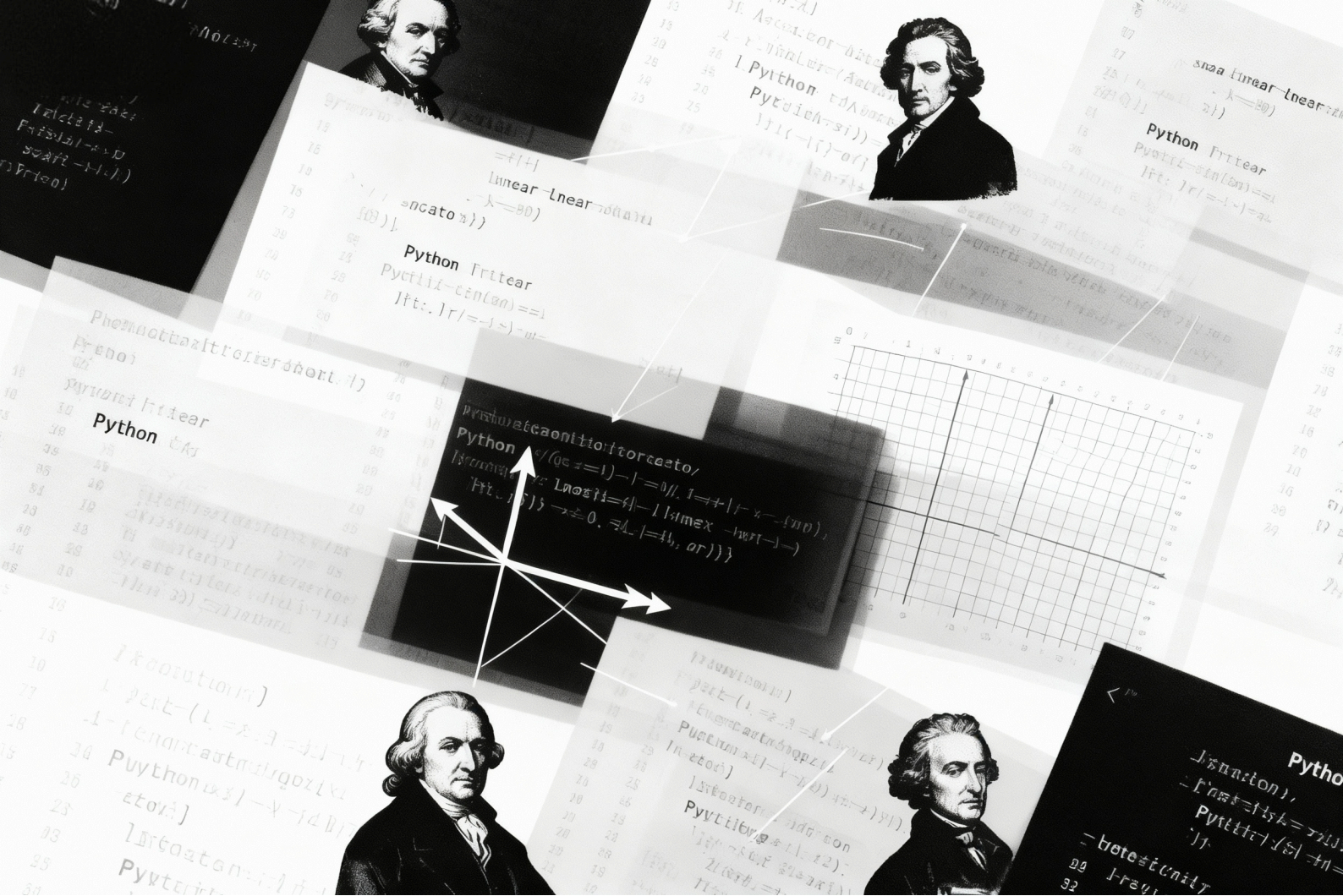




线性代数那些事儿

历史、几何、代码与人工智能前沿

GrokCV

作者：戴一冕 刘昊东 黄子豪 程明明

组织：南开大学媒体计算实验室

时间：2025 年 10 月 14 日

版本：V0.0.3

学习线性代数的终极目标，不是解题，而是获得一种驾驭复杂世界的思维范式

前言（一）

当前呈现给各位读者的，是本书在 2025 年 10 月 14 日的初始版本（V0.0.3）。它尚处于非常早期的草创阶段，远未达到理想中的完成度，在此先行说明。

最初的设想，是希望利用暑假相对完整的时间，将全书内容系统性地撰写完毕。然而事与愿违，整个暑期，我奔波于各种截止日期与差旅之间，没有得到一天完整的休息。因此，本书的撰写工作，只能以这种边教学边完善的模式进行。本书的电子版将持续更新和修正，恳请读者务必通过本书官方主页 <https://grokcv.ai/teaching/> 获取最新版本，以期获得最佳的阅读体验。

撰写此书的初衷，源于我青年时代作为一名学渣时，心中埋下的两颗种子。

第一颗种子，种于我的高一时期。那时，我在政治、历史、地理等文科科目中能够跻身学校前列，但在数学和物理上成绩非常一般。上课时经常打瞌睡，学习兴趣索然，作业也是草草应付，只交一部分作业了事。高二文理分科之际，我出人意料地选择了理科——如今已记不清确切的契机，只忆起高一暑假，我沉浸于多本数学史、物理史以及爱因斯坦、海森堡、冯·诺依曼等科学巨匠的传记之中。正是这些宏大叙事与先驱者的探索故事，为我揭示了公式与定理背后鲜活的人类求知历程，从而点燃了我对理性世界的好奇与向往。升入高二后，我甚至主动提前自学了微积分。尽管我的数学和物理成绩依旧平平，并未脱颖而出（究其原因，多半因练习量不足，我往往仅完成老师布置作业的三分之一），但我已不再如高一时那般抗拒。那时的我，曾暗自思忖：倘若老师在上课时，能将所授内容的演变历史娓娓道来，或许能激发更多学生的兴趣，从而提升教学成效。

第二颗种子，就是在大一线性代数课上种下的。我的本科是在南航度过的，线性代数由数学系教师执教，我学得一塌糊涂，全然不明所以：为何要学习线性代数？这些概念从何而来？学习它们又有何实际意义？直至考研之际，我从头自学麻省理工学院吉尔伯特·斯特朗教授的线性代数课程，方才稍稍领悟其对信息学科学生的价值。那时我就想，信息学科的线性代数课程，或许不应由数学系教师主讲，而应交由本专业教师负责。诚然，这可能不如数学系的严谨，但至少能让学生明了学习的缘由与用途。只有当一个人意识到知识的实用性，方能迸发真正的学习动力。我甚至设想，若有机会，我希望重新设计这门课程，自撰一本线性代数教程，让如我一般的学渣也能知其所以然。当然，我也深知，这不过是痴人说梦。线性代数作为基础课程，按照惯例，必然由数学系老师教授，即便有朝一日我成为大学教师，也不可能由工科教师承担，这不过是一个少年不切实际的幻想。

然而，命运的轨迹有时确实奇妙。我本是一名 211 高校的博士，求学历程磕磕绊绊，历时八年（延期两年）；博后阶段又耗费四年（再度延期两年）。这样一个“学业坎坷者”，竟有幸入职南开大学这样的 985 名校，已是天大的眷顾。更巧的是，南开工科的线性代数课程，竟由本专业教师执教！而学院领导又恰好指派我来上这门课！那一刻，我

深感这是一种宿命般的机缘。十几年前那个被数学困扰的少年许下的愿望，以一种不可思议的方式，在今天成为了我必须面对的责任。既然愿望已至眼前，我有必要写下这本书，来偿还当年的那份夙愿。

我希望这本书能够成为一本真正面向信息学科学生的线性代数教材，能够讲清楚“从哪来、到哪去、为何而学、如何能用”。因此，这本书会特别重视三件事：

1. 历史脉络与问题驱动：在叙述定理之前，先把问题与动机讲清楚，回答“它解决了谁的痛点”；
2. 计算直觉与工程可用：在推导之余，给出可运行的最小例子、数据与代码，帮助你“用起来、跑起来”；
3. 结构化的知识图谱：以信息学科为牵引，连接机器学习、信号处理、优化与几何的交叉地带，让概念互相照亮。

本书的成稿，得益于我的两位助教——刘昊东（西安交通大学，大二）和黄子豪（南开大学，大一）的鼎力相助。他们投入了大量心血，不仅贡献了绝大多数内容，还协助完善了配套幻灯片。没有他们的支持，此书与相关材料断不可能问世。英雄出少年，我对他们的感激，难以言表。

最后，我必须向选择这门课的 2025 级同学们，致以深深的歉意。你们不仅遇上了一位初执教鞭的菜鸟老师，一位曾经的数学学渣，还要面对这本尚在襁褓之中、内容与形式都将不断变化的教材。这无疑会给你们的学习带来诸多不便。感谢你们的耐心、包容与反馈——正是你们的使用与质疑，能让它尽快接近一部“可读、可学、可用”的教材。

教材会持续演进。我承诺：

1. 版本公开透明：每次更新都会给出变更记录与勘误；
2. 资源持续完善：配套的习题、答案、实验与数据将逐步上线；
3. 反馈快速闭环：欢迎通过课程主页 <https://grokcv.ai/teaching/> 提交问题与建议。

愿这本书，能为和我当年一样在数理门外徘徊的你，点一盏灯；也愿它成为你走向工程实践与科研探索的稳固台阶。若你从中获得哪怕一丝理解的欢喜与应用的把握，这本书便不虚此行。

戴一冕
于南开
2025 年初秋

前言（二）

如果你正在翻开这本书，心中或许正萦绕着一个与万千学子同样的困惑：线性代数，这门被誉为现代科学基石的学科，为何常常以抽象、繁杂的形象出现？我们埋首于矩阵的丛林，穿行于向量的迷雾，记忆着那些看似孤立的法则与定理，却时常忘记出发的目的——我们究竟为何而来，又要去向何方？我们努力学习“如何计算”，却很少有人阐明“为何如此”。

这个困惑，我也感同身受。我自己在初学时，也曾面对着一本本将知识点割裂讲解的教材感到迷茫：为何要学习行列式？它又如何与后来出现的矩阵产生关联？直到在题海中反复挣扎后，一幅关于线性代数的模糊图景才在脑海中慢慢拼凑完整。因此，当戴老师邀请我们以学生的身份，深度参与这本“非典型”教材的创作时，我欣然应允。希望能将自己作为学习者“拨开迷雾”的亲身经历，融入到这本书的字里行间。

我们的工作，始于一个简单而艰巨的任务：寻找一种能让初学者“顿悟”的讲解方式。我们翻阅了诸多经典教材，它们逻辑严谨，但常常将我们直接“空投”到公理和定义的陌生大陆；我们一头扎进了数学史的故纸堆，试图还原概念诞生的场景，但历史的珍珠散落各处，要如何才能将它们串成一条能真正指导学习的项链？

于是，我们决定将这条“探索路径”本身，作为本书的叙述主线。这本书的探索将会从一个基本问题开始：求解方程组。早在汉代的《九章算术》中，中国的先贤们便已在实践此事。然而，在很长一段时间里，人们的目标始终是计算出那个唯一的答案。直到一群卓越的数学家们完成了一次深刻的观念转变：不再只关注单个的“解”，而是审视所有解构成的“集合”及其内蕴的结构。正是这一次的灵光乍现，开启了一扇通往新世界的大门。我们会看到这个“解的集合”并非毫无规律，它拥有自身的规则，我们称之为“空间”。为了描述这个新世界，数学家们提炼出八条核心公理。这个过程本身就充满了故事——例如天才格拉斯曼，他的思想超越了整个时代，却在孤独中不被理解。我们相信，走进这些故事，能赋予冰冷的公式与定理以温度和灵魂。当然，有了“空间”这个静态的舞台尚显不足，我们还需要引入“运动”，于是“线性变换”应运而生。我们会发现，矩阵不再是孤立的数字方阵，而是为每一种旋转投影这样的“空间运动”颁发的唯一“身份标识”。在千变万化的运动中，我们如侦探般寻找那不变的线索——“特征值与特征向量”。最终，我们将为这个抽象世界装上“标尺”——“内积”，使其变得可以度量。在书中我们会看到通过一次次完美的视角切换，优雅地解决二次曲线的旋转问题，让复杂回归至简的数学之美。

当然，作为信息学科的学生，我们更致力于将历史与现实相连接，因为我们深知“能用”才是硬道理。书中的“代码视角”部分，将引导你在计算机上动手去实现这些经典思想。在书中我们会看到，十九世纪的抽象理论，如何化为几行简洁的 Python 代码，高效地解决实际问题。更为重要的是，现在是在互联网浪潮中，线性代数已经成为奠定现代人工智能的基石。例如，我们能用向量计算词语的意义，实现如“国王 - 男性 + 女性 $\approx$

女王”这样充满魅力的语义运算。在面对海量数据时，我们也可以利用 PCA 降维技术，从纷繁的信息中抓住核心特征，这背后正是特征向量思想的精彩应用。

所以，本书的愿景，远非是训练一部埋首于演算的机器。它并非一本线性代数的“使用手册”，而更像是一部“思想传记”，一份邀请你一同回到思想黎明的邀请函。我们希望你亲历伟大概念诞生时的心潮澎湃，触摸知识大厦奠基之时的坚实温度，进而将这份源自过往的深邃智慧，化为点亮当今数字世界的炬火。待你合上本书的最后一页，我们深信，线性代数在你眼中将焕然一新——它不再是一门刻板的学科，而是一次波澜壮阔的智识探险，一种足以重塑你世界观的深刻艺术。

黄子豪

于南开

2025 年 9 月 28 日

前言（三）

如果你此刻正读着这本《线性代数那些事儿》，或许和曾经的我一样，心里藏着一份对“数学”的无措——明明知道它是理工科的基石，却总在翻开教材时，被一行行冰冷的定理、一串串陌生的符号拦在门外。

我是数学专业的学生，自从大一入学以来就每天面对海量的数学知识。我至今记得大一刚接触数学时的模样：对着数学分析里的“连续与极限”发呆，盯着高等代数里的“欧氏空间”皱眉，手里的笔在草稿纸上画满了问号，却始终找不到“为什么要这样定义”“这些知识能用来做什么”的答案。那时的我，像在一片没有路标、没有温度的数学森林里打转，越努力刷题，越觉得自己离“数学”越来越远。

直到大一那个闷热的暑假，我在图书馆的旧书架上，偶然翻到了张筑生老师的《数学分析新讲》。当指尖触到泛黄的书页，当读到老师深入浅出的文字讲解，我突然有了一种“豁然开朗”的悸动：原来数学不是一堆孤立的公式，它可以被讲得这样温柔、这样透彻；原来那些让我头疼的概念，背后都藏着清晰的逻辑与温度。那天下午，我坐在图书馆的窗边，阳光落在书页上，连曾经觉得晦涩的定理，都仿佛有了呼吸。也是从那时起，我心里悄悄埋下一个念头：如果有一天，我能参与编写一本数学书，一定要让它像张老师的书那样，帮更多人走出“数学迷茫”。

可惜的是，我们大多时候遇到的教材，都少了这份“温度”。它们更像一本“定理清单”：先抛出定义，再罗列证明，最后附上几道例题，机械地灌输“是什么”，却很少回答“为什么”“怎么办”。我们学会了计算行列式，却不知道它最初是为了解决“方程组求解”而生；我们背熟了矩阵的运算规则，却不明白它能在“图像处理”“AI 算法”里发挥什么作用；我们对着习题集刷遍了题型，却从未亲手用代码实现过一次“用线性代数解决实际问题”。这样的学习，就像只记住了乐谱上的音符，却听不到整首曲子的旋律——枯燥，且没有灵魂。

所以，我们不想再做一本“冷冰冰的教材”，而是想打造一份“带读者走出数学迷雾的地图”：我们会沿着历史的脉络，把线性代数的“诞生故事”讲给你听，让你知道每一个概念的“来龙去脉”；我们会用最通俗的语言，把抽象的知识“拆解开”，让你不再为“看不懂定义”发愁；我们还会附上大量“代码实践案例”——从实现矩阵乘法，到解决线性规划问题，鼓励你亲手敲下代码，感受“数学知识落地为实用工具”的快乐。至于这本书的具体内容，前面几位的前序已经说得很详细，这里就不再赘述，只盼你翻开正文时，能感受到我们的用心。

写到这里，我又忍不住想起张筑生老师。他是北大的教授，被同学们亲切地称为“校园里的焦裕禄”。他不求名、不求利，只愿做“照亮学生数学之路的灯”。这份“甘为人梯”的精神，一直是我们编写这本书的底气——我们想沿着他的脚步，把这份对数学的热爱、对学生的负责，藏进每一页文字里。可惜篇幅有限，没法把张老师的故事一一讲完，但我真心希望你能在课后，上网搜一搜他的事迹——或许你会和我一样，从他的故

事里，读懂“做学问”与“做人”的温度。

最后，我们不奢望这本书能让你立刻成为“线性代数高手”，只希望它能帮你解开当初困扰我的那些疑惑：让你知道“线性代数是什么”，更明白“它为什么重要”“该怎么用”；让你不再觉得数学是“遥远的符号”，而是能帮你理解世界、解决问题的“朋友”。愿你翻开这本书时，能少一些迷茫，多一些豁然；愿你合上书时，能真正感受到——数学，从来都不是冰冷的，它藏着人类智慧的温度，也藏着照亮未来的力量。

刘昊东
于西交钱学森图书馆
2025.9.28

目录

第1章 代数简史	1
1.1 什么是代数?	1
1.2 代数发展时间线 (公元前 2000 年 – 19 世纪)	1
1.3 萌芽阶段 (公元前 2000 年 – 公元 500 年)	3
1.3.1 古巴比伦 (约公元前 1800 年)	3
1.3.2 丢番图 (约公元 200—284 年)	4
1.4 符号化与系统化 (公元 800 年 – 17 世纪)	6
1.4.1 花拉子米 (Al-Khwarizmi, 约 780–850 年)	6
1.4.2 中国宋元时期	9
1.5 符号体系的革命 (16 世纪 – 17 世纪)	13
1.5.1 韦达 (François Viète, 1540–1603)	13
1.5.2 笛卡尔 (René Descartes, 1596–1650)	14
1.6 结构代数与现代抽象阶段 (18 世纪 – 19 世纪)	16
1.6.1 阿贝尔 (Niels Henrik Abel, 1802–1829)	16
1.6.2 伽罗瓦 (Évariste Galois, 1811–1832)	17
1.6.3 哈密顿 (William Rowan Hamilton, 1805–1865)	18
1.6.4 布尔 (George Boole, 1815–1864)	19
1.7 小结	19
第2章 线性代数简史	21
2.1 什么是线性代数?	21
2.2 线性代数发展时间线	22
2.2.1 跨越千年的萌芽: 古代世界的智慧闪光	22
2.2.2 行列式: 判定方程命运的神秘数字	24
2.2.3 莱布尼茨 (Gottfried Wilhelm Leibniz, 1646-1716)	25
2.2.4 范德蒙德 (Vandermonde, 1735—1796)	25
2.2.5 克拉默 (Gabriel Cramer, 1704-1752)	26
2.3 近代革命: 结构思想的诞生 (19 世纪)	28
2.3.1 高斯 (Carl Friedrich Gauss, 1777-1855)	28
2.3.2 阿瑟·凯莱 (Arthur Cayley, 1821-1895)	29
2.3.2.1 命名与运算法则的创立	29
2.3.3 格拉斯曼与皮亚诺: 从几何实体到抽象空间的一跃	30
2.4 现代发展: 抽象浪潮与计算机革命 (20 世纪至今)	32
2.4.1 大卫·希尔伯特 (David Hilbert, 1862—1943)	32

2.4.2	图灵与冯·诺依曼	33
2.4.3	当代前沿：深度学习的“线性”基石	35
2.4.3.1	前沿创新：古典智慧的现代回响	35
第3章	行列式	38
3.1	行列式的起源与定义	38
3.1.1	历史引入：一个判别解的符号	38
3.1.2	核心定理：从低阶到 N 阶的严格定义	39
3.1.3	代码实现：莱布尼茨定义的代码逻辑	46
3.2	行列式的性质	49
3.2.1	历史引入：柯西与行列式理论的系统化	49
3.2.2	核心定理：行列式“运作”的五大基本法则	49
3.2.3	代码实现：用代码验证性质	62
3.3	行列式的计算方法	67
3.3.1	历史引入：拉普拉斯、范德蒙与展开定理	67
3.3.2	核心定理：代数余子式、拉普拉斯展开与特殊行列式	68
3.3.3	代码实现：从递归到高斯消元的算法之旅	78
3.4	行列式的应用	82
3.4.1	历史引入：克拉默法则的诞生与争议	82
3.4.2	核心定理：克拉默法则	83
3.4.3	代码实现：用代码求解线性方程组	86
3.4.4	行列式的算法	89
3.4.5	案例一： $(n+2)$ 阶特殊结构的行列式	89
3.4.6	案例二：一类特殊递推行列式的计算	91
3.4.7	案例三：行和相等型行列式的高效算法	93
3.4.8	案例四：三对角与常数填充型行列式	95
3.4.9	行列式算法的综合比较与选择策略	97
3.4.10	练习与思考	98
3.4.11	本章小结	101
第4章	矩阵	102
4.1	矩阵基础	102
4.1.1	历史引入：行列式的局限与矩阵的诞生	102
4.1.2	历史的岔路：从行列式到矩阵的飞跃	102
4.1.3	数学证明：矩阵的基本运算性质	103
4.1.4	几何角度：矩阵与线性变换	103
4.1.5	代码实现：矩阵的基本操作	103
4.2	矩阵运算与逆矩阵	106

4.2.1	历史引入：逆矩阵的提出与意义	106
4.2.2	核心定理：逆矩阵的定义与存在性	108
4.2.3	数学证明：逆矩阵的唯一性与性质	108
4.2.4	几何角度：逆矩阵与变换的可逆性	108
4.2.5	代码实现：逆矩阵的求解	108
4.3	矩阵的初等变换	111
4.3.1	历史引入：高斯消元法的精髓与代数化	111
4.3.2	核心定理：三类变换、初等矩阵与矩阵等价	112
4.3.3	数学证明：初等变换与初等矩阵的等价性	112
4.3.4	几何角度：剪切、反射与缩放的组合	112
4.3.5	代码实现：用代码模拟行变换与寻找逆矩阵	112
4.4	矩阵的转置	115
4.5	矩阵的转置	115
4.5.1	历史引入：凯莱的对称视角与对偶思想	115
4.5.2	核心定理：转置的运算法则与性质	117
4.5.3	数学证明：乘积转置定理 $(AB)^T = B^T A^T$	117
4.5.4	几何角度：变换的伴随与内积的守恒	117
4.5.5	代码实现：“T”属性的妙用	117
4.6	一些重要的矩阵	120
4.6.1	历史引入：从物理与几何中涌现的特殊结构	120
4.6.2	核心定理：对称、正交与对角矩阵的定义与性质	121
4.6.3	数学证明：特殊性质的推导	121
4.6.4	几何角度：旋转、缩放与正交拉伸	121
4.6.5	代码实现：特殊矩阵的构造与验证	121
4.7	矩阵的分块	124
4.7.1	历史引入：“分而治之”思想的代数体现	124
4.7.2	核心定理：分块矩阵的运算法则	126
4.7.3	数学证明：分块乘法法则的严谨性	126
4.7.4	几何角度：子空间的独立变换	126
4.7.5	代码实现：用代码拼接与分解矩阵	126
4.8	矩阵的概念	130
4.8.1	从实际问题到数学抽象	130
4.8.2	矩阵的定义与记号	131
4.8.3	矩阵的分类与典型实例	132
4.8.4	矩阵的基本运算	133
4.8.5	矩阵相等与增广矩阵	135
4.8.6	历史脉络与应用一瞥	135

4.8.7	矩阵与行列式：概念对照	136
4.8.8	小结	136
4.8.9	练习	136
4.9	矩阵的代数运算	138
4.9.1	线性运算：加法与数乘	138
4.9.2	矩阵乘法	140
4.9.3	方阵的多项式与行列式乘法性	142
4.9.4	典型证明与演算	143
4.9.5	小结	145
4.9.6	综合训练	145
4.10	逆矩阵与矩阵的初等变换	148
4.10.1	逆矩阵	148
4.10.1.1	定义与唯一性	148
4.10.1.2	逆矩阵的性质	149
4.10.1.3	伴随矩阵法求逆	149
4.10.1.4	逆矩阵与线性方程组	150
4.10.1.5	特殊情形与反例	151
4.10.2	矩阵的初等变换	152
4.10.2.1	初等变换的定义与类型	152
4.10.2.2	矩阵的等价与标准形	152
4.10.2.3	初等矩阵	153
4.10.2.4	Gauss-Jordan 消元法求逆矩阵	154
4.11	转置矩阵与一些重要的方阵	157
4.11.1	转置矩阵	157
4.11.2	一些重要的方阵（实数域）	158
4.11.3	选学：复矩阵、埃尔米特矩阵与酉矩阵	161
4.12	分块矩阵及其应用	163
4.12.1	分块矩阵的定义与表示	163
4.12.2	分块矩阵的运算规则	164
4.12.2.1	加法与数乘	164
4.12.2.2	乘法	164
4.12.2.3	转置	165
4.12.3	特殊分块矩阵	165
4.12.3.1	分块对角矩阵	165
4.12.3.2	分块三角矩阵	166
4.12.4	分块矩阵的初等变换	166
4.12.5	分块矩阵的应用	167

4.12.5.1	分块矩阵求逆	167
4.12.5.2	分块矩阵与行列式计算	168
4.12.5.3	分块矩阵在线性代数中的应用	168
第 5 章	线性方程组	171
5.1	向量组与矩阵的秩	171
5.1.1	向量组的秩	171
5.1.2	矩阵的秩	175
5.2	线性方程组的解法	181
5.2.1	非齐次线性方程组的解法	181
5.2.2	齐次线性方程组的解法	185
5.3	线性方程组解的结构	190
5.3.1	齐次线性方程组的基础解系	190
5.3.2	非齐次线性方程组的解的结构	192
5.3.3	典型命题与推论	194
第 6 章	伟大的抽象：线性空间	197
6.1	引言：从具体解到解空间	197
6.2	线性空间的公理体系：源于几何，归于抽象	198
6.2.1	一种新数学的诞生：格拉斯曼的远见	199
6.2.2	几何蓝图：从直观到封闭性	200
6.2.3	八条公理：线性结构的代数基石	201
6.3	抽象的统一性：向量空间的泛化实例	203
6.3.1	结构主义视角：究竟什么是“向量”？	203
6.3.2	向量空间的泛化实例	204
6.3.3	代码视角：用 Python 定义向量的抽象行为	205
6.3.4	应用前沿：为“意义”构建向量空间	206
6.4	n 维线性空间	207
6.4.1	空间的构造基石：张成、线性无关与基	207
6.4.2	空间的度量：维数	208
6.4.3	代码视角：用 NumPy 高效实现高维运算	209
6.4.4	应用前沿：应对“维数灾难”	210
第 7 章	伟大的动词：线性变换	213
7.1	线性变换的文法：一种描述“运动”的规则	213
7.1.1	历史的脉络：从几何运动到代数结构	214
7.1.2	线性变换的定义：保持结构的两个核心条件	214
7.1.3	变换的检验：哪些是线性的，哪些不是？	215

7.2	矩阵：线性变换的代数表示	216
7.2.1	思想的飞跃：由基的变换确定整个变换	217
7.2.2	构造变换矩阵：为变换建立档案	217
7.2.3	坐标的变换： $Y = AX$ 的推导与意义	218
7.2.4	相似矩阵：同一变换的不同代数面孔	219
7.3	特征值与特征向量：变换的不变本质	220
7.3.1	思想的演进：从天体运动到“本征”值	220
7.3.2	定义与求解：寻找不变方向的代数方法	221
7.3.3	对角化：寻找变换的“自然”坐标系	222
7.3.4	结论：从静态结构到动态变化	223
第 8 章	欧氏空间	224
8.1	欧氏空间：从抽象到可度量	224
8.1.1	历史的脉络：从几何测量到内积公理	225
8.1.2	内积的定义：赋予空间度量结构	225
8.1.3	长度、角度与正交：几何概念的代数重生	226
8.1.4	标准正交基与 Gram-Schmidt 过程	228
8.2	正交变换：空间中的刚性运动	229
8.2.1	正交变换的定义与等价刻画	230
8.2.2	正交矩阵：正交变换的代数身份证	231
8.2.3	正交变换的几何本质：旋转与反射	232
第 9 章	二次型	234
9.1	引言：超越线性，直面旋转的世界	234
9.2	n 元实二次型和标准型	236
9.2.1	思想的第一次飞跃：用矩阵“捕获”二次型	236
9.2.2	代数的力量：变量代换与合同变换	237
9.2.3	历史的回响：拉格朗日的配方法	238
9.2.4	不变量的荣耀：西尔维斯特惯性定理	240
9.3	正定二次型	241
9.3.1	动机：为何“恒为正”如此重要？	241
9.3.2	正定性的判别：从定义到实用准则	242
9.4	几何的守护者：正交变换	244
9.4.1	如何用代数语言刻画“保距”变换？	244
9.5	用正交变换化二次型为标准型	246
9.5.1	核心定理：主轴定理的诞生	246
9.5.2	几何的诠释：在自然的坐标系中看世界	247
9.5.3	一个完整的范例：让代数与几何共舞	248

附录 A Book 修订记录	251
附录 B Slides 修订记录	252

第1章 代数简史

1.1 什么是代数？

代数，这个词听起来或许有些抽象，但它的本质思想早已融入我们的日常思维。想象一下，当古人面对“我有3袋谷物，再添几袋才能凑够8袋？”这类问题时，他们实际上已经在进行最朴素的代数思考。

然而，仅仅停留在具体的数字上，人类的智慧便无法走远。每一次遇到类似问题，都需重新构思。代数的革命性飞跃，在于它为我们提供了一套超越具体数字的通用语言——符号。

定义 1.1 (代数 (现代视角))

代数是符号的数学，其词源本意便是“用符号代替数”。它通过一套强大的抽象工具，将人类从繁杂的个例中解放出来，去探索更本质的规律。

其核心力量体现在三个层面：

- 抽象化：不再纠结于“3袋”还是“5只羊”，而是用一个字母（如 x, y, a, b ）代表任意未知或已知的量，实现了从“特例”到“通则”的飞跃。
- 符号运算：将问题转化为由符号构成的方程或多项式，通过一套严谨的运算法则（如移项、合并），研究变量之间普遍存在的关系。
- 结构研究：当符号和运算足够成熟后，代数的目光转向了更高维度——不再仅仅关心“解”是什么，而是去探究运算规则本身所构成的体系，如群、环、域。这如同从学习单个词汇，上升到研究语法和文学。

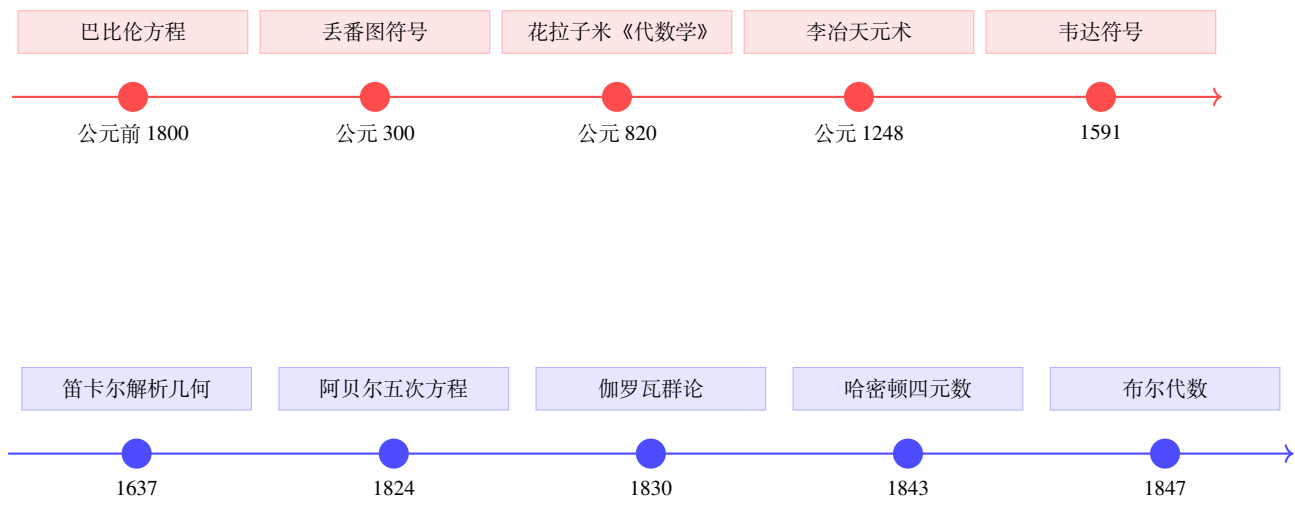
简而言之，代数完成了一次伟大的思想浓缩，它将

$$\text{“3袋谷物再添几袋可得8袋？”} \implies x + 3 = 8 (\implies x = 5)$$

这样的**具体问题**，升华为一个**通用模型**： $x + a = b$ 。一旦我们掌握了这个模型，所有“已知一部分和总量，求另一部分”的问题便迎刃而解。这，就是代数赋予我们的力量——驾驭复杂世界的思维范式。

1.2 代数发展时间线（公元前 2000 年 – 19 世纪）

代数的演化并非一蹴而就，它是一场跨越数千年、遍及全球文明的智慧接力。下面两条时间轴，勾勒出这条从具体求解到抽象结构的壮丽史诗。



从符号到结构：代数如何塑造线性代数的思维范式

这两条时间轴如同一部浓缩的史书，记录了代数思想的两次伟大跃迁。

第一次跃迁，是从“算术”到“符号”。在人类文明的黎明时期，巴比伦人在泥板上求解方程，展现了对线性关系最古老的直觉。然而，他们的数学是用冗长的文字描述的。直到丢番图首次引入缩写符号，代数才开始拥有自己独立的语言，从而与算术分道扬镳。随后，花拉子米系统化了方程的解法，李治的“天元术”在中国创造了独特的符号体系，最终由韦达完成了集大成的工作，用字母普遍地代表已知数和未知数。这数千年的积累，为我们今天所学的线性代数提供了最基础的概念——“变量”与“线性方程”。

第二次跃迁，是从“求解”到“结构”。17 世纪，笛卡尔的解析几何如一道闪电，将代数与几何这两个看似无关的世界连接起来。通过坐标系，几何图形的性质可以转化为代数方程的性质，反之亦然。这正是线性代数核心的几何直觉之源：向量、矩阵和线性变换都可以在这个框架下获得鲜活的生命。然而，故事并未就此结束。当阿贝尔和伽罗瓦探索高次方程的解时，他们发现问题的关键不在于计算，而在于解的对称性。这催生了“群”的概念，引导我们去思考线性空间中那些在变换下保持不变的性质。哈密顿的四元数则大胆地打破了乘法交换律，拓展了“数”的边界，预示了向量和矩阵等更广阔的代数结构。最终，布尔的逻辑代数将人类的思维规则本身也纳入了代数的范畴，为日后的计算机科学铺平了道路。

这些历史的足迹，不仅是数学知识的演进，更是一种思维方式的进化。学习线性代数，我们继承的正是这种从具体到抽象、从计算到结构的思维范式。在现代 AI 中，无论是神经网络中海量的矩阵乘法，还是数据降维时优雅的几何变换，其思想内核都深植于这段伟大的历史之中。

1.3 萌芽阶段（公元前 2000 年 – 公元 500 年）

1.3.1 古巴比伦（约公元前 1800 年）

在人类文明的摇篮——美索不达米亚平原，代数思想的一曙光出现在烧制的泥板上。古巴比伦的书吏们为了解决土地测量、税收计算和天文观测等实际问题，发展出了一套令人惊叹的计算体系。

- **人物与材料：**伟大的成就常常出自无名者之手。这些佚名的古代书吏，在 YBC 7289、Plimpton 322 等传世泥板上，为我们留下了最早的代数记录。
- **成就：**
 - **先进的记数法：**他们采用了基于 60 的六十进制位值制，这种制度的优越性在于 60 有大量的因子，便于进行分数运算。至今，我们仍在时间（60 分、60 秒）和角度（360 度）中使用其遗迹。
 - **惊人的计算精度：**在著名的 YBC 7289 泥板上，他们给出了 $\sqrt{2}$ 的一个近似值，换算成十进制后约为 1.41421296，与真实值 1.41421356 相比，误差仅为百万分之一。
 - **算法思想的雏形：**他们没有现代的公式，而是制作了详尽的**倒数表**、**平方表**，并通过巧妙的迭代算法来逼近平方根。
 - **方程的原始形态：**他们将实际的几何问题，如“一块土地的面积与一边的长度之和为 45”，转化为原始的二次方程，并给出了一套固定的求解步骤。
- **意义与局限：**古巴比伦数学的伟大之处在于，它**首次将现实世界的问题抽象为数学方程的雏形**，并用算法化的思想去求解。然而，他们的代数是“文辞代数”，完全用日常语言叙述，没有专门的符号，每一个问题都像在描述一个食谱，缺乏通用性。
- **应用：**土地测量、税收、天文计算（如预测月食）。

从美索不达米亚到古希腊：代数的抽象化之旅

古巴比伦人的工作，如同在黑暗中摸索的巨人，用强大的计算能力解决了实际问题。他们的迭代思想，在几千年后的今天，依然回响在机器学习的梯度下降法等算法之中。但要让代数成为一门真正的科学，就必须摆脱冗长的文字描述，创造一套简洁、普适的符号语言。

历史的接力棒，在近两千年后传到了古希腊。在这片崇尚逻辑与理性的土地上，一位名叫丢番图（Diophantus）的数学家，将扮演那个点燃符号革命火焰的关键角色。他的工作继承了巴比伦的实用主义精神，但又向前迈出了决定性的一步：**用符号来代表未知数**。正是这一步，让代数开始挣脱算术的束缚，朝着独立的学科方向大步迈进。

1.3.2 丢番图（约公元 200—284 年）

丢番图是罗马时代数学界的一颗璀璨明星。人们对丢番图的生活知之甚少，一般认为他生活在罗马时期的埃及亚历山大港，当时希腊文化的重镇。他生活的年代大概从公元 200 年或 214 年到 284 年或 298 年。他的生平如同一个谜，我们只能从一则流传后世的诗歌形式的墓志铭中窥见他的一生。

注 这则墓志铭本身就是一个绝妙的代数问题，它用诗意的语言，将一个人的生命历程编码成了一个线性方程。

“过路的人啊，这里埋葬着丢番图。

他生命的六分之一是幸福的童年，

又过了生命的十二分之一，他的脸颊长起了胡须；

在此之后，他生命的七分之一，他举行了婚礼；

婚后五年，他喜得贵子。

啊，不幸的孩子，他亲爱的儿子，其寿命只有父亲的一半。

儿子死后，丢番图在深切的悲痛中又活了四年，才结束了自己的生命。”

如果我们设丢番图的寿命为 x 岁，那么这首诗就可以“翻译”成以下的数学语言：

$$\underbrace{\frac{x}{6}}_{\text{童年}} + \underbrace{\frac{x}{12}}_{\text{青年}} + \underbrace{\frac{x}{7}}_{\text{婚前}} + \underbrace{5}_{\text{婚后生子}} + \underbrace{\frac{x}{2}}_{\text{儿子寿命}} + \underbrace{4}_{\text{丧子后}} = \underbrace{x}_{\text{总寿命}}$$

通过简单的移项和通分计算，我们可以解得 $x = 84$ 。丢番图用自己的一生，为我们生动地演示了如何将生活问题代数化。

- **著作：**他的传世巨著《算术》（*Arithmetica*）共有 13 卷，不幸的是，如今仅存 6 卷希腊文手稿和 4 卷后来的阿拉伯文译本。
- **历史定位：**他被誉为“代数学之父”，因为他终结了希腊数学以几何为绝对主导的传统，让代数作为一门独立的学科屹立于世。
- **核心突破：**

1. 符号体系革新——从“文辞代数”到“缩写代数”

- **未知数的符号化：**他首创性地使用希腊字母 ς （ σ / sigma 的古体）来表示未知数，这比用一个完整的词语（如“一堆”）来表示要高效得多。对于未知数的幂，他也创造了缩写符号，例如用 Δ^ς 表示平方（来自希腊语 δύναμις，意为“力量”），用 K^ς 表示立方（来自 κύβος，意为“立方体”）。
- **运算符号的引入：**他还引入了表示减法的符号 $\angle$ 和表示等于的词语“isos”（isos）。
- **历史意义：**虽然这套符号系统还不完备，看起来更像是个人化的简写，但它标志着代数史上一次质的飞跃。数学家们终于可以像写诗一样，用简洁的符号来表达和推演复杂的数学关系，从而将代数从几何图形的直观束缚中解放出来。

2. 不定方程研究——数论的先声

定理 1.1 (丢番图方程)

在《算术》中，丢番图系统地研究了一类特殊的方程：其未知数的个数多于方程的个数，并且要求解必须是整数或有理数。后人为了纪念他，将这类方程命名为“丢番图方程”。其一般形式为：

$$f(x_1, x_2, \dots, x_n) = 0 \quad (\text{要求解在整数 } \mathbb{Z} \text{ 或有理数 } \mathbb{Q} \text{ 集合中})$$



例题 1.1 丢番图在书中解决了大量此类问题，例如：

- **毕达哥拉斯三元组**：求解方程 $x^2 + y^2 = z^2$ 的整数解，如 $(3, 4, 5)$ 。
- **平方和分解**：将一个已知的数（如 25）分解为两个平方数之和，即求解 $a^2 + b^2 = 25$ ，得到有理数解 $(4, 3)$ 。
- **线性方程组**：求解 $x + y = 10, x - y = 2$ ，得到解 $x = 6, y = 4$ 。

值得注意的是，丢番图的解法极富创造性，但往往依赖于巧妙的构思，缺乏一种通用的方法论。

3. 代数方法论的奠基

- **移项与合并**：他已经熟练地运用了现代方程求解中的基本技巧，如将 $5x - 3 = 8$ 变形为 $5x = 11$ 。
- **消元与替换**：通过变量代换来简化复杂的方程。
- **时代的局限**：丢番图的思想超越了他的时代，但也受限于时代。他坚决拒绝负数解，认为它们是“荒谬的”，并且回避无理数，只在正有理数范围内寻找答案。

历史影响：

- 丢番图的《算术》在阿拉伯世界被珍藏和研究，后来在 16 世纪被翻译成拉丁文传入欧洲，直接点燃了文艺复兴时期数学的火炬。法国数学家韦达的符号代数改革深受其影响，而另一位伟大的数学家费马，正是在阅读《算术》时，在页边空白处写下了那个困扰了人类三百多年的“费马大定理”。
- 在现代，丢番图方程的研究已经发展成为数论的一个核心分支，其求解的复杂性也成为现代密码学（如 RSA 公钥密码体系）的安全基石。

从雅典到巴格达：符号革命的千年接力

丢番图的符号化尝试，如同在古希腊几何学统治的夜空中划过的一颗流星，虽然短暂，却照亮了前路。他用字母和缩写打破了文字叙述的桎梏，让代数开始挣脱几何的襁褓，拥有了独立行走的可能。然而，这种符号革新仍停留在个人化的简写阶段，未能形成一套普适的、可以机械化操作的运算规则。

历史的火种需要新的沃土来燃烧。当欧洲进入中世纪的沉寂时，丢番图的遗产穿越了近五百年的时空，在公元 9 世纪的巴格达“智慧宫”中被重新发现和诠释。在这里，一位来自波斯的伟大学者——花拉子米，将从《算术》中

汲取灵感，但他要走得更远。他不仅要解题，更要创造一套“解题的科学”。他将解方程的过程提炼为标准化的步骤（还原与对消），并巧妙地用几何图形来解释抽象的二次方程，最终使代数从一门解题的“艺术”升华为一门独立的“科学”。这场从“个人符号”到“系统规则”的进化，正是代数思想的一次关键跃迁。

1.4 符号化与系统化（公元 800 年 – 17 世纪）

1.4.1 花拉子米（Al-Khwarizmi，约 780–850 年）

在阿拉伯帝国的黄金时代，巴格达的“智慧宫”汇聚了世界顶级的学者。花拉子米正是其中最耀眼的明星之一，他的贡献深刻地塑造了我们今天所知的数学。

注 一个名字，两个学科的诞生。花拉子米的名字被拉丁化为 ALGORITHM，这正是现代计算机科学核心概念“算法”（algorithm）的词源。而他最重要著作的标题中的一个词 *al-jabr*，经过历史的演变，成为了“代数”（algebra）。

• 生平背景：

- 他是一位波斯裔的数学家、天文学家和地理学家，出生于中亚的花刺子模地区（今乌兹别克斯坦），其学术生涯的顶峰是在阿拔斯王朝的首都巴格达度过的。
- 当时的“智慧宫”是一个庞大的学术翻译和研究中心，它系统地翻译和吸收了古希腊、古印度和波斯的科学典籍，为花拉子米的创新提供了肥沃的土壤。
- **核心著作：**《还原与对消之书》（*Kitāb al-mukhtaṣar fī ḥisāb al-jabr wa-l-muqābala*，约公元 820 年），这本书的出现，标志着代数作为一门独立学科的正式诞生。

• 代数学的范式革命：

1. 方程的系统分类与通用解法

与丢番图不同，花拉子米的目标不是解决各种巧妙的难题，而是为所有二次方程提供一套标准化的“操作手册”。由于当时数学界仍未接受负数和零，他不得不将所有二次方程按照系数的正负，划分为六种基本类型。

定理 1.2 (二次方程六型分类)

设未知数为 x ，其平方为“一个平方” (x^2)，其本身为“一个根” (x)，常数为“一个数” (c)。并规定系数 a, b, c 均为正数，花拉子米将所有二次

方程归纳为以下六种标准形式：

$$\text{平方等于根:} \quad ax^2 = bx$$

$$\text{平方等于数:} \quad ax^2 = c$$

$$\text{根等于数:} \quad bx = c$$

$$\text{平方和根等于数:} \quad ax^2 + bx = c$$

$$\text{平方和数等于根:} \quad ax^2 + c = bx$$

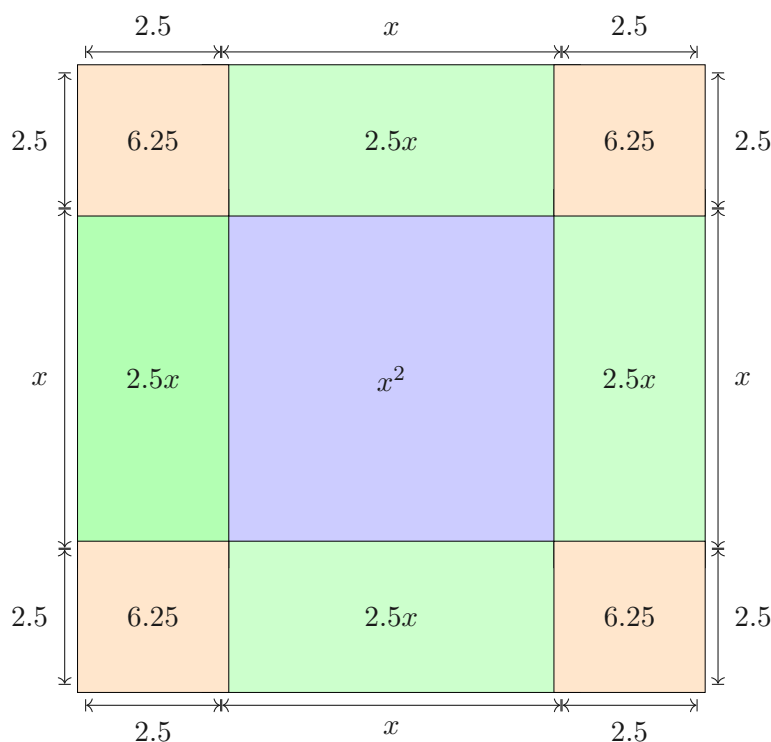
$$\text{根和数等于平方:} \quad bx + c = ax^2$$



为了让这些抽象的方程易于理解，花拉子米创造性地引入了几何证法，即我们今天熟知的“配方法”的几何解释。

证明 [几何证法]：以他书中的一个经典例子 $x^2 + 10x = 39$ (对应类型) 为例，他通过如下的几何构造给出了求解过程：

- (a). 构造核心正方形：首先，画一个边长为 x 的正方形，其面积为 x^2 。
- (b). 附加四个矩形：方程左边还有一项 $10x$ ，他巧妙地将其拆分为四个部分。在正方形的四个边上各附加一个长为 x 、宽为 $\frac{10}{4} = 2.5$ 的矩形。这四个矩形的面积之和恰好是 $4 \times (2.5 \times x) = 10x$ 。
- (c). “配方”成大正方形：此时，图形的四个角上还留有空缺。用四个边长为 2.5 的小正方形将它们填满。每个小正方形的面积是 $2.5^2 = 6.25$ ，总面积为 $4 \times 6.25 = 25$ 。
- (d). 建立面积等式：这样，我们就得到了一个边长为 $(x + 2.5 + 2.5) = (x + 5)$ 的大正方形。其总面积等于原有的面积 $x^2 + 10x$ (也就是 39)，加上我们新补上的四个小正方形的面积 25。所以， $(x + 5)^2 = 39 + 25 = 64$ 。
- (e). 开方求解：两边开方，得到 $x + 5 = 8$ ，因此解得 $x = 3$ 。(当时只取正根)



$$\begin{aligned}
 (x+5)^2 &= x^2 + 10x + 25 = 39 + 25 = 64 \\
 x+5 &= 8 \\
 x &= 3
 \end{aligned}$$

这种方法将抽象的代数运算转化为直观的几何拼接，极大地增强了理论的说服力。

2. 代数运算的奠基性规则

书名中的两个核心词汇，定义了代数运算最基本的操作：

- **Al-jabr (还原)**：这个词的原意是“接骨”或“恢复”。在数学中，它指移项操作，将一个负数项从等式的一边移到另一边，使其变为正数。例如，将 $2x^2 - 5x = 8$ “还原”成 $2x^2 = 5x + 8$ 。这个操作的目的是为了避免处理负数。
- **Al-muqābala (对消)**：意为“平衡”或“对消”。它指合并等式两边的同类项，以简化方程。例如，将 $2x^2 + 3x = x^2 + 7$ “对消”为 $x^2 + 3x = 7$ 。
- **历史意义**：这两条规则的提出，是代数史上的一座丰碑。它首次将方程的求解过程，从依赖灵感的艺术，转变为一套可以遵循的、机械化的程序，确立了“化简方程”作为代数核心任务的地位。

● 局限与超越：

- 与他的前辈一样，花拉子米的代数世界里只有正有理数解，负数和无理数仍被视为“不存在”或“荒谬”。
- 尽管如此，他的著作在 12 世纪被翻译成拉丁文传入欧洲，成为了之后几个世纪欧洲大学的标准教材，直接启发了斐波那契等数学家，并最终推动了负数

和代数符号体系的完善。

• 历史回响：

“是花拉子米，让代数从几何的婢女，变成了独立的王后。”

——数学史家卡茨（Victor J. Katz）

他不仅命名了代数，更定义了代数，为这门学科注入了算法的灵魂。

东方与西方的遥相辉映

当花拉子米的代数思想在阿拉伯世界和后来的欧洲生根发芽时，在遥远的东方，中国的数学家们正沿着一条截然不同但同样辉煌的道路，将代数推向了一个新的高峰。如果说花拉子米的贡献在于“系统化”，那么中国宋元时期的数学成就则在于“算法化”和“符号化”的极致。

在 13 世纪，当欧洲还在努力消化花拉子米的二次方程时，中国的数学家秦九韶和李冶等人，已经发展出求解任意高次方程的数值方法和独特的符号系统，其思想的深度和技术的先进性，遥遥领先于同时代的西方长达数百年。

1.4.2 中国宋元时期

注 宋元时期（10-14 世纪）是中国传统数学的巅峰。在商业繁荣、科技发展的推动下，秦九韶、李冶、杨辉、朱世杰等数学巨匠辈出。他们摆脱了对几何直观的依赖，发展出一套以算筹为计算工具，具有高度符号化和机械化特征的代数体系，集中体现在高次方程求解（天元术）和多元方程组消元（四元术）上。

• 秦九韶（约 1202—1261）

- **历史坐标：**他生活在南宋末年的乱世，是一位经历复杂的官员和数学家。1247 年，他在为母亲守孝期间，完成了不朽巨著《数书九章》，书中收录了 81 个来自实践的复杂问题，涵盖天文、历法、土木、赋役等领域。
- **增乘开方法：**高次方程的“流水线”解法面对形如 $a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0 = 0$ 这样的高次方程，欧洲数学家在 16 世纪还在为三、四次方程的求根公式苦苦挣扎，而秦九韶在《数书九章》中已经给出了一套普适的数值求解算法——“正负开方术”，后世称为**增乘开方法**。

定理 1.3 (增乘开方法)

这套算法在思想上与 19 世纪英国数学家霍纳（Horner）提出的方法完全一致，但却早了近 600 年！它是一种高效的迭代算法，可以求出高次方程的正实根，其过程高度程序化，非常适合在算筹（一种古代计算工具）上实现。其核心思想可以概括为“逐位逼近”，如同我们做长除法一样，从高位到低位，一位一位地把根“算”出来。

整个流程可以分解为四个关键步骤：

1. 估根 (立商)：首先，通过观察方程首项系数和常数项的数量级，对根的最高位数 (即“商”) 进行一个大致的估计。
2. 降次 (益)：将估算出的根的最高位数值，通过一系列“随乘随加”的操作，作用于原方程的系数，得到一个余式和一个新的、次数不变但系数更新的方程。这一步在代数上等价于对多项式 $P(x)$ 做变换，计算出 $P(y+d)$ 的系数，其中 d 是根的已确定部分。
3. 再估 (续商)：在新得到的方程基础上，用类似的方法，估计根的下一位数字。
4. 迭代：重复步骤 2 和 3，像剥洋葱一样，一层一层地求出根的每一位数字，直到达到所需的精度，或者余式为零。



例题 1.2 《数书九章》原题：遥度圆城 书中一个问题可以转化为求解一个四次方程：

$$-x^4 + 763200x^2 - 40642560000 = 0$$

(为了方便计算，我们乘以 -1，并写成现代形式)

$$x^4 - 763200x^2 + 40642560000 = 0$$

目标是求这个“圆城”的直径 x 。

Step 1: 估算首位 (立商) 秦九韶通过观察常数项 $40642560000 \approx 4 \times 10^{10}$ ，和 x^4 的关系，估计出根 x 大概在 800 到 900 之间。于是，他立商为 800 (百位位置 8)。

Step 2: 第一次迭代 (商 8) 接下来，他运用一套精巧的算筹布阵法 (即 Horner 算法的表格形式)，将方程的系数 ‘(1, 0, -763200, 0, 40642560000)’ 和商 ‘800’ 进行运算。

	1	0	-763200	0	40642560000
800	800	640000	-98560000	-78848000000	
	1	800	-123200	-98560000	-38205440000

运算后得到一个负的余数，说明 800 估得偏小了。同时，他也得到了一组新的系数 ‘(1, 800, -123200, -98560000)’，用于下一步计算。

Step 3: 第二次迭代 (商 40) 在新的系数基础上，秦九韶继续估算下一位。他发现商的十位应为 ‘4’ (即 40)。于是用 ‘40’ 对新的系数进行相同的“增乘”运算。

	...	...	...	...
40	...	...	...	0
	...	...	...	0

(为简化展示，此处省略具体计算过程) 经过这一轮运算，余数恰好为零！

Step 4: 得出结果 第一步的商是 800, 第二步的商是 40。将它们相加, 得到方程的根 $x = 800 + 40 = 840$ 。

小结: 整个过程只涉及基本的加法和乘法, 通过一套标准化的流程, 竟能精确地求解出如此复杂的高次方程。这不仅是计算技术的胜利, 更是算法思想的胜利, 堪称 13 世纪的“机械计算机”!

• 李冶 (1192-1279)

如果说秦九韶将代数的“算法化”推向了极致, 那么与他同时代的另一位伟大数学家李冶, 则在“符号化”方面取得了历史性的突破。

- **历史背景:** 李冶是金元时期的数学家。金朝灭亡后, 他隐居山中, 潜心著述, 在 1248 年完成了《测圆海镜》, 系统地创立了“天元术”。

定义 1.2 (天元术)

“天元术”是一种用来建立和求解高次方程的符号方法。它的核心思想是:

1. “立天元一为某某”: 首先, 选择问题中的一个未知量, 用符号“元”来表示它, 这完全等价于我们今天设未知数 x 。
2. 竖式多项式表示法: 李冶创造了一种独特的竖式方法来表示多项式。常数项写在中间, 记为“太”(代表太极, 万物之始)。“元”(即 x) 的一次幂的系数写在“太”的上一格, x^2 的系数再上一格, 以此类推。负数则在相应系数的数字上加一个斜划。

例如, 方程 $-x^2 + 260x - 6800 = 0$ 用天元术表示出来, 就是这样一个“算盘”式的布局:

(x^2 系数: -1)

(x^1 系数: 260)

(常数项定位符)

(x^0 系数: -6800)

这种表示法, 虽然形式上与今天不同, 但其本质已经是一个完全的符号系统, 并且非常适合在算筹上进行多项式的加减乘除运算。



• 天元的现代传承: 数学天元基金

为发挥我国数学研究潜力、推动其率先跻身世界先进水平, 国家领导人采纳了吴文俊、王元、程民德等十位老中青数学家组成的“二十一世纪中国数学展望”组委会建议, 自 1989 年起由国家财政拨专款设立“数学特别基金”(即“数学天元基金”), 用于资助国内特优数学人才、吸引海外优秀数学人才回国等。专款通过“带帽下达”至国家自然科学基金委员会, 由专家委员会统筹使用; 同时, 基金委成立了由 13 位知名数学家组成的“数学天元基金”学术领导小组, 负责指导、管理申请与评审等工作。“天元”得名, 既因李冶、朱世杰在十二世纪创造的“天元术”(作为古代高次方程求解的领先方法, 彰显

中国古代数学成就)，也因“天元”是围棋棋盘中心与冠军的象征，蕴含“高、远、优、始”的内涵——这一命名由华东师范大学张奠宙教授提议，获国家自然科学基金委员会认可：它与“推动数学创新、树立远大目标”的初衷高度契合，能激励数学界向世界先进水平迈进。

- **几何问题的代数化**：李冶在《测圆海镜》中，系统地利用天元术，将复杂的几何问题（主要是关于圆和直角三角形的各种求值问题）转化为标准的高次方程，然后再用类似于增乘开方法求解。

证明 [“几何代数化”范例]：一个经典的“勾股容圆”问题（求直角三角形内切圆的直径）：

1. 立天元：设内切圆直径为 x （“立天元一为圆径”）。
2. 列方程：利用三角形的几何关系（相似、面积等），将其他未知量（如勾、股、弦的各部分线段）都用含 x 的代数式表示出来。经过一系列推导，最终得到一个关于 x 的高次方程。
3. 求解：用开方术解出 x 的值。

这种“设未知数→列方程→解方程”的标准化流程，正是代数解决问题的精髓，其思想内核与几百年后笛卡尔的解析几何异曲同工。

- **历史影响与局限**：天元术的出现，标志着中国古代数学从以计算为中心，转向了以方程为中心，是一次深刻的范式革命。后来，朱世杰将其推广到可以处理四个未知数的“四元术”。然而，这种符号系统过于依赖算筹的物理形态，未能演化成更便于书写的横式字母符号，这在一定程度上限制了其进一步的抽象和传播。

● 宋元数学的启示

“中国古代数学，特别是宋元时期的数学，其思想和成就是辉煌的，它走的是一条与西方欧几里得公理化体系完全不同的、构造性的、算法化的道路。”

——吴文俊《数学机械化》

秦九韶的机械化算法与李冶的符号化尝试，共同揭示了数学的另一种力量——并非源于逻辑演绎，而是源于构造与计算。这种思想在计算机时代获得了新生，吴文俊院士开创的“数学机械化”正是在此基础上，将几何定理的证明转化为代数方程组的求解，让计算机可以自动证明定理。

从机械算法到符号宇宙：东西方思想的交汇

宋元数学的辉煌成就，如同算法的精密机械，将高次方程的求解过程拆解为标准化的流水线作业。秦九韶的增乘开方法和李冶的天元术，共同构建了一个以“程序化”解决问题为核心的代数世界。这是一种强大的、面向问题的思维范式。

然而，当朱世杰的四元术试图处理更复杂的多元方程组时，这种依赖算筹的体系触碰到了复杂性的天花板。数学的发展，亟需一场更深刻的革命——从“如何算”的算法层面，跃迁到“是什么”的语言层面。它需要一种不依赖任何计算工具、能够普遍描述数量关系的通用语言。

历史的巧合总是耐人寻味。当东方的算法成就因历史变迁而沉寂时，欧洲正经历着文艺复兴的洗礼。在 16 世纪的法国，一位名叫韦达的律师，在业余时间里，将为代数创造这样一种全新的语言。他将彻底抛弃用具体词语或特定符号表示未知数的做法，引入了用字母普遍代表“数”的思想。这不仅仅是一个记号的改变，它标志着代数研究的对象从“解出某个特定的数”，转变为“研究数与数之间的一般关系”。一个由纯粹符号构成的宇宙，即将诞生。

1.5 符号体系的革命 (16 世纪 – 17 世纪)

1.5.1 韦达 (François Viète, 1540–1603)

在 16 世纪的法国，代数迎来了一位关键的变革者——弗朗索瓦·韦达。他是一位职业律师和密码破译专家，数学只是他的业余爱好。然而，正是他在密码学中获得的“符号替换”思想，为代数带来了一场深刻的革命。

注 韦达的工作，标志着代数学从“解方程的技巧”正式升华为“研究方程结构的科学”。他的著作《分析方法入门》(1591) 为代数构建了第一个真正意义上的符号语言，使之成为一门普适的、抽象的学科。

- **符号化革命：从“算数”到“类数”** 韦达之前的代数，处理的都是具体的数字方程，如 $x^2 + 10x = 39$ 。韦达的革命性创举在于，他主张用字母来表示所有的数，无论是已知的还是未知的。

定理 1.4 (字母表示法则)

韦达提出了一个影响深远的约定：

- 用元音字母 (A, E, I, O, U) 表示未知量 (变量)。
- 用辅音字母 (B, C, D...) 表示已知量 (参数)。



这个看似简单的改变，却有着划时代的意义。从此，数学家们可以不再讨论“一个数”，而是讨论“一类数”。例如，花拉子米需要区分六种二次方程，但在韦达的体系下，它们都可以被统一成一个单一的、一般的形式。

例题 1.3 传统问题“一个数的平方加上这个数的 3 倍等于 4”，在不同代数阶段的表达方式：

- **文辞代数**：完全用语言描述。
- **缩写代数 (丢番图)**：用符号代表未知数，但系数仍是具体数字。

- **韦达的符号代数**： $A^2 + B \cdot A = C$ 。这里的 A 代表任何未知数，B 和 C 代表任何已知数。这不再是一个算术问题，而是一个关于“二次关系”这一普遍结构的研究。
- **方程理论的基石**： **韦达定理**正是因为有了符号化的系数，韦达才可能去探索方程的根与系数之间的普适关系。
- **韦达定理的诞生**：对于二次方程 $x^2 + px + q = 0$ 的两个根 x_1, x_2 ，韦达首次系统地证明了它们满足关系：

$$x_1 + x_2 = -p \quad \text{且} \quad x_1 x_2 = q$$

他还将这一关系推广到了更高次的方程，揭示了方程内部深刻的对称结构。这在之前的代数体系中是无法想象的。

- **密码破译的胜利**：韦达的代数威力在现实中得到了惊人的展现。在法国与西班牙的战争中，他成功破译了西班牙的复杂军事密码，令西班牙国王腓力二世大为震惊，甚至向教皇指控法国在使用“黑魔法”。1594 年，荷兰大使向法国国王亨利四世炫耀，说他们的数学家罗曼纽斯提出了一个全欧洲无人能解的 45 次方程。韦达当场就在几分钟内指出了方程的解法，并事后给出了全部 23 个正根。他所用的方法，正是巧妙地利用了三角函数 $\sin(45\theta)$ 的展开式与该方程的联系，这显示了其符号代数在统一不同数学分支上的巨大潜力。
- **历史局限与超越**
 - 韦达的思想虽然极具革命性，但他的符号系统仍有不完善之处。例如，他没有使用今天的‘+’和‘=’符号，幂的表示也比较繁琐（如 A^2 记作 $A \text{ quadratum}$ ）。
 - 他仍然拒绝承认负数解，认为它们是“伪解”。
 - 尽管如此，韦达为代数铺设的符号化跑道，很快将由一位更彻底的思想家——笛卡尔——来完成最后的冲刺。

1.5.2 笛卡尔 (René Descartes, 1596–1650)

笛卡尔，这位近代哲学的奠基人，以其名言“我思故我在”而闻名于世。在数学领域，他同样完成了一项“思想”的革命，他用代数语言精确地描述了整个几何世界。

注 传说，笛卡尔在病榻上静养时，看到天花板上有一只苍蝇在爬行。他突然想到：可以用苍蝇到两面墙的距离来唯一地确定它的位置。这个灵感，最终催生了现代科学的基石之一——坐标系。1637 年，他发表了《方法论》，其附录《几何学》的问世，宣告了“解析几何”的诞生。

- **解析几何：形与数的伟大统一**。笛卡尔的核心思想是建立“数”与“形”之间的一座桥梁。他证明了，平面上的任意一条曲线，都对应着一个关于两个变量的代数方程。

定理 1.5 (几何代数化基本定理)

通过引入一个直角坐标系，几何对象与代数方程之间可以建立一一对应的关系：

- 几何图形 $\iff$ 代数方程
- 几何变换 $\iff$ 代数运算

例如：

- 圆心在原点、半径为 r 的圆 $\iff$ 方程 $x^2 + y^2 = r^2$
- 顶点在原点、开口向上的抛物线 $\iff$ 方程 $y = ax^2$
- 椭圆 $\iff$ 方程 $\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1$



从此，研究几何图形的性质（如求切线、求交点）可以转化为解代数方程组。困扰了古希腊人上千年的三大几何作图难题，最终也是在解析几何的框架下，被证明为不可能的。这场“几何的代数化”，是数学史上最深刻的革命之一。

- **符号标准化：现代数学语言的定型。**如果说韦达是代数语言的创造者，那么笛卡尔就是这位语言的规范者和推广者。他在《几何学》中使用的符号系统，基本上就是我们今天沿用的系统。

- **字母规则的完善：**他改进了韦达的规则，约定用字母表中靠前的字母 a, b, c 来表示已知量，用靠后的字母 x, y, z 来表示未知量。这个习惯一直延续至今。
- **指数记法的发明：**他创造性地使用上标来表示幂，如用 x^3 来代替韦达繁琐的 x cubus。这个小小的改进，极大地简化了代数表达式的书写和运算。
- **运算符号的统一：**他系统地使用了‘+’，‘-’等运算符号，使得代数表达式的书写完全现代化。

- **哲学与科学方法的革命**

“所有可以归入‘量’或‘序’的问题，都可以用数学方法来处理。”——笛卡尔《指导哲理之原则》

笛卡尔的解析几何不仅仅是一项数学成就，它更是一种全新的世界观和方法论。他主张，所有科学问题都应该像几何问题一样，被分解为最简单的组成部分，然后用代数方法进行精确的推理和计算。这种思想深刻地影响了牛顿等后来的科学家，可以说，没有笛卡尔的解析几何，就没有牛顿的经典力学。

从符号革命到结构革命：代数学的范式跃迁

韦达的符号化和笛卡尔的几何化，共同为数学世界构建了一套前所未有的强大语言。代数从此拥有了坚实的基础，能够精确地描述和分析各种数量关系与空间形式。在这套语言的驱动下，微积分在牛顿和莱布尼茨的手中应运而生，人类分析自然现象的能力达到了新的高度。

然而，到了 18 世纪末，数学家们在攀登一座新的高峰时遇到了障碍——五

次及更高次的方程。几个世纪以来，人们已经找到了一、二、三、四次方程的通用求根公式，但五次方程的求根公式却迟迟未能发现。无数数学家尝试用更复杂的符号运算去攻克它，但都以失败告终。

问题的突破，需要一次全新的思想跃迁。两位年轻的数学家——阿贝尔和伽罗瓦——意识到，解方程的秘密，可能并不隐藏在“如何计算”的技巧中，而是在方程的根与根之间存在的一种看不见的“对称性”之中。他们将研究的焦点，从“求根”这一行为，转向了“根的结构”这一更深层次的对象。代数学即将迎来它的第三次，也是最深刻的一次革命：从研究“运算”，到研究“结构”。一个由“群”所主宰的全新代数世界，即将拉开序幕。

1.6 结构代数与现代抽象阶段 (18 世纪 – 19 世纪)

1.6.1 阿贝尔 (Niels Henrik Abel, 1802–1829)

在数学史上，很少有人像阿贝尔一样，在如此短暂和困苦的一生中，做出了如此巨大而深刻的贡献。这位挪威的天才数学家，以其惊人的洞察力，为持续了三百年的高次方程求根问题画上了一个否定的句号。

注 阿贝尔一生在贫困和疾病中挣扎，年仅 26 岁便因肺结核去世。在他去世两天后，一封来自柏林的信才姗姗来迟，信中通知他已被任命为柏林大学的教授。为了纪念他，挪威政府在 2002 年设立了数学界的顶级奖项——“阿贝尔奖”。

- **五次方程无解定理：一个时代的终结。** 自 16 世纪意大利数学家发现三、四次方程的求根公式以来，寻找五次方程的通用求根公式便成了无数数学家梦寐以求的目标。然而，阿贝尔证明了，这条路是走不通的。

定理 1.6 (阿贝尔-鲁菲尼定理)

一般的五次及更高次的代数方程，不存在“根式解”。也就是说，它的解无法像一元二次方程的 $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$ 那样，由方程的系数经过有限次的加、减、乘、除和开方运算来表示。



证明 [思想精髓：置换群的萌芽] 阿贝尔的证明是革命性的。他没有去尝试寻找一个公式，而是去分析“如果”一个根式解存在，那么方程的根必须满足什么样的性质。他发现，一个方程如果能用根式求解，那么它的所有根在进行某些特定的交换（置换）后，根与根之间的代数关系必须保持一种非常特殊的对称性。而对于一般的五次方程，其根的置换群（后来由伽罗瓦正式命名为 S_5 ）的结构过于复杂，不具备这种特殊的对称性。因此，通用的根式解不可能存在。

这个石破天惊的结论，宣告了古典代数学“以解方程为中心”的时代的终结。它告诉数学家们，仅仅依赖符号演算的技巧是有极限的，必须去探索代数对象背后更

深层次的“结构”。

- **阿贝尔群与阿贝尔积分。**除了五次方程，阿贝尔还在分析学领域做出了奠基性的工作。他研究的椭圆函数和阿贝尔积分，揭示了复变函数理论中深刻的代数结构。为了纪念他，后来的数学家将一类满足乘法交换律的群命名为“阿贝尔群”，这是群论中最基本、最重要的一类研究对象。

1.6.2 伽罗瓦 (Évariste Galois, 1811–1832)

如果说阿贝尔宣判了古典代数的死刑，那么伽罗瓦则为现代代数——即“抽象代数”——谱写了光辉的序曲。这位生命比阿贝尔更加短暂、更加富有戏剧性的法国天才，用“群”的概念，彻底揭示了方程可解性的本质。

注 伽罗瓦是一位思想激进的共和主义者，因政治活动两次入狱。20 岁时，他为了—位女性，陷入—场荣誉决斗。在决斗的前夜，他预感自己将不久于人世，于是彻夜不眠，奋笔疾书，将他关于方程论的全部思想精华浓缩在几十页手稿中，并潦草地在页边写下“我没有时间了”。第二天，他果然在决斗中腹部中弹，次日去世。这份传奇性的手稿，直到 14 年后才被数学家刘维尔发现其价值并公之于众，从此改变了代数学的进程。

- **伽罗瓦理论：用“群”解开方程的对称密码。**伽罗瓦继承并极大地发展了阿贝尔的思想。他天才地发现，每一个代数方程，都内蕴着一个描述其根的对称性的代数结构——这个结构就是“群”。

定义 1.3 (伽罗瓦群)

对于一个给定的多项式方程，其“伽罗瓦群”是这样—个集合：它包含了所有能够“重新排列”（置换）方程的根，但同时又保持根与根之间所有代数关系不变的变换。这个群的大小和结构，精确地刻画了方程的“对称性程度”。♣

基于这个概念，伽罗瓦建立了—个惊人的定理，完美地回答了“—个方程为何能（或不能）用根式求解”的终极问题。

定理 1.7 (伽罗瓦基本定理 (可解性判据))

—个代数方程可以用根式求解的充分必要条件是：它的伽罗瓦群是“可解群”。♥

所谓“可解群”，直观上讲，就是—个结构相对“简单”的群，它可以被逐级地分解为—些更基础的循环群。伽罗瓦证明了，对于 $n \geq 5$ 的—般 n 次方程，其伽罗瓦群 S_n 不是—个可解群，因此它们不存在通用的根式解。这个理论不仅重新证明了阿贝尔的结论，更提供了—个判断任意给定方程是否可解的通用“判据”。

- **结构思想的诞生。**伽罗瓦的贡献远不止于解决方程论。他开创了—种全新的数学思想范式：
 - **研究对象从“数”转向“结构”：**他让我们看到，研究—个数学对象（如方程），最深刻的方法是去研究它的对称性结构（群）。

- **一一对应思想**：他建立了方程的域扩张与伽罗瓦群子群之间的一一对应关系，使得我们可以用群论的语言来翻译和解决数域的问题。这种思想后来被推广到数学的各个分支，成为了整个现代数学的支柱。

1.6.3 哈密顿 (William Rowan Hamilton, 1805–1865)

当伽罗瓦在代数世界内部掀起结构革命时，爱尔兰的数学家和物理学家哈密顿，则从外部大胆地拓展了代数的边界。他创造了一种全新的“数”，并勇敢地打破了一条看似神圣不可侵犯的运算法则。

注 1843 年 10 月 16 日，哈密顿在与妻子散步走过都柏林的布鲁姆桥时，脑海中一道灵光闪过，他终于想通了如何将复数推广到更高维。激动之下，他立刻用小刀将这个关键的公式刻在了桥的石头上：

$$i^2 = j^2 = k^2 = ijk = -1$$

这个公式，宣告了“四元数”的诞生。如今，这座桥上镶嵌着一块纪念牌，记录着这一数学史上的顿悟时刻。

- **四元数：打破乘法交换律**。哈密顿长久以来试图寻找一种三维的“复数”，可以用来描述三维空间的旋转。但他发现，在三维空间中无法自治地定义乘法。最终，他在四维空间中找到了答案。

定义 1.4 (四元数)

一个四元数 q 可以写成 $q = a + bi + cj + dk$ ，其中 a, b, c, d 是实数， i, j, k 是三个虚数单位。它们的乘法规则除了哈密顿刻下的公式外，还包括：

$$ij = k, \quad jk = i, \quad ki = j$$

但最关键的是，它们的乘法顺序是不能颠倒的：

$$ji = -k, \quad kj = -i, \quad ik = -j$$

也就是说，对于四元数， $ij = -ji \neq ji$ 。



这是人类历史上第一个被系统研究的**非交换代数**。哈密顿的发现，打破了自古以来“乘法交换律”天经地义的思维定势。它告诉人们，数学家可以像立法者一样，根据需求去定义和创造全新的、具有不同运算法则的代数系统。这极大地解放了思想，为后来矩阵、向量等更广泛的代数结构的出现铺平了道路。

- **现代应用**。虽然四元数在诞生之初显得有些怪异，但它在今天却扮演着至关重要的角色。单位四元数可以非常高效、稳定地表示三维空间中的旋转，避免了欧拉角等方法中出现的“万向锁”问题。因此，在计算机图形学、3D 游戏引擎（如 Unity, Unreal）、航空航天（如航天器姿态控制）和机器人学中，四元数是核心的计算工具。

1.6.4 布尔 (George Boole, 1815–1864)

在 19 世纪中叶，另一位自学成才的英国数学家乔治·布尔，将代数的抽象化推向了终极——他将代数的方法应用于人类的逻辑思维本身。

注 布尔出身贫寒，只上过小学，通过自学成为了一名大学教授。他认为，人类的逻辑推理过程，也可以像代数运算一样，用一套符号和法则来精确描述。他的墓碑上刻着“纯粹数学之父”，以表彰他将思维规律本身纳入数学疆域的伟大创举。

- **布尔代数：逻辑的数学化** 在 1847 年和 1854 年，布尔发表了《逻辑的数学分析》和《思维的规律》，创立了一套全新的代数系统——布尔代数。

定义 1.5 (布尔代数)

这是一个只处理两个值的集合 $\{0, 1\}$ （或等价地，{真, 假}）的代数系统。它定义了三种基本运算：

- **与 (AND):** $x \wedge y$ 或 $x \cdot y$ 。只有当 x 和 y 都为 1 时，结果才为 1。
- **或 (OR):** $x \vee y$ 或 $x + y$ 。只要 x 或 y 中有任何一个为 1，结果就为 1。
- **非 (NOT):** $\neg x$ 或 $\bar{x}$ 。将 1 变为 0，0 变为 1。

布尔代数最独特的法则是幂等律： $x^2 = x \cdot x = x$ 。在逻辑上，这意味着“一个陈述的真实性，并不会因为它被重复而改变”。



布尔的工作，第一次成功地将亚里士多德以来的传统形式逻辑，转化为一个可以进行代数演算的数学系统。

- **信息时代的基石。** 布尔代数在它所处的时代，更像是一种哲学和数学上的奇思妙想。然而，在大约一百年后，它却成为了数字时代的理论基石。
- **香农的开关电路：** 1937 年，年轻的美国工程师克劳德·香农在他的硕士论文中指出，可以用布尔代数来完美地描述和设计电子开关电路。电路的“开”和“关”两种状态，正好对应布尔代数中的“1”和“0”。“与、或、非”运算，则对应于基本的逻辑门电路。
- **计算机的语言：** 香农的发现，将布尔的抽象逻辑与物理世界的电子元件连接了起来。从此，所有的计算机硬件设计，以及所有的计算机程序逻辑（如‘if...else’语句），其最底层的数学原理都是布尔代数。

可以说，没有布尔，就没有我们今天的信息时代。

1.7 小结

代数从古巴比伦的泥板上萌芽，到 19 世纪的抽象结构，其发展史是一部波澜壮阔的人类思维进化史。我们可以将其浓缩为三次伟大的思想跃迁：

具体计算 (巴比伦) → 符号语言 (韦达、笛卡尔) → 抽象结构 (伽罗瓦、哈密顿、布尔)

至此，代数的核心任务，已经完成了从“求解未知数”到“研究代数结构本身”的深刻转变。19 世纪之后，数学家们的研究焦点转向了群、环、域、向量空间等更为抽象的对象，开启了我们今天所称的“近世代数”的大门。

对线性代数学习的启示 我们为什么要花费篇章来回顾这段历史？因为理解历史，是理解线性代数本质的最佳途径。本书采用的几何-历史-代码三维框架，正是对这三次思想跃迁的现代回应：

- **几何视角**（如将行列式理解为有向体积）继承了笛卡尔“形数结合”的伟大思想，帮助我们摆脱纯粹符号运算的枯燥，建立直观的几何想象。
- **历史脉络**（如从《九章算术》的消元法到高斯消元法）让我们看到，矩阵、向量空间等“结构化”工具，是人类为了应对日益复杂的线性问题而必然发明的选择。
- **代码实践**（如用 PyTorch 或 NumPy 进行张量运算）则是将这些抽象的代数结构，转化为可以在计算机上运行和验证的实体模型，实现了从理论到实践的闭环。

AI 时代的代数思维 在今天，代数思想的重要性前所未有地凸显。因为深度学习的本质，就是在高维向量空间中寻找和学习复杂的代数映射：

- 神经网络的每一次前向传播，其核心都是一系列的矩阵乘法和向量加法，即 $\mathbf{y} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$ 。这正是在高维空间中进行线性变换和非线性激活。
- Transformer 模型中关键的自注意力机制 $Attention(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$ ，其本质是在向量空间中通过内积（点积）来衡量不同向量之间的“相关性”或“相似性”结构。

掌握这种结构化的代数思维，才能真正理解现代 AI 模型的工作原理，甚至去创造新的模型。例如，AlphaFold 预测蛋白质结构，其数学基础就是将氨基酸序列编码为李群和李代数中的元素，通过研究这些代数结构的几何性质来预测其在三维空间中的折叠形态。

因此，学习线性代数的终极目标，绝不仅仅是为了考试或解题，而是为了获得一种强大的、能够驾驭复杂世界的思维范式。凭借这种范式，我们能够看透现象的表象，洞察其背后的结构性规律，从而以更理性、更深刻的方式去认识世界、改造世界。

第2章 线性代数简史

2.1 什么是线性代数？

线性代数，这个听起来颇为抽象的学科，究竟是什么？从本质上讲，它是数学世界里一门关于“线性关系”的语言和艺术。它将纷繁复杂的多变量问题，通过一种优美而系统的方式，转化为我们可以理解和操作的对象——向量、矩阵和线性空间。

线性代数的核心

线性代数以“线性关系”为研究对象的数学分支。它通过代数方法与几何直观的结合，构建了一套描述和操作向量、矩阵、线性空间及线性变换的理论体系。

历史侧记

思想的飞跃：从算式到代数实体

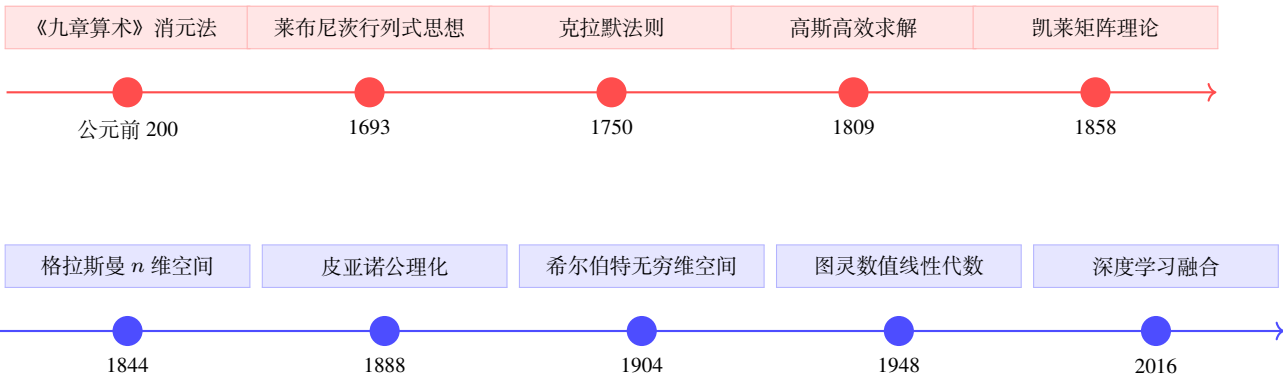
想象一下，当我们面对一个包含多个未知数的方程组时，例如中国古代数学典籍《九章算术》中的经典问题：“今有上禾三秉，中禾二秉，下禾一秉，实三十九斗...”，我们看到的不再是一堆散乱的算式，而是一个高度整合的结构。线性代数提供了一种强大的视角，让我们能将整个问题封装成一个简洁的矩阵方程：

$$\begin{bmatrix} 3 & 2 & 1 \\ 2 & 3 & 1 \\ 1 & 2 & 3 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 39 \\ 34 \\ 26 \end{bmatrix} \quad \text{即} \quad \mathbf{Ax} = \mathbf{b}$$

这不仅仅是书写上的简化，更是一次深刻的观念飞跃。它意味着整个方程组被视为一个独立的代数实体，我们可以像研究一个数字那样，去探讨它的性质，分析它的解是否存在、是否唯一，并洞察其解的内在结构。这便是线性代数的威力与魅力所在：化繁为简，洞见本质。

2.2 线性代数发展时间线

线性代数的演化，是一场围绕着“如何有效描述和解决线性问题”的智慧接力。下面两条时间轴，勾勒出这条从具体求解到抽象结构，再到现代应用的壮丽史诗。



这两条时间轴记录了线性代数思想的两次伟大跃迁。

第一次跃迁，是从“求解的技巧”到“研究的对象”。最初，线性代数的核心任务是解方程。从《九章算术》中蕴含的算法思想，到莱布尼茨捕捉到的、能判定方程组命运的“行列式”，再到克拉默给出的优美求解公式，其焦点始终在于“解”。然而，当高斯将消元法系统化，提供了一套无与伦比的实用算法后，数学家们的目光开始转向承载这一切的“系数方阵”本身。凯莱的登场，最终将矩阵确立为一个独立的代数实体，完成了这次深刻的视角转移。

第二次跃迁，是从“具体的对象”到“抽象的结构”。当矩阵和向量成为主角后，一场更深刻的抽象化开始了。格拉斯曼和皮亚诺将它们从几何和代数的具体形态中解放出来，用几条公理定义了普适的“线性空间”，线性代数从此获得了研究任意线性关系的统一框架。20 世纪，希尔伯特将其推广至无穷维，为量子力学铺平了道路。而图灵等人则在计算机时代开创了“数值线性代数”，专注于算法的效率与稳定。如今，这种结构化的思想与深度学习等前沿领域深度融合，继续焕发出新的生命力。

2.2.1 跨越千年的萌芽：古代世界的智慧闪光

线性代数的思想，并非一道划破夜空的闪电，由某位天才一蹴而就；它更像散落在世界不同角落的种子，在各自文明的土壤中独立生根、悄然发芽。从东方的华夏，到西方的希腊，再到南亚的印度，古老的先贤们为了解决土地丈量、粮食分配、天文历法等现实问题，不约而同地开始探索和总结关于“线性关系”的规律。这些早期的智慧闪光，共同汇成了线性代数漫长历史的源头。

历史侧记

中国古代的“方程术”：算法思想的巅峰

早在公元前 1 世纪左右的西汉时期，《九章算术》中记载的“方程术”，便展现了古代中国处理线性方程组的惊人智慧，堪称世界上最古老的系统性解法。其核心思想完全可以被视为现代线性代数的“临床前研究”。

1. **系数分离与矩阵表示**：“方程术”的第一步，是将方程组中的系数和常数项提取出来，用算筹（一种计算用的小棍）在算板上巧妙地排列成一个矩形阵列。这在形式上，已经与我们今天所说的“增广矩阵”别无二致。

$$\begin{bmatrix} 3 & 2 & 1 & 39 \\ 2 & 3 & 1 & 34 \\ 1 & 2 & 3 & 26 \end{bmatrix}$$

这种“数据”与“结构”分离的做法，是算法思想的精髓。它意味着解题的方法（算法）可以独立于具体的问题（数据）而存在，具有了普适性。

2. **“遍乘直除”的消元过程**：这是算法的核心。它包含一套被称为“遍乘直除”的标准化操作，目标是将矩阵化为“上三角”形式。例如，为了将第二行的第一个数“2”消为零，古人执行的操作等价于现代的初等行变换：

$$(\text{新第二行}) = 3 \times (\text{旧第二行}) - 2 \times (\text{第一行})$$

通过系统性地重复此过程，将矩阵削减为上三角形式，再自下而上逐一“回代”求解未知数。这套完整的、程序化的解法，不仅是古代世界的一项绝技，更蕴含了超越时代的算法灵魂。

为了让读者更直观地感受这种算法思想，我们不妨来看《九章算术》第八卷“方程”中的原文描述：

《九章算术·方程》原文节选

问曰：“今有上禾三秉，中禾二秉，下禾一秉，实三十九斗；上禾二秉，中禾三秉，下禾一秉，实三十四斗；上禾一秉，中禾二秉，下禾三秉，实二十六斗。问上、中、下禾实一秉各几何？”

方程术曰：“置上禾三秉，中禾二秉，下禾一秉，实三十九斗，于右方。中、左禾列如右方。以右行上禾遍乘中行而以直除。又乘其次，亦以直除。然以中行中禾不尽者遍乘左行而以直除。左方下禾不尽者，上为法，下为实。实即下禾之实。求中禾，以法乘中行下实，而除下禾之实。馀如中禾秉数而一，即中禾之实。求上禾亦以法乘右行下实，而除下禾、中禾之实。馀如上禾秉数而一，即上禾之实。实皆如法，各得一斗。”

答曰：“上禾一秉，九斗、四分斗之一 ($9\frac{1}{4}$ 斗)；中禾一秉，四斗、四分斗之一 ($4\frac{1}{4}$ 斗)；下禾一秉，二斗、四分斗之三 ($2\frac{3}{4}$ 斗)。”

历史侧记

古希腊的几何外衣

在地球的另一端，崇尚逻辑与形式之美的古希腊数学家们，则习惯于为一切代数问题披上几何的外衣。他们虽然没有发展出类似“方程术”的代数算法，但其思想同样触及了线性的核心。欧几里得在《几何原本》中对“线段定比分割”等问题的探讨，其本质就是一个简单的一元线性方程。他们将数视为线段的长度，将数的运算转化为几何图形的拼接与变换。这种“以形证数”的传统，虽然限制了代数符号的发展，却也为日后线性代数与几何空间的紧密结合，埋下了深刻的伏笔。

历史侧记

古印度的数论巧思

与此同时，古印度的数学家在数论领域高歌猛进，展现了对线性问题独特的洞察力。为了解决天文学和历法计算中的周期问题，婆罗摩笈多等学者提出了著名的“库塔卡”法（Kuttaka），用于巧妙地求解线性不定方程（即丢番图方程）的整数解。这种方法虽然关注点在于整数解，但其寻找解的系统性步骤，同样体现了将复杂问题“算法化”的努力。

这些古老的智慧，虽然形式各异——中国的算筹、希腊的图形、印度的公式——但它们殊途同归，共同指向了一个核心：**对线性规律的认识与系统化处理**。它们标志着人类思维的一次重要跃迁：不再满足于解决单个问题，而是开始尝试创造一套可以重复使用的、普适的“方法论”。

从具体计算到抽象工具的演进

当中国的先贤们在算板上娴熟地演绎算法，当希腊的哲人们在线段与面积中思索比例时，他们都在各自的道路上将线性思想推向了极致。当历史的指针拨向 17 世纪时，科学革命的浪潮席卷欧洲，天体力学等领域的复杂计算，迫切需要一种更通用、更深刻的理论语言，来系统性地分析和预测线性方程组解的性质。正是在这样的时代呼唤下，一个能够洞察方程组“灵魂”的全新概念——**行列式 (Determinant)**，开始酝酿成形。

2.2.2 行列式：判定方程命运的神秘数字

17 世纪末，当牛顿和莱布尼茨竞相铸造微积分这一强大工具时，莱布尼茨的思绪还在另一个更宏大的梦想中遨游——创造一种“通用计算语言”，使得人类所有的推理和争论都能像解数学题一样，通过计算得出唯一的、无可争辩的答案。正是在这个探索普适性符号与规则的宏伟蓝图下，他将目光投向了看似平凡的线性方程组。

2.2.3 莱布尼茨 (Gottfried Wilhelm Leibniz, 1646-1716)

德国博学家莱布尼茨在研究方程组消元的过程中，率先捕捉到了行列式的思想。他敏锐地发现，方程组有解的充要条件，隐藏在系数之间一个特定的组合关系中。

在 1693 年写给法国数学家洛必达的一封信中，莱布尼茨探讨了如何判断一个一般的线性方程组是否有唯一解。他并非简单地进行繁琐的计算，而是在消元的过程中，敏锐地寻找一种隐藏在系数背后的具有“判决权”的结构。

让我们重现他的思路。对于一个一般的二元一次方程组：

$$\begin{cases} a_{11}x + a_{12}y = b_1 \\ a_{21}x + a_{22}y = b_2 \end{cases}$$

为了消去变量 y ，我们将第一式乘以 a_{22} ，第二式乘以 a_{12} ，然后相减，便得到：

$$(a_{11}a_{22} - a_{21}a_{12})x = b_1a_{22} - b_2a_{12}$$

莱布尼茨注意到，左边 x 的系数 $(a_{11}a_{22} - a_{12}a_{21})$ 完全由方程组的系数唯一确定。这个数值至关重要：如果它不为零，我们就能顺利解出 x ，进而求得唯一的 y ；而如果它恰好为零，方程组的命运就会走向无解或无穷多解的岔路。

这个量，就如同二次方程根的判别式 $b^2 - 4ac$ 一样，能够在我们真正动手“求解”之前，就预先判定整个方程组的“命运”。莱布尼茨抓住了这个从系数方阵中提炼出的“命运判官”，这正是二阶行列式的最初形态。

$$\det(A) = \begin{vmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{vmatrix} = a_{11}a_{22} - a_{12}a_{21}$$

这一发现的意义是革命性的。它标志着数学家看待线性方程组的视角，发生了一次深刻的跃迁：**从一名埋头解题的“工匠”，转变为一位审视问题结构的“建筑师”**。关注点不再仅仅是“解是多少”，而是“解存在吗？唯一吗？”这类更本质的性质问题。行列式，这个与系数结构内在相关的“不变量”，正是开启这扇理论大门的第一把钥匙。

从灵感到理论的接力

莱布尼茨更像一位“预言家”，他预言了这片新大陆的存在，却未能亲自绘制出它的精确地图。他留下的关键问题是：**如何为任意 n 阶方阵给出一个通用的、严谨的行列式定义？** 数学界需要一位真正的“建筑师”，将这零散而闪光的思想蓝图，构造成一座宏伟坚实的理论大厦。历史的聚光灯，首先投向了法国数学家——范德蒙德。

2.2.4 范德蒙德 (Vandermonde, 1735—1796)

正是法国数学家亚历克西·泰奥菲尔·范德蒙德接过了这历史的接力棒。他并非一位高产的学者，一生只发表了四篇数学论文，但这四篇论文篇篇都闪耀着天才的光芒。在 1771 年提交给法国科学院的论文中，范德蒙德首次将行列式从解方程的“附属品”中

解放出来，作为一个独立的、值得研究的数学对象，并赋予了它第一个真正意义上的通用、严谨的定义。

范德蒙德的贡献：行列式的第一个组合定义

范德蒙德的思路是革命性的。他指出，一个 n 阶行列式的值，是由所有可能的、从矩阵中取出 n 个不同行不同列元素的乘积，再配以特定的正负号，最后求和得到的。

这个过程的核心在于回答两个问题：

1. 如何系统地取出所有这样的乘积项？

他引入了“排列” (Permutation) 的概念。对于 n 阶方阵，我们固定行序为 $1, 2, \dots, n$ ，而让列序在所有的 $n!$ 种排列 $\sigma = (\sigma(1), \sigma(2), \dots, \sigma(n))$ 中变化，从而构造出每一个乘积项 $a_{1\sigma(1)}a_{2\sigma(2)} \cdots a_{n\sigma(n)}$ 。

2. 如何确定每一项前面的正负号？

这是范德蒙德最天才的洞察。他指出，符号取决于排列的“奇偶性”。通过计算一个排列中的“逆序数” (Inversion, 即排列中所有逆序对的数目)，如果逆序数是偶数，该项就取正号；如果是奇数，就取负号。

综合起来，他给出了行列式的通用“配方”，也就是后世所熟知的莱布尼茨公式：

$$\det(A) = \sum_{\sigma \in S_n} \operatorname{sgn}(\sigma) a_{1\sigma(1)} a_{2\sigma(2)} \cdots a_{n\sigma(n)}$$

其中 $\sum$ 意为对所有 $n!$ 种排列求和， $\operatorname{sgn}(\sigma)$ 就是由逆序数奇偶性决定的符号 (+1 或 -1)。

范德蒙德的定义是一次伟大的思想飞跃。它将一个数值的计算，彻底转化为一个深刻的组合数学问题。行列式的本质，不再是某个消元过程的副产品，而是其内部元素之间排列与组合对称性的体现。

此外，范德蒙德还提出了一个以他名字命名的、在多项式插值等领域有着至关重要应用的特殊行列式——**范德蒙德行列式**，其优美的结构和简洁的计算结果，至今仍是线性代数中的一颗明珠。

$$\begin{vmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ 1 & x_3 & x_3^2 \end{vmatrix} = (x_2 - x_1)(x_3 - x_1)(x_3 - x_2)$$

可以说，是范德蒙德为行列式注入了严谨的数学灵魂，使其从一个模糊的直觉，转变为一个定义明确、性质清晰的数学实体。正是有了这个坚实的地基，拉普拉斯的展开定理和克拉默的求解法则才能被建立起来，并被证明在任意维度下都普遍成立。

2.2.5 克拉默 (Gabriel Cramer, 1704-1752)

如果说莱布尼茨、范德蒙德和拉普拉斯等人为行列式理论搭建了坚实的骨架，那么来自日内瓦的数学家加布里埃尔·克拉默 (Gabriel Cramer)，则为这座宏伟的建筑砌上了

最耀眼的穹顶。在 1750 年出版的《线性分析导论》一书中，他发表了一个石破天惊的成果，将行列式这个抽象的理论工具，直接与线性方程组的求解完美地联系起来。

定理：克拉默法则 (Cramer's Rule, 1750 年)

对于一个含有 n 个未知数的 n 元线性方程组 $\mathbf{Ax} = \mathbf{b}$ ，如果其系数矩阵的行列式 $\det(A)$ 不为零，那么该方程组有且仅有唯一解。

这个唯一解的每一个分量 x_i 都可以由两个行列式的商精确给出：

$$x_i = \frac{\det(A_i)}{\det(A)}$$

其中， A_i 是将系数矩阵 A 的第 i 列用常数项向量 $\mathbf{b}$ 替换后得到的新矩阵。

以二元方程组为例： 对于方程组 $\begin{cases} a_{11}x + a_{12}y = b_1 \\ a_{21}x + a_{22}y = b_2 \end{cases}$ ，其解为：

$$x = \frac{\begin{vmatrix} b_1 & a_{12} \\ b_2 & a_{22} \end{vmatrix}}{\begin{vmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{vmatrix}}, \quad y = \frac{\begin{vmatrix} a_{11} & b_1 \\ a_{21} & b_2 \end{vmatrix}}{\begin{vmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{vmatrix}}$$

克拉默法则的问世，是行列式理论的一座丰碑，其核心价值在当时是无与伦比的理论之美。

首先，它为线性方程组的解提供了一份无可辩驳的“出生证明”。在此之前，人们通过消元等操作求解，但操作过程的繁琐往往会掩盖解的本质。而克拉默法则以一种极其优雅的方式宣告：只要分母的行列式 $\det(A)$ 不为零，解就必然存在且唯一。它将解的存在性问题，转化为了一个可以明确计算的数值是否为零的判断题，这在数学的严谨性上是巨大的飞跃。

其次，它给出了一个逻辑上完备的封闭解 (Closed-form solution)。在那个崇尚理性与秩序的启蒙时代，这样一个不依赖于任何近似或迭代、能够“一击即中”给出精确解的普适公式，满足了当时数学家对和谐与完美的全部想象。至此，线性方程组的求解在理论层面似乎已经达到了“终极”形态，行列式理论的大厦也初步建成。

从“完美公式”到“高效算法”的转向

克拉默法则造出了一把理论上完美的“屠龙刀”，却重得无人能举。问题的根源在于其惊人的计算量。计算一个 n 阶行列式，如果按照其定义（莱布尼茨公式）展开，大约需要进行 $n!$ (n 的阶乘) 次乘法运算。这意味着：

- 解一个 3×3 的方程组，尚可应付。
- 解一个 10×10 的方程组，需要计算 11 个 10 阶行列式，总计算量将是天文数字 ($10! = 3,628,800$)。
- 而解一个仅仅 25×25 的方程组，用克拉默法则所需的计算次数，已经远远超过了宇宙中所有原子的总数！

这种“阶乘爆炸”式的复杂度，使得克拉默法则在面对稍大规模的现实问题时，瞬间从一个“完美公式”变成了一个“计算灾难”。它是一个完美的理论工具，让我们得以洞察解的性质；但它绝不是一个有效的计算方法，无法指导我们真正在现实世界中解决问题。

克拉默法则的“优美而无用”，清晰地揭示了理论的完备性与计算的可行性是两个截然不同的挑战。数学的发展，迫切需要一把真正轻便、锋利的武器。

2.3 近代革命：结构思想的诞生 (19 世纪)

2.3.1 高斯 (Carl Friedrich Gauss, 1777-1855)

被誉为“数学王子”的德国数学家高斯，正是实现这一历史性转向的巨人。他将古老的消元思想进行了严格化和系统化，使其成为第一个在实践中真正高效、普适的求解算法。

- **背景与著作：**在 1809 年出版的著作《天体运动理论》中，为了解决天文学中预测行星轨道的复杂计算问题（这本质上是一个最小二乘法问题，需要求解线性方程组），高斯系统地阐述了他的消元法。
- **核心思想：从公式到流程**
 - **算法化：**高斯消元法的思想核心，是放弃寻找一个封闭的“表达式”，转而设计一套标准的“程序”。它通过一系列规范的初等行变换（倍加、互换、倍乘），将原方程组的增广矩阵逐步转化为一个等价但极其简单的“行阶梯形”，从而可以轻松地自下而上回代求解。
 - **效率革命：**与克拉默法则的阶乘级复杂度相比，高斯消元法的计算量大致与未知数数量的立方成正比。这在计算上是天壤之别，使得求解拥有数十甚至上百个未知数的方程组首次成为可能。
- **历史意义：**高斯消元法是连接线性代数理论与大规模科学计算的第一座坚实桥梁。它标志着线性代数的研究重心，开始从纯粹的理论构造，转向兼顾计算效率的算法设计。

从“完美公式”到“高效算法”的转向

克拉默法则清晰地揭示了理论的完备性与计算的可行性是两个截然不同的问题。既然“公式化求解”这条路在现实中走不通，那么解决大规模线性方程组的实用出路究竟在何方？历史的聚光灯自然地从“寻找一个普适的公式”转向了“设计一套高效的算法”。

2.3.2 阿瑟·凯莱 (Arthur Cayley, 1821-1895)

高斯的算法终结了“如何求解”的古典难题，但 19 世纪的数学家们很快就面对一个更深层次的议题。在几何学、数论和物理学中，他们越来越频繁地遇到“**线性变换**”，将空间中的一个向量（或点），通过一套线性规则，映射为另一个向量。例如，一个简单的二维旋转可以表示为：

$$\begin{cases} x' = x \cos \theta - y \sin \theta \\ y' = x \sin \theta + y \cos \theta \end{cases}$$

当数学家们想要研究连续进行两次或多次变换（比如先旋转再拉伸）的复合效应时，他们不得不将一套方程代入另一套，过程冗长乏味，极易出错，并且完全掩盖了变换本身的几何精髓。他们迫切需要一种新的语言和工具，来简洁地表达和操作这些“变换”本身。

历史侧记

英国数学家阿瑟·凯莱，一位主业是律师、却在业余时间推动了数学革命的天才，正是这场变革的旗手。他敏锐地意识到，在上述的线性变换中，真正的主角并非变量 x, y, x', y' ，而是那一组系数本身所构成的矩形阵列。

$$\begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

凯莱的革命性思想在于：这个数字阵列，不应再被看作是储存系数的“表格”，而应被视为一个独立的、可以进行运算的数学“实体” (Entity) 或“算子” (Operator)。它本身就完整地代表了“旋转”这个几何操作。

为了实现这一构想，凯莱在 1858 年发表了奠基性的论文《矩阵理论的研究报告》 (A Memoir on the Theory of Matrices)，系统地构建了这门全新的代数学分支。

2.3.2.1 命名与运算法则的创立

凯莱首先为这个新的数学实体赋予了一个名字——“**矩阵**” (Matrix)，其词源来自拉丁语，意为“子宫”或“孕育之所”，寓意着它是能够孕育出行列式等其他数学对象的母体。

紧接着，他为矩阵定义了一套完整的运算法则：

- **加法与数乘**：他定义了直观的矩阵加法和数乘，这对应于变换效果的叠加与缩放。
- **矩阵乘法**：这是凯莱最天才的创造。他定义的矩阵乘法规则（即我们今天熟知的“行乘以列”）看似古怪，但其动机却无比清晰：**两个矩阵的乘积，必须精确对应于两个线性变换的连续复合**。这个定义使得代数运算与几何操作完美地统一起来。

意义与突破：一个新代数世界的开启。

当凯莱为矩阵这门新艺术定义了完备的运算法则后，他就如同发现了一片新大陆的探险家，开始勘探这片土地上独特的自然法则。而他所揭示出的性质，深刻地改变了整个代数学的面貌。

首先，也是最颠覆性的发现是，**矩阵乘法并不遵循交换律**。在凯莱之前的所有代数系统中， $a \times b$ 总是等于 $b \times a$ 。但他证明了，对于大多数矩阵而言， $AB \neq BA$ 。这个在当时看来惊世骇俗的结论，却并非一个凭空创造的古怪规则。它恰恰完美地呼应了现实世界的几何真理：在空间中，先进行 A 变换（如拉伸），再进行 B 变换（如旋转），其最终效果与先 B 后 A 的顺序截然不同。矩阵代数的这种“非交换性”，第一次如此深刻地、如此优美地刻画了空间变换的内在结构。

其次，凯莱为“撤销一次变换”这一几何直觉，赋予了坚实的代数形式——“**逆矩阵**”(A^{-1})。在此基础上，他完成了一次伟大的理论统一，将他崭新的矩阵理论与古典的行列式理论紧密地联系在了一起。他清晰地证明：一个矩阵所代表的变换能否被“撤销”（即矩阵是否可逆），完全取决于其行列式是否为零。他给出的二阶逆矩阵公式，更是这一深刻联系的完美展现：

$$A^{-1} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix} = \frac{1}{\det(A)} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

这个公式如同一座桥梁，将一个动态的代数操作（求逆）和一个静态的代数量（行列式）紧密地“焊接”在了一起，为两者数百年的各自发展找到了最深刻的归宿。

历史侧记

同时代的巨匠：西尔维斯特与柯西

凯莱的革命性工作并非在真空中完成，而是与时代精英们思想共振的结果。与他亦师亦友的**詹姆斯·约瑟夫·西尔维斯特 (James Joseph Sylvester)** 是一位“术语创造大师”，正是他创造了“矩阵”(Matrix) 这个词并推荐给了凯莱，我们今天仍在使用的“秩”(rank)、“不变量”(invariant) 等核心词汇，也都出自他的贡献。而更早些时候，法国数学家**奥古斯丁·路易·柯西 (Augustin-Louis Cauchy)** 已经在行列式领域做出了集大成的工作，他不仅证明了行列式的乘法定理 $\det(AB) = \det(A)\det(B)$ ，还引入了“特征方程” $\det(A - \lambda I) = 0$ 的概念，并证明了实对称矩阵的特征值必为实数，这些都为凯莱的矩阵理论提供了肥沃的土壤。

可以说，是凯莱最终完成了线性代数史上一次关键的视角转移：从研究“关于线性方程组的计算技巧”，转向了研究“承载这些技巧的代数结构本身”。矩阵，从此作为一个拥有独立生命、深刻内涵的数学实体，登上了历史的中心舞台。

2.3.3 格拉斯曼与皮亚诺：从几何实体到抽象空间的一跃

凯莱将矩阵确立为代数舞台的中心后，一个根本性的问题浮现出来：这些矩阵算子，它们作用的对象——“向量”，到底是什么？在当时大多数数学家的脑海里，向量依然是

那个被禁锢在二维或三维物理空间中的“带箭头的线段”。然而，数学的发展早已暗示，许多其他事物（如多项式、函数）也表现出类似“向量”的性质。线性代数需要一次彻底的思想解放，挣脱三维几何的“肉身”，去拥抱一个更广阔、更抽象的“灵魂”。

这次伟大的抽象，由两位跨越时代的数学家以接力的方式完成。

赫尔曼·格拉斯曼 (Hermann Grassmann, 1809-1877) 是一位德国偏远小镇的中学教师，但他的思想却领先于他所处的整个时代。他不仅仅是数学家，更是哲学家和语言学家。他毕生追求一个宏大的目标：创造一种普适的“扩张论”，一种能够描述任何“广延量”的通用数学框架。

历史侧记

在 1844 年出版的同名著作《扩张论》中，格拉斯曼做出了石破天惊的创举。他完全抛弃了从笛卡尔坐标系入手的传统思路，而是从最基本的哲学概念出发，构建了一个全新的代数世界：

- **挣脱维度束缚**：他认为，空间的本质不在于我们能“看见”的三个维度，而在于其中元素的运算法则。他首次系统地提出了“ n 维线性空间”的构想，并清晰地定义了其中元素（即 n 维向量）的加法和数乘运算。
- **定义核心概念**：在此基础上，他进一步定义了诸如**线性无关**、**子空间**、**基底**、**维度**、**内积（点积）**和**外积**等一系列我们今天无比熟悉的核心概念。

这本质上是在宣告：任何满足这些基本运算法则的元素集合，无论它看起来是什么样，都应该被一视同仁地当作一个“空间”来研究。

然而，格拉斯曼的命运是一位思想先知的典型悲剧。他的著作充满了晦涩的哲学思辨和独创的符号，行文风格对于当时习惯于从具体计算和几何直观出发的数学家来说，如同一本天书。他的工作在长达几十年的时间里几乎无人问津，被学界彻底忽视。

历史的尘埃不会永远掩盖金子。几十年后，当数学界自身经历了“严谨化运动”，对抽象结构和公理化方法的认识日益成熟时，格拉斯曼的深邃思想终于等来了它的“解码者”和“立法者”——意大利数学家朱塞佩·皮亚诺 (Giuseppe Peano, 1858-1932)。

皮亚诺是一位逻辑学家和公理化大师，他敏锐地意识到格拉斯曼思想的巨大潜力。他所做的，是将格拉斯曼那略显庞杂的哲学体系，提炼成一座逻辑严密、无懈可击的数学大厦。

皮亚诺的贡献：线性空间的公理化定义 (1888 年)

皮亚诺在前人工作的基础上，于 1888 年历史性地给出了我们今天教科书中所学的线性空间（或称向量空间）的公理化定义。他指出，一个集合 V 构成一个向量空间，当且仅当其中的元素（即“向量”）和相关的“标量”满足以下八条简洁、优美且不容置疑的公理（如加法交换律、结合律，数乘分配律等）。

这一定义的伟大之处在于其无与伦比的普适性。它完成了一次彻底的“去身份化”：一个对象是不是“向量”，不再取决于它是否是“带箭头的线段”，而只取决

于它是否遵守这八条“交通规则”。

从此以后：

- 几何世界中的**有向线段**是向量。
- 代数世界中的 n 元有序数组 $(x_1, x_2, \dots, x_n)$ 是向量。
- 函数世界中，所有从 $[a, b]$ 区间到实数的**连续函数**集合，也是一个向量空间。
- 物理世界中，量子力学的**波函数**也生活在向量空间中。

所有这些看似风马牛不相及的数学对象，从此都可以被纳入同一个理论框架下进行研究。线性代数，也因此从研究“解方程”和“矩阵”的专门理论，升华为一门研究“线性结构”的普适性基础学科。

可以说，格拉斯曼是那位独自穿越沙漠、瞥见了新世界宏伟景象的先知，而皮亚诺则是那位随后赶到、用精确的语言和工具为这片新世界绘制出第一份可靠地图的伟大测绘师。他们二人的思想接力，最终为“空间”这一概念赋予了现代数学的灵魂。

2.4 现代发展：抽象浪潮与计算机革命 (20 世纪至今)

2.4.1 大卫·希尔伯特 (David Hilbert, 1862—1943)

进入 20 世纪，线性代数在有限维的世界里似乎已经功德圆满。然而，数学的前沿永不停歇，新的挑战来自于数学分析领域，特别是对“**积分方程**”的研究。这是一种全新的、更为复杂的方程，它要求数学家们将线性代数的视角，投向一个超乎想象的新疆域——无穷维。

历史侧记

时代的需求：从求解“有限个数”到求解“未知函数”

我们熟悉的线性方程组 $\mathbf{Ax} = \mathbf{b}$ ，是在求解一个含有 n 个未知数 $(x_1, \dots, x_n)$ 的**有限维向量**。然而，在许多物理和工程问题中，未知量不是一个有限的数组，而是一个**连续函数** $f(t)$ 。求解这类未知函数的方程，被称为“积分方程”。

一个函数 $f(t)$ ，可以被看作是一个拥有无穷多个“分量”的向量（即它在定义域上每一点的函数值）。因此，对积分方程的研究，天然地要求数学家们将线性代数的思想——例如线性变换、基底、特征值等——从我们熟悉的有限维空间，推广到一个由无数函数构成的**无穷维空间**之中。

在这历史性的转折点上，德国数学巨匠**大卫·希尔伯特 (David Hilbert, 1862-1943)** 扮演了思想领袖和新世界开创者的角色。他以非凡的洞察力，看穿了有限维线性方程组和无穷维积分方程之间深刻的结构相似性，并着手为这个无穷维世界建立坚实的数学基础。

希尔伯特空间：融合代数与分析的无穷维世界

希尔伯特创立的理论，构建了一个被称为“希尔伯特空间”(Hilbert Space) 的数学结构。它是一个为无穷维量身定做的、性质优良的“超级”线性空间，除了满足基本的线性公理外，还额外满足两个至关重要的条件：

1. **拥有内积 (Inner Product)**：它将我们熟悉的向量点积，推广到了无穷维。例如，对于函数空间，两个函数 $f(t)$ 和 $g(t)$ 的内积可以定义为 $\langle f, g \rangle = \int f(t)g(t)dt$ 。有了内积，我们就能在这个无穷维空间中，同样可以精确地度量“向量”（即函数）的**长度、夹角**，并定义“**正交**”（垂直）等关键几何概念。
2. **满足完备性 (Completeness)**：这是一个来自分析学的核心要求，通俗地讲，它保证了这个空间是“**没有孔洞的**”。如果空间中的一列向量（函数）无限逼近某个极限，那么这个极限本身也保证“居住”在这个空间之内。这个性质是至关重要的，它确保了所有微积分中的极限运算，都可以在这个空间中安全、自洽地进行。

可以说，希尔伯特空间是一个将线性代数的几何直观，与数学分析的极限理论，完美融合在一起的、坚实可靠的无穷维舞台。

历史侧记**历史的巧合：量子革命的数学基石**

希尔伯特空间的诞生，不仅直接催生了数学的一个重要新分支——**泛函分析 (Functional Analysis)**，更令人拍案叫绝的是，它似乎是上帝为当时刚刚兴起的、彻底颠覆人类世界观的**量子力学**，提前准备好的完美数学语言。

在量子世界中，一个物理系统（如一个电子）的状态，不再由牛顿力学中的位置和速度来描述，而是由希尔伯特空间中的一个“**态向量**”（即物理学家所说的波函数）来完整描述。而我们能够测量的一切物理量（如能量、动量、位置），则对应着作用在该空间上的“**线性算子**”。物理的测量过程，在数学上被诠释为线性算子作用于态向量，而测量的可能结果，就是该算子的特征值。

希尔伯特在纯粹的数学探索中构建的抽象宫殿，竟与物理学家在最前沿实验中窥探到的微观世界的基本规则，发生了惊人的重叠。这成为了 20 世纪科学史上最壮丽的篇章之一。

2.4.2 图灵与冯·诺依曼

在希尔伯特等人将线性代数的理论推向无穷维的抽象顶峰之时，另一场更为“接地气”的革命，在第二次世界大战的硝烟中被点燃了。战争对弹道计算、密码破译（如图灵破解恩尼格玛密码机）和原子弹设计（冯·诺依曼参与的曼哈顿计划）提出了前所未有的计算需求。面对浩如烟海的线性方程组，人力计算已是杯水车薪。正是在这种极端

需求的催化下，电子计算机应运而生。

计算机的诞生，宣告了一个全新纪元的到来。线性代数不再仅仅是数学家在纸上推演的智力游戏，而是成为了驱动现代科学与工程的核心引擎。然而，当数学家们兴奋地将经典算法搬上这些机器时，却遭遇了两个前所未有的强大“敌人”。

历史侧记

新时代的两个核心挑战

1. “效率”的诅咒：在纸笔时代，一个公式是否“优美”远比它是否“好算”重要。但在每秒能执行数百万次运算的计算机面前，算法的计算量——即运算次数的多少——成为了决定性的生死线。一个理论上可行的算法，如果计算量过大（如克莱姆法则的阶乘级复杂度），在计算机眼中与“不可行”无异。

2. “稳定”的梦魇：这是更隐蔽、更危险的敌人。计算机无法像数学家一样处理无限精确的实数，它只能使用有限位数的“浮点数”来近似表达。这意味着每一步运算，都可能产生微小的“舍入误差”。这就好比用一把略有瑕疵的尺子去建造一座摩天大楼，每一次微小的测量误差，在经过成千上万次的累积和放大后，都可能导致整座大厦的倾斜甚至坍塌。一个在理论上无懈可击的算法，在实际运算中可能因为数值不稳定而给出一个荒谬的错误答案。

在这场与“效率”和“稳定”的全新战斗中，**艾伦·图灵 (Alan Turing)** 和 **约翰·冯·诺依曼 (John von Neumann)** 等计算机科学的先驱们，开创了一门至关重要的应用分支——**数值线性代数**。他们以及后继者们发展的核心思想，就是通过“**矩阵分解**”来驯服那些庞大而“病态”的矩阵。

核心思想：矩阵分解——化整为零的艺术

矩阵分解的哲学，是一种极致的“分而治之”。它主张不要直接去攻击一个结构复杂、性质恶劣的“敌军主阵地”（原始矩阵 A ），而是通过精巧的数学变换，将其拆解为多个结构简单、性质优良的“小分队”（如三角矩阵、正交矩阵等）的乘积。

$$A = M_1 M_2 \cdots M_k$$

如此一来，一个棘手的求解问题（如求 A^{-1} ），就转化为了一系列更容易、更快速、更稳定的子问题。这是现代数值算法的灵魂。其中，两种最基础也最重要的分解是：

• LU 分解：高斯消元的现代化身

由图灵等人系统性分析和完善的 LU 分解，可以看作是高斯消元法的“预处理”版本。它将矩阵 A 分解为一个下三角矩阵 (L , Lower) 和一个上三角矩阵 (U , Upper) 的乘积。求解三角矩阵方程极其迅速且稳定。LU 分解因此成为了求解中等规模线性方程组、计算行列式等的标准工业流程。

• QR 分解：数值稳定性的守护神

由阿尔斯通·霍斯霍尔德 (Alston Householder) 等人发展的 QR 分解，则将矩阵 A 分解为一个正交矩阵 (Q , Orthogonal) 和一个上三角矩阵 (R , Right-triangular) 的乘积。正交矩阵 Q 对应于空间中的旋转或反射，其最大的优点是它**不会放大误差**（如同一个完美的刚体转动），因此具有无与伦比的数值稳定性。QR 分解在求解最小二乘问题（如数据拟合）、计算矩阵特征值等对精度要求极高的领域，是当之无愧的王者。

从此，线性代数的发展进入了一个理论与实践深度融合的新阶段。数学家们不仅要创造优美的理论，更要设计出在有限精度计算机上依然稳健、高效的算法。这门古老学科，也因此成为了连接抽象数学世界与具体工程应用的、最不可或缺的桥梁。

2.4.3 当代前沿：深度学习的“线性”基石

如果说 20 世纪的计算机革命将线性代数推向了大规模科学计算的中心，那么 21 世纪的深度学习浪潮，则让线性代数成为了驱动人工智能革命的、无处不在的“底层操作系统”。表面上看，神经网络以其强大的“非线性”拟合能力著称，但剥开其层层外衣，其骨架和血肉几乎完全由线性代数构成。

从本质上讲，一个深度神经网络的“前向传播”过程，就是将输入数据（一个高维向量）进行一连串“**线性变换 + 非线性激活**”的复合操作。其中，每一个网络层的核心，都是一次大规模的矩阵乘法：

$$\mathbf{y} = f(\mathbf{W}\mathbf{x} + \mathbf{b})$$

这里， $\mathbf{x}$ 是输入向量， $\mathbf{W}$ 是该层的权重矩阵， $\mathbf{b}$ 是偏置向量，而 f 是一个简单的非线性激活函数（如 ReLU）。整个“学习”或“训练”的过程（即反向传播），本质上也是利用微积分和链式法则，来计算损失函数对权重矩阵 $\mathbf{W}$ 中每一个元素的偏导数——这本身就是一个庞大的矩阵运算过程。

可以说，深度学习的硬件（如 GPU 和 TPU）从根本上就是为并行执行大规模矩阵和向量运算而设计的“线性代数加速器”。没有线性代数提供的这套简洁而强大的语言，以及高效的数值计算方法，深度学习模型根本无法被构建和训练。

如果说 20 世纪的计算机革命将线性代数推向了大规模科学计算的中心，那么 21 世纪的深度学习浪潮，则让线性代数成为了人工智能革命无处不在的“底层操作系统”。

2.4.3.1 前沿创新：古典智慧的现代回响

线性代数不仅为深度学习提供了基础运算，其深刻的理论，如矩阵分解、秩、特征值等，更是成为了解决前沿问题的灵感源泉。

LoRA：用“低秩分解”驯服大语言模型

近年来，像 GPT 这样的大语言模型（LLM）动辄拥有上千亿参数，对其进行微调需要消耗惊人的计算资源。其核心挑战在于，需要更新一个极其庞大的权重矩阵 $\mathbf{W}$ 。

为了解决这个问题，研究者们从经典的线性代数理论中汲取智慧，提出了“**低秩适配**”（Low-Rank Adaptation, **LoRA**）技术。其核心假设是：模型微调所产生的权重变化量 $\Delta\mathbf{W}$ ，本质上是一个“**低秩**”（Low-Rank）矩阵。这意味着，这个巨大的变化矩阵，可以用两个规模小得多的“瘦高”和“矮胖”矩阵 $\mathbf{B}$ 和 $\mathbf{A}$ 的乘积来近似：

$$\underset{(d \times k)}{\Delta\mathbf{W}} \approx \underset{(d \times r)}{\mathbf{B}} \cdot \underset{(r \times k)}{\mathbf{A}}$$

其中，秩 r 远小于原始维度 d 和 k 。例如，我们不再需要训练一个 10000×10000 （一亿）参数的 $\Delta\mathbf{W}$ ，而只需要训练一个 10000×8 的 $\mathbf{B}$ 和一个 8×10000 的 $\mathbf{A}$ ，总参数量骤降至十六万，实现了超过 99% 的参数压缩。LoRA 的巨大成功，雄辩地证明了矩阵秩和分解这些古典概念，在解决当今最前沿的 AI 问题中，依然拥有何等强大的威力。

除了在模型结构上进行创新，线性代数的思想也深刻地影响着如何将传统科学计算与深度学习进行融合，尤其是在解决“**线性逆问题**”的领域。

逆问题的挑战与压缩感知的曙光

在科学与工程中，许多问题分为“正问题”和“逆问题”。**正问题**是“由因求果”，通常比较容易，例如知道人体的内部结构（因），去模拟计算它产生的 CT 图像（果）。而**逆问题**则是“由果溯因”，即通过观测到的结果（如 CT 图像），去反推未知的内部成因（人体结构），这通常极其困难。

为什么逆问题难解？主要原因在于其“**不适定性**”（Ill-posedness）：

- **解不唯一**：我们获得的数据（果）往往远少于需要重建的未知量（因），导致一个结果可能对应多个成因。这在数学上是一个严重的“**欠定**”问题。
- **对噪声敏感**：测量数据中微小的噪声或误差，可能会在反推过程中被急剧放大，导致重建结果面目全非，充满伪影。

压缩感知理论的意义何在？21 世纪初兴起的“**压缩感知**”（Compressed Sensing, CS）理论为此带来了革命性的曙光。它在数学上严格证明：如果未知的成因（如医学图像）本身是“**稀疏的**”（即在某个变换域下，其绝大部分分量都为零或接近于零），那么我们就可以打破传统采样定理的限制，仅用远少于常规所需的数据量，就能从看似不完整的结果中，**完美地**反推出唯一的、真实的成因。

这一理论的意义是颠覆性的，它直接催生了磁共振成像的加速技术，使得扫描时间大幅缩短；也深刻影响了雷达成像、无线通信等诸多领域。而压缩感知的核心，正是将一个原本无解的欠定线性逆问题，转化为一个有解的、带有稀疏性约

束的优化问题。

深度展开：模型驱动与数据驱动的结合

如何高效求解这类带有复杂约束的优化问题呢？经典的算法之一是“**交替方向乘子法**”(Alternating Direction Method of Multipliers, **ADMM**)。近年来，一个重要的研究方向，便是将这类经典的、由数学模型驱动算法，与数据驱动的深度学习相结合。由西安交通大学徐宗本院士团队等人提出的 **ADMM-Net**，正是这一思想的典范。

他们创造性地将解决压缩感知磁共振成像 (CS-MRI) 问题的 **ADMM** 算法的整个迭代过程，“**展开**”成一个深度神经网络的层次结构。这种设计的巧妙之处在于：

- **保留模型可解释性**：网络的整体架构不再是一个随意的“黑箱”，它具有清晰的数学物理解释，因为其结构源于一个成熟的优化算法。
- **引入数据驱动优化**：传统 **ADMM** 算法中那些需要人工反复调试的、棘手的超参数（如正则化项、步长等），现在都可以作为网络的可训练参数，通过端到端的方式，让网络从海量数据中自动学习到最优值。

这一“模型驱动”与“数据驱动”深度融合的框架，不仅在线性逆问题的求解上取得了突破性成果，也为其他基于优化算法求解的科学问题，提供了一个激动人心的未来方向。

结论

线性代数的历史，是一部从具体问题出发，不断追求更深层次抽象、更普适结构的壮丽史诗。从古代解方程的实用技巧，到作为理论基石的行列式，再到描述变换的独立实体——矩阵，最终升华为研究一切线性结构的“空间”的艺术。在计算机时代，这门古老的学科与数值计算和人工智能深度融合，从幕后走上台前，成为了驱动现代科技革命的核心引擎。

理解这段跨越千年的思想演进史，我们才能真正把握这门学科的灵魂：它教会我们如何将复杂问题化繁为简，如何在看似无关的事物中洞察共通的结构，并最终运用这种结构化的思想去理解、构建和改造我们所处的世界。

第3章 行列式

3.1 行列式的起源与定义

3.1.1 历史引入：一个判别解的符号

在现代数学的宏伟殿堂中，“行列式”常以其复杂的定义和计算规则让初学者望而生畏。然而，任何抽象概念的诞生，往往源于一个具体而朴素的动机。对于行列式而言，这个动机就是：**我们能否找到一个“符号”或者“表达式”，仅通过考察线性方程组的系数，就能预判其解的性质？**

让我们回到 17 世纪的数学家们所面临的日常问题。考虑一个最简单的二元线性方程组：

$$\begin{cases} a_1x + b_1y = c_1 \\ a_2x + b_2y = c_2 \end{cases}$$

通过初中的代数方法，我们可以消去变量 y 来求解 x 。将第一式两边乘以 b_2 ，第二式两边乘以 b_1 ，得到：

$$\begin{cases} a_1b_2x + b_1b_2y = c_1b_2 \\ a_2b_1x + b_1b_2y = c_2b_1 \end{cases}$$

两式相减，我们得到：

$$(a_1b_2 - a_2b_1)x = c_1b_2 - c_2b_1$$

于是，变量 x 的解为：

$$x = \frac{c_1b_2 - c_2b_1}{a_1b_2 - a_2b_1}$$

同理，我们可以求得 y 的解：

$$y = \frac{a_1c_2 - a_2c_1}{a_1b_2 - a_2b_1}$$

观察这两个解的表达式，一个至关重要的结构浮出水面：它们拥有一个完全相同的分母—— $a_1b_2 - a_2b_1$ 。这个表达式完全由方程组的系数 a_1, b_1, a_2, b_2 决定。

要点提炼

这个表达式 $a_1b_2 - a_2b_1$ 正是解能否存在的“判官”。

- 如果 $a_1b_2 - a_2b_1 \neq 0$ ，那么分母不为零，方程组有且仅有唯一的解。
- 如果 $a_1b_2 - a_2b_1 = 0$ ，分母为零，计算将无法进行。这意味着方程组的解要么不存在（两直线平行），要么存在无穷多个（两直线重合）。

因此，这个完全由系数构成的数值表达式，就像一个“判别式”，无需我们费力去求出完整的解，就能预先判定解的“命运”。它正是行列式概念最原始、最核心的雏形——

一个判别解的符号。

历史侧记

历史上，正是循着这一思路，数学家们开启了对这个“判别符号”的探索。

- **莱布尼茨 (Gottfried Leibniz, 1693)**：在研究三元线性方程组时，他第一次系统地构造了一个由系数组成的表达式，并明确指出，当这个表达式为零时，方程组的解将存在问题。他所构造的，正是三阶行列式的原型。
- **关孝和 (Seki Kowa, 1683)**：几乎在同一时期，甚至更早，日本数学家关孝和也在其著作中独立地提出了类似的概念，并将其用于解决更高阶的方程组。

他们共同的贡献在于，将这个“判别符号”从具体方程的求解过程中剥离出来，视作一个具有独立意义的数学对象来研究。这个对象，就是我们今天所称的“行列式”。现在，让我们从这个历史的源头出发，在下一节中为这一重要的数学对象赋予严格的、现代的定义。

3.1.2 核心定理：从低阶到 N 阶的严格定义

二、三阶行列式

上一节我们看到，为了判断二元线性方程组解的性质，一个关键的代数表达式 $a_1b_2 - a_2b_1$ 自然地浮现出来。这个完全由系数决定的“判别符号”，就是行列式最原始的形态。现在，我们开始为其建立严谨的数学框架。

首先，我们给出二阶行列式的正式定义。

定义 3.1 (二阶行列式)

由四个数 $a_{11}, a_{12}, a_{21}, a_{22}$ 排成两行两列，所确定的二阶行列式定义为：

$$\begin{vmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{vmatrix} = a_{11}a_{22} - a_{12}a_{21}.$$

记为：

$$|A|, \text{ 或者, } \det A$$



要点提炼

一张表看懂行列式和矩阵：把四个数 $a_{11}, a_{12}, a_{21}, a_{22}$ 排成两行两列，可以确定一个二阶行列式：

$$\begin{vmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{vmatrix} = a_{11}a_{22} - a_{12}a_{21}.$$

同理，把四个系数写成矩形的“表”，也可以定义出矩阵：

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{cases} \text{每一行叫行向量} \\ \text{每一列叫列向量} \\ \text{整个表叫矩阵 } A \end{cases}$$

对于矩阵，划掉一个元素所在的行与列，剩下的数就是它的**余子式**；再乘棋盘符号 $(-1)^{\text{行号}+\text{列号}}$ 得到**代数余子式**。二阶情形一眼口算：

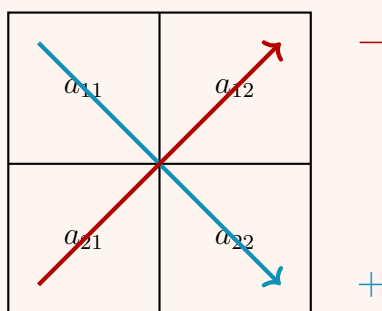
$$M_{11} = a_{22}, \quad A_{11} = +a_{22};$$

$$M_{12} = a_{21}, \quad A_{12} = -a_{21}.$$

要点提炼

对角线法则（仅适用于二阶行列式）

$$\begin{vmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{vmatrix} = \underbrace{a_{11}a_{22}}_{\text{主对角线乘积}} - \underbrace{a_{12}a_{21}}_{\text{副对角线乘积}}.$$



记忆口诀：主对角线乘积减去副对角线乘积。

这个想法可以自然地推广到三元线性方程组，从而得到三阶行列式的表达式。

定义 3.2 (三阶行列式)

$$\begin{vmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{vmatrix} = a_{11}a_{22}a_{33} + a_{12}a_{23}a_{31} + a_{13}a_{21}a_{32} - a_{11}a_{23}a_{32} - a_{12}a_{21}a_{33} - a_{13}a_{22}a_{31}.$$

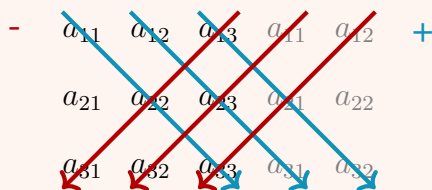


要点提炼

沙路法则 (Sarrus' Rule) 是一种便于记忆三阶行列式计算的方法：

1. 将行列式的前两列复制并写到第三列右侧。
2. 三条主对角线方向（从左下到右上）上三个数的乘积之和，减去三条副对

角线方向（从右上到左下）上三个数的乘积之和。



注意：沙路法则仅适用于二阶和三阶行列式。

发现通往 N 阶的规律

对角线法则和沙路法则是非常方便的记忆技巧，但它们都只是低阶的“特例”，无法推广到四阶及以上。要找到一个普适的定义，我们必须深入分析这些展开式的内在结构。

1. 每一项的构成规律是什么？

仔细观察二阶和三阶的展开式，我们会发现一个共同的模式：

- 二阶展开式： $a_{11}a_{22}$ 和 $a_{12}a_{21}$ 。
- 三阶展开式： $a_{11}a_{22}a_{33}$, $a_{12}a_{23}a_{31}$ 等等。

每一项都是从矩阵的**不同行、不同列**中各取一个元素相乘得到的。例如，在 $a_{12}a_{23}a_{31}$ 中，行标是 (1, 2, 3)，列标是 (2, 3, 1)，行列标都没有重复。如果我们固定行标按 $(1, 2, \dots, n)$ 的顺序选取，那么每一项的列标组合就构成了数字 $\{1, 2, \dots, n\}$ 的一个**排列** (Permutation)。

- 二阶行列式：涉及列标排列 (1, 2) 和 (2, 1)，共 $2! = 2$ 项。
- 三阶行列式：涉及列标排列 (1, 2, 3), (2, 3, 1), (3, 1, 2), (1, 3, 2), (2, 1, 3), (3, 2, 1)，共 $3! = 6$ 项。

2. 每一项的符号由什么决定？

我们已经解决了“有哪些项”的问题，但更核心的难题是：这些项前面的正负号是如何确定的？对角线法则在 $n \geq 4$ 时将完全失效。一定存在一个更深刻的规则。让我们再次检视列标排列与符号的关系：

- 二阶：
 - 排列 (1, 2) (自然有序) $\rightarrow$ 符号为 +
 - 排列 (2, 1) (一次交换即可恢复有序) $\rightarrow$ 符号为 -
- 三阶：
 - 排列 (1, 2, 3), (2, 3, 1), (3, 1, 2) (偶数次交换恢复有序) $\rightarrow$ 符号为 +
 - 排列 (1, 3, 2), (2, 1, 3), (3, 2, 1) (奇数次交换恢复有序) $\rightarrow$ 符号为 -

我们大胆猜测：符号的正负，与列标排列的“混乱程度”或“无序程度”有关！

为了精确地描述这种“混乱程度”，数学家引入了一个关键概念——**逆序数** (Inversion Number)。一个排列的逆序数如果是偶数，我们就给它正号；如果是奇数，就给它负号。

至此，通往 n 阶行列式定义的道路已经清晰：我们需要系统地学习**排列**的知识，并掌握计算其**逆序数**的方法，从而为 $n!$ 个乘积项中的每一项赋予正确的符号。这正是下

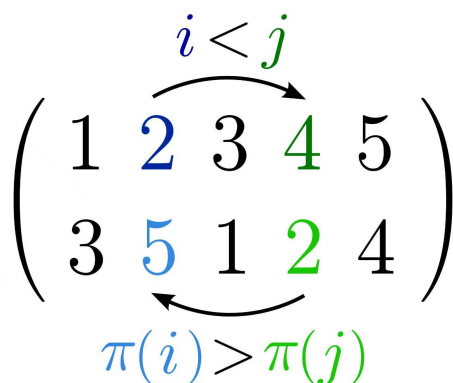


图 3.1: 逆序对的直观示意图：当下标 $i < j$ 时，其对应的值却满足 $\pi(i) > \pi(j)$ 。图中以 $i = 2, j = 4$ 为例，展示了一个逆序对 $(5, 2)$ 。

一节我们要解决的问题。

排列、逆序数与行列式的符号决定

为了将行列式的概念推广到 n 阶，我们需要引入**排列** (Permutation) 和**逆序数** (Inversion Number) 的概念，它们决定了行列式展开式中每一项的符号。

排列与逆序数

定义 3.3 (逆序对)

给定排列 $\sigma = (\sigma(1), \sigma(2), \dots, \sigma(n))$ ，若下标 $i < j$ 而数值 $\sigma(i) > \sigma(j)$ ，则称 $(\sigma(i), \sigma(j))$ 构成一个逆序对。^a

^a $\sigma(k)$ 表示排列中第 k 个位置上的数值，初学者常把它当成“下标”本身，其实它是排列后的元素值。

例题 3.1 在 $\sigma = (3, 1, 4, 2)$ 中：

- 对于数值 3 和 1 来说，对应索引下标 $1 < 2$ 且数值 $3 > 1 \Rightarrow (3, 1)$ 是逆序对；
- 对于数值 1 和 2 来说，对应索引下标 $2 < 4$ 且数值 $1 < 2 \Rightarrow (1, 2)$ 不是逆序对。

定义 3.4 (一个数的逆序数)

排列 σ 中，元素 $\sigma(k)$ 的逆序数等于排在它前面且比它大的元素个数，记为 $\tau_k(\sigma)$ 。

例题 3.2 继续 $\sigma = (3, 1, 4, 2)$ ：

- $\sigma(1) = 3$ ：前面无数， $\tau_1 = 0$ ；
- $\sigma(2) = 1$ ：前面有 $3 > 1$ ， $\tau_2 = 1$ ；
- $\sigma(3) = 4$ ：前面 3, 1 均小于 4， $\tau_3 = 0$ ；
- $\sigma(4) = 2$ ：前面 $3 > 2$ 且 $4 > 2$ ， $\tau_4 = 2$ 。

定义 3.5 (排列的逆序数与符号)

把排列 σ 中所有元素的逆序数相加, 得到 排列的逆序数

$$\tau(\sigma) = \sum_{k=1}^n \tau_k(\sigma).$$

若 $\tau(\sigma)$ 为奇数, 称 σ 为 奇排列; 若为偶数, 称 偶排列。定义排列的 符号

$$\operatorname{sgn}(\sigma) = (-1)^{\tau(\sigma)}.$$



例题 3.3 对 $\sigma = (3, 1, 4, 2)$ 已算得各 $\tau_k = (0, 1, 0, 2)$, 故

$$\tau(\sigma) = 0 + 1 + 0 + 2 = 3 \quad (\text{奇排列}), \quad \operatorname{sgn}(\sigma) = -1.$$

例题 3.4 求排列的逆序数 求排列 3, 2, 5, 1, 4 的逆序数。

解:

- 数字 3 前无比它大的数, 逆序数为 0。
- 数字 2 前有 1 个比它大的数 (3), 逆序数为 1。
- 数字 5 前无比它大的数, 逆序数为 0。
- 数字 1 前有 3 个比它大的数 (3, 2, 5), 逆序数为 3。
- 数字 4 前有 1 个比它大的数 (5), 逆序数为 1。

故此排列的总逆序数 $\tau = 0 + 1 + 0 + 3 + 1 = 5$, 为奇排列, 符号 $\operatorname{sgn}(\sigma) = -1$ 。

命题 3.1 (对换改变奇偶性)

对排列进行一次对换 (交换任意两个元素的位置), 会改变排列的奇偶性。



证明 设排列 $\sigma = (\dots, a, \dots, b, \dots)$, 其中 a 位于下标 i , b 位于下标 j ($i < j$)。

Step 1: 相邻对换若 $j = i + 1$, 则交换 a 与 b 只影响这对相邻元素。

- 若 (a, b) 原为顺序, 则对换后新增 1 个逆序;
- 若 (a, b) 原为逆序, 则对换后减少 1 个逆序。

无论哪种情况, 逆序数 τ 改变 ± 1 , 奇偶性反转。

Step 2: 一般对换 (冒泡实现) 将 b 逐步左移, 再把 a 逐步右移, 如下图所示 (每条竖线表示一次相邻对换):

a					b			
-----	--	--	--	--	-----	--	--	--

初始

a			b				
-----	--	--	-----	--	--	--	--

1 次相邻对换

a		b					
-----	--	-----	--	--	--	--	--

2 次

b	a						
-----	-----	--	--	--	--	--	--

 $j-i$ 次

b		a					
-----	--	-----	--	--	--	--	--

 $j-i+1$ 次

b				a			
-----	--	--	--	-----	--	--	--

 $2(j-i)-1$ 次

每一步仅交换相邻两元素，故均为相邻对换。把 b 从位置 j 移到 i 需 $j-i$ 步；此时 a 的位置下标是 $i+1$ ，再把 a 从位置 i 移到 j 又需 $j-(i+1) = j-i-1$ 步，共 $2(j-i)-1$ 次移动。

$$2(j-i)-1,$$

恰为奇数。由 Step 1，每步反转奇偶性，因此奇数次反转最终仍反转奇偶性。一般对换亦改变排列的奇偶性。

注 逆序数和排列的奇偶性是理解行列式定义的关键。在一般 n 阶行列式的定义中，每一项的符号正是由列标排列的奇偶性决定的，即 $\text{sgn}(\sigma)$ 。

更进一步，它保证了“用对换把排列化为自然序”所需的对换次数之奇偶性与具体路径无关，从而使 $\text{sgn}(\sigma)$ 是良定义的（即不管怎么算，最后算出的结果都是一样的）；这也为后续“行列式的多重线性与反对称性”的证明提供了不可或缺的工具。

n 阶行列式的莱布尼茨定义

经过前面的铺垫，我们已经打造了定义 n 阶行列式所需的所有精密零件：

1. **项的构成**：由来自不同行、不同列的 n 个元素构成的乘积，其列标构成了一个 n 元排列 σ 。
2. **项的符号**：由排列 σ 的奇偶性决定，即其符号 $\text{sgn}(\sigma)$ 。

现在，是时候将这些零件组装起来，构建行列式这座宏伟大厦的最终蓝图了。这个集大成的定义，正是以其思想的前驱者莱布尼茨的名字命名的。

定义 3.6 (n 阶行列式 (莱布尼茨定义))

设 $A = (a_{ij})$ 是一个 $n \times n$ 矩阵, 则其行列式定义为:

$$\det(A) = \sum_{\sigma \in S_n} \operatorname{sgn}(\sigma) \cdot a_{1\sigma(1)} a_{2\sigma(2)} \cdots a_{n\sigma(n)} = \sum_{\sigma \in S_n} \operatorname{sgn}(\sigma) \prod_{i=1}^n a_{i,\sigma(i)}. \quad (3.1)$$

其中:

- S_n 表示所有 n 元排列的集合。
- σ 是 S_n 中的一个排列。
- $\operatorname{sgn}(\sigma)$ 是排列 σ 的符号。
- 乘积 $\prod_{i=1}^n a_{i,\sigma(i)}$ 表示从矩阵 A 的每一行、每一列中恰好选取一个元素相乘。
原因: 因为 σ 是双射 (即一一对应), 所以行指标 $1, 2, \dots, n$ 与列指标 $\sigma(1), \sigma(2), \dots, \sigma(n)$ 各自恰好出现一次, 既不会重复选中同一列, 也不会漏掉任何一列; 同理, 行号也绝无重复或遗漏。这正是行列式展开式中“合法项”必须满足的基本前提——每一项都对应一条“完美匹配”。

**注 定义剖析:**

1. **项数:** 求和符号 $\sum_{\sigma \in S_n}$ 意味着共有 $n!$ 项。
2. **每一项的构成:** 每一项都是矩阵中位于**不同行、不同列**的 n 个元素的乘积。
3. **符号规则:** 每一项的符号由其列标排列 σ 的奇偶性决定。这也等价于由行标排列的奇偶性决定 (因为 $\det(A) = \det(A^T)$)。
4. **兼容性:** 此定义与之前给出的二、三阶行列式的定义完全一致。例如, 二阶行列式中的 $a_{11}a_{22}$ 对应排列 $(1, 2)$ (偶排列, 符号为正), $a_{12}a_{21}$ 对应排列 $(2, 1)$ (奇排列, 符号为负)。

例题 3.5 反对角线行列式 计算以下 4 阶行列式:

$$D = \begin{vmatrix} 0 & 0 & 0 & 4 \\ 0 & 0 & 3 & 0 \\ 0 & 2 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{vmatrix}.$$

解: 根据行列式的定义, 需要找出所有非零的项。观察发现, 只有元素 $a_{14}, a_{23}, a_{32}, a_{41}$ 不为零, 且它们恰好位于不同的行和列。因此, 只有一项可能非零: $a_{14}a_{23}a_{32}a_{41} = 4 \times 3 \times 2 \times 1 = 24$ 。现在确定这项的符号。这项对应的列标排列是 $\sigma = (4, 3, 2, 1)$ 。计算其逆序数:

$$\begin{aligned} \tau(\sigma) &= \tau(4, 3, 2, 1) \\ &= (43, 2, 1) + (32, 1) + (21) \\ &= 3 + 2 + 1 = 6. \end{aligned}$$

逆序数 $\tau(\sigma) = 6$ 是偶数, 故 $\operatorname{sgn}(\sigma) = +1$ 。因此, 该行列式的值 $D = 24$ 。

3.1.3 代码实现：莱布尼茨定义的代码逻辑

我们在前面的小节中已经探讨了行列式的严格数学定义——莱布尼茨公式。这个公式虽然在形式上高度抽象，但它却为我们提供了一套最原始、最直接的“暴力”计算方法。其核心思想可以分解为三个步骤：**遍历所有排列、计算每一项的乘积、赋予正确的符号并求和**。

代码视角

莱布尼茨公式为：

$$\det(A) = \sum_{\sigma \in S_n} \text{sgn}(\sigma) \prod_{i=1}^n a_{i,\sigma(i)}$$

要将这个公式翻译成代码，我们需要解决两个关键问题：

1. 如何生成所有的排列 $\sigma \in S_n$?
2. 如何为每一个排列 σ 计算其符号 $\text{sgn}(\sigma)$?

对于问题 (1)，我们可以利用算法生成一个序列 $\{0, 1, \dots, n-1\}$ 的所有全排列。对于问题 (2)，符号 $\text{sgn}(\sigma)$ 由排列 σ 的逆序数决定。一个排列的逆序数是指排列中所有逆序对的总数（即满足 $i < j$ 但 $\sigma(i) > \sigma(j)$ 的数对 (i, j) ）。如果一个排列的逆序数为偶数，其符号为 $+1$ ；如果为奇数，其符号为 -1 。

下面的 Python 代码完整地实现了这一逻辑。

```
import itertools

def determinant_leibniz(matrix):
    n = len(matrix)
    if n == 0 or len(matrix[0]) != n:
        raise ValueError("输入必须是一个 n x n 的方阵")

    # 获取所有列索引的全排列
    # 例如 n=3, indices = [0, 1, 2]
    # permutations 将是 [(0,1,2), (0,2,1), (1,0,2), ...]
    indices = list(range(n))
    permutations = list(itertools.permutations(indices))

    total_determinant = 0

    for p in permutations:
        # 1. 计算当前排列的符号 sgn(p)
        # 通过计算逆序数来实现
```

```

        inversions = 0
    for i in range(n):
        for j in range(i + 1, n):
            if p[i] > p[j]:
                inversions += 1

    sign = (-1)**inversions

    # 2. 计算连乘项 product
    # product = a[0, p[0]] * a[1, p[1]] * ...
    product = 1
    for i in range(n):
        product *= matrix[i][p[i]]

    # 3. 将带符号的乘积累加到总和中
    total_determinant += sign * product

return total_determinant

# --- 示例 ---
if __name__ == '__main__':
    # 一个 3x3 矩阵
    mat = [[1, 2, 3],
            [4, 5, 6],
            [7, 8, 9]]

    # 它的行列式应该是 0
    print(f"矩阵:\n{mat}")
    print(f"通过莱布尼茨公式计算的行列式: {determinant_leibniz(mat)}")

    # 一个 2x2 矩阵
    mat2 = [[3, 8],
            [4, 6]]

    #  $3*6 - 8*4 = 18 - 32 = -14$ 
    print(f"\n矩阵:\n{mat2}")
    print(f"通过莱布尼茨公式计算的行列式: {determinant_leibniz(mat2)}")

```

算法复杂度警示：仅适用于教学与微型矩阵

莱布尼茨定义的代码实现虽然完美地对应了其数学理论，但在计算效率上却是灾难性的。一个 $n \times n$ 矩阵，其排列总数有 $n!$ (n 的阶乘) 种。算法的整体时间复杂度大致为 $O(n! \cdot n)$ 。

这意味着，计算一个 10×10 矩阵的行列式就需要进行超过 3600 万次排列的计算，而对于一个 20×20 的矩阵，其计算量将达到天文数字，远超现代计算机的承受能力。因此，这种方法在实际工程应用中是完全不可行的，它更多地是作为一种理论定义和教学工具存在。在后续章节中，我们将学习更高效的计算方法，如利用行列式性质（高斯消元法）进行计算。

3.2 行列式的性质

3.2.1 历史引入：柯西与行列式理论的系统化

在 19 世纪之前，行列式在数学家的手中更像是一系列巧妙的计算“秘诀”，与求解线性方程组紧密相连。莱布尼茨、克拉默、拉普拉斯等先驱已经揭示了它的存在和部分威力，但这些知识是零散的、非系统的，缺乏一个统一的理论框架。行列式真正从一个“解题工具”蜕变为一门独立的、严谨的数学理论，很大程度上要归功于法国伟大的数学家——奥古斯丁-路易·柯西 (Augustin-Louis Cauchy)。

历史侧记

柯西在 1812 年向法兰西科学院提交了一篇长达 84 页的鸿篇巨著，这篇论文被视为行列式理论发展史上的里程碑。在这项工作中，柯西首次对行列式的理论进行了全面而系统的阐述，为其后续发展奠定了坚实的代数基础。

他的主要贡献可以概括为以下几个方面：

- **命名与推广**：柯西是第一批在现代意义上使用“行列式” (determinant) 这一术语的数学家之一，赋予了这个数学对象一个正式的名称。他不再局限于二阶或三阶的特殊情况，而是致力于研究 n 阶行列式的一般性质。
- **性质的严格证明**：柯西系统地证明了行列式的一系列基本性质。例如，他严格证明了交换行列式的任意两行或两列，行列式的值会变号，以及方阵转置后其行列式的值保持不变。
- **乘法定理的创立**：在所有性质中，柯西最重要的贡献莫过于证明了“**行列式乘法定理**”。

3.2.2 核心定理：行列式“运作”的五大基本法则

正如上一节所述，正是柯西的开创性工作，将行列式从零散的技巧整合为一门系统的理论。他所证明的一系列性质，正是我们能够高效计算和深刻理解行列式的基石。

这些优美而强大的性质，使我们不必每次都依赖项数高达 $n!$ 的莱布尼茨定义来进行繁琐的计算，同时也深刻地揭示了行列式作为“有向体积函数”的几何本质。本节将系统阐述其中最核心的五大法则。

性质一：转置不变性（行与列的对称性）

行列式的第一个，也是最根本的性质，是其定义在行与列上的完美对称性。这直接导出了行列式的值在转置（行列互换）后保持不变的结论。

定理 3.1 (转置不变)

一个 n 阶行列式的值，等于其转置行列式的值。其中，转置行列式是指将原行列式的行与列完全互换后得到的新行列式。



证明 我们的目标是证明一个行列式 D 和它的转置行列式 D^T 的值相等。核心策略是：从 D^T 的定义出发，通过一系列代数变形，最终证明它的展开式和 D 的展开式是完全一样的。

贯穿证明的案例

为了让抽象的符号变得具体，我们在证明的每一步都将以这个简单的二阶行列式 D 为例进行对照说明：

$$D = \begin{vmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{vmatrix}, \quad \text{其转置行列式为} \quad D^T = \begin{vmatrix} a_{11} & a_{21} \\ a_{12} & a_{22} \end{vmatrix}$$

我们已知 $D = a_{11}a_{22} - a_{12}a_{21}$ 。我们的目标是证明 D^T 经过一系列变形后，也会得到这个结果。

第一步：写出转置行列式 D^T 的定义式

- 理论层面：我们首先回顾转置的定义：如果原行列式 D 的元素是 a_{ij} ，那么其转置行列式 D^T 的第 i 行、第 j 列的元素 a'_{ij} 就等于原行列式第 j 行、第 i 列的元素 a_{ji} ，即 $a'_{ij} = a_{ji}$ 。现在，我们将莱布尼茨定义应用于 D^T ：

$$D^T = \sum_{\sigma \in S_n} \text{sgn}(\sigma) \prod_{i=1}^n a'_{i, \sigma(i)}$$

将 $a'_{i, \sigma(i)} = a_{\sigma(i), i}$ 代入上式，我们得到：

$$D^T = \sum_{\sigma \in S_n} \text{sgn}(\sigma) \prod_{i=1}^n a_{\sigma(i), i} \quad (3.2)$$

这个公式的含义是：对所有排列 σ 求和，每一项的乘积，是让列标 i 从 1 取到 n ，而对应的行标则是 $\sigma(i)$ 。

- 案例对照 ($n=2$)：对于我们的例子， $D^T = \begin{vmatrix} a'_{11} & a'_{12} \\ a'_{21} & a'_{22} \end{vmatrix} = \begin{vmatrix} a_{11} & a_{21} \\ a_{12} & a_{22} \end{vmatrix}$ 。排列集合 S_2 只有两个排列： $\sigma_1 = (1, 2)$ (偶排列, $\text{sgn} = +1$) 和 $\sigma_2 = (2, 1)$ (奇排列, $\text{sgn} = -1$)。将它们代入式 (3.2)：

$$\begin{aligned} D^T &= \text{sgn}(1, 2) \cdot (a_{1,1}a_{2,2}) + \text{sgn}(2, 1) \cdot (a_{2,1}a_{1,2}) \\ &= (+1) \cdot a_{11}a_{22} + (-1) \cdot a_{21}a_{12} \\ &= a_{11}a_{22} - a_{21}a_{12} \end{aligned}$$

我们已经得到了最终答案，但为了演示证明过程，我们将继续对理论公式进行变形，看看它如何与这个具体结果对应。

第二步：重新整理乘积项的顺序

- 理论层面：在式 (3.2) 的任意一个乘积项 $\prod_{i=1}^n a_{\sigma(i),i}$ 中，因子的顺序可以任意调换。我们的目标是，把它重新排成“行标”按 $1, 2, \dots, n$ 自然顺序排列的形式。为此，我们需要引入逆排列 $\pi = \sigma^{-1}$ 。

什么是逆排列？

如果一个排列 σ 回答了问题：“第 i 个位置上是什么值？”，记为 值 = $\sigma(\text{位置})$ ；那么它的逆排列 σ^{-1} 就回答相反的问题：“值 j 在第几个位置上？”，记为 位置 = $\sigma^{-1}(\text{值})$ 。

在乘积项 $a_{\sigma(i),i}$ 中，如果令行标 $j = \sigma(i)$ ，那么根据逆排列的定义，列标 i 就可以表示为 $i = \sigma^{-1}(j) = \pi(j)$ 。当我们重新整理整个乘积，让行标 j 从 1 遍历到 n 时，对应的列标就是 $\pi(j)$ 。因此，乘积可以改写为：

$$\prod_{i=1}^n a_{\sigma(i),i} = \prod_{j=1}^n a_{j,\pi(j)}$$

- 案例对照 ($n = 2$)：我们来看 D^T 中由 $\sigma = (2, 1)$ 产生的项，其乘积为 $a_{2,1}a_{1,2}$ 。
 - 排列 $\sigma = (2, 1)$ 的逆排列 $\pi = \sigma^{-1}$ 也是 $(2, 1)$ 。
 - 我们的目标是把乘积 $a_{2,1}a_{1,2}$ 改写成行标为 $(1, 2)$ 的顺序，即 $a_{1,2}a_{2,1}$ 。
 - 手动排序很简单： $a_{2,1}a_{1,2} = a_{1,2}a_{2,1}$ 。
 - 用逆排列公式验证： $a_{1,\pi(1)}a_{2,\pi(2)} = a_{1,2}a_{2,1}$ 。两者完全一致。

第三步：处理符号并得出结论

- 理论层面：一个基础结论是：一个排列和它的逆排列的逆序数具有相同的奇偶性。这意味着它们的符号是相同的： $\text{sgn}(\sigma) = \text{sgn}(\sigma^{-1}) = \text{sgn}(\pi)$ 。将第二步的结论代回 D^T 的表达式：

$$D^T = \sum_{\sigma \in S_n} \text{sgn}(\pi) \prod_{j=1}^n a_{j,\pi(j)}$$

当 σ 遍历所有排列时，其逆排列 π 也会不重不漏地遍历所有排列。因此，我们可以用 π 作为求和变量：

$$D^T = \sum_{\pi \in S_n} \text{sgn}(\pi) \prod_{j=1}^n a_{j,\pi(j)}$$

这个最终的表达式，与原行列式 D 的定义 $\sum_{\sigma \in S_n} \text{sgn}(\sigma) \prod_{i=1}^n a_{i,\sigma(i)}$ 在数学结构上是完全等价的。

- 案例对照 ($n = 2$)：我们已经知道 $D^T = \text{sgn}(1, 2)a_{1,1}a_{2,2} + \text{sgn}(2, 1)a_{2,1}a_{1,2}$ 。经过第二步和第三步的转换，它变为：

$$\begin{aligned} D^T &= \text{sgn}(\pi_1)a_{1,\pi_1(1)}a_{2,\pi_1(2)} + \text{sgn}(\pi_2)a_{1,\pi_2(1)}a_{2,\pi_2(2)} \\ &= \text{sgn}(1, 2)a_{1,1}a_{2,2} + \text{sgn}(2, 1)a_{1,2}a_{2,1} \\ &= (+1) \cdot a_{11}a_{22} + (-1) \cdot a_{12}a_{21} \\ &= a_{11}a_{22} - a_{12}a_{21} \end{aligned}$$

这个结果正是原行列式 D 的定义式。我们通过具体的例子，一步步验证了抽象的证明过程是完全成立的。

注 对称迁移原则这个定理的意义极为深远。它从根本上确立了行列式中“行”与“列”的平等地位。因此，任何只涉及“行”的性质，都可以被直接、无条件地推广到“列”上，反之亦然。这使得我们可以节省一半的证明工作，是后续所有性质的基石。

性质二：交换两行（列）——行列式变号

行列式作为交替函数（Alternating Function）的核心特性，在行（或列）被交换时，会改变其符号。这是行列式最重要的性质之一。

定理 3.2 (交换变号)

设矩阵 B 是由方阵 A 交换其第 s 行与第 t 行 ($s \neq t$) 所得的矩阵，则

$$\det(B) = -\det(A).$$

由于转置不改变行列式的值，此性质对列交换同样成立。



证明 我们的策略是，证明 $\det(B)$ 展开式中的每一项都恰好是 $\det(A)$ 展开式中某一项的相反数。如果每一项都变号了，那么它们的总和自然也变号了。

第一步：写出 $\det(B)$ 的一般项

根据行列式的定义， $\det(B)$ 是对所有列标排列求和的结果。我们从中任意取出一个由排列 $\pi = (j_1, j_2, \dots, j_s, \dots, j_t, \dots, j_n)$ 决定的项，记为 T_B ：

$$T_B = \text{sgn}(\pi) \cdot b_{1,j_1} \cdots b_{s,j_s} \cdots b_{t,j_t} \cdots b_{n,j_n}$$

由于矩阵 B 是由 A 交换 s, t 两行得到的，所以其元素关系为 $b_{s,k} = a_{t,k}$, $b_{t,k} = a_{s,k}$ ，而其他行保持不变 $b_{ik} = a_{ik}$ ($i \neq s, t$)。我们将这个关系代入 T_B ：

$$T_B = \text{sgn}(\pi) \cdot a_{1,j_1} \cdots a_{t,j_s} \cdots a_{s,j_t} \cdots a_{n,j_n}$$

第二步：找到 T_B 在 $\det(A)$ 中的对应项

观察 T_B 的乘积部分，它包含了和 $\det(A)$ 中某个项完全相同的 n 个元素。为了找到它在 $\det(A)$ 中的对应项，我们把这个乘积的因子重新排序，使其行标恢复为 $1, 2, \dots, s, \dots, t, \dots, n$ 的自然顺序：

$$\text{乘积部分} = a_{1,j_1} \cdots a_{s,j_t} \cdots a_{t,j_s} \cdots a_{n,j_n}$$

这个重新排序后的乘积，正是 $\det(A)$ 中由列标排列 $\sigma = (j_1, \dots, j_t, \dots, j_s, \dots, j_n)$ 所决定的项的乘积部分；换句话说，它把原来第 s 行选出的元素 a_{t,j_s} 放到了行标 s 的位置，而把第 t 行选出的元素 a_{s,j_t} 放到了行标 t 的位置，其余行保持不变，从而恰好对应 $\det(A)$ 展开式中排列 σ 的那一项。

我们将这一个在 $\det(A)$ 中对应的项记为 T_A 。

第三步：比较两对应项的符号

我们来比较排列 π 和 σ :

$$\pi = (j_1, \dots, j_s, \dots, j_t, \dots, j_n)$$

$$\sigma = (j_1, \dots, j_t, \dots, j_s, \dots, j_n)$$

很明显, 排列 σ 是由排列 π 经过一次对换 (交换第 s 位和第 t 位的元素) 得到的。根据“对换改变排列奇偶性”的引理, 它们的符号必然相反:

$$\operatorname{sgn}(\sigma) = -\operatorname{sgn}(\pi)$$

现在我们可以把 T_A 和 T_B 完整地写在一起进行比较:

$$T_A = \operatorname{sgn}(\sigma) \cdot (a_{1,j_1} \cdots a_{s,j_t} \cdots a_{t,j_s} \cdots a_{n,j_n})$$

$$T_B = \operatorname{sgn}(\pi) \cdot (a_{1,j_1} \cdots a_{t,j_s} \cdots a_{s,j_t} \cdots a_{n,j_n})$$

由于两者的乘积部分相同, 而 $\operatorname{sgn}(\pi) = -\operatorname{sgn}(\sigma)$, 我们得到:

$$T_B = -T_A$$

结论: 我们已经证明, $\det(B)$ 展开式中的任意一项, 都等于 $\det(A)$ 中某一项的相反数。由于这种一一对应的关系覆盖了所有 $n!$ 个项, 因此两行列式的总和也必然互为相反数, 即 $\det(B) = -\det(A)$ 。

例题 3.6 数值验证 我们来通过一个具体的例子, 一步步验证这个定理。给定矩阵

$$A = \begin{bmatrix} 1 & 7 & 5 \\ 6 & 6 & 2 \\ 3 & 5 & 8 \end{bmatrix}$$

我们将它的第 2 行和第 3 行交换, 得到新矩阵 B :

$$B = \begin{bmatrix} 1 & 7 & 5 \\ 3 & 5 & 8 \\ 6 & 6 & 2 \end{bmatrix}$$

第一步: 计算原行列式 $\det(A)$

我们可以使用沙路法则或者按第一行展开。这里按第一行展开:

$$\begin{aligned} \det(A) &= 1 \cdot \begin{vmatrix} 6 & 2 \\ 5 & 8 \end{vmatrix} - 7 \cdot \begin{vmatrix} 6 & 2 \\ 3 & 8 \end{vmatrix} + 5 \cdot \begin{vmatrix} 6 & 6 \\ 3 & 5 \end{vmatrix} \\ &= 1 \cdot (6 \times 8 - 2 \times 5) - 7 \cdot (6 \times 8 - 2 \times 3) + 5 \cdot (6 \times 5 - 6 \times 3) \\ &= 1 \cdot (48 - 10) - 7 \cdot (48 - 6) + 5 \cdot (30 - 18) \\ &= 1 \cdot (38) - 7 \cdot (42) + 5 \cdot (12) \\ &= 38 - 294 + 60 \\ &= -196 \end{aligned}$$

第二步: 计算新行列式 $\det(B)$

同样按第一行展开：

$$\begin{aligned}
 \det(B) &= 1 \cdot \begin{vmatrix} 5 & 8 \\ 6 & 2 \end{vmatrix} - 7 \cdot \begin{vmatrix} 3 & 8 \\ 6 & 2 \end{vmatrix} + 5 \cdot \begin{vmatrix} 3 & 5 \\ 6 & 6 \end{vmatrix} \\
 &= 1 \cdot (5 \times 2 - 8 \times 6) - 7 \cdot (3 \times 2 - 8 \times 6) + 5 \cdot (3 \times 6 - 5 \times 6) \\
 &= 1 \cdot (10 - 48) - 7 \cdot (6 - 48) + 5 \cdot (18 - 30) \\
 &= 1 \cdot (-38) - 7 \cdot (-42) + 5 \cdot (-12) \\
 &= -38 + 294 - 60 \\
 &= 196
 \end{aligned}$$

第三步：比较结果

我们计算得到 $\det(A) = -196$ 和 $\det(B) = 196$ 。可以清楚地看到, $\det(B) = -\det(A)$ 。这个具体的数值计算结果, 完美地验证了我们的定理。

性质三：线性性（倍乘与可加性）

行列式最重要的代数属性之一是它的“多重线性性”（Multilinearity）。这个词听起来很抽象, 但它指的是行列式对于它的某一行（或某一列）来说, 表现出优美的线性特征。我们可以把它分解成两个更简单的性质来理解：倍乘性和可加性。

1. 单行（列）的倍乘性

定理 3.3 (单行倍乘)

如果将行列式中某一行中的所有元素都乘以一个常数 k , 那么新的行列式的值等于原行列式的值乘以 k 。

$$\begin{vmatrix} \vdots & \vdots & \vdots \\ k \cdot a_{i1} & k \cdot a_{i2} & \cdots & k \cdot a_{in} \\ \vdots & \vdots & \vdots \end{vmatrix} = k \cdot \begin{vmatrix} \vdots & \vdots & \vdots \\ a_{i1} & a_{i2} & \cdots & a_{in} \\ \vdots & \vdots & \vdots \end{vmatrix}$$



证明 这个性质可以从行列式的定义式直接得出。让我们一步步来看：

1. 回顾定义：行列式的值是所有取自不同行、不同列的元素乘积项的代数和：

$$\det(A) = \sum_{\sigma \in S_n} \text{sgn}(\sigma) \cdot a_{1,\sigma(1)} a_{2,\sigma(2)} \cdots a_{n,\sigma(n)}$$

2. 观察每一项：在这个求和式中, 任意一项都必须恰好包含一个来自第 i 行的元素, 即 $a_{i,\sigma(i)}$ 。
3. 进行替换：现在, 我们将第 i 行的所有元素乘以 k 得到新矩阵 B 。那么 B 的元素满足 $b_{ik} = k \cdot a_{ik}$ 。对于 $\det(B)$ 展开式中的任意一项, 其乘积变为：

$$a_{1,\sigma(1)} \cdots (k \cdot a_{i,\sigma(i)}) \cdots a_{n,\sigma(n)}$$

4. 提取公因数：由于 k 是一个常数，我们可以把它从乘积中提出来：

$$k \cdot (a_{1,\sigma(1)} \cdots a_{i,\sigma(i)} \cdots a_{n,\sigma(n)})$$

5. 得出结论：这意味着， $\det(B)$ 展开式中的每一项都恰好是 $\det(A)$ 对应项的 k 倍。根据分配律，我们可以将这个公因数 k 从整个求和式中提出来：

$$\det(B) = \sum_{\sigma \in S_n} \text{sgn}(\sigma) \cdot [k \cdot (\dots)] = k \cdot \left(\sum_{\sigma \in S_n} \text{sgn}(\sigma) \cdot (\dots) \right) = k \cdot \det(A)$$

证明完毕。

注 重要区分：单行倍乘 vs. 矩阵数乘

初学者极易混淆这两个概念。

- **单行倍乘**：只有一个行（或列）被乘以 k 。

$$\det(\dots, k\mathbf{r}_i, \dots) = k \cdot \det(\dots, \mathbf{r}_i, \dots)$$

- **矩阵数乘**：矩阵的所有元素都乘以 k 。这意味着矩阵的**每一行**都被乘以了 k 。根据单行倍乘性质，我们需要把每一行的 k 都提取出来，共 n 次。

$$\det(kA) = k^n \det(A)$$

2. 单行（列）的可加性

定理 3.4 (单行可加)

如果行列式中某一行（或列）的每个元素都可以写成两数之和，那么这个行列式可以分解为两个行列式之和。

$$\begin{vmatrix} \vdots & \vdots & & \\ b_1 + c_1 & b_2 + c_2 & \cdots & \\ \vdots & \vdots & & \end{vmatrix} = \begin{vmatrix} \vdots & \vdots & & \\ b_1 & b_2 & \cdots & \\ \vdots & \vdots & & \end{vmatrix} + \begin{vmatrix} \vdots & \vdots & & \\ c_1 & c_2 & \cdots & \\ \vdots & \vdots & & \end{vmatrix}$$



证明 这个性质的证明同样源于定义式和乘法分配律。

1. 写出一般项：设第 i 行的元素是 $a_{ij} = b_{ij} + c_{ij}$ 。行列式展开式中的任意一项为：

$$\text{sgn}(\sigma) \cdot a_{1,\sigma(1)} \cdots (b_{i,\sigma(i)} + c_{i,\sigma(i)}) \cdots a_{n,\sigma(n)}$$

2. 应用分配律：根据乘法对加法的分配律，这个乘积项可以被拆分成两部分之和：

$$\text{sgn}(\sigma) \cdot (a_{1,\sigma(1)} \cdots b_{i,\sigma(i)} \cdots) + \text{sgn}(\sigma) \cdot (a_{1,\sigma(1)} \cdots c_{i,\sigma(i)} \cdots)$$

3. 拆分总和：由于展开式中的每一项都可以这样拆分，因此整个行列式的总和也可以拆分为两个总和：

$$\det(A) = \sum (\text{第一部分}) + \sum (\text{第二部分})$$

4. 识别新行列式：观察这两个新的总和，第一个总和正是将原行列式第 i 行换成 $(b_{i1}, \dots, b_{in})$ 后得到的新行列式的定义式；第二个总和则是将第 i 行换成 $(c_{i1}, \dots, c_{in})$ 后的行列式的定义式。

定理得证。

例题 3.7 线性性质的直观应用 我们通过一个简单的例子来验证这两个性质。计算行列

$$\text{式 } D = \begin{vmatrix} 4 & 10 \\ 3 & 5 \end{vmatrix}.$$

直接计算: $D = 4 \times 5 - 10 \times 3 = 20 - 30 = -10$ 。

应用性质一 (倍乘性): 我们观察到第一行有公因数 2。

$$D = \begin{vmatrix} 2 \cdot 2 & 2 \cdot 5 \\ 3 & 5 \end{vmatrix} \stackrel{\text{性质三}}{=} 2 \cdot \begin{vmatrix} 2 & 5 \\ 3 & 5 \end{vmatrix}$$

现在计算这个更简单的行列式:

$$2 \cdot (2 \times 5 - 5 \times 3) = 2 \cdot (10 - 15) = 2 \cdot (-5) = -10.$$

结果与直接计算完全一致。

应用性质二 (可加性): 我们可以将第一行 $(4, 10)$ 拆分为两个更简单的向量之和, 例如 $(1, 0) + (3, 10)$ 。

$$D = \begin{vmatrix} 1+3 & 0+10 \\ 3 & 5 \end{vmatrix} \stackrel{\text{性质三}}{=} \begin{vmatrix} 1 & 0 \\ 3 & 5 \end{vmatrix} + \begin{vmatrix} 3 & 10 \\ 3 & 5 \end{vmatrix}$$

现在分别计算这两个行列式:

$$\text{第一个行列式} = 1 \times 5 - 0 \times 3 = 5$$

$$\text{第二个行列式} = 3 \times 5 - 10 \times 3 = 15 - 30 = -15$$

将两者相加: $5 + (-15) = -10$ 。结果再次与直接计算一致。这个例子清晰地展示了如何利用线性性质来分解和简化行列式的计算。

性质四: 零值判定定理

行列式为零的情况蕴含着矩阵线性相关的重要信息。

定理 3.5 (零值充要条件)

行列式 $\det(A) = 0$ 当且仅当以下任一情况发生:

- (i) A 中存在一行 (或一列) 元素全为零;
- (ii) A 中存在两行 (或两列) 完全相同;
- (iii) A 中存在两行 (或两列) 元素成比例。



证明

- (i) 若第 i 行全零, 则莱布尼茨定义中每一项都包含因子 $a_{i,\sigma(i)} = 0$, 故和为 0。
- (ii) 设第 i 行与第 j 行相同。交换这两行, 根据定理 3.2, 有 $\det(A) = -\det(A)$, 故 $2\det(A) = 0$, 所以 $\det(A) = 0$ 。
- (iii) 若第 i 行与第 j 行成比例, 设 $\mathbf{r}_i = k\mathbf{r}_j$ 。则由定理 3.3, 可提取因子 $k: \det(\dots, k\mathbf{r}_j, \dots, \mathbf{r}_j, \dots) = k \det(\dots, \mathbf{r}_j, \dots, \mathbf{r}_j, \dots)$ 。由情形 (ii), 后者为 0。

反之，若 $\det(A) = 0$ ，则 A 的列向量组（或行向量组）线性相关，必然导致上述至少一种情况发生（或可经初等变换化为上述情况）。

性质五：倍加不变性（剪切变换）

这是行列式计算中最常用且重要的性质，它对应几何上的剪切变换，不改变体积。

定理 3.6 (倍加不变)

将矩阵 A 的第 j 行的 k 倍加到第 i 行 ($i \neq j$) 上，行列式的值不变。即：

$$\mathbf{r}_i \leftarrow \mathbf{r}_i + k\mathbf{r}_j \implies \det(\dots, \mathbf{r}_i + k\mathbf{r}_j, \dots, \mathbf{r}_j, \dots) = \det(A).$$

对列变换有相同结论： $\mathbf{c}_i \leftarrow \mathbf{c}_i + k\mathbf{c}_j$ 行列式也不变。



证明 由定理 3.4，新行列式可分解：

$$\det(\dots, \mathbf{r}_i + k\mathbf{r}_j, \dots, \mathbf{r}_j, \dots) = \det(\dots, \mathbf{r}_i, \dots, \mathbf{r}_j, \dots) + k \det(\dots, \mathbf{r}_j, \dots, \mathbf{r}_j, \dots).$$

右边第二项包含两行相同（第 i 行和第 j 行都是 $\mathbf{r}_j$ 或它的倍数），由定理 3.5 知其值为 0。故上式等于 $\det(A)$ 。

例题 3.8 高斯消元求行列式 计算 $\begin{vmatrix} 1 & 2 & 3 \\ 2 & 3 & 4 \\ 3 & 4 & 5 \end{vmatrix}$ 。解：用倍加不变化三角：

$$\xrightarrow{r_2-2r_1, r_3-3r_1} \begin{vmatrix} 1 & 2 & 3 \\ 0 & -1 & -2 \\ 0 & -2 & -4 \end{vmatrix} \xrightarrow{r_3-2r_2} \begin{vmatrix} 1 & 2 & 3 \\ 0 & -1 & -2 \\ 0 & 0 & 0 \end{vmatrix} = 0.$$

五大性质总结与计算策略

要点提炼

行列式五大核心性质总结

1. **转置不变性**： $\det(A) = \det(A^T)$
2. **交换变号**：交换两行（列），行列式变号
3. **线性性**
 - **单行倍乘**：某一行（列）乘以 k ，行列式变为原来的 k 倍
 - **单行可加**：某一行（列）是两向量之和，行列式可分解为两行列式之和
4. **零值判定**：全零行/相同行/比例行 $\Rightarrow \det = 0$
5. **倍加不变**：某行的 k 倍加到另一行，行列式不变

基于这些性质，我们可以发展出高效计算行列式的通用策略。**注**：高斯消元法（化

三角形法)

计算行列式最有效的方法之一是**高斯消元法**：

1. 利用**倍加不变**性质，将矩阵化为上三角（或下三角）矩阵。
2. 在化三角过程中，可能需要交换两行以获取非零主元。每次交换需记录**符号改变**（定理 3.2）。
3. 化三角过程中，可能需提取某行的公因子（定理 3.3），需记录这些**系数**。
4. 三角矩阵的行列式等于其主对角线元素的乘积（见例 ??）。
5. 最终行列式值 = (主对角线乘积) × (所有行交换产生的符号因子) × (所有提取的系数之积的倒数)。

此方法是许多计算机代数系统计算行列式的基础。

典型例题

例题 3.9 化三角法求值 计算行列式：

$$D = \begin{vmatrix} 1 & 2 & 3 \\ 2 & 3 & 4 \\ 3 & 4 & 5 \end{vmatrix}.$$

解：

$$\begin{aligned} & \begin{vmatrix} 1 & 2 & 3 \\ 2 & 3 & 4 \\ 3 & 4 & 5 \end{vmatrix} \\ & \xrightarrow{r_2-2r_1, r_3-3r_1} \begin{vmatrix} 1 & 2 & 3 \\ 0 & -1 & -2 \\ 0 & -2 & -4 \end{vmatrix} \\ & \xrightarrow{r_3-2r_2} \begin{vmatrix} 1 & 2 & 3 \\ 0 & -1 & -2 \\ 0 & 0 & 0 \end{vmatrix} = 0. \end{aligned}$$

最后得到一个全零行，根据定理 3.5， $D = 0$ 。

例题 3.10“幂次平移”不变性的再证 设 $D = \det(a_{ij})_{n \times n}$ ， $D' = \det(a_{ij}b^{i-j})_{n \times n}$ 。证明： $D' = D$ 。证（利用性质）：观察 D' 的每一行。第 i 行的每个元素都含有因子 b^i 。由定理 3.3，每行可提取公因子 b^i ，故 $D' = (b^1 b^2 \cdots b^n) \det(a_{ij}b^{-j}) = b^{n(n+1)/2} \det(a_{ij}b^{-j})$ 。再观察每一列。第 j 列的每个元素都含有因子 b^{-j} 。每列可提取公因子 b^{-j} ，故 $\det(a_{ij}b^{-j}) = (b^{-1}b^{-2} \cdots b^{-n}) \det(a_{ij}) = b^{-n(n+1)/2} D$ 。因此， $D' = b^{n(n+1)/2} \cdot b^{-n(n+1)/2} D = D$ 。

例题 3.11 对称常数矩阵的行列式 计算 n 阶行列式：

$$D_n = \begin{vmatrix} a & b & \cdots & b \\ b & a & \cdots & b \\ \vdots & \vdots & \ddots & \vdots \\ b & b & \cdots & a \end{vmatrix}.$$

解（巧用倍加不变和可加性）：

$$\begin{aligned} D_n &= \begin{vmatrix} a + (n-1)b & b & \cdots & b \\ a + (n-1)b & a & \cdots & b \\ \vdots & \vdots & \ddots & \vdots \\ a + (n-1)b & b & \cdots & a \end{vmatrix} && \text{(将所有列加到第 1 列, 定理 3.6)} \\ &= [a + (n-1)b] \begin{vmatrix} 1 & b & \cdots & b \\ 1 & a & \cdots & b \\ \vdots & \vdots & \ddots & \vdots \\ 1 & b & \cdots & a \end{vmatrix} && \text{(第 1 列提取公因子)} \\ &= [a + (n-1)b] \begin{vmatrix} 1 & b & \cdots & b \\ 0 & a-b & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & a-b \end{vmatrix} && \text{(第 1 行乘以 } -b \text{ 加到后续各行, 定理 3.6).} \end{aligned}$$

这是一个上三角行列式，其值为主对角线元素乘积：

$$D_n = [a + (n-1)b] (a-b)^{n-1}.$$

例题 3.12 可拆项与秩判定 计算行列式：

$$D = \begin{vmatrix} x_1 + 1 & x_2 + 1 & \cdots & x_n + 1 \\ x_1 + 2 & x_2 + 2 & \cdots & x_n + 2 \\ \vdots & \vdots & \ddots & \vdots \\ x_1 + n & x_2 + n & \cdots & x_n + n \end{vmatrix}.$$

解（利用可加性分析秩）：将每一列拆分为两个列向量之和（定理 3.4），例如第 j 列：

$$\begin{bmatrix} x_j + 1 \\ x_j + 2 \\ \vdots \\ x_j + n \end{bmatrix} = \begin{bmatrix} x_j \\ x_j \\ \vdots \\ x_j \end{bmatrix} + \begin{bmatrix} 1 \\ 2 \\ \vdots \\ n \end{bmatrix}. \quad \text{因此, 整个矩阵的列向量可以由两组向量张成: } \mathbf{u} = \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix} \quad (\text{各 } x_j \text{ 的系数) 和 } \mathbf{v} = \begin{bmatrix} 1 \\ 2 \\ \vdots \\ n \end{bmatrix}.$$

这意味着矩阵的列空间维数（即秩）最多为 2。当 $n > 2$ 时，列向

量线性相关, 根据定理 3.5, $D = 0$ 。当 $n = 2$ 时, 可直接计算: $D_2 = \begin{vmatrix} x_1 + 1 & x_2 + 1 \\ x_1 + 2 & x_2 + 2 \end{vmatrix} = (x_1 + 1)(x_2 + 2) - (x_2 + 1)(x_1 + 2) = x_2 - x_1$ 。

例题 3.13 三阶行列式变换 证明:

$$\det \begin{bmatrix} 1 & a & a^2 \\ 1 & b & b^2 \\ 1 & c & c^2 \end{bmatrix} = (b-a)(c-a)(c-b).$$

证

进行行变换, 然后按第一行展开:

$$\begin{vmatrix} 1 & a & a^2 \\ 1 & b & b^2 \\ 1 & c & c^2 \end{vmatrix} = \begin{vmatrix} 1 & a & a^2 \\ 0 & b-a & b^2-a^2 \\ 0 & c-a & c^2-a^2 \end{vmatrix} = \begin{vmatrix} b-a & b^2-a^2 \\ c-a & c^2-a^2 \end{vmatrix} = (b-a)(c-a) \begin{vmatrix} 1 & b+a \\ 1 & c+a \end{vmatrix} = (b-a)(c-a)(c-b).$$

例题 3.14 上三角行列式 证明: 上三角行列式的值等于其主对角线上元素的乘积。

$$D = \begin{vmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ & a_{22} & \cdots & a_{2n} \\ & & \ddots & \vdots \\ & & & a_{nn} \end{vmatrix} = a_{11}a_{22} \cdots a_{nn}.$$

证: 根据行列式的定义, 每一项都是取自不同行不同列的 n 个元素的乘积。考虑第一行, 如果选取的不是 a_{11} , 而是某个 $a_{1j}(j > 1)$, 那么由于矩阵下三角部分 (主对角线以下) 全为零, 即 $a_{i1} = 0$ (对于 $i > 1$), 后续行中将无法再选取第一列的元素 (因为每列只能选一个)。但更重要的是, 如果要构成非零项, 每一行选取的元素其列号必须不能小于该行的行号太多 (否则会选到零)。实际上, 唯一可能非零的项就是选取主对角线上的元素 $a_{11}, a_{22}, \dots, a_{nn}$ 的乘积。这项对应的排列是恒等排列 $\sigma = (1, 2, \dots, n)$, 其逆序数为 0 (偶排列), 符号为正。因此, $D = a_{11}a_{22} \cdots a_{nn}$ 。

类似地, 下三角行列式和 **diagonal** 行列式 (对角行列式) 的值也等于其主对角线上元素的乘积。

例题 3.15 数值型下三角行列式 计算:

$$\begin{vmatrix} 8 & 0 & 0 & 0 \\ 6 & 5 & 0 & 0 \\ 1 & 2 & 4 & 0 \\ 3 & 2 & 1 & 4 \end{vmatrix}.$$

解: 这是一个下三角行列式, 其值等于主对角线元素的乘积。

$$D = 8 \times 5 \times 4 \times 4 = 640.$$

例题 3.16 一般形式的反对角行列式 证明：反对角行列式（次对角线不为零）的值为

$$D = \begin{vmatrix} 0 & \cdots & 0 & a_{1n} \\ 0 & \cdots & a_{2,n-1} & 0 \\ \vdots & \vdots & \vdots & \vdots \\ a_{n1} & \cdots & 0 & 0 \end{vmatrix} = (-1)^{\frac{n(n-1)}{2}} a_{1n} a_{2,n-1} \cdots a_{n1}.$$

证：与例 1 类似，非零项只有一项： $a_{1n}a_{2,n-1}\cdots a_{n1}$ 。这项对应的列标排列为 $\sigma = (n, n-1, \dots, 2, 1)$ 。其逆序数为

$$\tau(\sigma) = \tau(n, n-1, \dots, 1) = (n-1) + (n-2) + \cdots + 1 + 0 = \frac{n(n-1)}{2}.$$

因此符号为 $(-1)^{\frac{n(n-1)}{2}}$ 。故 $D = (-1)^{\frac{n(n-1)}{2}} a_{1n} a_{2,n-1} \cdots a_{n1}$ 。

例题 3.17 判断项是否为行列式的项并确定符号 判断下列乘积是否为 5 阶行列式 $\det(a_{ij})$ 的项，若是，确定其符号：

1. $a_{14}a_{23}a_{31}a_{42}a_{55}$;
2. $a_{14}a_{23}a_{31}a_{42}a_{56}$;
3. $-a_{32}a_{43}a_{14}a_{51}a_{25}a_{66}$ （按 6 阶行列式讨论）。

证明 [解]

1. 检查行列位置

行标：(1, 2, 3, 4, 5) 列标：(4, 3, 1, 2, 5)。

各列指标互不相同，且行列一一对应，故为 5 阶行列式的一项。

确定符号

列标排列 $\sigma = (4, 3, 1, 2, 5)$ 的逆序数

$$\tau(\sigma) = 3 + 2 + 0 + 0 + 0 = 5 \quad (\text{奇排列}).$$

因此该项符号为 $(-1)^5 = -1$ 。

2. 检查行列位置

行标：(1, 2, 3, 4, 5) 列标：(4, 3, 1, 2, 6)。

列标出现 6，超出 5 阶行列式范围，不是合法项。

3. 检查行列位置

行标：(3, 4, 1, 5, 2, 6) 列标：(2, 3, 4, 1, 5, 6)。

行列指标均各不重复，为 6 阶行列式的一项。

确定符号

列标排列 $\sigma = (2, 3, 4, 1, 5, 6)$ 的逆序数

$$\tau(\sigma) = 0 + 0 + 0 + 3 + 0 + 0 = 3 \quad (\text{奇排列}).$$

根据定义，该项本身符号为 $(-1)^3 = -1$ 。

题目在乘积前又加一负号，整体符号变为 $(-1) \times (-1) = +1$ ，与标准展开式不符，故不是行列式定义中的标准项。

注意：判断时通常先看元素是否来自不同行不同列，然后计算列标排列的逆序数决

定符号，最后与所给符号对比。

例题 3.18 特殊反对角行列式 计算：

$$D_n = \begin{vmatrix} 0 & 0 & \cdots & 0 & 1 \\ 0 & 0 & \cdots & 2 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & n-1 & \cdots & 0 & 0 \\ n & 0 & \cdots & 0 & 0 \end{vmatrix}.$$

解：这个行列式是**例题：一般形式的反对角行列式**的特殊情况，其中 $a_{1n} = 1, a_{2,n-1} = 2, \dots, a_{n1} = n$ 。根据这个例题的结论：

$$D_n = (-1)^{\frac{n(n-1)}{2}} (1 \times 2 \times \cdots \times n) = (-1)^{\frac{n(n-1)}{2}} n!.$$

要点提炼

行列式计算策略小结

- **观察先行：**先观察是否有特殊结构（如三角、范德蒙德、循环矩阵等）。
- **高斯消元：**无特殊结构时，优先使用行（列）倍加变换化三角阵。
- **灵活拆项：**当某行（列）可表示为和的形式时，考虑拆项（定理 3.4）。
- **秩的判断：**若明显存在线性相关的行（列），则行列式为零（定理 3.5）。
- **递归降阶：**适用于稀疏矩阵或 0 元素多的矩阵，按行（列）展开。

注：常见误区与提醒

- **误区一：** $\det(A+B) = \det(A) + \det(B)$? **错误！**行列式不具有这种加性。
- **误区二：** $\det(kA) = k \det(A)$? **错误！**应为 $\det(kA) = k^n \det(A)$ 。
- **误区三：**进行倍加行变换 $r_i \leftarrow r_i + kr_j$ 时，误以为行列式值改变。**正确：**不变（定理 3.6）。
- **提醒：**行交换会改变符号（定理 3.2），计算过程中需仔细跟踪符号变化。
- **提醒：**化三角法是计算行列式的通用有效方法，其理论基础就是本节的性质。

3.2.3 代码实现：用代码验证性质

理论学习之后，最好的巩固方式莫过于亲手实践。在本节中，我们将使用主流的深度学习框架 PyTorch，通过其强大的张量 (Tensor) 运算功能来直观地验证前文讨论的行列式核心性质。PyTorch 提供了 ‘torch.linalg.det()’ 函数来快速计算方阵的行列式，让我们得以从繁琐的手动计算中解放出来，专注于性质本身。

代码视角

我们将通过代码来验证以下几个关键性质：

- **性质 1：**方阵转置，行列式值不变，即 $\det(A) = \det(A^T)$ 。

- **性质 2:** 矩阵乘积的行列式等于行列式的乘积，即 $\det(AB) = \det(A) \cdot \det(B)$ 。
- **性质 3:** 行列式为 0 的几种情况（例如，两行或两列线性相关）。
- **性质 4:** 交换任意两行或两列，行列式变号。

下面的 Python 代码使用 PyTorch 完整地实现了这些性质的验证。

```
import torch

# --- 准备工作：定义两个 3x3 的示例张量 A 和 B ---
# 注意：行列式计算通常在浮点数上进行，因此指定 dtype=torch.float32
A = torch.tensor([[1, 2, 3],
                  [3, 0, 1],
                  [1, 2, 1]], dtype=torch.float32)

B = torch.tensor([[4, 1, 5],
                  [2, 3, 6],
                  [8, 7, 9]], dtype=torch.float32)

# 计算 A 和 B 的行列式以备后用
det_A = torch.linalg.det(A)
det_B = torch.linalg.det(B)

print(f"det(A) = {det_A:.2f}")
print(f"det(B) = {det_B:.2f}\n")

# --- 1. 验证性质一：det(A) = det(A^T) ---
print("---- 验证性质一：转置 ----")
# A 的转置张量
A_T = A.T
det_A_T = torch.linalg.det(A_T)
print(f"det(A^T) = {det_A_T:.2f}")
# 检查两者是否近似相等（考虑到浮点数精度问题）
print(f"性质验证：{torch.isclose(det_A, det_A_T)}\n")

# --- 2. 验证性质二：det(AB) = det(A) * det(B) ---
```



```

print("--- 验证性质二： 矩阵乘法 ---")
# 矩阵 AB 的乘积
AB = A @ B
det_AB = torch.linalg.det(AB)
print(f"det(A) * det(B) = {det_A * det_B:.2f}")
print(f"det(AB) = {det_AB:.2f}")
print(f"性质验证: {torch.isclose(det_AB, det_A * det_B)}\n")

# --- 3. 验证性质三： 行列式为 0 的情况 ---
print("--- 验证性质三： 行列式为 0 ---")
# 构造一个两行成比例的张量 C (第3行 = 2 * 第1行)
C = torch.tensor([[1, 2, 3],
                  [4, 5, 6],
                  [2, 4, 6]], dtype=torch.float32)
det_C = torch.linalg.det(C)
print(f"张量 C (两行成比例):\n{C}")
# 行列式理论上为 0, 计算结果会是一个非常接近 0 的浮点数
print(f"det(C) = {det_C:.2f} (理论值为 0)\n")

# --- 4. 验证性质四： 交换两行, 行列式变号 ---
print("--- 验证性质四： 交换两行 ---")
# 构造一个新张量 D, 通过交换 A 的第1行和第2行得到
D = A.clone() # 先克隆一份 A
D[[0, 1]] = D[[1, 0]] # 交换第0行和第1行
det_D = torch.linalg.det(D)
print(f"张量 A:\n{A}")
print(f"det(A) = {det_A:.2f}\n")
print(f"张量 D (交换 A 的前两行):\n{D}")
print(f"det(D) = {det_D:.2f}")
print(f"性质验证 (det(D) == -det(A)): {torch.isclose(det_D, -det_A)}\n")

```

代码运行输出

将上述代码运行后, 得到的输出结果如下:

代码视角

```
det(A) = 4.00
det(B) = -80.00

--- 验证性质一：转置 ---
det(A^T) = 4.00
性质验证: True

--- 验证性质二：矩阵乘法 ---
det(A) * det(B) = -320.00
det(AB) = -320.00
性质验证: True

--- 验证性质三：行列式为 0 ---
张量 C (两行成比例):
tensor([[1., 2., 3.],
        [4., 5., 6.],
        [2., 4., 6.]])
det(C) = 0.00 (理论值为 0)

--- 验证性质四：交换两行 ---
张量 A:
tensor([[1., 2., 3.],
        [3., 0., 1.],
        [1., 2., 1.]])
det(A) = 4.00

张量 D (交换 A 的前两行):
tensor([[3., 0., 1.],
        [1., 2., 3.],
        [1., 2., 1.]])
det(D) = -4.00
性质验证 (det(D) == -det(A)): True
```

结果分析

从运行输出可以清晰地看到，代码成功验证了所有四个性质：

- **转置不变性:** 矩阵 A 的行列式 $\det(A) = 4.00$, 其转置矩阵 A^T 的行列式 $\det(A^T) = 4.00$, 两者相等。
- **乘法性质:** $\det(A) \cdot \det(B)$ 的计算结果为 -320.00 , 而矩阵乘积 AB 的行列式 $\det(AB)$ 同样为 -320.00 , 两者相等。
- **奇异矩阵:** 对于矩阵 C , 其第三行是第一行的两倍 (线性相关), 计算得到的行列式为 0.00 , 符合理论值。
- **行交换性质:** 矩阵 A 的行列式为 4.00 。将其前两行交换得到矩阵 D 后, 其行列式 $\det(D) = -4.00$, 恰好是原行列式的相反数。

所有验证步骤中的 ‘`torch.isclose()`’ 函数返回值均为 `True`, 这表明在考虑浮点数计算精度的情况下, 理论性质得到了有效的实验验证。

3.3 行列式的计算方法

3.3.1 历史引入：拉普拉斯、范德蒙与展开定理

随着行列式在求解线性方程组中的应用日益增多，数学家们迫切需要一种比莱布尼茨公式更实用、更具结构化的计算方法。直接使用莱布尼茨公式计算高阶行列式，其计算量会随着阶数的增加而发生“组合爆炸”，这在实践中是不可行的。18 世纪后期，两位法国数学家分别从不同角度推动了行列式计算方法的重大突破，他们的工作共同构成了我们今天所知的“展开定理”的基石。

历史侧记

皮埃尔-西蒙·拉普拉斯 (Pierre-Simon Laplace)

在 1772 年，伟大的数学家和天文学家拉普拉斯提出了一种计算行列式的通用方法，这种方法巧妙地将一个高阶行列式的计算，递归地分解为若干个低一阶行列式的计算。这个方法后来被称为“**拉普拉斯展开**” (Laplace Expansion) 或“**余子式展开**” (Cofactor Expansion)。

拉普拉斯的洞见在于，一个 $n \times n$ 行列式的值，可以沿着它的任意一行或任意一列进行“展开”。展开的过程就是将该行（或列）的每一个元素，乘以一个带符号的、阶数更低的行列式（即它的**代数余子式**），然后将所有这些乘积相加。这种方法不仅大大简化了手算过程，更重要的是，它提供了一种清晰的、可编程的递归算法。

历史侧记

亚历山大-泰奥菲尔·范德蒙 (Alexandre-Théophile Vandermonde)

几乎在同一时期（1771 年），范德蒙也对行列式理论做出了开创性的贡献。与当时大多数将行列式视为解方程工具的数学家不同，范德蒙是第一批将行列式作为一个**独立的函数**来研究的学者。他致力于探索行列式本身的代数结构和性质，而不是仅仅把它当作一个计算过程。

他引入了一套关于行列式下标的系统记号，并研究了当某些元素为 0 时行列式如何化简。虽然他没有像拉普拉斯那样明确提出一个通用的展开公式，但他对行列式结构的深刻分析，为后来的理论发展（包括拉普拉斯展开的严格化）铺平了道路。今天，一种以他名字命名的特殊行列式“**范德蒙行列式**”仍在各个领域发挥着重要作用。

要点提炼

核心思想：降维与递归

拉普拉斯和范德蒙工作的共同之处，在于他们都抓住了行列式计算的核心思想——**降维**。无论是拉普拉斯的展开定理，还是范德蒙对结构的分析，其本质都是将一个复杂的 n 阶问题，转化为若干个相对简单的 $n-1$ 阶问题。

这种递归的思想，是整个行列式计算理论的基石。它告诉我们，无论行列式有多高的阶数，我们总能通过一步步的“展开”，最终将其归结为我们熟知的二阶或三阶行列式的计算。本章后续介绍的核心定理和具体计算方法，正是这一思想的直接体现。

3.3.2 核心定理：代数余子式、拉普拉斯展开与特殊行列式

上一小节我们看到，拉普拉斯和范德蒙为我们指明了计算高阶行列式的核心思想——“降维与递归”。为了将这个卓越的思想转化为精确、可操作的数学工具，本节将系统地介绍实现这一目标所需的理论基石和具体方法。

我们将首先定义实现“降阶”的关键概念——**余子式与代数余子式**；然后在此基础上，给出将行列式沿任意行（列）展开的**拉普拉斯展开定理**；最后，介绍一些利用这些工具计算特殊行列式的技巧。

要点提炼

- 余子式与代数余子式：降阶计算的核心工具
- 展开定理：高阶行列式化为低阶行列式的和
- 正交关系：同行（列）代数余子式的内在正交性
- 拉普拉斯展开： k 阶子式的一般化展开

余子式与代数余子式：降阶的基石

为了将高阶行列式转化为低阶行列式计算，我们需要先定义两个核心概念。

定义 3.7 (余子式与代数余子式)

设 $A = (a_{ij})_{n \times n}$ 是一个 n 阶方阵。

1. 元素 a_{ij} 的余子式 (Minor)，记作 M_{ij} ，是指从 A 中划去第 i 行和第 j 列后，剩下的 $(n-1)$ 阶行列式。
2. 元素 a_{ij} 的代数余子式 (Cofactor)，记作 A_{ij} ，定义为：

$$A_{ij} = (-1)^{i+j} M_{ij}.$$



注： $(-1)^{i+j}$ 的符号呈现出棋盘格样的正负交替 pattern：左上角 $(1,1)$ 位置为 $+$ ，然后像国际象棋棋盘一样，相邻格子的符号相反。计算时，可先求出余子式 M_{ij} ，再乘以对应位置的符号 $(-1)^{i+j}$ 。

例题 3.19 对于四阶行列式

$$D = \begin{vmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{vmatrix},$$

元素 a_{23} 的余子式为：

$$M_{23} = \begin{vmatrix} a_{11} & a_{12} & a_{14} \\ a_{31} & a_{32} & a_{34} \\ a_{41} & a_{42} & a_{44} \end{vmatrix},$$

其代数余子式为：

$$A_{23} = (-1)^{2+3} M_{23} = -M_{23}.$$

行列式按一行（列）展开定理

有了代数余子式的概念，我们现在可以给出行列式计算中最强大的工具之一——按一行（列）展开定理。这个定理的本质，是将一个高阶行列式的计算，“降维”成若干个低一阶行列式的计算。

定理 3.7 (行列式按一行（列）展开)

一个 n 阶行列式的值，等于其任意一行（或一列）的各个元素，与它们各自对应的代数余子式的乘积之和。

- 按第 i 行展开：

$$D = a_{i1}A_{i1} + a_{i2}A_{i2} + \cdots + a_{in}A_{in} = \sum_{k=1}^n a_{ik}A_{ik}$$

- 按第 j 列展开：

$$D = a_{1j}A_{1j} + a_{2j}A_{2j} + \cdots + a_{nj}A_{nj} = \sum_{k=1}^n a_{kj}A_{kj}$$



证明 我们以“按行展开”为例进行证明，“按列展开”可由转置不变性直接得到。证明的核心思路是：利用之前学过的“单行可加性”性质，巧妙地将一个复杂的行列式拆解为多个简单的行列式之和，从而推导出展开公式。

第一步：拆分目标行

- 理论层面：我们任意选定第 i 行，它的行向量 $(a_{i1}, a_{i2}, \dots, a_{in})$ 可以看作是 n 个“只有一个非零元素”的向量之和：

$$(a_{i1}, \dots, a_{in}) = (a_{i1}, 0, \dots, 0) + (0, a_{i2}, \dots, 0) + \cdots + (0, \dots, 0, a_{in})$$

- 案例对照 (以 3 阶行列式按第 1 行展开为例)：我们将第一行向量 (a_{11}, a_{12}, a_{13}) 拆

分为三个向量的和：

$$(a_{11}, a_{12}, a_{13}) = (a_{11}, 0, 0) + (0, a_{12}, 0) + (0, 0, a_{13})$$

第二步：利用“单行可加性”拆分行列式

- 理论层面：根据行列式的“单行可加性”性质，既然第 i 行可以拆分为 n 个向量的和，那么原行列式就可以拆分为 n 个新行列式的和。在这 n 个新行列式中，第 k 个行列式的第 i 行只有 a_{ik} 这一个非零元素，其余元素均为 0。
- 案例对照：将原行列式 D 拆分为三个行列式 D_1, D_2, D_3 之和：

$$D = \begin{vmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{vmatrix} = \begin{vmatrix} a_{11} & 0 & 0 \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{vmatrix} + \begin{vmatrix} 0 & a_{12} & 0 \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{vmatrix} + \begin{vmatrix} 0 & 0 & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{vmatrix}$$

第三步：化简拆分后的每个行列式

- 理论层面：现在我们来处理其中任意一个行列式，比如第 k 个，它的第 i 行只有 a_{ik} 非零。对于这类行列式，有一个重要的引理：

引理

若一个行列式的第 i 行只有第 k 个元素 a_{ik} 不为零，则该行列式的值等于 a_{ik} 乘以其自身的代数余子式 A_{ik} 。即：

$$\text{值为 } a_{ik}A_{ik} = a_{ik} \cdot (-1)^{i+k}M_{ik}$$

这个引理的证明思路是：通过 $i-1$ 次相邻换行和 $k-1$ 次相邻换列，可以将元素 a_{ik} “移动”到左上角的 $(1,1)$ 位置。每次交换都会给行列式乘以 -1 ，总共乘以了 $(-1)^{(i-1)+(k-1)} = (-1)^{i+k-2} = (-1)^{i+k}$ 。移动后，行列式变为左上角是 a_{ik} ，第一行和第一列其余元素为 0 的形式，其值等于 a_{ik} 乘以右下角的 $n-1$ 阶行列式，而这个 $n-1$ 阶行列式正是 a_{ik} 的余子式 M_{ik} 。

- 案例对照：应用上述引理，我们可以分别计算 D_1, D_2, D_3 的值：

$$\begin{aligned} D_1 &= a_{11} \cdot A_{11} = a_{11} \cdot (-1)^{1+1}M_{11} = a_{11} \begin{vmatrix} a_{22} & a_{23} \\ a_{32} & a_{33} \end{vmatrix} \\ D_2 &= a_{12} \cdot A_{12} = a_{12} \cdot (-1)^{1+2}M_{12} = -a_{12} \begin{vmatrix} a_{21} & a_{23} \\ a_{31} & a_{33} \end{vmatrix} \\ D_3 &= a_{13} \cdot A_{13} = a_{13} \cdot (-1)^{1+3}M_{13} = a_{13} \begin{vmatrix} a_{21} & a_{22} \\ a_{31} & a_{32} \end{vmatrix} \end{aligned}$$

第四步：求和得到最终公式

- 理论层面：将所有 n 个简化后的行列式的值相加，我们就得到了最终的按第 i 行展开的公式：

$$D = a_{i1}A_{i1} + a_{i2}A_{i2} + \cdots + a_{in}A_{in} = \sum_{k=1}^n a_{ik}A_{ik}$$

证明完毕。

- 案例对照：将 D_1, D_2, D_3 的值加起来，我们得到了 3×3 行列式按第一行展开的公式，这与我们熟知的写法完全一致：

$$D = D_1 + D_2 + D_3 = a_{11}A_{11} + a_{12}A_{12} + a_{13}A_{13}$$

例题 3.20 三阶行列式按第一行展开的标准写法 根据我们刚刚证明的定理，一个三阶行列式可以这样展开：

$$\begin{vmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{vmatrix} = a_{11} \cdot A_{11} + a_{12} \cdot A_{12} + a_{13} \cdot A_{13}$$

将代数余子式 $A_{ij} = (-1)^{i+j}M_{ij}$ 代入，并注意各项符号的交替： $(-1)^{1+1} = +1, (-1)^{1+2} = -1, (-1)^{1+3} = +1$ 。

$$= a_{11} \begin{vmatrix} a_{22} & a_{23} \\ a_{32} & a_{33} \end{vmatrix} - a_{12} \begin{vmatrix} a_{21} & a_{23} \\ a_{31} & a_{33} \end{vmatrix} + a_{13} \begin{vmatrix} a_{21} & a_{22} \\ a_{31} & a_{32} \end{vmatrix}$$

注 实用技巧：展开时，应优先选择零元素最多的行或列。因为零元素对应的项 $a_{ij}A_{ij} = 0 \cdot A_{ij} = 0$ ，可以避免不必要的计算，从而显著简化问题。

代数余子式的正交关系

代数余子式之间存在着一种深刻且优美的内在关系，通常被称为“正交关系”。这一定理不仅在理论上很重要，更是推导逆矩阵公式（通过伴随矩阵）的基石。

定理 3.8 (代数余子式的正交关系)

一个行列式中，某一行（列）的各元素与其自身对应的代数余子式乘积之和，等于行列式的值；而某一行（列）的各元素与另一行（列）对应的代数余子式乘积之和，恒等于零。

我们可以用克罗内克 delta 符号 δ_{ij} 将这两种情况统一起来：

$$\begin{aligned} \text{按行: } \sum_{k=1}^n a_{ik}A_{jk} &= \det(A)\delta_{ij} = \begin{cases} \det(A), & \text{当 } i = j \\ 0, & \text{当 } i \neq j \end{cases} \\ \text{按列: } \sum_{k=1}^n a_{ki}A_{kj} &= \det(A)\delta_{ij} = \begin{cases} \det(A), & \text{当 } i = j \\ 0, & \text{当 } i \neq j \end{cases} \end{aligned}$$

其中 δ_{ij} 定义为：

$$\delta_{ij} = \begin{cases} 1 & \text{当 } i = j \\ 0 & \text{当 } i \neq j \end{cases}$$

证明 我们将按行的情况分两种情形讨论：

情形一：当 $i = j$ 时我们要证明的式子是 $\sum_{k=1}^n a_{ik}A_{ik} = \det(A)$ 。这正是我们上一节证明的“行列式按第 i 行展开”定理本身，所以结论自然成立。

情形二：当 $i \neq j$ 时我们要证明的式子是 $\sum_{k=1}^n a_{ik}A_{jk} = 0$ 。这里的证明非常巧妙。我们不是直接计算这个和式，而是构造一个新的行列式 D' ，并证明这个和式恰好是 D' 的展开式，而 D' 的值又必然为 0。

1. 构造一个新的行列式 D' ：我们基于原行列式 D 构造一个新行列式 D' ，方法是：将 D 的第 j 行，用 D 的第 i 行的元素来替换。 D' 的其余各行与 D 完全相同。

$$D' = \begin{vmatrix} \cdots & \cdots & \cdots \\ a_{i1} & a_{i2} & \cdots & a_{in} \\ \cdots & \cdots & \cdots \\ a_{i1} & a_{i2} & \cdots & a_{in} \\ \cdots & \cdots & \cdots \end{vmatrix}$$

2. 计算 D' 的值：由于行列式 D' 中存在两行完全相同（第 i 行和第 j 行），根据行列式的性质四，其值必然为零。

$$\det(D') = 0$$

3. 按第 j 行展开 D' ：现在，我们对 D' 使用“按行展开”定理，选择对其第 j 行进行展开。

$$\det(D') = \sum_{k=1}^n (\text{第 } j \text{ 行第 } k \text{ 列的元素}) \cdot (\text{对应的代数余子式})$$

- D' 第 j 行的元素是什么？根据我们的构造，它们是 $a_{i1}, a_{i2}, \dots, a_{in}$ 。
- D' 第 j 行元素的代数余子式是什么？代数余子式 A'_{jk} 的计算，需要划掉第 j 行和第 k 列。因为 D' 与原行列式 D 只有第 j 行不同，所以当我们划掉第 j 行后，计算余子式所用的其余 $n-1$ 行，两者是完全一样的。因此， $A'_{jk} = A_{jk}$ 。将这两部分代入展开式，我们得到：

$$\det(D') = \sum_{k=1}^n a_{ik}A_{jk}$$

4. 得出结论：我们从两个不同的角度得到了 $\det(D')$ 的表达式：

$$\det(D') = 0 \quad \text{并且} \quad \det(D') = \sum_{k=1}^n a_{ik}A_{jk}$$

因此，必然有：

$$\sum_{k=1}^n a_{ik}A_{jk} = 0$$

证明完毕。按列的情况同理可证。

例题 3.21

3 × 3 案例验证：让我们通过一个具体的例子来感受一下 $i \neq j$ 时的证明过程。设原

行列式为 $D = \begin{vmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{vmatrix}$ 。我们来验证和式： $a_{11}A_{21} + a_{12}A_{22} + a_{13}A_{23}$ （即 $i = 1, j = 2$ ）的值。

1. 写出和式:

$$\begin{aligned} \sum_{k=1}^3 a_{1k} A_{2k} &= a_{11}(-M_{21}) + a_{12}(+M_{22}) + a_{13}(-M_{23}) \\ &= -a_{11} \begin{vmatrix} a_{12} & a_{13} \\ a_{32} & a_{33} \end{vmatrix} + a_{12} \begin{vmatrix} a_{11} & a_{13} \\ a_{31} & a_{33} \end{vmatrix} - a_{13} \begin{vmatrix} a_{11} & a_{12} \\ a_{31} & a_{32} \end{vmatrix} \end{aligned}$$

2. 构造新行列式 D' : 我们构造一个新行列式 D' , 方法是: 将 D 的第 2 行替换为第 1 行。

$$D' = \begin{vmatrix} a_{11} & a_{12} & a_{13} \\ a_{11} & a_{12} & a_{13} \\ a_{31} & a_{32} & a_{33} \end{vmatrix}$$

因为 D' 有两行相同, 所以 $\det(D') = 0$ 。

3. 按第 2 行展开 D' : 我们对 D' 按其第 2 行展开。注意, 第 2 行的元素是 (a_{11}, a_{12}, a_{13}) , 而代数余子式与原行列式 D 的代数余子式 A_{2k} 相同。

$$\begin{aligned} \det(D') &= a_{11} \cdot A'_{21} + a_{12} \cdot A'_{22} + a_{13} \cdot A'_{23} \\ &= a_{11} \cdot A_{21} + a_{12} \cdot A_{22} + a_{13} \cdot A_{23} \end{aligned}$$

4. 得出结论: 我们看到, 我们想计算的和式, 恰好等于行列式 D' 的值。而我们又知道 D' 的值必为 0。因此, 这个和式的值就是 0。

拉普拉斯展开: k 阶子式的一般化展开

行列式按一行(列)展开可以推广到按 k 行(列)展开, 这就是强大的拉普拉斯展开 (Laplace Expansion), 它提供了更大的灵活性。

定义 3.8 (k 阶子式与对应的代数余子式)

在 n 阶行列式 $\det(A)$ 中, 任意选取 k 行 $i_1 < i_2 < \cdots < i_k$ 和 k 列 $j_1 < j_2 < \cdots < j_k$, 这些行列交叉处的元素按原顺序排列形成的 k 阶行列式, 称为一个 k 阶子式, 记作 $M(i_1, \dots, i_k | j_1, \dots, j_k)$ 。划去这 k 行 k 列后, 剩下的 $(n-k)$ 阶行列式称为该 k 阶子式的余子式。定义其对应的代数余子式为:

$$A(i_1, \dots, i_k | j_1, \dots, j_k) = (-1)^{(i_1 + \cdots + i_k) + (j_1 + \cdots + j_k)} \cdot N,$$

其中 N 是余子式。



定理 3.9 (拉普拉斯展开定理)

在 n 阶行列式 $\det(A)$ 中, 取定任意 k 行 (或 k 列), 则行列式等于所有这些 k 阶子式与其对应的代数余子式的乘积之和:

$$\det(A) = \sum M(i_1, \dots, i_k | j_1, \dots, j_k) \cdot A(i_1, \dots, i_k | j_1, \dots, j_k),$$

求和遍及所有从 n 列 (或 n 行) 中选取 k 列 (或 k 行) 的组合。



证明 定理的证明可通过组合数学的方法，或对 k 进行数学归纳法。其基本思想是将选取的 k 行（列）的所有可能子式与其补集（余子式）的贡献进行组合，利用行列式的定义和排列性质即可得证。

注 当 $k = 1$ 时，拉普拉斯展开退化为按一行（列）展开定理。拉普拉斯展开提供了更大的自由度，有时选择特定的 k 行（列）（例如包含较多零的行列）可以极大简化计算。

特别地，当矩阵具有分块形式时，拉普拉斯展开尤为高效：

- 对于分块上三角矩阵 $\begin{bmatrix} A & B \\ O & D \end{bmatrix}$ ，有 $\det = \det(A) \cdot \det(D)$
- 对于分块下三角矩阵 $\begin{bmatrix} A & O \\ C & D \end{bmatrix}$ ，有 $\det = \det(A) \cdot \det(D)$
- 对于分块对角矩阵 $\begin{bmatrix} A & O \\ O & D \end{bmatrix}$ ，有 $\det = \det(A) \cdot \det(D)$

这些都可以看作是拉普拉斯展开的特例。

小结

要点提炼

本节核心要点

- **余子式与代数余子式**： $M_{ij}, A_{ij} = (-1)^{i+j} M_{ij}$ 。
- **展开定理**： $\det(A) = \sum_{k=1}^n a_{ik} A_{ik} = \sum_{k=1}^n a_{kj} A_{kj}$ 。
- **正交关系**： $\sum_k a_{ik} A_{jk} = \det(A) \delta_{ij}$ ，伴随矩阵 $\text{adj}(A)$ ，逆矩阵公式 $A^{-1} = \frac{\text{adj}(A)}{\det(A)}$ 。
- **拉普拉斯展开**：选取 k 行（列），用 k 阶子式及其代数余子式展开。
- **范德蒙行列式**： $V_n = \prod_{1 \leq j < i \leq n} (x_i - x_j)$ 。
- **计算策略**：观察 $\rightarrow$ 创造零 $\rightarrow$ 选择零多的行/列展开 $\rightarrow$ 递归降阶；巧妙利用分块、范德蒙等特殊结构。

行列式按行（列）展开的理论不仅是一种计算工具，更体现了“化归”的数学思想——将复杂的高阶问题转化为简单的低阶问题。这一思想在后续的矩阵特征值计算、相似对角化等问题中将继续发挥重要作用。同时，行列式作为线性变换的“体积缩放因子”，其展开定理也从代数角度揭示了这种缩放是如何由各分量（代数余子式）组合而成的。

实战技巧：降阶计算策略与范例

行列式展开定理的核心价值在于提供了一套系统且强大的降阶计算策略。以下通过几个典型例子展示如何灵活运用这些定理。

例题 3.22 优先选择零多的行或列 计算行列式：

$$D = \begin{vmatrix} 3 & 0 & 2 & -1 \\ 1 & 0 & 0 & 5 \\ 0 & -1 & 0 & 2 \\ -2 & 1 & 0 & 3 \end{vmatrix}.$$

观察发现第 3 列零元素最多（只有一个非零元素），故按第 3 列展开：

$$D = a_{13}A_{13} + a_{23}A_{23} + a_{33}A_{33} + a_{43}A_{43} = 2 \cdot A_{13} + 0 \cdot A_{23} + 0 \cdot A_{33} + 0 \cdot A_{43} = 2 \cdot A_{13}.$$

其中 $a_{13} = 2$, $A_{13} = (-1)^{1+3}M_{13} = M_{13}$ 。

$$M_{13} = \begin{vmatrix} 1 & 0 & 5 \\ 0 & -1 & 2 \\ -2 & 1 & 3 \end{vmatrix}.$$

计算此三阶行列式（可按第一行展开）：

$$M_{13} = 1 \cdot \begin{vmatrix} -1 & 2 \\ 1 & 3 \end{vmatrix} - 0 \cdot \begin{vmatrix} 0 & 2 \\ -2 & 3 \end{vmatrix} + 5 \cdot \begin{vmatrix} 0 & -1 \\ -2 & 1 \end{vmatrix} = 1 \cdot (-5) + 5 \cdot (-2) = -5 - 10 = -15.$$

因此， $D = 2 \times (-15) = -30$ 。

要点提炼

降阶法优选策略：

1. **观察先行：** 首先寻找零元素最多的行或列。
2. **创造零元：** 如果没有零元素丰富的行或列，先使用行列式的性质（特别是倍加不变性： $r_i \leftarrow r_i + kr_j$ ）进行行变换，制造出更多的零元素。
3. **递归降阶：** 按选定的行或列展开后，对得到的低阶行列式重复上述过程，直至化为二阶或三阶行列式。

例题 3.23 利用拉普拉斯展开处理分块矩阵 计算行列式：

$$D = \begin{vmatrix} 2 & 3 & 0 & 0 \\ 1 & 4 & 0 & 0 \\ -1 & 2 & 5 & 6 \\ 3 & 1 & 7 & 8 \end{vmatrix}.$$

此矩阵呈分块上三角形： $\begin{bmatrix} A & O \\ C & D \end{bmatrix}$ ，其中 $A = \begin{bmatrix} 2 & 3 \\ 1 & 4 \end{bmatrix}$, $C = \begin{bmatrix} -1 & 2 \\ 3 & 1 \end{bmatrix}$, $D = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$ 。选择前两行进行拉普拉斯展开（ $k=2$ ），非零的 2 阶子式只有 A （其他的主子式都有全为 0 的列），故：

$$\det \begin{bmatrix} A & O \\ C & D \end{bmatrix} = \det(A) \cdot \det(D).$$

因此，

$$D = \det(A) \cdot \det(D) = (2 \times 4 - 3 \times 1) \cdot (5 \times 8 - 6 \times 7) = (8 - 3) \cdot (40 - 42) = 5 \times (-2) = -10.$$

范德蒙行列式：一个强大的公式

范德蒙行列式是一种特殊形式的行列式，它具有简洁漂亮的计算公式，在多项式插值、线性无关性判断等问题中应用广泛。

定义 3.9 (范德蒙行列式)

n 阶范德蒙行列式 (Vandermonde Determinant) 定义为：

$$V_n(x_1, x_2, \dots, x_n) = \begin{vmatrix} 1 & 1 & 1 & \cdots & 1 \\ x_1 & x_2 & x_3 & \cdots & x_n \\ x_1^2 & x_2^2 & x_3^2 & \cdots & x_n^2 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ x_1^{n-1} & x_2^{n-1} & x_3^{n-1} & \cdots & x_n^{n-1} \end{vmatrix}.$$

其值由以下公式给出：

$$V_n(x_1, x_2, \dots, x_n) = \prod_{1 \leq j < i \leq n} (x_i - x_j).$$



证明 [归纳证明要点]

1. 基础步骤： $n = 2$ 时， $V_2 = \begin{vmatrix} 1 & 1 \\ x_1 & x_2 \end{vmatrix} = x_2 - x_1$ ，公式成立。
2. 归纳假设：假设公式对 $n - 1$ 阶范德蒙行列式成立。
3. 归纳步骤：对于 n 阶范德蒙行列式 $V_n(x_1, \dots, x_n)$ ，从第 n 行开始，依次将每一行减去上一行的 x_1 倍：

$$r_n \leftarrow r_n - x_1 r_{n-1}, \quad r_{n-1} \leftarrow r_{n-1} - x_1 r_{n-2}, \quad \dots, \quad r_2 \leftarrow r_2 - x_1 r_1.$$

这样操作后，第一列除第一个元素外全为零，且第 j 列 ($j > 1$) 的第 i 行元素变为 $x_j^{i-1}(x_j - x_1)$ 。按第一列展开，并提取每列的公因子 $(x_j - x_1)$ ，得到：

$$V_n = (x_2 - x_1)(x_3 - x_1) \cdots (x_n - x_1) \cdot V_{n-1}(x_2, x_3, \dots, x_n).$$

代入归纳假设，即得证。

例题 3.24 计算范德蒙行列式 计算 $V = \begin{vmatrix} 1 & 1 & 1 \\ a & b & c \\ a^2 & b^2 & c^2 \end{vmatrix}$ 。根据公式：

$$V = (b - a)(c - a)(c - b).$$

几何视角

范德蒙行列式不为零（即所有 x_i 互不相等）的充要条件是向量组 $(1, x_i, x_i^2, \dots, x_i^{n-1})$ 线性无关。这等价于说存在唯一的次数小于 n 的多项式经过 n 个不同的点 (x_i, y_i) （多项式插值），深刻揭示了行列式、线性无关和多项式插值之间的内在联系。

历史侧记

范德蒙行列式以法国数学家 Alexandre-Théophile Vandermonde 命名，尽管其在其著作中并未明确给出此行列式，但由于他在行列式发展中的先驱性工作，此行列式被冠以其名。范德蒙是第一个系统研究行列式性质并将其应用于线性方程组、多项式理论的数学家之一。

综合例题与思维进阶

以下例题综合运用行列式的性质和展开定理，展示了解决更复杂问题时的思维流程。

例题 3.25 五阶行列式的计算 计算

$$D = \begin{vmatrix} 5 & -3 & 1 & 2 & 0 \\ 1 & 7 & 2 & 5 & 2 \\ 0 & 2 & 3 & 1 & 0 \\ 0 & 4 & -1 & 4 & 0 \\ 0 & 2 & 3 & -5 & 0 \end{vmatrix}.$$

观察发现第 5 列仅 $a_{25} = 2$ 非零，故按第 5 列展开：

$$D = a_{25}A_{25} = 2 \cdot (-1)^{2+5}M_{25} = -2M_{25}.$$

M_{25} 是划去第 2 行第 5 列后的 4 阶行列式：

$$M_{25} = \begin{vmatrix} 5 & -3 & 1 & 2 \\ 0 & 2 & 3 & 1 \\ 0 & 4 & -1 & 4 \\ 0 & 2 & 3 & -5 \end{vmatrix}.$$

此行列式第 1 列仅 $a_{11} = 5$ 非零，故再按第 1 列展开：

$$M_{25} = 5 \cdot (-1)^{1+1} \begin{vmatrix} 2 & 3 & 1 \\ 4 & -1 & 4 \\ 2 & 3 & -5 \end{vmatrix} = 5 \cdot \begin{vmatrix} 2 & 3 & 1 \\ 4 & -1 & 4 \\ 2 & 3 & -5 \end{vmatrix}.$$

计算三阶行列式：

$$\begin{vmatrix} 2 & 3 & 1 \\ 4 & -1 & 4 \\ 2 & 3 & -5 \end{vmatrix} = 2 \cdot (-1) \cdot (-5) + 3 \cdot 4 \cdot 2 + 1 \cdot 4 \cdot 3 - 1 \cdot (-1) \cdot 2 - 3 \cdot 4 \cdot (-5) - 2 \cdot 4 \cdot 3 = 10 + 24 + 12 + 2 + 60 - 24 = 84.$$

因此， $M_{25} = 5 \times 84 = 420$ ， $D = -2 \times 420 = -840$ 。

例题 3.26 结构化行列式的巧算 计算

$$D = \begin{vmatrix} a & 0 & a & 0 & a \\ b & 0 & c & 0 & d \\ b^2 & 0 & c^2 & 0 & d^2 \\ 0 & ab & 0 & bc & 0 \\ 0 & cd & 0 & da & 0 \end{vmatrix}.$$

这个行列式结构特殊。将列重新排序（奇数列在前，偶数列在后）： C_1, C_3, C_5, C_2, C_4 。这是一个列置换，其符号为 $(-1)^\tau$ ，其中 τ 是排列 $(1, 3, 5, 2, 4)$ 的逆序数。计算逆序数： $\tau(1, 3, 5, 2, 4) = 0 + 1 + 2 + 0 = 3$ （奇），故符号为 -1 。新行列式为：

$$D' = - \begin{vmatrix} a & a & a & 0 & 0 \\ b & c & d & 0 & 0 \\ b^2 & c^2 & d^2 & 0 & 0 \\ 0 & 0 & 0 & ab & bc \\ 0 & 0 & 0 & cd & da \end{vmatrix}.$$

这是一个分块对角矩阵： $\begin{bmatrix} M & O \\ O & N \end{bmatrix}$ ，其中

$$M = \begin{bmatrix} a & a & a \\ b & c & d \\ b^2 & c^2 & d^2 \end{bmatrix}, \quad N = \begin{bmatrix} ab & bc \\ cd & da \end{bmatrix}.$$

分块对角矩阵的行列式等于各主对角线块行列式之积： $D' = -\det(M) \cdot \det(N)$ 。计算 $\det(M)$ ：从第一行提取公因子 a ，并识别出是范德蒙行列式的转置：

$$\det(M) = a \begin{vmatrix} 1 & 1 & 1 \\ b & c & d \\ b^2 & c^2 & d^2 \end{vmatrix} = a(c-b)(d-b)(d-c).$$

计算 $\det(N)$ ：

$$\det(N) = (ab)(da) - (bc)(cd) = a^2bd - bc^2d = bd(a^2 - c^2).$$

因此，

$$\begin{aligned} D = -D' &= -[-\det(M)\det(N)] = \det(M)\det(N) = [a(c-b)(d-b)(d-c)] \cdot [bd(a^2 - c^2)] \\ &= abd(c-b)(d-b)(d-c)(a-c)(a+c). \end{aligned}$$

3.3.3 代码实现：从递归到高斯消元的算法之旅

计算行列式，我们有两条截然不同的算法路径。一条是“理论之路”，它严格遵循拉普拉斯展开定理，通过递归将问题层层分解，思路清晰，代码优美；另一条是“效率之路”，它巧妙地利用行变换的性质，通过高斯消元法将矩阵简化，追求极致的计算性能。让我们依次探索这两条路径的实现逻辑。

代码视角

算法一：递归法 (Laplace 展开)

这种方法是拉普拉斯展开定理最直接的代码翻译。其核心思想是，一个 $n \times n$ 矩阵的行列式，可以表示为它第一行所有元素与其对应代数余子式的乘积之和。而代数余子式本身又包含一个 $(n-1) \times (n-1)$ 矩阵的行列式，从而形成了一个天然的递归结构。

```
import numpy as np

# 辅助函数：获取余子矩阵 (Minor Matrix)
def get_minor(matrix, row, col):
    """ 删除指定行和列，返回余子矩阵 """
    return np.delete(np.delete(matrix, row, axis=0), col, axis=1)

def determinant_recursive(matrix):
    [cite_start]""" 使用拉普拉斯展开递归计算行列式 [cite: 1152, 1160] """
    n = matrix.shape[0]

    # 递归的基准情况 (Base Case)
    if n == 1:
        return matrix[0, 0]

    det = 0
    # 沿第一行展开
    for col_idx in range(n):
        # 计算代数余子式
        cofactor = ((-1) ** col_idx) *
            determinant_recursive(get_minor(matrix, 0, col_idx))
        # 累加求和
        det += matrix[0, col_idx] * cofactor

    return det

# --- 测试 ---
A = np.array([[1, 2, 3],
               [3, 0, 1],
               [1, 2, 1]])
```

```
print(f"递归法计算 det(A) = {determinant_recursive(A)}")
```

代码视角

算法二：高斯消元法 (Row Reduction)

递归法虽然直观，但其时间复杂度高达 $O(n!)$ ，在实际应用中几乎不可行。更高效的方法是利用行列式的性质，通过高斯消元将矩阵转化为一个上三角矩阵。

这个算法的基石是两条关键性质：

- **性质 1:** 将某一行或某一列的倍数加到另一行或另一列上，行列式的值不变。
- **性质 2:** 交换任意两行或两列，行列式的值变号。
- **性质 3:** 三角矩阵的行列式等于其对角线元素的乘积。

算法的思路是：通过行变换将矩阵化为上三角形式，同时记录行交换的次数。最终行列式的值就等于变换后上三角矩阵的对角线元素之积，再乘以 $(-1)^{\text{交换次数}}$ 。

```
def determinant_gaussian(matrix):
    """ 使用高斯消元法计算行列式 """
    A = matrix.copy().astype(float) # 复制矩阵并转为浮点数以保证精度
    n = A.shape[0]
    det_sign = 1.0 # 用于记录行交换的符号变化

    for i in range(n):
        # 寻找主元 (pivot)
        pivot = i
        while pivot < n and np.isclose(A[pivot, i], 0):
            pivot += 1

        if pivot == n: # 如果当前列全为0，则行列式为0
            return 0

        if pivot != i:
            # 交换行，改变行列式符号
            A[[i, pivot]] = A[[pivot, i]]
            det_sign *= -1

        # 消元过程 (将当前列在主元下方的元素变为0)
        for j in range(i + 1, n):
```



```

        factor = A[j, i] / A[i, i]
        A[j, i:] -= factor * A[i, i:]

    # 计算对角线元素的乘积
    diagonal_prod = np.prod(np.diag(A))

    return det_sign * diagonal_prod

# --- 测试 ---
A = np.array([[1, 2, 3],
              [3, 0, 1],
              [1, 2, 1]])
print(f"高斯消元法计算 det(A) = {determinant_gaussian(A)}")

```

注意

我们的算法之旅从理论走向了实践，也从优雅走向了高效。

- **递归法**: 完美体现了数学定义，但时间复杂度为 $O(n!)$ 。对于一个 20×20 的矩阵，其计算量是天文数字。
- **高斯消元法**: 利用了行列式的内在性质，将时间复杂度革命性地降低到了 $O(n^3)$ 。对于 20×20 的矩阵，计算瞬间即可完成。

这个对比鲜明地告诉我们，在计算科学中，一个聪明的算法远比最强大的硬件更为重要。现代科学计算库（如 NumPy 和 PyTorch）内部计算行列式时，正是采用了基于高斯消元（或更优化的 LU 分解）的高效算法。

3.4 行列式的应用

3.4.1 历史引入：克拉默法则的诞生与争议

行列式的概念从诞生之初，就与求解线性方程组的命运紧密相连。在数学家们告别了逐一消元的繁琐操作后，他们渴望找到一个普适的、优美的“公式”，能够直接给出任意 n 元线性方程组的解。这一历史性的突破，最终以“克拉默法则”(Cramer's Rule) 的形式载入史册，但它的诞生与命名，却伴随着一段有趣的学术争议。

历史侧记

克拉默法则的诞生

1750 年，瑞士数学家加布里埃尔·克拉默 (Gabriel Cramer) 在其著作《线性分析导论》中，系统地介绍了一种利用行列式求解线性方程组的方法。这个方法非常巧妙：对于一个有 n 个方程和 n 个未知数的线性方程组，只要其系数行列式不为零，那么每个未知数的值都可以表示为两个行列式的商。

这个公式的出现，在理论上是一个巨大的进步。它首次将线性方程组的解与行列式这个代数对象完全、清晰地联系在一起，提供了一个“一劳永逸”的解题范式。由于克拉默清晰的阐述和通用的表达形式，这个法则很快便以他的名字流传开来。

历史的争议：谁是第一人？

尽管这个法则以克拉默命名，但他并非最早发现它的人。

- **麦克劳林 (Colin Maclaurin)**: 苏格兰数学家麦克劳林早在 1729 年就已经在他的著作中阐述了求解二元和三元线性方程组的类似法则，其思想与克拉默法则完全一致。然而，他的著作直到 1748 年（他去世后）才出版，且其符号表示不如克拉默通用，因此未能产生广泛影响。
- **莱布尼茨 (Gottfried Leibniz)**: 如果追溯得更早，行列式的先驱莱布尼茨在 1693 年与洛必达的通信中，已经勾勒出了利用系数表达式（即行列式）来判定方程组解的存在性的思想，这可以说是克拉默法则最原始的萌芽。但他的研究成果并未公开发表。

在科学史上，一项发现往往以其最清晰、最系统的阐述者或最有效的传播者的名字命名，而非最早的发现者。克拉默法则便是一个典型的例子。

现代争议：理论价值 vs. 计算效率

除了历史上的归属争议，克拉默法则在现代数学和计算领域也面临着另一场“争议”——关于其实用价值的讨论。

理论上的优雅: 克拉默法则无疑是优美的。它为线性方程组的解提供了一个简洁的闭合表达式，在理论分析和数学证明中（例如，证明解的存在性和唯一性、解对系数的依赖关系等）非常有用。

计算上的低效: 然而，从计算的角度来看，克拉默法则是一个“灾难”。求解一个 n 阶线性方程组，需要计算 $n+1$ 个 n 阶行列式。我们已经知道，使用拉普拉斯展开法计算一个 n 阶行列式的时间复杂度是 $O(n!)$ 。这意味着，对于一个稍大一点的系统（例如 10×10 ），使用克拉默法则的计算量将是天文数字。相比之下，我们将在后续章节学习的高斯消元法，其时间复杂度仅为 $O(n^3)$ ，效率天差地别。因此，克拉默法则在今天更多地被看作一个重要的理论工具，而非一个实用的计算算法。它完美地展示了行列式在线性代数理论中的核心地位，但在解决实际问题时，我们通常会选择更高效率的数值方法。

3.4.2 核心定理：克拉默法则

经过上一节的介绍，我们了解了克拉默法则的历史背景及其在理论与实践中的不同角色。现在，我们来给出它的严格数学表述。克拉默法则以一种极为优美的形式，将线性方程组的解与行列式紧密地联系在一起。

定理 3.10 (克拉默法则 (Cramer's Rule))

对于一个包含 n 个方程和 n 个未知数 $(x_1, x_2, \dots, x_n)$ 的线性方程组：

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2 \\ \vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n = b_n \end{cases}$$

记其系数矩阵为 A ，常数项向量为 b 。令 $D = \det(A)$ 为该方程组的系数行列式。

如果 $D \neq 0$ ，则该方程组有且仅有唯一解，其解可以表示为：

$$x_1 = \frac{D_1}{D}, \quad x_2 = \frac{D_2}{D}, \quad \dots, \quad x_n = \frac{D_n}{D}$$

其中， D_j ($j = 1, 2, \dots, n$) 是将系数行列式 D 中的第 j 列元素，替换为方程组右端的常数项 $b_1, b_2, \dots, b_n$ 后得到的 n 阶行列式。即：

$$D_j = \begin{vmatrix} a_{11} & \cdots & a_{1,j-1} & b_1 & a_{1,j+1} & \cdots & a_{1n} \\ a_{21} & \cdots & a_{2,j-1} & b_2 & a_{2,j+1} & \cdots & a_{2n} \\ \vdots & & \vdots & \vdots & \vdots & & \vdots \\ a_{n1} & \cdots & a_{n,j-1} & b_n & a_{n,j+1} & \cdots & a_{nn} \end{vmatrix}$$



解题步骤

使用克拉默法则求解线性方程组的步骤清晰明了：

1. **计算系数行列式 D** : 写出方程组的系数矩阵 A , 并计算其行列式 $D = \det(A)$ 。
2. **判断解的性质**:
 - 若 $D \neq 0$, 则方程组有唯一解, 可继续下一步。
 - 若 $D = 0$, 则克拉默法则不适用。此时方程组可能无解, 也可能有无穷多解。
3. **计算 D_j** : 依次将 D 的第 j 列替换为常数项向量, 得到 n 个新的行列式 $D_1, D_2, \dots, D_n$, 并计算它们的值。
4. **求得解**: 根据公式 $x_j = \frac{D_j}{D}$ 计算出每个未知数的值。

例题 3.27 使用克拉默法则求解三元线性方程组 求解方程组：

$$\begin{cases} x_1 + 2x_2 + 3x_3 = 14 \\ 2x_1 + x_2 + 2x_3 = 10 \\ 3x_1 + 4x_2 + x_3 = 20 \end{cases}$$

解：

1. **计算系数行列式 D** :

$$D = \begin{vmatrix} 1 & 2 & 3 \\ 2 & 1 & 2 \\ 3 & 4 & 1 \end{vmatrix} = 1(1-8) - 2(2-6) + 3(8-3) = -7 + 8 + 15 = 16$$

因为 $D = 16 \neq 0$, 所以方程组有唯一解。

2. **计算 D_1, D_2, D_3** :

$$D_1 = \begin{vmatrix} 14 & 2 & 3 \\ 10 & 1 & 2 \\ 20 & 4 & 1 \end{vmatrix} = 14(1-8) - 2(10-40) + 3(40-20) = -98 + 60 + 60 = 22$$

$$D_2 = \begin{vmatrix} 1 & 14 & 3 \\ 2 & 10 & 2 \\ 3 & 20 & 1 \end{vmatrix} = 1(10-40) - 14(2-6) + 3(40-30) = -30 + 56 + 30 = 56$$

$$D_3 = \begin{vmatrix} 1 & 2 & 14 \\ 2 & 1 & 10 \\ 3 & 4 & 20 \end{vmatrix} = 1(20-40) - 2(40-30) + 14(8-3) = -20 - 20 + 70 = 30$$

3. **求得解**:

$$x_1 = \frac{D_1}{D} = \frac{22}{16} = \frac{11}{8}$$

$$x_2 = \frac{D_2}{D} = \frac{56}{16} = \frac{7}{2}$$

$$x_3 = \frac{D_3}{D} = \frac{30}{16} = \frac{15}{8}$$

定义 3.10 (伴随矩阵 (adjoint Matrix))

设 $A = (a_{ij})_{n \times n}$, 其代数余子式矩阵为

$$C = (A_{ij})_{n \times n}, \quad A_{ij} = (-1)^{i+j} M_{ij}.$$

把 C 转置一次, 就得到 A 的伴随矩阵:

$$C^T = (A_{ji})_{n \times n}.$$

**定理 3.11 (伴随矩阵的核心性质)**

对任意方阵 A , 有

$$A \cdot \text{adj}(A) = \text{adj}(A) \cdot A = \det(A) I_n.$$



证明 [简要证明]

1. 当 $i = j$ (对角元) $(A \cdot \text{adj}(A))_{ii} = \sum_{k=1}^n a_{ik} A_{ik}$ 这正是按第 i 行展开 $\det A$ 的定义, 故

$$(A \cdot \text{adj}(A))_{ii} = \det A.$$

2. 当 $i \neq j$ (非对角元) $(A \cdot \text{adj}(A))_{ij} = \sum_{k=1}^n a_{ik} A_{jk}$ 相当于把 A 的第 j 行替换成第 i 行后, 再按第 j 行展开。此时矩阵有两行相同, 行列式为 0, 故

$$(A \cdot \text{adj}(A))_{ij} = 0.$$

综合两点, 矩阵 $A \cdot \text{adj}(A)$ 的对角线全为 $\det A$, 非对角线全为 0, 即

$$A \cdot \text{adj}(A) = \det(A) I_n.$$

同理可证 $\text{adj}(A) \cdot A$ 相同。

克拉默法则的证明思路

克拉默法则的证明巧妙地利用了行列式按列展开的性质和伴随矩阵的性质。

1. 将方程组写成矩阵形式 $A\mathbf{x} = \mathbf{b}$ 。
2. 用 A 的伴随矩阵 $\text{adj}(A)$ 左乘等式两边, 得到 $\text{adj}(A)A\mathbf{x} = \text{adj}(A)\mathbf{b}$ 。
3. 根据伴随矩阵的核心性质, 我们有 $\text{adj}(A)A = \det(A)I$, 其中 I 是单位矩阵。
4. 因此, 等式变为 $\det(A)I\mathbf{x} = \text{adj}(A)\mathbf{b}$, 即 $D \cdot \mathbf{x} = \text{adj}(A)\mathbf{b}$ 。
5. 观察这个矩阵等式的第 j 个分量, 我们得到 $D \cdot x_j = (\text{adj}(A)\mathbf{b})_j$ 。
6. $(\text{adj}(A)\mathbf{b})_j$ 恰好是 $\text{adj}(A)$ 的第 j 行与向量 $\mathbf{b}$ 的点积, 这等价于将行列式 D 的第 j 列换成 $\mathbf{b}$ 后按该列展开的结果, 即 D_j 。
7. 于是我们证明了 $D \cdot x_j = D_j$, 当 $D \neq 0$ 时, 即可得到 $x_j = \frac{D_j}{D}$ 。

这个证明完美地体现了行列式、伴随矩阵和线性方程组解之间的深刻代数联系。

3.4.3 代码实现：用代码求解线性方程组

理论学习之后，我们将亲手实现用代码求解线性方程组。这部分内容将完美地诠释“克拉默法则的争议”：我们将分别实现理论上优雅的克拉默法则，以及实践中高效的库函数解法，让您直观地感受到理论与实践之间的差异。

我们将求解以下线性方程组：

$$\begin{cases} x_1 + 2x_2 + 3x_3 = 6 \\ 3x_1 + 0x_2 + x_3 = 4 \\ x_1 + 2x_2 + x_3 = 4 \end{cases} \implies \begin{bmatrix} 1 & 2 & 3 \\ 3 & 0 & 1 \\ 1 & 2 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 6 \\ 4 \\ 4 \end{bmatrix}$$

其中， $A = \begin{bmatrix} 1 & 2 & 3 \\ 3 & 0 & 1 \\ 1 & 2 & 1 \end{bmatrix}$ ， $b = \begin{bmatrix} 6 \\ 4 \\ 4 \end{bmatrix}$ 。

代码视角

[解法一：克拉默法则的编程实现] 根据克拉默法则，解 x_i 等于一个特殊构造的矩阵 A_i 的行列式除以原系数矩阵 A 的行列式，即 $x_i = \frac{\det(A_i)}{\det(A)}$ 。其中， A_i 是将矩阵 A 的第 i 列替换为向量 b 后得到的新矩阵。

下面的代码完整实现了这个逻辑。

```
import torch

def solve_cramer(A, b):
    """ 使用克拉默法则求解线性方程组 Ax = b """
    # 确保输入是浮点数张量
    A = A.to(dtype=torch.float32)
    b = b.to(dtype=torch.float32)

    # 首先计算系数矩阵A的行列式
    det_A = torch.linalg.det(A)
    if torch.isclose(det_A, torch.tensor(0.0)):
        print("系数矩阵行列式为0，方程组无唯一解。")
        return None

    n = A.shape[0]
```



```

solutions = torch.zeros(n)

# 循环计算每个未知数
for i in range(n):
    # 构造矩阵 Ai
    Ai = A.clone()
    Ai[:, i] = b # 将第 i 列替换为向量 b

    # 计算 det(Ai) 并求解 xi
    det_Ai = torch.linalg.det(Ai)
    solutions[i] = det_Ai / det_A

return solutions

# --- 测试 ---
A = torch.tensor([[1, 2, 3],
                  [3, 0, 1],
                  [1, 2, 1]])

b = torch.tensor([6, 4, 4])

solution_cramer = solve_cramer(A, b)
print(f"使用克拉默法则求解结果 x = {solution_cramer}")

```

代码视角

[解法二：使用 PyTorch 内置求解器] 在实际应用中，我们几乎从不手动实现克拉默法则来求解线性方程组。所有主流的科学计算库都提供了高度优化的内置函数。在 PyTorch 中，这个函数是 ‘torch.linalg.solve()’。它基于数值稳定性更好、计算效率更高的算法（如 LU 分解），是解决此类问题的标准方法。

```

# --- 测试 ---
# 同样使用上面的矩阵 A 和向量 b

# 直接调用 torch.linalg.solve()
# 注意：需要将 b 变形为列向量形式 (n, 1)
solution_solver = torch.linalg.solve(A, b.view(-1, 1))

```

```
# .flatten() 是为了让输出格式与方法一保持一致
print(f"使用内置求解器求解结果 x = {solution_solver.flatten()}")
```

克拉默法则的“现代争议”：理论 vs. 实践

通过上面的代码实现，我们可以清晰地看到克拉默法则的“争议”所在：

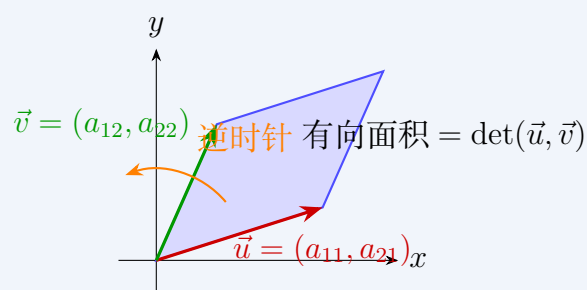
- **理论实现的价值：**solve-cramer 函数让我们将抽象的数学公式转化为了具体的代码逻辑，这是一个非常有价值的学习过程，它加深了我们对行列式应用的理解。
- **实践中的性能陷阱：**克拉默法则的计算量是巨大的。求解一个 $n \times n$ 的系统需要计算 $n + 1$ 个 n 阶行列式。使用高斯消元法计算行列式的时间复杂度是 $O(n^3)$ ，那么克拉默法则的总复杂度将达到 $O(n^4)$ ，远高于内置求解器 $O(n^3)$ 的复杂度。如果行列式用递归法计算，复杂度更是高达 $O((n + 1)!)$ 。

结论：尽管克拉默法则在理论上极为重要，但在代码实现和解决实际问题时，我们应当始终优先使用像 ‘torch.linalg.solve()’ 这样经过高度优化的库函数。

本章习题精解

几何视角

二阶行列式具有深刻的几何意义。其绝对值等于由矩阵的两个列向量（或行向量）在 $\mathbb{R}^2$ 中所张成的平行四边形的面积。若行列式的值为正，则表示两个向量满足右手定则（从第一个向量到第二个向量为逆时针旋转）；若为负，则表示左手定则（顺时针旋转）；若为零，则两向量共线（平行四边形退化为一条线），这意味着线性方程组可能无解或有无穷多解。行列式从诞生之初就同时具备了代数和几何的双重身份。



3.4.4 行列式的算法

要点提炼

行列式计算策略的三层境界

- **观察结构**：识别特殊模式（稀疏、三角、对称、循环等）
- **选择算法**：在直接展开、递归降阶、高斯消元、分解法间权衡
- **验证优化**：数值检验、复杂度分析与几何解释

行列式的计算既是线性代数的基础技能，也是衡量算法效率的经典问题。从 19 世纪的手工计算到现代的计算机算法，行列式的计算方法经历了从数学技巧到系统算法的发展。本节将通过几个典型案例，展示不同情境下的算法选择与优化策略。

3.4.5 案例一： $(n+2)$ 阶特殊结构的行列式

考虑如下具有特殊结构的行列式：

$$D_{n+2} = \begin{vmatrix} a & 0 & 0 & \cdots & 0 & 1 \\ 0 & a & 0 & \cdots & 0 & 1 \\ 0 & 0 & a & \cdots & 0 & 1 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & a & 1 \\ 1 & 0 & 0 & \cdots & 0 & a \end{vmatrix}.$$

定理 3.12

对任意实数 a 与整数 $n \geq 1$ ，有

$$D_{n+2} = a^n(a^2 - 1).$$



证明 [证法一：按第一行展开（递归降阶）] 按第一行展开：

$$D_{n+2} = a \cdot M_{11} + (-1)^{1+n+2} \cdot 1 \cdot M_{1,n+2}.$$

其中 M_{11} 是右下角的 $(n+1)$ 阶下三角行列式， $M_{11} = a^{n+1}$ ； $M_{1,n+2}$ 是删去第一行和最后一列后的行列式：

$$M_{1,n+2} = \begin{vmatrix} 0 & a & 0 & \cdots & 0 \\ 0 & 0 & a & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & a \\ 1 & 0 & 0 & \cdots & 0 \end{vmatrix} = (-1)^n a^n.$$

代入得：

$$D_{n+2} = a \cdot a^{n+1} + (-1)^{n+3} \cdot (-1)^n a^n = a^{n+2} - a^n = a^n(a^2 - 1).$$

证明 [证法二：Laplace 展开（分块处理）] 选择第 1 行和第 $(n+2)$ 行，以及第 1 列和第 $(n+2)$ 列进行 Laplace 展开。对应的 2 阶子式为：

$$\begin{vmatrix} a & 1 \\ 1 & a \end{vmatrix} = a^2 - 1,$$

其余子式为 a^n ，符号因子为 $(-1)^{(1+(n+2))+(1+(n+2))} = 1$ ，故

$$D_{n+2} = (a^2 - 1) \cdot a^n.$$

证明 [证法三：列变换（创造零元素）] 将所有列加到第 1 列： $c_1 \leftarrow \sum_{j=1}^{n+2} c_j$ ，则第 1 列元素全变为 $a+1$ 。提取公因子 $(a+1)$ 后，对第 2 至第 $n+2$ 行执行行变换 $r_i \leftarrow r_i - r_1$ ，将矩阵化为上三角阵，主对角线元素为 $1, a, \dots, a, a-1$ ，其乘积为 $a^n(a-1)$ 。故

$$D_{n+2} = (a+1) \cdot a^n(a-1) = a^n(a^2 - 1).$$

要点提炼

算法选择建议：

- **递归展开**：适合结构化稀疏矩阵，但时间复杂度为 $O(n!)$ ，仅适用于中小规模。
- **Laplace 展开**：当能选择少量行列使剩余部分成为分块对角时高效。
- **初等变换**：通用性强，时间复杂度为 $O(n^3)$ ，适合数值计算，尤其是大型矩阵。

历史侧记

行列式的算法发展经历了多个阶段。19 世纪，数学家们主要使用拉普拉斯展开等组合方法。20 世纪中期，随着计算机的出现，基于高斯消元的高效数值算法（复杂度 $O(n^3)$ ）成为主流。对于整数或精确计算，Bareiss 算法（1968）能在避免分数的情况下计算行列式。近年来，随机算法和并行算法也在发展中。

代码视角

[Python/NumPy 验证]

```
import numpy as np

def special_determinant(n, a):
    """构造并计算n+2阶特殊行列式"""
    # 构造矩阵
    mat = np.zeros((n+2, n+2))
    np.fill_diagonal(mat, a)      # 主对角线元素为a
    mat[:-1, -1] = 1             # 最后一列（除最后一行）为1
    mat[-1, 0] = 1               # 最后一行第一列为1
```

```

# 计算行列式
det_value = np.linalg.det(mat)
# 理论值
theoretical_value = (a**n) * (a**2 - 1)
return det_value, theoretical_value

# 测试示例: n=3, a=2
n_val = 3
a_val = 2
det_val, theory_val = special_determinant(n_val, a_val)
print(f"计算值: {det_val:.0f}, 理论值: {theory_val:.0f}")
# 输出: 计算值: 24, 理论值: 24

```

3.4.6 案例二：一类特殊递推行列式的计算

递推关系是计算行列式的强大工具，特别适用于具有特定模式的矩阵。

考虑如下 n 阶行列式：

$$D_n = \begin{vmatrix} x & 0 & 0 & \cdots & 0 & a_n \\ -1 & x & 0 & \cdots & 0 & a_{n-1} \\ 0 & -1 & x & \cdots & 0 & a_{n-2} \\ \vdots & \vdots & \ddots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \ddots & x & a_2 \\ 0 & 0 & 0 & \cdots & -1 & x + a_1 \end{vmatrix}, \quad (n \geq 1), \quad \text{我们定义 } D_0 = 1.$$

定理 3.13

该行列式满足递推关系：

$$D_n = xD_{n-1} + a_n.$$



证明 我们对行列式 D_n 按第一行进行展开。第一行只有两个非零元素，分别是第一列的 x 和第 n 列的 a_n 。

根据行列式展开定理：

$$D_n = (-1)^{1+1} \cdot x \cdot M_{11} + (-1)^{1+n} \cdot a_n \cdot M_{1n}.$$

其中 M_{11} 和 M_{1n} 分别是元素 $(1, 1)$ 和 $(1, n)$ 的余子式。

1. 计算 M_{11} : 删去第一行第一列后, 得到的 $(n-1)$ 阶矩阵为:

$$M_{11} = \begin{vmatrix} x & 0 & \cdots & a_{n-1} \\ -1 & x & \cdots & a_{n-2} \\ \vdots & \ddots & \vdots & \vdots \\ 0 & \cdots & x & a_2 \\ 0 & \cdots & -1 & x + a_1 \end{vmatrix}_{(n-1) \times (n-1)}$$

观察这个行列式的结构, 它与 D_{n-1} 的定义完全一致。因此, $M_{11} = D_{n-1}$ 。

2. 计算 M_{1n} : 删去第一行第 n 列后, 得到的 $(n-1)$ 阶矩阵为:

$$M_{1n} = \begin{vmatrix} -1 & x & 0 & \cdots & 0 \\ 0 & -1 & x & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & 0 & \ddots & x \\ 0 & 0 & 0 & \cdots & -1 \end{vmatrix}_{(n-1) \times (n-1)}$$

这是一个上三角矩阵。其行列式的值等于主对角线上所有元素的乘积。主对角线上的元素全为 -1 。因此, $M_{1n} = (-1)^{n-1}$ 。

3. 合并结果: 将 M_{11} 和 M_{1n} 的值代入展开式:

$$D_n = x \cdot D_{n-1} + (-1)^{1+n} \cdot a_n \cdot (-1)^{n-1}$$

$$D_n = xD_{n-1} + (-1)^{2n} \cdot a_n$$

因为 $2n$ 是偶数, 所以 $(-1)^{2n} = 1$ 。最终得到递推关系:

$$D_n = xD_{n-1} + a_n.$$

定理得证。

推论 3.1 (显式表达式)

该行列式的显式 (通项) 表达式为:

$$D_n = x^n + a_1x^{n-1} + a_2x^{n-2} + \cdots + a_{n-1}x + a_n = \sum_{k=0}^n a_k x^{n-k},$$

其中我们定义 $a_0 = 1$ 。



证明 我们将使用数学归纳法来证明这个显式表达式。

1. 基础情况 (Base Case): 当 $n = 1$ 时, 根据行列式定义:

$$D_1 = \begin{vmatrix} x + a_1 \end{vmatrix} = x + a_1.$$

根据推论的公式:

$$D_1 = \sum_{k=0}^1 a_k x^{1-k} = a_0 x^1 + a_1 x^0 = 1 \cdot x + a_1 = x + a_1.$$

两者相符, 基础情况成立。

2. 归纳假设 (Inductive Hypothesis): 假设当 $n = m$ 时结论成立, 即:

$$D_m = x^m + a_1x^{m-1} + a_2x^{m-2} + \cdots + a_m = \sum_{k=0}^m a_k x^{m-k}.$$

3. 归纳步骤 (Inductive Step): 我们需要证明当 $n = m + 1$ 时结论也成立。根据我们已经证明的定理 3.13, 我们有:

$$D_{m+1} = xD_m + a_{m+1}.$$

现在, 我们将归纳假设代入上式:

$$\begin{aligned} D_{m+1} &= x \left(\sum_{k=0}^m a_k x^{m-k} \right) + a_{m+1} \\ D_{m+1} &= \sum_{k=0}^m a_k x^{m-k+1} + a_{m+1} \end{aligned}$$

展开求和项:

$$D_{m+1} = (a_0x^{m+1} + a_1x^m + a_2x^{m-1} + \cdots + a_mx^1) + a_{m+1}$$

将 a_{m+1} 写入求和式中, 并回顾 $a_0 = 1$:

$$\begin{aligned} D_{m+1} &= 1 \cdot x^{m+1} + a_1x^m + a_2x^{m-1} + \cdots + a_mx + a_{m+1} \\ D_{m+1} &= \sum_{k=0}^{m+1} a_k x^{(m+1)-k}. \end{aligned}$$

这与当 $n = m + 1$ 时推论公式的形式完全一致。

根据数学归纳法原理, 该显式表达式对所有 $n \geq 1$ 成立。

例题 3.28 当 $n = 3$ 时: 根据显式表达式:

$$D_3 = a_0x^3 + a_1x^2 + a_2x + a_3 = x^3 + a_1x^2 + a_2x + a_3.$$

我们也可以使用递推关系来验证:

$$D_3 = xD_2 + a_3$$

而 $D_2 = xD_1 + a_2 = x(x + a_1) + a_2 = x^2 + a_1x + a_2$ 。代入 D_3 的递推式:

$$D_3 = x(x^2 + a_1x + a_2) + a_3 = x^3 + a_1x^2 + a_2x + a_3.$$

两种计算方法结果一致。

要点提炼

递推法的复杂度分析: 递推关系将 n 阶行列式的计算转化为 $n - 1$ 阶问题, 时间复杂度为 $O(n)$, 远优于直接展开的 $O(n!)$ 。这种方法特别适用于三对角矩阵或具有类似重复结构的矩阵。

3.4.7 案例三: 行和相等型行列式的高效算法

行和相等型行列式具有特殊的结构, 可以通过巧妙的变换简化计算。

问题探讨

计算行列式：

$$D_n = \begin{vmatrix} a_1 + b & a_2 & a_3 & \cdots & a_n \\ a_1 & a_2 + b & a_3 & \cdots & a_n \\ a_1 & a_2 & a_3 + b & \cdots & a_n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_1 & a_2 & a_3 & \cdots & a_n + b \end{vmatrix}, \quad (b \neq 0).$$

定理 3.14

$$D_n = b^{n-1} \left(\sum_{i=1}^n a_i + b \right).$$



证明 [证法一：行变换法] 将所有行加到第一行：

$$r_1 \leftarrow \sum_{i=1}^n r_i,$$

则第一行变为 $(\sum_{i=1}^n a_i + b, \sum_{i=1}^n a_i + b, \dots, \sum_{i=1}^n a_i + b)$ 。提取公因子后，用第一行消去其他行的对应元素，得到上三角矩阵，主对角线元素为 $1, b, b, \dots, b$ ，故行列式为 $b^{n-1} (\sum_{i=1}^n a_i + b)$ 。

证明 [证法二：加边法] 将行列式升阶为 $n+1$ 阶：

$$D_n = \begin{vmatrix} 1 & a_1 & a_2 & \cdots & a_n \\ 0 & a_1 + b & a_2 & \cdots & a_n \\ 0 & a_1 & a_2 + b & \cdots & a_n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & a_1 & a_2 & \cdots & a_n + b \end{vmatrix}.$$

通过初等变换，可以化为分块上三角矩阵，从而得证。

证明 [证法三：特征值法] 注意到该矩阵可以写成：

$$A = bI_n + \mathbf{a}\mathbf{1}^T,$$

其中 $\mathbf{a} = (a_1, a_2, \dots, a_n)^T$, $\mathbf{1} = (1, 1, \dots, 1)^T$ 。根据矩阵行列式引理：

$$\det(A) = b^{n-1}(b + \mathbf{1}^T \mathbf{a}) = b^{n-1} \left(b + \sum_{i=1}^n a_i \right).$$

要点提炼

算法比较：

- **行变换法：**直观易懂，时间复杂度 $O(n^3)$ ，适合手工计算。
- **加边法：**技巧性强，能处理更一般的行和相等型问题。

- **特征值法**：基于矩阵分解，理论深刻，计算复杂度低 ($O(n)$)，适合大规模数值计算。

代码视角

[行和相等型行列式的数值计算]

```
import numpy as np

def row_sum_equal_determinant(a_list, b):
    """计算行和相等型行列式"""
    n = len(a_list)
    # 构造矩阵
    A = np.array(a_list).reshape(1, n) # 行向量
    matrix = b * np.eye(n) + np.ones((n, 1)).dot(A)
    det_value = np.linalg.det(matrix)
    # 理论值
    theoretical_value = (b ** (n-1)) * (np.sum(a_list) + b)
    return det_value, theoretical_value

# 测试示例
a_vals = [1, 2, 3, 4]
b_val = 2
det_val, theory_val = row_sum_equal_determinant(a_vals, b_val)
print(f"计算值: {det_val:.2f}, 理论值: {theory_val:.2f}")
# 输出: 计算值: 80.00, 理论值: 80.00
```

3.4.8 案例四：三对角与常数填充型行列式

这类行列式在数值计算和物理建模中常见，有特定的高效算法。

考虑行列式：

$$D_n = \begin{vmatrix} a & c & c & \cdots & c \\ b & a & c & \cdots & c \\ b & b & a & \cdots & c \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ b & b & b & \cdots & a \end{vmatrix}.$$

命题 3.2 (递推关系)

该行列式满足递推关系：

$$D_n = (a - c)D_{n-1} + c(a - b)^{n-1}.$$

若 $a = c$ ，则 $D_n = c(a - b)^{n-1}$ 。



证明 通过行变换 $r_i \leftarrow r_i - r_{i-1}$ ($i = n, n-1, \dots, 2$)，将矩阵化为几乎上三角的形式，然后按第一列展开，即可得到递推关系。

注[特殊情形与闭式解]

- **特例** $b = c$ ：此时 $D_n = [a + (n-1)b](a - b)^{n-1}$ 。
- **特例** $a = c$ ： $D_n = c(a - b)^{n-1}$ 。
- **一般情形**：可通过求解递推关系的特征方程得到闭式解。

要点提炼

三对角矩阵的专用算法：对于严格的三对角矩阵（仅主对角线和相邻对角线非零），存在专用的 $O(n)$ 算法（Thomas 算法），比一般的高斯消元法更高效。

代码视角

[常数填充型行列式的计算]

```
import numpy as np

def constant_fill_determinant(n, a, b, c):
    """计算常数填充型行列式"""
    # 构造矩阵
    matrix = np.full((n, n), c, dtype=float)
    for i in range(n):
        for j in range(n):
            if i > j:
                matrix[i, j] = b
            elif i == j:
                matrix[i, j] = a
    # 计算行列式
    det_value = np.linalg.det(matrix)
    return det_value

# 测试特殊情形：b=c
n_val = 4
a_val, b_val, c_val = 5, 2, 2
```

```
det_val = constant_fill_determinant(n_val, a_val, b_val, c_val)
theoretical_val = (a_val + (n_val-1)*b_val) * (a_val - b_val)**(n_val-1)
print(f"计算值: {det_val:.2f}, 理论值: {theoretical_val:.2f}")
# 输出: 计算值: 189.00, 理论值: 189.00
```

3.4.9 行列式算法的综合比较与选择策略

行列式计算有多种算法，选择取决于矩阵结构和计算需求。

表 3.1: 行列式算法比较

算法	时间复杂度	适用场景	稳定性
拉普拉斯展开	$O(n!)$	小型矩阵 ($n \leq 6$), 理论分析	精确
高斯消元法	$O(n^3)$	一般中型矩阵, 数值计算	依赖主元选择
分块矩阵法	$O(n^3)$	分块矩阵, 稀疏矩阵	良好
递推法	$O(n)$	三对角、循环等特殊结构	精确
特征值法	$O(n^3)$	对称矩阵, 理论分析	良好
Bareiss 算法	$O(n^3)$	整数矩阵, 精确计算	精确

要点提炼

算法选择指南

- 1. 小规模矩阵 ($n \leq 6$): 优先考虑拉普拉斯展开，易于实现且精确。
- 2. 中型稠密矩阵 ($6 < n \leq 1000$): 使用高斯消元法（部分主元），数值稳定。
- 3. 特殊结构矩阵: 根据结构选择专用算法（递推、分块等）。
- 4. 整数或精确计算: 考虑 Bareiss 算法或符号计算。
- 5. 超大规模矩阵 ($n > 10000$): 使用迭代法或随机算法，或考虑稀疏性。

历史侧记

行列式算法的历史反映了计算数学的发展。从 Cramer 的展开法则到 Gauss 的消元法，再到 Bareiss 的整数算法，每一步进步都解决了特定问题。现代计算机代数系统（如 Mathematica、Maple）通常结合多种算法，根据输入矩阵自动选择最佳方法。

代码视角

[NumPy 中的行列式计算实现] NumPy 使用 LU 分解计算行列式，时间复杂度为 $O(n^3)$ ，基于 LAPACK 库，支持多种数值类型：

```
import numpy as np

# 随机矩阵行列式计算
```

```

n = 1000
A = np.random.rand(n, n)
det_A = np.linalg.det(A) # 使用LU分解

# 对于整数矩阵，可使用精确计算（但较慢）
from scipy import linalg
A_int = np.random.randint(0, 10, (n, n))
det_A_int = linalg.det(A_int) # 仍使用浮点运算，可能溢出

# 对于非常大或病态矩阵，考虑对数行列式或迭代法
log_det = np.linalg.slogdet(A) # 避免数值溢出

```

3.4.10 练习与思考

 **练习 3.1 变换的综合运用** 计算行列式：

$$D_n = \begin{vmatrix} 1 & 2 & 3 & \cdots & n-1 & n \\ 1 & 0 & 3 & \cdots & n-1 & n \\ 1 & 2 & 0 & \cdots & n-1 & n \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 1 & 2 & 3 & \cdots & 0 & n \\ 1 & 2 & 3 & \cdots & n-1 & 0 \end{vmatrix}.$$

提示：使用行变换 $r_i \leftarrow r_i - r_1$ ($i = 2, \dots, n$)。结果为 $(-1)^{n-1}n!$ 。

证明 Step 1. 对第 i 行 ($i \geq 2$) 做 $r_i \leftarrow r_i - r_1$ 得

$$D_n = \begin{vmatrix} 1 & 2 & 3 & \cdots & n-1 & n \\ 0 & -2 & 0 & \cdots & 0 & 0 \\ 0 & 0 & -3 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & -(n-1) & 0 \\ 0 & 0 & 0 & \cdots & 0 & -n \end{vmatrix}.$$

Step 2. 得到上三角行列式，故结果为： $(-1)^{n-1}n!$ 。

 **练习 3.2 对称矩阵的行列式** 设 $\mathbf{a} = (a_1, a_2, \dots, a_n)^T$ 为一个列向量。计算如下 $n \times n$ 实对

称矩阵的行列式 D_n , 该矩阵可表示为 $xI + \mathbf{a}\mathbf{a}^T$:

$$D_n = \begin{vmatrix} x + a_1^2 & a_1 a_2 & a_1 a_3 & \cdots & a_1 a_n \\ a_2 a_1 & x + a_2^2 & a_2 a_3 & \cdots & a_2 a_n \\ a_3 a_1 & a_3 a_2 & x + a_3^2 & \cdots & a_3 a_n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_n a_1 & a_n a_2 & a_n a_3 & \cdots & x + a_n^2 \end{vmatrix}.$$

做法: 使用加边法, 通过构造一个巧妙的 $n+1$ 阶行列式来求解。

答案: $D_n = x^{n-1} (x + \sum_{i=1}^n a_i^2)$.

证明 [详细步骤] 我们采用加边法来计算。考虑如下的 $n+1$ 阶行列式 P :

$$P = \begin{vmatrix} 1 & -a_1 & -a_2 & \cdots & -a_n \\ a_1 & x & 0 & \cdots & 0 \\ a_2 & 0 & x & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_n & 0 & 0 & \cdots & x \end{vmatrix}.$$

第一步: 利用初等行变换。对行列式 P 的第 i 行 ($i = 2, \dots, n+1$) 执行行变换 $r_i \leftarrow r_i - a_{i-1}r_1$ 。经过这些行变换, 行列式 P 变为一个上三角分块矩阵:

$$P = \begin{vmatrix} 1 & -a_1 & -a_2 & \cdots & -a_n \\ 0 & x + a_1^2 & a_1 a_2 & \cdots & a_1 a_n \\ 0 & a_2 a_1 & x + a_2^2 & \cdots & a_2 a_n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & a_n a_1 & a_n a_2 & \cdots & x + a_n^2 \end{vmatrix}.$$

其行列式等于对角线上块的行列式之积:

$$P = 1 \cdot \begin{vmatrix} x + a_1^2 & a_1 a_2 & \cdots & a_1 a_n \\ a_2 a_1 & x + a_2^2 & \cdots & a_2 a_n \\ \vdots & \vdots & \ddots & \vdots \\ a_n a_1 & a_n a_2 & \cdots & x + a_n^2 \end{vmatrix} = 1 \cdot D_n = D_n.$$

第二步: 按第一行展开。按第一行展开, 行列式 P 等于:

$$P = 1 \cdot C_{11} + \sum_{j=1}^n (-a_j) C_{1,j+1} = 1 \cdot x^n + \sum_{j=1}^n (-a_j) (-1)^{1+(j+1)} M_{1,j+1}$$

其中 C_{ij} 和 M_{ij} 分别是代数余子式和余子式。我们来计算余子式 $M_{1,j+1}$ 。这是一个 $n \times n$ 的行列式, 其第一列为 $(a_1, \dots, a_n)^T$, 其余列为对角矩阵 xI_n 去掉第 j 列后所构成的。在 $M_{1,j+1}$ 矩阵中, 第 j 行除了第一个元素 a_j 外, 其余元素均为 0。因此, 我们可以按第 j 行展开 $M_{1,j+1}$:

$$M_{1,j+1} = a_j \cdot (-1)^{j+1} \cdot \det(M')$$

其中 M' 是一个 $(n-1) \times (n-1)$ 的对角矩阵, 其对角线元素均为 x 。所以 $\det(M') = x^{n-1}$ 。

因此, $M_{1,j+1} = a_j(-1)^{j+1}x^{n-1}$ 。代入 P 的表达式:

$$\begin{aligned} P &= x^n + \sum_{j=1}^n (-a_j)(-1)^{j+2} (a_j(-1)^{j+1}x^{n-1}) \\ P &= x^n + \sum_{j=1}^n (-a_j^2 x^{n-1})(-1)^{2j+3} = x^n + \sum_{j=1}^n (-a_j^2 x^{n-1})(-1) \\ P &= x^n + x^{n-1} \sum_{j=1}^n a_j^2. \end{aligned}$$

综上, 我们得到:

$$D_n = x^n + x^{n-1} \sum_{i=1}^n a_i^2.$$

提取公因子 x^{n-1} , 得到最终答案:

$$D_n = x^{n-1} \left(x + \sum_{i=1}^n a_i^2 \right).$$

 **练习 3.3 循环矩阵的行列式** 计算如下 $n \times n$ 循环矩阵的行列式 D_n :

$$D_n = \begin{vmatrix} a & b & b & \cdots & b \\ b & a & b & \cdots & b \\ b & b & a & \cdots & b \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ b & b & b & \cdots & a \end{vmatrix}.$$

做法: 将所有列 (或所有行) 加到第一列 (或第一行), 提取公因子后进行化简。

答案: $D_n = [a + (n-1)b](a-b)^{n-1}$.

证明 [详细步骤]

Step 1. 将所有列加到第一列。

对 $j = 2, 3, \dots, n$, 执行列变换 $c_1 \leftarrow c_1 + c_j$ 。每一行的元素和都是 $a + (n-1)b$ 。因此, 变换后的第一列所有元素都等于 $a + (n-1)b$ 。

$$D_n = \begin{vmatrix} a + (n-1)b & b & b & \cdots & b \\ a + (n-1)b & a & b & \cdots & b \\ a + (n-1)b & b & a & \cdots & b \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a + (n-1)b & b & b & \cdots & a \end{vmatrix}.$$

Step 2. 从第一列提取公因子。

令 $S = a + (n - 1)b$ 。将 S 从第一列提出。

$$D_n = S \cdot \begin{vmatrix} 1 & b & b & \cdots & b \\ 1 & a & b & \cdots & b \\ 1 & b & a & \cdots & b \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & b & b & \cdots & a \end{vmatrix}.$$

Step 3. 利用第一列制造零元素。

为了简化行列式，我们对 $i = 2, 3, \dots, n$ 执行行变换 $r_i \leftarrow r_i - r_1$ 。这会将矩阵化为上三角形式。

$$D_n = S \cdot \begin{vmatrix} 1 & b & b & \cdots & b \\ 0 & a-b & 0 & \cdots & 0 \\ 0 & 0 & a-b & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & a-b \end{vmatrix}.$$

Step 4. 计算上三角行列式。

得到的矩阵是一个上三角矩阵。其行列式等于对角线元素的乘积。

$$D_n = S \cdot (1 \cdot (a - b) \cdot (a - b) \cdots (a - b)) = S \cdot (a - b)^{n-1}.$$

Step 5. 代入公因子 S 并汇总。

将 $S = a + (n - 1)b$ 代回，得到最终结果：

$$D_n = [a + (n - 1)b](a - b)^{n-1}.$$

3.4.11 本章小结

要点提炼

行列式算法的核心思想

- **结构决定算法**：识别矩阵结构是选择高效算法的关键。
- **化归与递归**：将复杂问题转化为简单问题，大规模问题转化为小规模问题。
- **数值稳定性**：关注算法的数值稳定性，特别是病态矩阵的处理。
- **精确与近似**：根据需求选择精确算法或数值近似算法。

行列式的计算不仅是线性代数的基础技能，也是连接数学理论与计算实践的桥梁。从手工计算到高性能计算，行列式算法的发展体现了计算数学的进步。随着机器学习、科学计算的发展，行列式在新领域（如生成模型、量子计算）中继续发挥重要作用，其算法也在不断革新。

第4章 矩阵

4.1 矩阵基础

4.1.1 历史引入：行列式的局限与矩阵的诞生

4.1.2 历史的岔路：从行列式到矩阵的飞跃

在深入探索矩阵的广阔天地之前，我们先回顾已经掌握的工具——行列式。它是一个定义在方阵上的重要函数，能判断线性方程组解的性质，也能度量线性变换对空间体积的影响。然而，当面对更复杂的线性问题时，行列式的局限性便显现出来：它将一个丰富的方阵压缩为单一数值，丢失了大量结构信息；它无法处理更普遍的非方阵情形；其运算体系也无法直接对应线性变换的复合。

为了突破这些局限，数学需要一次深刻的范式革命。这场革命的主角，是两位志趣相投、性格迥异的挚友——维多利亚时代英国最杰出的数学双星：**阿瑟·凯莱**（Arthur Cayley）与**詹姆斯·西尔维斯特**（James Sylvester）。

他们的友谊与合作长达四十年，共同奠定了矩阵理论的基石。凯莱是一位高产的数学家，同时也是一名执业长达14年的律师。他为人严谨、内敛，逻辑缜密，习惯于将复杂的思想系统化、代数化。而西尔维斯特则更像一位数学界的“诗人”，他热情奔放，才华横溢，痴迷于为新的数学概念创造富有想象力的词汇。

正是这位“诗人”西尔维斯特，在1850年首次赋予了那个矩形数表一个充满生命力的名字。

历史侧记

“矩阵”的诞生：一个充满母性光辉的词汇

西尔维斯特并非凭空创造了“矩阵”这个词。在拉丁语中，“Matrix”的本意是“子宫”或“孕育生命之物”。他之所以选择这个词，是因为他观察到，在一个大的矩形数表中，我们可以通过划掉任意的行和列，得到一个更小的方阵，并计算出它的行列式。

在他看来，这个大的矩形数表就像一个“母体”，能够“孕育”出无数个小的行列式。因此，他富有诗意地将其命名为 **Matrix**。这个名字不仅形象地描述了子式与母体的关系，也预示着这个新概念将拥有强大的衍生和创造能力。

如果说西尔维斯特为这个新生儿取了名字，那么系统地教会它“说话”和“行动”的，则是他的挚友凯莱。凯莱并未将矩阵仅仅看作一个被动的数表，他敏锐地意识到，矩阵本身可以被视为一个独立的、可进行运算的代数实体。

在1858年，凯莱发表了那篇被誉为“矩阵理论出生证明”的鸿篇巨著——《矩阵理论纪要》（A Memoir on the Theory of Matrices）。在这篇论文中，他系统地定义了矩阵

的加法、数乘，尤其是至关重要的**矩阵乘法**。他所定义的乘法规则并非随意设计的，其动机是让矩阵的乘积能够完美地对应几何空间中**线性变换的复合**。当一个变换 T_2 作用于另一个变换 T_1 的结果之上时，其总效果恰好对应于代表这两个变换的矩阵 M_2 与 M_1 的乘积。

至此，线性代数的核心飞跃完成了。矩阵不再仅仅是记录线性方程组系数的工具，它已经化身为线性变换本身在代数世界的“化身”或“代理人”。方程组 $Ax = b$ 不再只是一堆方程的简写，它变成了一个深刻的代数陈述：“算子 A 作用于向量 x ，得到结果 b ”。

可以说，从行列式到矩阵，是线性代数在核心认知上的一次重要发展。它标志着研究重点从一个描述变换结果的**静态数值属性**（行列式），转向了一个能够完整描述变换过程并进行运算的**动态代数结构**（矩阵）。本章，我们将深入学习这一由凯莱和西尔维斯特共同开启的，强大而普适的数学语言。

4.1.3 数学证明：矩阵的基本运算性质

证明矩阵加法、数乘、乘法的封闭性、结合律、分配律等基本性质。

4.1.4 几何角度：矩阵与线性变换

从几何角度解释矩阵如何表示线性变换，举例二维、三维空间中的旋转、缩放、投影等。

4.1.5 代码实现：矩阵的基本操作

理论学习之后，我们将通过具体的代码实践来巩固对矩阵基本运算的理解。现代深度学习框架，如 PyTorch，其核心数据结构——张量 (Tensor)，在设计上与矩阵的代数体系完美契合。使用 PyTorch，我们可以非常直观地实现和验证矩阵的创建、加法、数乘、转置和乘法等基本操作。

代码视角

下面的代码展示了如何在 PyTorch 中执行这些核心运算。

```
import torch

# --- 1. 矩阵的创建 ---
# 创建两个 2x3 的矩阵 A 和 B
# PyTorch 中，矩阵通常表示为二维张量 (Tensor)
A = torch.tensor([[1, 2, 3],
                  [4, 5, 6]], dtype=torch.float32)
```

```

B = torch.tensor([[7, 8, 9],
                  [10, 11, 12]], dtype=torch.float32)

print("---- 矩阵创建 ----")
print("矩阵 A:\n", A)
print("矩阵 B:\n", B)

# --- 2. 矩阵加法 ---
# 只有同型矩阵才能相加
C_add = A + B
print("\n---- 矩阵加法 (A + B) ----")
print("结果:\n", C_add)

# --- 3. 标量乘法 (数乘) ---
C_scalar = 3 * A
print("\n---- 标量乘法 (3 * A) ----")
print("结果:\n", C_scalar)

# --- 4. 矩阵转置 ---
# A 是 2x3 矩阵, A的转置是 3x2 矩阵
A_T = A.T
print("\n---- 矩阵转置 (A.T) ----")
print("A 的转置:\n", A_T)

# --- 5. 矩阵乘法 ---
# 矩阵乘法要求 A 的列数等于 B 的行数。
# 这里 A 是 2x3, B 是 2x3, A@B 会报错。
# 我们需要一个新的矩阵 B_T (3x2) 才能与 A (2x3) 相乘
B_T = B.T
C_mul = A @ B_T # 使用 @ 运算符进行矩阵乘法

print("\n---- 矩阵乘法 (A @ B.T) ----")
print(f"A (2x3) @ B.T (3x2) 的结果 (2x2):\n", C_mul)

# 警示: * 运算符执行的是逐元素乘法, 而非矩阵乘法!

```



```
C_elementwise = A * B
print("\n--- 警示：逐元素乘法 (A * B) ---")
print("结果:\n", C_elementwise)
```

代码视角

代码运行输出

--- 矩阵创建 ---

矩阵 A:

```
tensor([[1., 2., 3.],
        [4., 5., 6.]])
```

矩阵 B:

```
tensor([[ 7.,  8.,  9.],
        [10., 11., 12.]])
```

--- 矩阵加法 (A + B) ---

结果:

```
tensor([[ 8., 10., 12.],
        [14., 16., 18.]])
```

--- 标量乘法 (3 * A) ---

结果:

```
tensor([[ 3.,  6.,  9.],
        [12., 15., 18.]])
```

--- 矩阵转置 (A.T) ---

A 的转置:

```
tensor([[1., 4.],
        [2., 5.],
        [3., 6.]])
```

--- 矩阵乘法 (A @ B.T) ---

A (2x3) @ B.T (3x2) 的结果 (2x2):

```

tensor([[ 50.,  68.],
        [122., 167.]])

--- 警示：逐元素乘法 (A * B) ---
结果：
tensor([[ 7., 16., 27.],
        [40., 55., 72.]])

```

代码解读

上述代码清晰地展示了矩阵的基本运算与 PyTorch 张量操作的对应关系：

- **创建**：使用 ‘`torch.tensor()`’ 可以从 Python 列表中直接创建矩阵。
- **加法与数乘**：PyTorch 重载了 ‘+’ 和 ‘*’ (与标量) 运算符，使其行为与线性代数中的定义完全一致，直观且易用。
- **转置**：通过访问 ‘`.T`’ 属性即可轻松获得转置矩阵。
- **矩阵乘法**：核心运算使用 ‘@’ 运算符 (或 ‘`torch.matmul()`’)。代码中特别强调了矩阵乘法对维度的要求，并通过一个例子展示了当维度不匹配时需要先进行转置操作。
- **重要区分**：代码最后一部分通过一个警示，明确了 ‘@’ (矩阵乘法) 和 ‘*’ (逐元素乘法) 在 PyTorch (以及 NumPy) 中的根本区别，这是初学者极易混淆的关键点。

通过这段代码，我们不仅学习了如何在实践中操作矩阵，也为后续利用 PyTorch 探索更复杂的线性代数概念（如线性变换、特征分解等）打下了坚实的基础。

4.2 矩阵运算与逆矩阵

4.2.1 历史引入：逆矩阵的提出与意义

在我们学习了矩阵的加法与乘法之后，一个自然而然的问题便浮现出来：矩阵有“除法”吗？在普通算术中，除以一个数 a ，等价于乘以它的倒数 a^{-1} 。这个“倒数”的关键性质是 $a \cdot a^{-1} = 1$ 。那么，对于一个矩阵 A ，是否存在一个与之对应的特殊矩阵，我们姑且称之为 A^{-1} ，使得它们的乘积恰好是矩阵世界里的“1”——单位矩阵 E 呢？

这个探索“矩阵倒数”的疑问，与一个更古老、更实际的问题——求解线性方程组——不期而遇，并擦出了思想的火花。我们将线性方程组写为简洁的矩阵形式 $A\mathbf{x} = \mathbf{b}$ ，这本身就是一个巨大的进步。但数学家们并未止步于此，他们追求的是一种更彻底的、结构化的解法。

一个优雅得令人着迷的设想是：如果我们能像解普通方程 $ax = b$ 那样，通过两边乘

以 a^{-1} 得到 $x = a^{-1}b$ 来求解，那么我们是否也能对矩阵方程 $A\mathbf{x} = \mathbf{b}$ 做类似的操作呢？如果存在一个矩阵 A^{-1} ，我们只需在方程两边左乘这个矩阵，就能“消去” A ，从而直接得到解：

$$A^{-1}(A\mathbf{x}) = A^{-1}\mathbf{b} \implies (A^{-1}A)\mathbf{x} = A^{-1}\mathbf{b} \implies E\mathbf{x} = A^{-1}\mathbf{b} \implies \mathbf{x} = A^{-1}\mathbf{b}.$$

这个设想是革命性的。它将一个复杂的、需要反复消元的求解过程，转化成了一个更为优雅的、一步到位的矩阵乘法。这个神奇的矩阵 A^{-1} ，就是我们即将深入学习的**逆矩阵** (Inverse Matrix)。

历史侧记

凯莱的远见：构建完整的矩阵代数

将这一思想形式化并赋予其坚实理论基础的，正是矩阵理论的奠基人——**阿瑟·凯莱**。凯莱的雄心不止于将矩阵作为一种简便的记号，他致力于构建一个全新的、自洽的代数体系。在这个体系中，仅仅有加法和乘法是远远不够的。

如同数字世界需要“1”作为乘法单位元一样，矩阵代数需要**单位矩阵** E 。而更关键的是，为了让这个代数结构更加丰满和强大（例如，形成群的结构），就需要有“逆元素”的存在。凯莱在 1858 年的经典论文中，正式定义了方阵 A 的逆矩阵 A^{-1} ，它必须满足关系式：

$$AA^{-1} = A^{-1}A = E.$$

这个定义为矩阵代数的世界补上了最后一块关键的拼图，使得“矩阵除法”以“乘以逆矩阵”的形式得以实现，并为线性方程组的理论解法提供了坚实的代数根基。

这个“矩阵的倒数”——逆矩阵的提出，其意义是深远且多维度的。

逆矩阵的划时代意义

- **代数上的完备性**：它为矩阵乘法引入了“逆运算”的概念，使得矩阵代数结构更加完整，类似于从只有乘法的算术发展到有除法的算术。
- **方程求解的革新**：它在理论上提供了一种求解线性方程组的全新范式，将问题核心从“解方程”转化为“求逆矩阵”。虽然在数值计算上未必最高效，但其理论形式 $\mathbf{x} = A^{-1}\mathbf{b}$ 极为简洁，深刻地影响了整个线性代数的理论发展。
- **几何变换的可逆性**：如果一个矩阵 A 代表一个空间变换（如旋转、缩放、剪切），那么它的逆矩阵 A^{-1} 就代表了与之完全相反的“撤销”或“复原”操作。一个变换是否可逆，在几何上直观地对应了其矩阵是否存在逆矩阵。

当然，这个优雅的设想也立刻带来了一系列深刻的数学问题：是不是所有的方阵都存在逆矩阵？如果存在，它是否唯一？我们又该如何计算出这个逆矩阵呢？这些问题，正是我们接下来要通过严格的数学定义和定理来逐一解答的核心内容。

4.2.2 核心定理：逆矩阵的定义与存在性

给出逆矩阵的定义，说明可逆矩阵的判别条件（行列式不为零）。

4.2.3 数学证明：逆矩阵的唯一性与性质

证明逆矩阵的唯一性，推导逆矩阵的基本性质（如 $(AB)^{-1} = B^{-1}A^{-1}$ 等）。

4.2.4 几何角度：逆矩阵与变换的可逆性

从几何角度解释逆矩阵对应的线性变换的可逆性，举例说明。

4.2.5 代码实现：逆矩阵的求解

在理论上理解了逆矩阵的定义与意义后，我们将在代码中实践如何求解它。幸运的是，我们无需手动实现复杂的伴随矩阵法或高斯-若尔当消元法。所有主流的科学计算库都提供了高效且数值稳定的函数来直接计算逆矩阵。在 PyTorch 中，这个核心函数是 `torch.linalg.inv()`。

本节将展示如何使用该函数求解一个可逆矩阵的逆，并通过矩阵乘法验证其正确性。同时，我们也将探讨当一个矩阵不可逆（奇异）时，代码会如何响应。

代码视角

下面的代码演示了求解可逆矩阵与处理奇异矩阵两种情况。

```
import torch

# --- 1. 求解一个可逆矩阵的逆 ---
# 创建一个 3x3 的可逆矩阵 A
# 它的行列式不为零
A = torch.tensor([[1., 2., 0.],
                  [3., 5., 1.],
                  [4., 7., 2.]]) # 使用浮点数以保证计算精度

print("--- 求解可逆矩阵 ---")
print("原始矩阵 A:\n", A)

# 使用 torch.linalg.inv() 计算逆矩阵
A_inv = torch.linalg.inv(A)
print("\n计算得到的逆矩阵 A_inv:\n", A_inv)
```

```
# --- 2. 验证逆矩阵的正确性 ---
# 根据定义, A @ A_inv 应该等于单位矩阵 E
identity_check = A @ A_inv
print("\n验证 A @ A_inv:\n", identity_check)
print("注意: 由于浮点数精度限制, 结果不是很完美的单位矩阵。")

# --- 3. 尝试求解一个不可逆 (奇异) 矩阵 ---
# 创建一个奇异矩阵 B (第三行是第一行的两倍)
# 它的行列式为 0
B = torch.tensor([[1., 2., 3.],
                  [4., 5., 6.],
                  [2., 4., 6.]])

print("\n--- 尝试求解奇异矩阵 ---")
print("奇异矩阵 B:\n", B)

try:
    B_inv = torch.linalg.inv(B)
    print("\nB 的逆矩阵:\n", B_inv)
except torch.linalg.LinAlgError as e:
    print(f"\n求解失败, 正如预期。PyTorch 抛出错误: {e}")
```

代码视角

代码运行输出

```
--- 求解可逆矩阵 ---
原始矩阵 A:
tensor([[1., 2., 0.],
        [3., 5., 1.],
        [4., 7., 2.]])

计算得到的逆矩阵 A_inv:
tensor([[-3.,  4., -2.],
```

```
[ 2., -2.,  1.],
[ 1., -1.,  1.]])
```

验证 $A @ A_inv$:

```
tensor([[1.0000, 0.0000, 0.0000],
        [0.0000, 1.0000, 0.0000],
        [0.0000, 0.0000, 1.0000]])
```

注意：由于浮点数精度限制，结果不是很完美的单位矩阵。

--- 尝试求解奇异矩阵 ---

奇异矩阵 B:

```
tensor([[1., 2., 3.],
        [4., 5., 6.],
        [2., 4., 6.]])
```

求解失败，正如预期。PyTorch 抛出错误：LU, Singular matrix

代码解读

- **核心函数**：求解逆矩阵的关键是 `torch.linalg.inv(A)`。这个函数封装了高效的数值算法（通常基于 LU 分解），是求解逆矩阵的标准实践。
- **验证是关键**：如何确认计算结果的正确性？我们利用了逆矩阵的根本定义： $AA^{-1} = E$ 。代码中计算了 `A @ A_inv`，其结果非常接近单位矩阵，从而有力地验证了 `A_inv` 确实是 A 的逆。输出结果中的微小误差是浮点数运算的正常现象。
- **理论与实践的呼应**：代码的第二部分完美地演示了理论的“边界”。当一个矩阵的行列式为零（即奇异矩阵）时，它在理论上没有逆矩阵。在实践中，`torch.linalg.inv()` 会尝试进行计算，但当它发现矩阵是奇异的时，会抛出一个 `LinAlgError: Singular matrix` 的异常。这清晰地告诉我们，代码的行为与数学理论是完全一致的。
- **编程健壮性**：通过 `try...except` 语句块，我们优雅地捕获了这个预料之中的错误，而不是让程序崩溃。这是编写健壮数值代码的良好习惯。

这段代码不仅展示了如何“做”，更重要的是解释了如何“验证”以及当理论条件不满足时会“发生什么”，构成了一个完整的学习闭环。

4.3 矩阵的初等变换

4.3.1 历史引入：高斯消元法的精髓与代数化

在逆矩阵为我们描绘出求解线性方程组 $A\mathbf{x} = \mathbf{b}$ 的优雅理论蓝图 ($\mathbf{x} = A^{-1}\mathbf{b}$) 之后，一个现实的、甚至有些“扫兴”的问题摆在了我们面前：我们该如何高效地计算出 A^{-1} 呢？答案，出人意料地，又将我们带回了那个最古老、最质朴的起点——逐行消元。

长久以来，解线性方程组更像是一门手艺，依赖于代数直觉和反复的变量替换。然而，随着方程组规模的增大，这种“作坊式”的方法变得混乱且容易出错。人们迫切需要一种标准化的、如同工业流水线般精确的流程。

历史侧记

高斯的系统化：从“术”到“法”

尽管逐行消元的思想可以追溯到公元前的中国古代数学著作《九章算术》中的“方程术”，但将其提炼并推广为一种普适方法的荣誉，通常归功于 19 世纪的“数学王子”——卡尔·弗里德里希·高斯 (Carl Friedrich Gauss)。

高斯在处理天体轨道计算等大规模数据问题时，将这种消元法发展成了一套清晰、系统的步骤。其核心目标非常明确：通过一系列规范化的操作，将一个任意的线性方程组（或其系数矩阵），转化为一个等价的、但形式上极其简单的“**阶梯形**”或“**上三角形**”。一旦达到这种形式，方程组的解便可以通过简单的“回代”过程逐一求出。高斯的工作，标志着解线性方程组从一门零散的“技艺”，演变为一门有章可循的“方法”。

然而，故事并未在此结束。19 世纪中叶，当凯莱等人将矩阵作为一个独立的代数实体来研究时，他们以一种全新的、更抽象的眼光重新审视了高斯的方法。他们发现，高斯消元法中所有看似复杂的操作，无论多么曲折，都可以被分解为三种最基本、最纯粹的动作。这些动作，就是**初等行变换**。

从解题技巧到代数算法的飞跃

高斯消元法的精髓，被抽象为了以下三种“原子操作”：

1. **对换 (Swap)**: 交换任意两行（方程）的位置。
2. **倍乘 (Scale)**: 将某一行（方程）的所有系数乘以一个非零常数。
3. **倍加 (Pivot)**: 将某一行（方程）的倍数加到另一行（方程）上。

这个发现是革命性的。它意味着，任何线性方程组的求解过程，都可以被描述为对系数矩阵施加的一系列有限次的、标准化的行变换。一个依赖于经验和技巧的“解题过程”，至此被彻底转化为了一套严谨、可机械执行的“**代数算法**”。

这一“代数化”的飞跃，其意义远超于简化手工计算。它为计算机的诞生铺平了道路。计算机不理解“直觉”或“技巧”，但它能以极高的速度和精度执行定义明确的、确定性的指令。初等变换恰好就是这样一套完美的指令集。

因此，当我们将一个矩阵输入计算机，命令它求解线性方程组或计算逆矩阵时，其底层执行的核心逻辑，正是源自高斯消元法精髓的这一系列初等变换。它们是连接人类数学智慧与机器强大算力之间的桥梁。在本节中，我们将深入学习这三种变换的严格定义，以及它们在矩阵代数中所扮演的核心角色。

4.3.2 核心定理：三类变换、初等矩阵与矩阵等价

4.3.3 数学证明：初等变换与初等矩阵的等价性

4.3.4 几何角度：剪切、反射与缩放的组合

4.3.5 代码实现：用代码模拟行变换与寻找逆矩阵

我们已经知道，任何初等行变换都等价于左乘一个相应的初等矩阵。这个性质不仅是理论基石，更启发了一种寻找逆矩阵的算法思想。本节将通过 PyTorch 代码，分两步来演示这一过程。

首先，我们将构造三类初等矩阵，并验证左乘它们确实能实现对应的行变换。然后，我们将应用这一思想，通过一系列初等变换，同步作用于目标矩阵和单位矩阵，从而“构造”出逆矩阵。

第一部分：用初等矩阵模拟行变换

这一步的目的是验证“左乘初等矩阵等价于行变换”这一核心理论。

代码视角

```
import torch

# 定义一个 3x3 的目标矩阵 A
A = torch.tensor([[1., 2., 3.],
                  [4., 5., 6.],
                  [7., 8., 9.]])

print("--- 原始矩阵 A ---")
print(A, "\n")

# 1. 对换变换：交换第1行(r0)和第3行(r2)
# 构造对换矩阵 E_swap：从单位矩阵开始，交换对应的行
E_swap = torch.eye(3)
E_swap[[0, 2]] = E_swap[[2, 0]]
```

```

print("--- 1. 对换变换 (r0 <-> r2) ---")
print("对换矩阵 E_swap:\n", E_swap)
print("E_swap @ A 的结果:\n", E_swap @ A, "\n")

# 2. 倍乘变换: 将第2行(r1)乘以 5
# 构造倍乘矩阵 E_scale: 从单位矩阵开始, 修改对角线元素
E_scale = torch.eye(3)
E_scale[1, 1] = 5
print("--- 2. 倍乘变换 (r1 <- 5*r1) ---")
print("倍乘矩阵 E_scale:\n", E_scale)
print("E_scale @ A 的结果:\n", E_scale @ A, "\n")

# 3. 倍加变换: 将第1行(r0)的 -3 倍加到第3行(r2)上 (r2 <- r2 - 3*r0)
# 构造倍加矩阵 E_replace: 从单位矩阵开始, 修改非对角线元素
E_replace = torch.eye(3)
E_replace[2, 0] = -3
print("--- 3. 倍加变换 (r2 <- r2 - 3*r0) ---")
print("倍加矩阵 E_replace:\n", E_replace)
print("E_replace @ A 的结果:\n", E_replace @ A)

```

第二部分：寻找逆矩阵的算法思想与实现

我们的目标是找到一个变换矩阵 P ，使得 $PA = E$ 。这个 P 就是 A^{-1} 。我们可以通过一系列初等矩阵的连乘来构造 P 。

算法思想

1. 我们的目标是找到一系列初等矩阵 $E_k, \dots, E_1$ ，使得 $E_k \cdots E_1 A = E$ 。
2. 那么，逆矩阵就是 $A^{-1} = E_k \cdots E_1$ 。
3. 如何计算这个连乘积呢？我们可以从单位矩阵 E 开始，依次左乘这些初等矩阵： $A^{-1} = E_k \cdots E_1 E$ 。
4. 这启发我们：我们可以同时维护两个矩阵，一个是待变换的目标矩阵 A ，另一个是用于记录变换的“追踪矩阵”（初始为 E ）。对 A 施加的任何一次行变换（左乘 E_i ），都**同步地**对“追踪矩阵”施加。当 A 变为 E 时，“追踪矩阵”就变成了 A^{-1} 。

下面，我们对一个简单的 2×2 矩阵 $A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ 应用这个思想。

代码视角

```

# 目标矩阵 A
A_current = torch.tensor([[1., 2.],
                           [3., 4.]])

# “追踪矩阵”，初始为单位矩阵
Tracker_M = torch.eye(2)

print("---- 初始状态 ----")
print("目标矩阵 A:\n", A_current)
print("追踪矩阵:\n", Tracker_M, "\n")

# 步骤 1: 将 A 的第一列化为 (1, 0), 操作:  $r_1 \leftarrow r_1 - 3r_0$ 
E1 = torch.tensor([[1., 0.], [-3., 1.]])
A_current = E1 @ A_current
Tracker_M = E1 @ Tracker_M # 对追踪矩阵施加相同变换
print("---- 步骤 1:  $r_1 \leftarrow r_1 - 3r_0$  ----")
print("变换后 A:\n", A_current)
print("变换后追踪矩阵:\n", Tracker_M, "\n")

# 步骤 2: 将 A 的对角线元素化为 1, 操作:  $r_1 \leftarrow -0.5r_1$ 
E2 = torch.tensor([[1., 0.], [0., -0.5]])
A_current = E2 @ A_current
Tracker_M = E2 @ Tracker_M
print("---- 步骤 2:  $r_1 \leftarrow -0.5r_1$  ----")
print("变换后 A:\n", A_current)
print("变换后追踪矩阵:\n", Tracker_M, "\n")

# 步骤 3: 将 A 的第二列化为 (0, 1), 操作:  $r_0 \leftarrow r_0 - 2r_1$ 
E3 = torch.tensor([[1., -2.], [0., 1.]])
A_current = E3 @ A_current
Tracker_M = E3 @ Tracker_M
print("---- 步骤 3:  $r_0 \leftarrow r_0 - 2r_1$  ----")
print("最终 A (已是单位矩阵):\n", A_current)
print("最终追踪矩阵 (即 A 的逆):\n", Tracker_M, "\n")

# --- 最终结果与验证 ---

```

```
print("--- 最终结果与验证 ---")
print("通过算法计算得到的逆矩阵:\n", Tracker_M)

# 使用内置函数验证
A_original = torch.tensor([[1., 2.], [3., 4.]])
A_inv_torch = torch.linalg.inv(A_original)
print("\nPyTorch 内置函数计算的逆矩阵:\n", A_inv_torch)
```

代码解读

- **变换的代数本质：**第一部分代码直观地确认了核心理论：对矩阵 A 施加一次初等行变换，其效果与用对应的初等矩阵 E 左乘 A 完全相同。
- **算法的实现：**第二部分代码将这个思想付诸实践。我们并未引入新的概念，而是始终围绕“左乘初等矩阵”这一工具。通过同步变换目标矩阵和单位矩阵，我们最终在“追踪矩阵”中构造出了逆矩阵。
- **理论的深刻联系：**整个过程在代数上等价于计算了总变换矩阵 $P = E_3 E_2 E_1$ 。我们看到 $PA = E$ ，同时 $PE = P$ 。这清晰地表明，将 A 变为单位矩阵的变换序列，同样会将单位矩阵变为 A 的逆。
- **教学意义：**这个实现方式的价值在于，它向学生揭示了求逆算法的底层逻辑，即通过一系列基本变换来“抵消”原矩阵。这为后续正式学习高斯-若尔当消元法和增广矩阵（它们本质上是这一过程的更高效记法）打下了坚实的直观基础。

4.4 矩阵的转置

4.5 矩阵的转置

4.5.1 历史引入：凯莱的对称视角与对偶思想

在矩阵的众多运算中，转置（Transpose）或许是形式上最简单、最直观的操作之一：不过是将矩阵的行与列进行一次优雅的互换。然而，正是这个看似平凡的“翻转”，在矩阵理论的奠基人阿瑟·凯莱眼中，却开启了一扇通往矩阵结构内部秘密的大门。

当凯莱系统地构建矩阵代数时，他自然会问一个深刻的问题：这个“翻-转”后的新矩阵 A^T ，与原矩阵 A 之间存在着怎样的内在联系？通过对这个问题的探索，他揭示了数学中一些极为重要和普适的结构。

历史侧记

凯莱的分类学：对称与反对称世界的开启

凯莱发现，通过比较一个矩阵与其自身的转置，可以对矩阵进行一种根本性的分类。

- 当一个方阵“翻转”后与自身完全相同时，即 $A = A^T$ ，他识别出了一类具有完美内部平衡的矩阵。他将其命名为**对称矩阵** (Symmetric Matrix)。这类矩阵并非凭空构造，它们恰好是描述物理学中二次型（如能量、转动惯量）和几何学中二次曲线的核心数学工具。
- 相应地，当一个方阵“翻转”后恰好等于自身的负矩阵，即 $A = -A^T$ ，凯莱发现了另一类奇特的结构——**反对称矩阵** (Anti-symmetric Matrix)。这类矩阵后来被证明与描述旋转和角速度的数学对象（如李代数）密切相关。可以说，转置操作就像一面“魔镜”，它让凯莱首次窥见了矩阵世界中泾渭分明的两大阵营——对称与反对称，这为后来的矩阵分解和谱理论等领域奠定了基础。

然而，凯莱和后来的数学家们逐渐意识到，转置的意义远不止于此。它不仅仅是一种分类工具，更是一种揭示矩阵深层几何功能的“钥匙”。它触及了线性代数中一个更为抽象和核心的概念——**对偶性** (Duality)。

转置的深层本质：对偶与伴随

为了理解这一点，我们可以从一个新的视角来看待向量：

1. **向量的双重身份**：一个列向量 $\mathbf{x}$ 通常被看作空间中的一个点或一个箭头，它是一个几何**实体**。而它的转置 $\mathbf{x}^T$ (一个行向量)，则可以被看作一种**测量工具**。当这个“工具”作用于另一个列向量 $\mathbf{y}$ 时（即执行乘法 $\mathbf{x}^T \mathbf{y}$ ），它会得出一个标量——内积。从这个意义上讲，行向量是列向量的“对偶” (Dual)。
2. **变换的“影子”——伴随算子**：转置矩阵的真正威力，体现在它与内积的相互作用中。对于任意两个向量 $\mathbf{x}, \mathbf{y}$ 和一个矩阵 A ，它们之间存在一个恒等式：

$$\langle A\mathbf{x}, \mathbf{y} \rangle = (A\mathbf{x})^T \mathbf{y} = \mathbf{x}^T A^T \mathbf{y} = \langle \mathbf{x}, A^T \mathbf{y} \rangle$$

这个公式的直观解释是：对向量 $\mathbf{x}$ 先施加变换 A ，再用 $\mathbf{y}$ 去“测量”它，其结果 **完全等同于** 对“测量工具” $\mathbf{y}$ 先施加一个“伴随变换” A^T ，再用这个新的测量工具去测量原始的向量 $\mathbf{x}$ 。

因此，转置矩阵 A^T 并非一个孤立的存在，它是在内积世界中与 A 成对出现的“影子”或“伴随者” (Adjoint)。

综上所述，凯莱对转置的研究，引领我们走上了一条从直观到深刻的认知之路。它始于一个简单的行列互换操作，引出了对称与反对称这两个重要的矩阵家族，最终揭示了转置作为伴随算子，在维持内积结构不变性中所扮演的核心角色。理解了转置的这一

深层意义，我们才能真正把握后续学习中正交矩阵、谱定理等内容的精髓。

4.5.2 核心定理：转置的运算法则与性质

4.5.3 数学证明：乘积转置定理 $(AB)^T = B^T A^T$

4.5.4 几何角度：变换的伴随与内积的守恒

4.5.5 代码实现：“.T”属性的妙用

理论上，矩阵的转置是一个基础但至关重要的操作。在编程实践中，得益于现代科学计算库的优雅设计，执行这一操作变得异常简单。在 PyTorch (以及 NumPy) 中，我们无需手动交换元素，只需访问矩阵对象的 ‘.T’ 属性即可。

本节将通过一个具体的数值例子，不仅展示 ‘.T’ 属性的便捷性，更重要的是，用代码来验证转置运算中最核心、也最容易出错的性质——乘积转置定理 $(AB)^T = B^T A^T$ 。

代码视角

为了验证 $(AB)^T = B^T A^T$ ，我们将分别计算等式的左边 (LHS) 和右边 (RHS)，然后比较它们是否相等。

```
import torch

# --- 1. 定义两个维度适合相乘的矩阵 A 和 B ---
# A 是一个 2x3 矩阵
A = torch.tensor([[1., 2., 3.],
                  [4., 5., 6.]])

# B 是一个 3x2 矩阵
B = torch.tensor([[7., 8.],
                  [9., 10.],
                  [11., 12.]])

print("--- 原始矩阵 ---")
print("矩阵 A (2x3):\n", A)
print("矩阵 B (3x2):\n", B)

# --- 2. 计算等式左边 (LHS): (A @ B).T ---
```

```

# 首先计算矩阵乘积 A @ B
C = A @ B
# 然后对结果进行转置
LHS = C.T # 或者直接写作 (A @ B).T

print("\n--- 计算等式左边 (LHS) ---")
print("A @ B 的结果 (2x2):\n", C)
print("(A @ B).T 的结果:\n", LHS)

# --- 3. 计算等式右边 (RHS): B.T @ A.T ---
# 首先分别计算 A 和 B 的转置
A_T = A.T
B_T = B.T
# 然后注意顺序, 用 B.T 乘以 A.T
RHS = B_T @ A_T

print("\n--- 计算等式右边 (RHS) ---")
print("B.T (2x3):\n", B_T)
print("A.T (3x2):\n", A_T)
print("B.T @ A.T 的结果:\n", RHS)

# --- 4. 验证 LHS 与 RHS 是否相等 ---
# 使用 torch.allclose() 来比较浮点数张量, 它能处理微小的精度误差
are_equal = torch.allclose(LHS, RHS)

print("\n--- 验证结果 ---")
print(f"LHS 与 RHS 是否相等? -> {are_equal}")

```

代码视角

代码运行输出

--- 原始矩阵 ---

矩阵 A (2x3):

```
tensor([[1., 2., 3.],  
        [4., 5., 6.]])
```

矩阵 B (3x2):

```
tensor([[ 7.,  8.],  
        [ 9., 10.],  
        [11., 12.]])
```

--- 计算等式左边 (LHS) ---

A @ B 的结果 (2x2):

```
tensor([[ 58.,  64.],  
        [139., 154.]])
```

(A @ B).T 的结果:

```
tensor([[ 58., 139.],  
        [ 64., 154.]])
```

--- 计算等式右边 (RHS) ---

B.T (2x3):

```
tensor([[ 7.,  9., 11.],  
        [ 8., 10., 12.]])
```

A.T (3x2):

```
tensor([[1., 4.],  
        [2., 5.],  
        [3., 6.]])
```

B.T @ A.T 的结果:

```
tensor([[ 58., 139.],  
        [ 64., 154.]])
```

--- 验证结果 ---

LHS 与 RHS 是否相等? -> True

代码解读

- **操作的简洁性**：代码清晰地展示了，无论是对于 2×3 的矩阵 ‘A’ 还是 3×2 的矩阵 ‘B’，获取其转置都只需要简单地调用 ‘.T’ 属性。这极大地简化了编程实现。
- **验证的逻辑**：我们严格遵循了数学证明的思路。代码分别计算了等式 $(AB)^T$ 和 $B^T A^T$ 两侧的表达式，并将最终结果进行比较。‘torch.allclose()’ 函数返回 ‘True’，从数值上无可辩驳地证实了该定理的正确性。
- **“穿脱原则”的直观体现**：这段代码最核心的教学意义在于，它让抽象的“穿脱原则”或“逆序相乘”变得具体可见。为了计算右侧 ‘RHS’，我们必须严格遵守 $B_T @ A_T$ 的顺序，而不是 $A_T @ B_T$ （后者甚至会因为维度不匹配而报错）。亲手编写并运行这段代码，能让学习者对这个核心性质留下极其深刻的印象。

4.6 一些重要的矩阵

4.6.1 历史引入：从物理与几何中涌现的特殊结构

在线性代数的学习中，我们会遇到一系列具有特殊名称和性质的矩阵，如对称矩阵、正交矩阵等。这些特殊结构并非数学家为了理论完备性而随意定义的，恰恰相反，它们大多是在描述和解决具体物理与几何问题时，作为核心数学模型被发现和提炼出来的。这些矩阵的特殊性质，直接反映了现实世界中某些深刻的内在规律。

历史侧记

对称性：物理系统内在属性的数学表达

对称矩阵的定义 $A^T = A$ （即 $a_{ij} = a_{ji}$ ）在形式上非常简洁，但其重要性源于它能够精确描述物理系统中普遍存在的互易性（Reciprocity）。

- **经典力学中的转动惯量**：当分析一个三维刚体的转动时，我们需要一个 3×3 的矩阵来描述其转动惯量，称为惯量张量 I 。其中，非对角元素 I_{xy} 表示物体绕 y 轴转动时，在 x 轴方向上产生的惯性效应。物理定律表明，这个效应是互易的，即 $I_{xy} = I_{yx}$ 。因此，惯量张量必然是一个**对称矩阵**。对称性在这里不是一个数学选项，而是物理现实的直接反映。
- **量子力学中的可观测量**：在量子力学中，所有可被测量的物理量（如能量、动量、位置）都由一类特殊的算子表示。当这些算子在有限维空间中用矩阵表示时，它们必须是**埃尔米特矩阵**（在实数域上，就是对称矩阵）。其根本原因是，物理测量的结果必须是实数。数学上可以证明，只有对称（或埃尔米特）矩阵才能保证其所有特征值（代表可能的测量结果）都是实数。因此，对称结构是描述物理系统静态属性（如惯性）和可观测量（如能量）的必要数学形式。

历史侧记

正交性：几何空间保距变换的代数语言

与描述系统内在属性的对称矩阵不同，另一类重要的矩阵——正交矩阵，源于对空间**变换**过程的研究。特别是那些不改变物体形状和大小的“刚体变换”。

- **几何中的旋转与反射**：在欧几里得空间中，当我们对一个物体进行旋转（Rotation）或反射（Reflection）时，物体上任意两点间的距离和任意两条线段间的夹角都保持不变。这类变换被称为**保距变换**（Isometry）。
- **代数上的定义**：数学家发现，能够实现这类保距变换的线性算子，其对应的矩阵 Q 必须满足一个严格的条件： $Q^T Q = E$ （单位矩阵）。这类矩阵就被定义为**正交矩阵**。这个代数条件的几何意义是，矩阵 Q 的所有列向量（或行向量）构成了一组**标准正交基**——它们两两垂直，且长度均为 1。

从几何上看，一个正交变换的作用，就是将空间中的标准坐标系 $(\mathbf{i}, \mathbf{j}, \mathbf{k})$ 变换到一个新的、但同样是标准正交的坐标系。这正是旋转和反射操作的本质。因此，正交矩阵是描述刚体运动和坐标系变换最自然、最精确的代数语言。

结构与功能：特殊矩阵的两大来源

通过以上例子，我们可以清晰地看到特殊矩阵结构的两大主要来源：

- **对称矩阵**：通常用于描述一个物理或几何**系统**的**内在属性**，其结构源于物理定律的互易性或可观测量的实数要求。
- **正交矩阵**：通常用于描述作用于一个系统之上的**变换或操作**，其结构源于几何空间中长度、角度等度量保持不变的要求。

理解了这些特殊矩阵的物理和几何起源，将有助于我们更深刻地把握它们在后续理论学习和实际应用中的核心作用。

4.6.2 核心定理：对称、正交与对角矩阵的定义与性质

4.6.3 数学证明：特殊性质的推导

4.6.4 几何角度：旋转、缩放与正交拉伸

4.6.5 代码实现：特殊矩阵的构造与验证

理论学习之后，我们将通过代码实践来构造并验证本节讨论的几类重要矩阵。PyTorch 提供了丰富的函数和操作，使得创建具有特殊结构的矩阵以及检验它们的性质变得非常直观和便捷。

本节将演示如何构造对角矩阵、对称矩阵和正交矩阵，并利用它们的定义来编写代码进行验证。

代码视角

```

import torch
import numpy as np # For pi

# --- 1. 对角矩阵 (Diagonal Matrix) ---
# 使用 torch.diag() 可以从一个一维向量创建对角矩阵
diag_elements = torch.tensor([5., -2., 3.])
D = torch.diag(diag_elements)

print("--- 1. 对角矩阵 ---")
print("构造的对角矩阵 D:\n", D, "\n")

# --- 2. 对称矩阵 (Symmetric Matrix) ---
# 构造方法：对于任意方阵 A, A + A.T 必然是对称的。
# 或者 A @ A.T 也是对称的。我们使用后者。
A = torch.tensor([[1., 2., 3.],
                  [4., 5., 6.]]) # 一个 2x3 的非方阵
S = A @ A.T # 结果是一个 2x2 的对称矩阵

print("\n--- 2. 对称矩阵 ---")
print("任意矩阵 A:\n", A)
print("通过 A @ A.T 构造的对称矩阵 S:\n", S)

# 验证对称性：检查 S 是否等于其转置 S.T
S_T = S.T
is_symmetric = torch.allclose(S, S_T)
print("\n验证 S == S.T:", is_symmetric)

# --- 3. 正交矩阵 (Orthogonal Matrix) ---
# 构造方法：以一个 2D 旋转矩阵为例，它是典型的正交矩阵
theta = torch.tensor(np.pi / 4) # 旋转 45 度
c, s = torch.cos(theta), torch.sin(theta)
Q = torch.tensor([[c, -s],

```



```

[s, c]])

print("\n--- 3. 正交矩阵 ---")
print("构造的旋转矩阵 Q (旋转45度):\n", Q)

# 验证正交性: 检查 Q.T @ Q 是否等于单位矩阵 E
Q_T_Q = Q.T @ Q
E = torch.eye(2) # 构造一个 2x2 的单位矩阵

is_orthogonal = torch.allclose(Q_T_Q, E)
print("\n计算 Q.T @ Q 的结果:\n", Q_T_Q)
print("验证 Q.T @ Q == E:", is_orthogonal)

```

代码视角

代码运行输出

```

--- 1. 对角矩阵 ---
构造的对角矩阵 D:
tensor([[ 5.,  0.,  0.],
        [ 0., -2.,  0.],
        [ 0.,  0.,  3.]])

--- 2. 对称矩阵 ---
任意矩阵 A:
tensor([[1., 2., 3.],
        [4., 5., 6.]])
通过 A @ A.T 构造的对称矩阵 S:
tensor([[14., 32.],
        [32., 77.]])

验证 S == S.T: True

--- 3. 正交矩阵 ---

```

构造的旋转矩阵 Q (旋转45度):

```
tensor([[ 0.7071, -0.7071],
        [ 0.7071,  0.7071]])
```

计算 $Q.T @ Q$ 的结果:

```
tensor([[1.0000, 0.0000],
        [0.0000, 1.0000]])
```

验证 $Q.T @ Q == E$: True

代码解读

- **对角矩阵的构造**: ‘torch.diag()’ 是一个非常方便的函数, 它接受一个一维张量 (向量), 并将其元素放置在一个零矩阵的主对角线上, 从而快速生成对角矩阵。
- **对称矩阵的构造与验证**: 我们没有手动创建一个对称矩阵, 而是展示了一种更通用的构造方法: 对于任意矩阵 A (甚至非方阵), 乘积 AA^T 必然是对称的。代码通过计算 ‘ $A @ A.T$ ’ 生成了对称矩阵 ‘ S ’, 并通过 ‘torch.allclose($S, S.T$)’ 验证了其 ‘ $S = S^T$ ’ 的核心性质。
- **正交矩阵的构造与验证**: 我们以一个具体的几何实例——2D 旋转矩阵——来构造正交矩阵。这是正交矩阵最典型的应用场景之一。验证过程严格遵循了其代数定义: 计算 ‘ $Q.T @ Q$ ’ 并检查结果是否为单位矩阵。代码输出显示, 计算结果在浮点数精度范围内与单位矩阵完全一致。
- **‘torch.allclose()’ 的重要性**: 在比较浮点数矩阵时, 直接使用 ‘==’ 可能会因为微小的精度误差而导致 ‘False’。‘torch.allclose()’ 函数是专门为此设计的, 它在一定的容忍度 (tolerance) 内比较两个张量是否 “足够接近”, 是进行数值验证的标准做法。

这段代码不仅展示了如何生成这些特殊矩阵, 更重要的是, 它教会了我们如何用代码的语言来 “检验” 一个矩阵是否满足其数学定义, 这是连接理论与实践的关键一步。

4.7 矩阵的分块

4.7.1 历史引入: “分而治之” 思想的代数体现

“分而治之” (Divide and Conquer) 是人类解决复杂问题时最古老、也最有效的思想之一。从军事策略到大型工程, 再到现代计算机算法, 其核心都是将一个庞大到难以处理的问题, 分解为若干个规模更小、结构相似且易于解决的子问题, 最后再将子问题的解合并, 得到原问题的解。

在矩阵理论中, 这一思想的直接体现就是**矩阵分块** (Block Matrix) 技术。它允许我

们将一个高阶矩阵视作一个由更小的“子矩阵块”构成的“超级矩阵”，从而在更高层次上洞察其结构并简化运算。

历史侧记

前计算机时代：化繁为简的数学技艺

在没有计算机辅助的年代，处理高阶矩阵（例如，一个 10×10 矩阵）的行列式或逆矩阵计算是一项极其繁重且极易出错的工作。数学家们自然地开始寻找简化计算的捷径。他们发现，如果一个大矩阵的某些区域恰好是零，那么计算就可以被大大简化。

矩阵分块的思想，正是在这一背景下由拉普拉斯、舒尔（Issai Schur）等数学家系统化。例如，他们证明了对于一个分块三角矩阵，其行列式等于对角线上子块行列式的乘积：

$$\begin{vmatrix} A & B \\ O & D \end{vmatrix} = \det(A) \cdot \det(D)$$

这个公式的意义是革命性的。它将一个高阶行列式的计算，成功地“分而治之”为两个独立的、阶数更低的行列式计算。对于手工计算而言，这极大地降低了问题的复杂度。在当时，矩阵分块主要是一种依赖于矩阵特殊结构（如含有零块）的、巧妙的**计算技巧**。

历史侧记

计算机时代：高性能计算的基石

随着计算机的诞生，矩阵运算从手工时代迈入了自动化时代。人们可能会认为，既然计算机算力强大，这种“投机取巧”的分块技巧就不再重要了。然而，事实恰恰相反：矩阵分块在计算机科学中迎来了第二次生命，并从一种“技巧”升华为一种核心的**架构思想**。

这主要是由现代计算机的硬件结构决定的：

- **缓存优化 (Cache Optimization)**: CPU 访问高速缓存 (Cache) 的速度远快于访问主内存 (RAM)。当处理一个巨大的矩阵时，如果按元素逐个访问，会导致频繁的“缓存未命中”，效率低下。但如果将矩阵分块，就可以将一个小块完整地读入高速缓存，在缓存内完成所有相关计算后，再写回主内存。这种策略能极大提升计算速度，是所有高性能线性代数库（如 BLAS, LAPACK）的核心优化手段。
- **并行计算 (Parallel Computing)**: 现代计算机通常拥有多个处理器核心。为了利用所有核心，必须将计算任务分解。矩阵分块为此提供了完美的解决方案。一个大的矩阵乘法 $C = AB$ 可以被分解为多个小的子块矩阵乘法，每个子任务被分配到一个独立的处理器核心上并行执行，最后再将结果拼接起来。这使得处理超大规模矩阵成为可能。

从技巧到架构的升华

矩阵分块思想的演变，清晰地体现了数学与计算科学的互动：

- **在前计算机时代**，它是一种依赖于特殊结构的**计算技巧**，目的是降低人类手工计算的复杂度。
- **在计算机时代**，它演变成为一种通用的**数据结构与算法设计思想**，目的是优化硬件性能（缓存）和实现并行计算，以应对超大规模问题的挑战。

因此，学习矩阵分块不仅是在学习一套代数运算法则，更是在理解一种贯穿于整个科学与工程领域的、强大的“分而治之”的思维范式。

4.7.2 核心理论：分块矩阵的运算法则

4.7.3 数学证明：分块乘法法则的严谨性

4.7.4 几何角度：子空间的独立变换

4.7.5 代码实现：用代码拼接与分解矩阵

矩阵分块的“分而治之”思想在编程实践中得到了完美的体现。通过将大矩阵分解为小块，我们不仅可以在概念上简化问题，还能在特定情况下大幅提升计算效率。本节将使用 PyTorch 来演示如何构造分块矩阵，并从两个层面验证其有效性：

1. **一般情况**：验证分块矩阵的乘法规则与标准矩阵乘法的结果完全一致。
2. **特殊情况**：展示对于分块对角矩阵，其求逆和行列式计算可以被分解为对角线上子块的独立计算。

第一部分：构造分块矩阵并验证乘法规则

我们将构造两个分块矩阵 M 和 N ，然后分别用标准方法和分块方法计算它们的乘积，最后验证结果是否相同。

代码视角

[Python (PyTorch) 实现]

```
import torch

# --- 1. 定义子矩阵块 ---
# 定义 M 的四个子块
A = torch.tensor([[1., 2.], [3., 4.]]) # 2x2
B = torch.tensor([[5.], [6.]])         # 2x1
C = torch.tensor([[7., 8.]])           # 1x2
```

```

D = torch.tensor([[9.]])          # 1x1

# 定义 N 的四个子块（维度需与 M 匹配以进行分块乘法）
E = torch.tensor([[10., 11.], [12., 13.]]) # 2x2
F = torch.tensor([[14.], [15.]])          # 2x1
G = torch.tensor([[16., 17.]])            # 1x2
H = torch.tensor([[18.]])                 # 1x1

# --- 2. 使用 torch.cat 构造完整的 M 和 N ---
# 构造 M
top_M = torch.cat((A, B), dim=1)          # dim=1 表示水平拼接
bottom_M = torch.cat((C, D), dim=1)
M = torch.cat((top_M, bottom_M), dim=0) # dim=0 表示垂直拼接

# 构造 N
top_N = torch.cat((E, F), dim=1)
bottom_N = torch.cat((G, H), dim=1)
N = torch.cat((top_N, bottom_N), dim=0)

print("---- 构造的完整矩阵 ----")
print("矩阵 M (3x3):\n", M)
print("矩阵 N (3x3):\n", N, "\n")

# --- 3. 标准方法：直接计算 M @ N ---
MN_standard = M @ N
print("---- 标准方法计算 M @ N ----")
print("结果:\n", MN_standard, "\n")

# --- 4. 分块方法：根据公式计算 M @ N ---
# P = A@E + B@G
P = A @ E + B @ G
# Q = A@F + B@H
Q = A @ F + B @ H
# R = C@E + D@G
R = C @ E + D @ G
# S = C@F + D@H

```

```

S = C @ F + D @ H

# 使用 torch.cat 拼接结果子块
top_MN_block = torch.cat((P, Q), dim=1)
bottom_MN_block = torch.cat((R, S), dim=1)
MN_block = torch.cat((top_MN_block, bottom_MN_block), dim=0)

print("---- 分块方法计算 M @ N ----")
print("结果:\n", MN_block, "\n")

# --- 5. 验证结果 ---
is_equal = torch.allclose(MN_standard, MN_block)
print(f"--- 验证 ---")
print(f"两种方法的结果是否一致? -> {is_equal}")

```

第二部分：分块对角矩阵的简化运算

现在，我们将构造一个分块对角矩阵，并展示其求逆和行列式计算如何被简化。

代码视角

```

# --- 1. 定义对角线上的子块 ---
B1 = torch.tensor([[1., 2.], [3., 4.]]) # 2x2 可逆矩阵
B2 = torch.tensor([[5.]])                # 1x1 可逆矩阵

# 使用 torch.block_diag 快速构造分块对角矩阵
# (也可以用 torch.cat 和零矩阵手动构造)
M_diag = torch.block_diag(B1, B2)

print("\n--- 分块对角矩阵 ---")
print("分块对角矩阵 M_diag:\n", M_diag, "\n")

# --- 2. 求逆：比较两种方法 ---
# 方法一：直接对整个矩阵求逆
inv_M_standard = torch.linalg.inv(M_diag)

# 方法二：分别对子块求逆，再拼接

```



```

inv_B1 = torch.linalg.inv(B1)
inv_B2 = torch.linalg.inv(B2)
inv_M_block = torch.block_diag(inv_B1, inv_B2)

print("---- 求逆矩阵 ----")
print("标准方法求逆:\n", inv_M_standard)
print("分块方法求逆:\n", inv_M_block)
print(f"两种求逆方法结果是否一致? -> {torch.allclose(inv_M_standard, inv_M_block)}")

# --- 3. 行列式: 比较两种方法 ---
# 方法一: 直接计算整个矩阵的行列式
det_M_standard = torch.linalg.det(M_diag)

# 方法二: 计算子块行列式的乘积
det_M_block = torch.linalg.det(B1) * torch.linalg.det(B2)

print("---- 求行列式 ----")
print(f"标准方法求行列式: {det_M_standard:.2f}")
print(f"分块方法求行列式: {det_M_block:.2f}")
print(f"两种行列式计算结果是否一致? -> {torch.isclose(det_M_standard, det_M_block)}")

```

代码解读

- **矩阵的拼接**: ‘torch.cat()’ 是构造分块矩阵的基础工具。通过设置 ‘dim=1’ (按列, 水平拼接) 和 ‘dim=0’ (按行, 垂直拼接), 我们可以像搭积木一样将任意子块组合成一个大矩阵。
- **乘法法则的验证**: 第一部分的代码有力地证明了分块矩阵乘法法则的正确性。无论是直接对 3×3 矩阵进行标准乘法, 还是遵循分块乘法公式 ($A@E + B@G, \dots$) 对子块进行运算再拼接, 最终得到的结果是完全一致的。
- **分块对角矩阵的优势**: 第二部分是“分而治之”思想威力的绝佳展示。
 - 对于**求逆**, 我们无需处理整个 3×3 矩阵, 而是独立地处理一个 2×2 和一个 1×1 的小矩阵, 计算量大大降低。
 - 对于**行列式**, 计算同样被分解为两个低阶行列式的乘积。

代码中 ‘torch.block_diag()’ 函数的运用, 也展示了 PyTorch 对此类特殊结构的原生支持。

- **实践意义**：这些示例说明，在处理具有明显分块结构（尤其是含大量零块）的大规模矩阵时，识别并利用其分块性质，是设计高效算法的关键。

4.8 矩阵的概念

要点提炼

本节核心目标

1. 理解矩阵作为数据组织与线性变换的数学抽象
2. 掌握矩阵的严格定义、表示方法与特殊类型
3. 熟练运用矩阵的基本运算规则与性质
4. 厘清矩阵与行列式的本质区别与联系

4.8.1 从实际问题到数学抽象

矩阵的概念并非凭空产生，它源于对现实世界多维数据与线性关系进行高效表达与处理的迫切需求。

注[矩阵的三大作用]

- **数据组织**：将多维数据规整为“行 × 列”的矩形数表，便于系统化处理与操作。
- **计算效率**：为批量处理线性运算（如多元一次方程组）提供简洁的代数语言。
- **统一表达**：从线性方程组、网络关系到几何变换，皆可用矩阵统一描述。

例题 4.1 线性方程组的矩阵表示 考虑线性方程组

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2 \\ \vdots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n = b_m. \end{cases}$$

其全部信息可浓缩于两个矩阵：

$$\begin{aligned} \bullet \text{ 系数矩阵: } \mathbf{A} = (a_{ij})_{m \times n} &= \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix} \\ \bullet \text{ 增广矩阵: } [\mathbf{A} \mid \mathbf{b}] &= \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ a_{21} & a_{22} & \cdots & a_{2n} & b_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} & b_m \end{pmatrix} \end{aligned}$$

其中 $\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}$ 为未知向量, $\mathbf{b} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{pmatrix}$ 为常数项向量。方程组可简洁地表示为 $\mathbf{Ax} = \mathbf{b}$ 。

例题 4.2 网络的邻接矩阵 在图论中, 有向图 (如航线网络、超链接网络) 可用矩阵简洁表示。设图有 n 个顶点, 其邻接矩阵 $\mathbf{M} = (m_{ij})_{n \times n}$ 定义为:

$$m_{ij} = \begin{cases} 1, & \text{存在从顶点 } i \text{ 到 } j \text{ 的有向边} \\ 0, & \text{不存在从顶点 } i \text{ 到 } j \text{ 的有向边} \end{cases}$$

命题 4.1 (矩阵幂的路径计数意义)

对于有向图的邻接矩阵 $\mathbf{M}$, 其 k 次幂 $\mathbf{M}^k$ 的 (i, j) 元等于从顶点 i 到顶点 j 的长度为 k 的有向路径总数。



历史侧记

矩阵的思想萌芽可追溯至中国古代《九章算术》中的“方程术”。术语“矩阵”(Matrix) 由英国数学家詹姆斯·西尔维斯特 (James Sylvester) 于 1850 年首次提出, 其数学形式则由阿瑟·凯莱 (Arthur Cayley) 于 1858 年系统阐述并发展。凯莱被誉为矩阵理论的奠基人, 他确立了矩阵的代数运算规则。

4.8.2 矩阵的定义与记号

定义 4.1 (矩阵)

设 $\mathbb{F}$ 是一个数域 (通常取实数域 $\mathbb{R}$ 或复数域 $\mathbb{C}$), 由 $m \times n$ 个数 $a_{ij} \in \mathbb{F}$ ($i = 1, 2, \dots, m; j = 1, 2, \dots, n$) 排成的 m 行 n 列的数表称为一个矩阵 (Matrix):

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}$$

可简记为 $\mathbf{A} = (a_{ij})_{m \times n}$, 或 $\mathbf{A} \in \mathbb{F}^{m \times n}$ 。 a_{ij} 称为矩阵 $\mathbf{A}$ 的 (i, j) 元。



注[记号与术语]

- 矩阵常用大写粗体字母表示, 如 $\mathbf{A}, \mathbf{B}, \mathbf{C}$ 。
- 矩阵的元素用小写字母加双下标表示, 如 a_{ij} 表示第 i 行第 j 列的元素。
- 矩阵的维度常用下标强调, 如 $\mathbf{A}_{m \times n}$ 。
- 元素全为实数的矩阵称为**实矩阵**, 元素全为复数的矩阵称为**复矩阵**。

4.8.3 矩阵的分类与典型实例

矩阵可根据其元素、形状和特殊性质进行分类。

4.8.3.0.1 按元素类型

- **实矩阵**: 所有元素均为实数, $\mathbf{A} \in \mathbb{R}^{m \times n}$ 。
- **复矩阵**: 至少有一个元素为复数, $\mathbf{A} \in \mathbb{C}^{m \times n}$ 。

4.8.3.0.2 按形状与结构

- **行矩阵** (行向量): 仅有一行, $\mathbf{R} \in \mathbb{F}^{1 \times n}$, 如 $\mathbf{R} = (r_1 \ r_2 \ \cdots \ r_n)$ 。
- **列矩阵** (列向量): 仅有一列, $\mathbf{C} \in \mathbb{F}^{m \times 1}$, 如 $\mathbf{C} = \begin{pmatrix} c_1 \\ c_2 \\ \vdots \\ c_m \end{pmatrix}$ 。
- **方阵**: 行数与列数相等 ($m = n$) 的矩阵, 称为 n 阶方阵, 记作 $\mathbf{A}_n$ 。方阵有其特殊性质, 如可计算行列式和迹。
- **零矩阵**: 所有元素均为零的矩阵, 记作 $\mathbf{O}_{m \times n}$ 或简记为 $\mathbf{0}$ 。不同维度的零矩阵不相等。

- **对角矩阵**: 非主对角线元素全为零的方阵, 形如 $\mathbf{D} = \text{diag}(d_1, d_2, \dots, d_n) = \begin{pmatrix} d_1 & 0 & \cdots & 0 \\ 0 & d_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & d_n \end{pmatrix}$ 。
- **单位矩阵**: 主对角线元素全为 1 的对角矩阵, 记作 $\mathbf{I}_n$ 或 $\mathbf{E}_n$ 。即 $\mathbf{I}_n = \text{diag}(1, 1, \dots, 1)$ 。
- **三角矩阵**:
 - **上三角矩阵**: 主对角线以下元素全为零的方阵。
 - **下三角矩阵**: 主对角线以上元素全为零的方阵。
- **对称矩阵**: 满足 $\mathbf{A}^\top = \mathbf{A}$ 的方阵, 即 $a_{ij} = a_{ji}$ 对所有 i, j 成立。
- **反对称矩阵**: 满足 $\mathbf{A}^\top = -\mathbf{A}$ 的方阵, 即 $a_{ij} = -a_{ji}$, 且主对角线元素必为零。

定义 4.2 (方阵的迹)

对于 n 阶方阵 $\mathbf{A} = (a_{ij})_{n \times n}$, 其迹 (Trace) 定义为所有主对角线元素之和:

$$\text{tr}(\mathbf{A}) = a_{11} + a_{22} + \cdots + a_{nn} = \sum_{i=1}^n a_{ii}$$



例题 4.3 特殊方阵的性质

- **对角矩阵的幂运算**: 若 $\mathbf{D} = \text{diag}(d_1, d_2, \dots, d_n)$, 则 $\mathbf{D}^k = \text{diag}(d_1^k, d_2^k, \dots, d_n^k)$ 。
- **单位矩阵的恒等性质**: 对任意 $\mathbf{A}_{m \times n}$, 有 $\mathbf{I}_m \mathbf{A} = \mathbf{A} \mathbf{I}_n = \mathbf{A}$ 。
- **三角矩阵的行列式**: 上 (下) 三角矩阵的行列式等于其主对角线元素的乘积。

4.8.4 矩阵的基本运算

矩阵的运算赋予其强大的表达能力，主要包括线性运算、乘法和转置。

4.8.4.0.1 加法与数乘（线性运算）

- **加法**：仅当两个矩阵同型（即行、列数分别相同）时才能进行。设 $\mathbf{A} = (a_{ij})_{m \times n}$, $\mathbf{B} = (b_{ij})_{m \times n}$ ，则

$$\mathbf{A} + \mathbf{B} = (a_{ij} + b_{ij})_{m \times n}$$

- **数乘**：数 $\lambda \in \mathbb{F}$ 与矩阵 $\mathbf{A} = (a_{ij})_{m \times n}$ 的乘积定义为

$$\lambda \mathbf{A} = (\lambda a_{ij})_{m \times n}$$

线性运算满足以下运算律（ $\mathbf{A}, \mathbf{B}, \mathbf{C}$ 同型， λ, μ 为数）：

- 加法交换律： $\mathbf{A} + \mathbf{B} = \mathbf{B} + \mathbf{A}$
- 加法结合律： $(\mathbf{A} + \mathbf{B}) + \mathbf{C} = \mathbf{A} + (\mathbf{B} + \mathbf{C})$
- 零矩阵性质： $\mathbf{A} + \mathbf{O} = \mathbf{A}$
- 数乘结合律： $(\lambda\mu)\mathbf{A} = \lambda(\mu\mathbf{A})$
- 数乘分配律： $\lambda(\mathbf{A} + \mathbf{B}) = \lambda\mathbf{A} + \lambda\mathbf{B}$, $(\lambda + \mu)\mathbf{A} = \lambda\mathbf{A} + \mu\mathbf{A}$

4.8.4.0.2 转置 将矩阵 $\mathbf{A} = (a_{ij})_{m \times n}$ 的行列互换，得到其转置矩阵 $\mathbf{A}^\top = (a_{ji})_{n \times m}$ 。转置运算满足（假设运算可行）：

- $(\mathbf{A}^\top)^\top = \mathbf{A}$
- $(\mathbf{A} + \mathbf{B})^\top = \mathbf{A}^\top + \mathbf{B}^\top$
- $(\lambda\mathbf{A})^\top = \lambda\mathbf{A}^\top$
- $(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top$

4.8.4.0.3 矩阵乘法 矩阵乘法是核心运算，但其定义并非元素对应相乘。

定义 4.3 (矩阵乘法)

设 $\mathbf{A} = (a_{ik})_{m \times p}$, $\mathbf{B} = (b_{kj})_{p \times n}$ ，则乘积 $\mathbf{C} = \mathbf{AB}$ 是一个 $m \times n$ 矩阵，其 (i, j) 元为

$$c_{ij} = \sum_{k=1}^p a_{ik} b_{kj}$$

即 $\mathbf{C}$ 的第 i 行第 j 列元素是 $\mathbf{A}$ 的第 i 行与 $\mathbf{B}$ 的第 j 列对应元素的乘积之和。



注[乘法注意事项]

- **维度要求**： $\mathbf{A}$ 的列数必须等于 $\mathbf{B}$ 的行数。
- **非交换性**：一般地， $\mathbf{AB} \neq \mathbf{BA}$ 。即使可乘，结果也可能不同。
- **零因子**：存在非零矩阵 $\mathbf{A}, \mathbf{B}$ 使得 $\mathbf{AB} = \mathbf{O}$ 。
- **消去律不成立**：由 $\mathbf{AB} = \mathbf{AC}$ 且 $\mathbf{A} \neq \mathbf{O}$ 不能推出 $\mathbf{B} = \mathbf{C}$ 。

矩阵乘法满足以下运算律（假设运算可行）：

- 乘法结合律: $(\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC})$
- 乘法对加法分配律: $\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{AB} + \mathbf{AC}$, $(\mathbf{B} + \mathbf{C})\mathbf{A} = \mathbf{BA} + \mathbf{CA}$
- 数乘结合律: $\lambda(\mathbf{AB}) = (\lambda\mathbf{A})\mathbf{B} = \mathbf{A}(\lambda\mathbf{B})$
- 单位矩阵恒等性: $\mathbf{I}_m\mathbf{A}_{m \times n} = \mathbf{A}_{m \times n}\mathbf{I}_n = \mathbf{A}$

代码视角

Python/NumPy 中的矩阵运算 NumPy 是 Python 中进行矩阵运算的强大工具。

```
import numpy as np

# 矩阵定义
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

# 加法
C_add = A + B

# 数乘
C_scalar = 3 * A

# 转置
A_transpose = A.T

# 矩阵乘法 (使用 @ 运算符或 dot 函数)
C_matmul = A @ B # 或 np.dot(A, B)

# 注意: * 在 NumPy 中是逐元素相乘, 非矩阵乘法!
C_elementwise = A * B

print("A + B:\n", C_add)
print("3 * A:\n", C_scalar)
print("A.T:\n", A_transpose)
print("A @ B (矩阵乘法):\n", C_matmul)
print("A * B (逐元素乘):\n", C_elementwise)
```


4.8.5 矩阵相等与增广矩阵

定义 4.4 (矩阵相等)

两个矩阵 $\mathbf{A} = (a_{ij})$ 和 $\mathbf{B} = (b_{ij})$ 称为相等，记作 $\mathbf{A} = \mathbf{B}$ ，当且仅当它们满足：

1. 同型：具有相同的行数和列数。
2. 对应元素相等：对所有 i, j ，有 $a_{ij} = b_{ij}$ 。



例题 4.4 通过矩阵相等求解未知数 设 $\mathbf{A} = \begin{pmatrix} x+y & 2 \\ 1 & x-y \end{pmatrix}$, $\mathbf{B} = \begin{pmatrix} 3 & 2 \\ 1 & 1 \end{pmatrix}$ 。若 $\mathbf{A} = \mathbf{B}$ ，则：

$$\begin{cases} x+y=3 \\ x-y=1 \end{cases} \Rightarrow \begin{cases} x=2 \\ y=1 \end{cases}$$

注[增广矩阵与线性方程组] 对于线性方程组 $\mathbf{Ax} = \mathbf{b}$ ，其增广矩阵 $[\mathbf{A} \mid \mathbf{b}]$ 包含了方程组的全部信息。高斯消元法对增广矩阵进行行初等变换，等价于用一系列初等矩阵左乘该方程组，从而求解。

4.8.6 历史脉络与应用一瞥

历史侧记

矩阵的思想源远流长。其萌芽可追溯至中国古代《九章算术》中的“方程术”，用矩形阵列表示线性方程组并进行消元。19 世纪，英国数学家阿瑟·凯莱 (Arthur Cayley) 系统提出了矩阵的概念及其代数运算规则 (1858 年)。他与詹姆斯·西尔维斯特 (James Sylvester) 共同奠定了矩阵理论的基础。行列式的发展则更早，源于关孝和、莱布尼茨等人对线性方程组解的研究。20 世纪以来，矩阵理论在量子力学、计算机科学等领域发挥了巨大作用。

例题 4.5 矩阵的现代应用掠影 矩阵是描述复杂结构与变换的通用语言。

- **计算机图形学**：3D 变换（旋转、缩放、平移）均可用矩阵表示。物体顶点的坐标变换通过矩阵乘法实现。
- **数据分析与机器学习**：数据集常表示为矩阵（行样本，列特征）。主成分分析 (PCA)、奇异值分解 (SVD) 等核心算法 heavily rely on matrix operations.
- **经济学与投入产出分析**：里昂惕夫投入产出模型用矩阵描述各部门间的产品流向与需求关系。
- **网页排序与 PageRank**：将互联网超链接结构表示为矩阵，通过计算矩阵的主特征向量为网页排序。

4.8.7 矩阵与行列式：概念对照

尽管矩阵与行列式在形式上有关联（都涉及数表），但它们的数学本质、运算和用途截然不同。

特征	矩阵	行列式
数学本质	线性变换的数表/算子	方阵的标量函数（体积缩放因子）
对象范围	$m \times n$ （任意形状）	仅对 $n \times n$ 方阵定义
核心运算	加法、数乘、乘法（非交换）	按行（列）展开，结果为数值
相等性	同型且对应元素相等	数值相等即相等
初等变换	行/列变换，秩保持	行变换改变值，用于求值
典型用途	表示变换、系统建模、数据存储	判断可逆性、计算体积、解方程组（克莱姆法则）

几何视角

为何行列式仅对方阵定义？行列式的几何意义是线性变换对空间体积的**缩放因子**。只有从 $\mathbb{R}^n$ 到 $\mathbb{R}^n$ 的线性变换（对应方阵）才谈得上对原始空间（ n 维）体积的缩放。对于非方阵 $\mathbf{A}_{m \times n}$ ($m \neq n$)，其变换 $\mathbb{R}^n \rightarrow \mathbb{R}^m$ 涉及维度变化，无法定义统一的体积缩放比例。

4.8.8 小结

- 矩阵是组织多维数据、表达线性关系的强大数学工具，其概念源于解决实际问题的需求。
- 掌握矩阵的严格定义（数表）、表示法（元素、下标）、基本类型（方阵、向量、零矩阵、单位矩阵、对角矩阵、三角矩阵等）及其性质。
- 熟练掌握矩阵的线性运算（加、数乘）、乘法（规则、性质、注意事项）和转置运算。
- 理解矩阵相等的概念。
- 明晰矩阵与行列式的根本区别：矩阵是**对象**，行列式是方阵的**数值函数**。
- 了解矩阵深厚的历史背景及其在现代科技中的广泛应用。

4.8.9 练习

 **练习 4.1** 将下列线性方程组表示为矩阵形式 $\mathbf{Ax} = \mathbf{b}$ ，并写出其增广矩阵 $[\mathbf{A} \mid \mathbf{b}]$ ：

$$\begin{cases} 2x_1 - x_2 + 3x_3 = 1 \\ -x_1 + 4x_2 + 2x_3 = 0 \\ x_1 + 2x_2 - x_3 = 5 \end{cases}$$

 **练习 4.2** 给定有向图顶点集 $V = \{1, 2, 3, 4\}$ ，弧集为 $\{1 \rightarrow 2, 2 \rightarrow 1, 3 \rightarrow 4, 4 \rightarrow 3\}$ 。

1. 写出该有向图的邻接矩阵 $\mathbf{M}$ 。
2. 计算 $\mathbf{M}^2$ 。
3. 解释 $\mathbf{M}^2$ 中 $(1, 1)$ 元和 $(1, 2)$ 元的含义。

 **练习 4.3** 判断下列命题的真伪，并简要说明理由：

1. 若矩阵 $\mathbf{A}$ 和 $\mathbf{B}$ 满足 $\mathbf{AB}$ 可乘，则 $\mathbf{BA}$ 必定也可乘。
2. 若 $\mathbf{A}$ 是上三角矩阵，则 $\det(\mathbf{A})$ 等于其主对角线元素的乘积。
3. 对任意矩阵 $\mathbf{A} \in \mathbb{R}^{m \times n}$ ，乘积 $\mathbf{A}^\top \mathbf{A}$ 总是方阵。

 **练习 4.4 计算与理解** 设对角矩阵 $\mathbf{D} = \text{diag}(1, 2, 3)$ ，置换矩阵 $\mathbf{P} = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}$ 。

1. 计算 $\mathbf{P}^\top \mathbf{D} \mathbf{P}$ 。
2. 结合 $\mathbf{P}$ 的作用（它交换了坐标系的哪两个轴？），解释计算结果 $\mathbf{P}^\top \mathbf{D} \mathbf{P}$ 的意义。

代码视角

可选：NumPy 验证

```
import numpy as np

# 练习2：邻接矩阵与幂
M = np.array([[0, 1, 0, 0],
              [1, 0, 0, 0],
              [0, 0, 0, 1],
              [0, 0, 1, 0]], dtype=int)

M_squared = M @ M
print("M^2:\n", M_squared)

# 练习4：置换相似
D = np.diag([1, 2, 3])
P = np.array([[0, 1, 0],
              [1, 0, 0],
              [0, 0, 1]])

result = P.T @ D @ P
print("P^T D P:\n", result)
```

4.9 矩阵的代数运算

要点提炼

本节目标

- 掌握矩阵的线性运算（加法、数乘）与乘法的定义与成立条件；
- 熟悉基本代数性质与常见“陷阱”（不交换、不可消去、零因子）；
- 了解矩阵多项式 $f(A)$ 的构造与性质；
- 会用行列式乘法性 $\det(AB) = \det(A)\det(B)$ ；
- 能处理若干典型演算与证明题（如 $(AB)^2 = E$ 的充要条件、Jordan 块幂等）。

4.9.1 线性运算：加法与数乘

矩阵的线性运算是矩阵代数中最基本且直观的运算，包括加法和数乘，它们为矩阵提供了线性结构。

定义 4.5 (矩阵加法)

设 $\mathbf{A} = (a_{ij})_{m \times n}$ 与 $\mathbf{B} = (b_{ij})_{m \times n}$ 是两个同型矩阵（即行数和列数分别相同），则它们的和 $\mathbf{A} + \mathbf{B}$ 定义为对应元素相加：

$$\mathbf{A} + \mathbf{B} = (a_{ij} + b_{ij})_{m \times n}.$$



注 矩阵加法要求两个矩阵必须是同型的。矩阵的减法可定义为 $\mathbf{A} - \mathbf{B} = \mathbf{A} + (-\mathbf{B})$ ，其中 $-\mathbf{B}$ 是 $\mathbf{B}$ 的负矩阵。

定义 4.6 (数乘)

给定标量 $\lambda \in \mathbb{F}$ （数域），矩阵 $\mathbf{A} = (a_{ij})_{m \times n}$ 与 λ 的数乘定义为：

$$\lambda \mathbf{A} = (\lambda a_{ij})_{m \times n}.$$

特别地，若 $\mathbf{E} = E_n$ 是 n 阶单位矩阵，则 $\lambda \mathbf{E}$ 称为数量矩阵。



定理 4.1 (线性运算的代数性质)

矩阵的加法和数乘运算 (合称线性运算) 满足以下规律: 对任意同型矩阵 $\mathbf{A}, \mathbf{B}, \mathbf{C}$ 及标量 $\lambda, \mu \in \mathbb{F}$, 有

$$\mathbf{A} + \mathbf{B} = \mathbf{B} + \mathbf{A} \quad (\text{加法交换律})$$

$$(\mathbf{A} + \mathbf{B}) + \mathbf{C} = \mathbf{A} + (\mathbf{B} + \mathbf{C}) \quad (\text{加法结合律})$$

$$\mathbf{A} + \mathbf{0} = \mathbf{A} \quad (\text{零矩阵性质})$$

$$\mathbf{A} + (-\mathbf{A}) = \mathbf{0}$$

$$\lambda(\mathbf{A} + \mathbf{B}) = \lambda\mathbf{A} + \lambda\mathbf{B} \quad (\text{数乘对加法的分配律})$$

$$(\lambda + \mu)\mathbf{A} = \lambda\mathbf{A} + \mu\mathbf{A}$$

$$\lambda(\mu\mathbf{A}) = (\lambda\mu)\mathbf{A} \quad (\text{数乘结合律})$$

**例题 4.6**

$$\begin{pmatrix} 12 & 3 & 5 \\ 1 & 9 & 0 \\ 3 & 6 & 8 \end{pmatrix} + \begin{pmatrix} 1 & 8 & -9 \\ 6 & -5 & 4 \\ -1 & 2 & 1 \end{pmatrix} = \begin{pmatrix} 13 & 11 & -4 \\ 7 & 4 & 4 \\ 2 & 8 & 9 \end{pmatrix}.$$

定理 4.2 (方阵的数乘与行列式)

若 $\mathbf{A}$ 为 n 阶方阵、 $k \in \mathbb{F}$, 则

$$\det(k\mathbf{A}) = k^n \det(\mathbf{A}).$$

**定义 4.7 (线性组合)**

设 $\mathbf{A}_1, \dots, \mathbf{A}_m$ 是一组同型矩阵, $\lambda_1, \dots, \lambda_m \in \mathbb{F}$, 则称

$$\mathbf{B} = \sum_{j=1}^m \lambda_j \mathbf{A}_j$$

为矩阵 $\mathbf{A}_1, \dots, \mathbf{A}_m$ 的一个线性组合。



例题 4.7 二阶矩阵的基分解 所有 2×2 矩阵构成的集合是一个线性空间。其一组自然的基为: $\mathbf{M}_1 = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}, \mathbf{M}_2 = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}, \mathbf{M}_3 = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}, \mathbf{M}_4 = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}$. 任意二阶矩阵 $\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}$ 都可唯一地表示为该基的线性组合: $\mathbf{A} = a_{11}\mathbf{M}_1 + a_{12}\mathbf{M}_2 + a_{21}\mathbf{M}_3 + a_{22}\mathbf{M}_4$.

例题 4.8 设 $\mathbf{A} = \begin{pmatrix} 3 & 2 \\ 4 & 1 \\ 1 & 5 \end{pmatrix}, \mathbf{B} = \begin{pmatrix} 1 & 3 \\ 2 & 4 \\ 3 & 1 \end{pmatrix}$, 若 $\mathbf{A} + 3\mathbf{X} = 2\mathbf{B}$, 求解 $\mathbf{X}$.

解:

$$3\mathbf{X} = 2\mathbf{B} - \mathbf{A}$$

$$\begin{aligned}\mathbf{X} &= \frac{1}{3}(2\mathbf{B} - \mathbf{A}) = \frac{1}{3} \left(2 \begin{pmatrix} 1 & 3 \\ 2 & 4 \\ 3 & 1 \end{pmatrix} - \begin{pmatrix} 3 & 2 \\ 4 & 1 \\ 1 & 5 \end{pmatrix} \right) \\ &= \frac{1}{3} \begin{pmatrix} 2-3 & 6-2 \\ 4-4 & 8-1 \\ 6-1 & 2-5 \end{pmatrix} = \frac{1}{3} \begin{pmatrix} -1 & 4 \\ 0 & 7 \\ 5 & -3 \end{pmatrix}.\end{aligned}$$

4.9.2 矩阵乘法

矩阵乘法是矩阵代数中核心且功能强大的运算，它并非元素的对应相乘，而是行与列的内积，这使得它能简洁地表示线性变换的复合、线性方程组等。

定义 4.8 (矩阵乘法)

设 $\mathbf{A} = (a_{ik})_{m \times n} \in \mathbb{F}^{m \times n}$ 、 $\mathbf{B} = (b_{kj})_{n \times s} \in \mathbb{F}^{n \times s}$ ，则定义它们的乘积 $\mathbf{C} = \mathbf{AB}$ 为一个 $m \times s$ 矩阵，其 (i, j) 元为：

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj} \quad (1 \leq i \leq m, 1 \leq j \leq s).$$

即， c_{ij} 是 $\mathbf{A}$ 的第 i 行与 $\mathbf{B}$ 的第 j 列对应元素的乘积之和。



注 矩阵乘法成立的维度条件至关重要：左矩阵的列数必须等于右矩阵的行数。乘积矩阵的行数等于左矩阵的行数，列数等于右矩阵的列数。

例题 4.9 计算乘积 设

$$\mathbf{A} = \begin{pmatrix} 1 & 0 & 1 & 2 \\ -1 & -1 & 3 & 0 \\ 0 & 5 & 1 & -4 \end{pmatrix}_{3 \times 4}, \quad \mathbf{B} = \begin{pmatrix} -1 & 2 & 1 \\ 3 & -1 & 1 \\ -1 & 2 & 1 \\ 0 & 3 & 4 \end{pmatrix}_{4 \times 3}.$$

则它们的乘积 $\mathbf{C} = \mathbf{AB}$ 是一个 3×3 矩阵：

$$\mathbf{C} = \begin{pmatrix} (1)(-1) + (0)(3) + (1)(-1) + (2)(0) & (1)(2) + (0)(-1) + (1)(2) + (2)(3) & (1)(1) + (0)(1) + (1)(1) + (2)(4) \\ (-1)(-1) + (-1)(3) + (3)(-1) + (0)(0) & (-1)(2) + (-1)(-1) + (3)(2) + (0)(3) & (-1)(1) + (-1)(1) + (3)(1) + (0)(4) \\ (0)(-1) + (5)(3) + (1)(-1) + (-4)(0) & (0)(2) + (5)(-1) + (1)(2) + (-4)(3) & (0)(1) + (5)(1) + (-4)(4) \end{pmatrix}.$$

例题 4.10 行向量与列向量

- **内积**：若 $\mathbf{a} = [a_1, \dots, a_n] \in \mathbb{F}^{1 \times n}$ (行向量)， $\mathbf{b} = [b_1, \dots, b_n]^T \in \mathbb{F}^{n \times 1}$ (列向量)，则它们的乘积 $\mathbf{ab}$ 是一个标量 (内积)：

$$\mathbf{ab} = \sum_{i=1}^n a_i b_i \in \mathbb{F}.$$

- **外积**：反之， $\mathbf{ba}$ 是一个 $n \times n$ 矩阵（外积）：

$$\mathbf{ba} = (b_i a_j)_{n \times n}.$$

外积矩阵的秩至多为 1。

例题 4.11 线性方程组的矩阵形式 令 $m \times n$ 矩阵 $\mathbf{A} = (a_{ij})$ 为方程组的系数矩阵， $n \times 1$ 列向量 $\mathbf{x} = (x_j)$ 为未知数向量， $m \times 1$ 列向量 $\mathbf{b} = (b_i)$ 为常数项向量。则线性方程组

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2 \\ \vdots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n = b_m \end{cases}$$

可简洁地表示为矩阵方程：

$$\mathbf{Ax} = \mathbf{b}.$$

定理 4.3 (乘法的基本代数律)

矩阵乘法满足以下运算律（假设运算在维度上可行， λ 为标量）：

$$(\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC}) \quad (\text{结合律})$$

$$\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{AB} + \mathbf{AC} \quad (\text{左分配律})$$

$$(\mathbf{B} + \mathbf{C})\mathbf{A} = \mathbf{BA} + \mathbf{CA} \quad (\text{右分配律})$$

$$\lambda(\mathbf{AB}) = (\lambda\mathbf{A})\mathbf{B} = \mathbf{A}(\lambda\mathbf{B})$$

$$\mathbf{E}_m \mathbf{A} = \mathbf{A}, \quad \mathbf{A} \mathbf{E}_n = \mathbf{A} \quad (\text{单位矩阵的恒等性})$$

对于 n 阶方阵 $\mathbf{A}$ ，可定义其幂运算： $\mathbf{A}^k = \underbrace{\mathbf{A} \cdots \mathbf{A}}_{k \text{ 次}}$ (k 为正整数)，并约定 $\mathbf{A}^0 = \mathbf{E}_n$ 。

幂运算满足：

$$\mathbf{A}^m \mathbf{A}^k = \mathbf{A}^{m+k}, \quad (\mathbf{A}^m)^k = \mathbf{A}^{mk}.$$



注意

三大“陷阱”矩阵乘法与数的乘法有显著区别，需特别注意以下陷阱：

- **不满足交换律**：一般情况下 $\mathbf{AB} \neq \mathbf{BA}$ 。例如：设 $\mathbf{A} = \begin{pmatrix} 1 & 1 \\ -1 & -1 \end{pmatrix}$ ， $\mathbf{B} = \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}$ ，则 $\mathbf{AB} = \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix} = \mathbf{0}$ ，而 $\mathbf{BA} = \begin{pmatrix} 2 & 2 \\ -2 & -2 \end{pmatrix} \neq \mathbf{0}$ 。
- **存在零因子**： $\mathbf{AB} = \mathbf{0}$ 不能推出 $\mathbf{A} = \mathbf{0}$ 或 $\mathbf{B} = \mathbf{0}$ （见上例）。
- **不满足消去律**：由 $\mathbf{AB} = \mathbf{AC}$ 且 $\mathbf{A} \neq \mathbf{0}$ ，不能推出 $\mathbf{B} = \mathbf{C}$ ；同理，由 $\mathbf{BA} = \mathbf{CA}$ 且 $\mathbf{A} \neq \mathbf{0}$ ，也不能推出 $\mathbf{B} = \mathbf{C}$ 。

历史侧记

矩阵乘法的现代定义由英国数学家阿瑟·凯莱 (Arthur Cayley) 于 1858 年在其论文《矩阵理论备忘录》中系统提出并阐述。虽然矩阵加法的思想可追溯至更早，但凯莱的工作确立了矩阵乘法的运算规则及其与线性变换复合的联系，奠定了矩阵理论的基础。

代码视角

[Python/NumPy 中的矩阵乘法] 在 NumPy 中，使用 '@' 运算符或 'np.dot()' 函数进行矩阵乘法，这与 '*' 表示的逐元素乘法不同。

```
import numpy as np

A = np.array([[1, 0, 1, 2],
              [-1, -1, 3, 0],
              [0, 5, 1, -4]])
B = np.array([[-1, 2, 1],
              [3, -1, 1],
              [-1, 2, 1],
              [0, 3, 4]])

# 矩阵乘法
C = A @ B # 或 np.dot(A, B)
print("Matrix product A @ B:\n", C)

# 对比逐元素乘法（会出错，因为不同型）
# D = A * B # 这将引发错误，或得到非预期结果
```

4.9.3 方阵的多项式与行列式乘法性

当矩阵是方阵时，我们可以定义其多项式函数，并且行列式具有一个极其重要的性质：乘法性。

定义 4.9 (矩阵多项式)

设 $\mathbf{A}$ 是 n 阶方阵， $f(\lambda) = a_m \lambda^m + a_{m-1} \lambda^{m-1} + \cdots + a_1 \lambda + a_0$ 是一个一元多项式，则定义矩阵多项式：

$$f(\mathbf{A}) = a_m \mathbf{A}^m + a_{m-1} \mathbf{A}^{m-1} + \cdots + a_1 \mathbf{A} + a_0 E_n.$$



命题 4.2 (多项式代数)

设 $f(\lambda), g(\lambda)$ 是多项式, $h(\lambda) = f(\lambda) + g(\lambda)$, $s(\lambda) = f(\lambda)g(\lambda)$ 。则有:

$$h(\mathbf{A}) = f(\mathbf{A}) + g(\mathbf{A}), \quad s(\mathbf{A}) = f(\mathbf{A})g(\mathbf{A}) = g(\mathbf{A})f(\mathbf{A}).$$

注意: 由于矩阵乘法不可交换, 一般情况下降法不成立, 但多项式在同一个矩阵上的乘法总是可交换的。

**定理 4.4 (行列式的乘法性)**

若 $\mathbf{A}, \mathbf{B}$ 为同阶方阵, 则它们的乘积的行列式满足:

$$\det(\mathbf{AB}) = \det(\mathbf{A}) \det(\mathbf{B}).$$



证明 [证明思路 (分块矩阵法)] 构造分块矩阵:

$$\mathbf{H} = \begin{pmatrix} \mathbf{A} & \mathbf{0} \\ -\mathbf{E} & \mathbf{B} \end{pmatrix}.$$

对 $\mathbf{H}$ 进行列变换: 将第 1 块列乘以 $\mathbf{B}$ 加到第 2 块列上, 得到:

$$\mathbf{H}' = \begin{pmatrix} \mathbf{A} & \mathbf{AB} \\ -\mathbf{E} & \mathbf{0} \end{pmatrix}.$$

由于列变换不改变行列式的值, 故 $\det(\mathbf{H}) = \det(\mathbf{H}')$ 。另一方面, $\mathbf{H}$ 是分块下三角矩阵, $\mathbf{H}'$ 是分块上三角矩阵, 它们的行列式均可表示为分块对角元行列式之积:

$$\det(\mathbf{H}) = \det(\mathbf{A}) \det(\mathbf{B}),$$

$$\det(\mathbf{H}') = \det(\mathbf{AB}) \det(-\mathbf{E}) = \det(\mathbf{AB})(-1)^n \det(\mathbf{E}) = (-1)^n \det(\mathbf{AB}).$$

因此有 $\det(\mathbf{A}) \det(\mathbf{B}) = (-1)^n \det(\mathbf{AB})$ 。仔细观察最初的构造和变换, 或者考虑在 $\mathbf{H}$ 的行左边乘上 $\begin{pmatrix} \mathbf{E} & \mathbf{0} \\ \mathbf{0} & -\mathbf{E} \end{pmatrix}$ (其行列式为 $(-1)^n$) 并调整证明, 亦可直接得到 $\det(\mathbf{AB}) = \det(\mathbf{A}) \det(\mathbf{B})$ 。更标准的证明可利用行列式对行的多重线性性和交错性。

注 行列式乘法性是行列式理论的一个核心结论。它意味着: 即使矩阵乘法不可交换, 但矩阵乘积的行列式却等于行列式的乘积 (这是一个可交换的标量运算)。这也说明, 即使 $\mathbf{AB} \neq \mathbf{BA}$, 但 $\det(\mathbf{AB}) = \det(\mathbf{BA})$ 总是成立。

4.9.4 典型证明与演算

本部分通过几个典型例子展示矩阵运算在证明和计算中的应用。

定理 4.5

设方阵 $\mathbf{A}, \mathbf{B}$ 满足 $\mathbf{A}^2 = \mathbf{E}$ (对合), $\mathbf{B}^2 = \mathbf{E}$ 。则

$$(\mathbf{AB})^2 = \mathbf{E} \iff \mathbf{AB} = \mathbf{BA}.$$



证明 “ $\Leftarrow$ ”: 若 $AB = BA$, 则

$$(AB)^2 = ABAB = A(BA)B = A(AB)B = A^2B^2 = EE = E.$$

“ $\Rightarrow$ ”: 若 $(AB)^2 = E$, 则

$$AB = (AB)^{-1} = B^{-1}A^{-1}.$$

由于 $A^2 = E$ 且 $B^2 = E$, 故 $A^{-1} = A$, $B^{-1} = B$ (对合矩阵的逆等于自身)。代入上式得:

$$AB = B^{-1}A^{-1} = BA.$$

定理 4.6

若方阵 A, B 满足 $AB + BA = E$ 且 $A^2 = 0$ (或 $B^2 = 0$), 则 AB 是幂等矩阵, 即

$$(AB)^2 = AB.$$



证明 已知 $AB + BA = E$ 且 $A^2 = 0$ 。左乘 A : $A(AB + BA) = AE$, 得 $A^2B + ABA = A$ 。由于 $A^2 = 0$, 故 $ABA = A$ 。右乘 B : $(ABA)B = AB$, 即 $AB(AB) = AB$ 。因此 $(AB)^2 = AB$ 。

例题 4.12“全 -1”矩阵的幂 设 A 为 4×4 矩阵, 且所有元素均为 -1 。令 J 为全 1 的 4×4 矩阵, 则 $A = -J$ 。已知 $J^2 = 4J$ (因为每行每列求和再相乘)。则

$$A^2 = (-J)^2 = J^2 = 4J = -4(-J) = -4A.$$

这是一个递推关系: $A^2 = -4A$ 。两边左乘 A : $A^3 = -4A^2 = (-4)^2A$ 。以此类推, 可得通项公式:

$$A^k = (-4)^{k-1}A \quad (k \geq 1).$$

例题 4.13 Jordan 块的幂 Jordan 块在矩阵理论中非常重要。考虑一个 3×3 的 Jordan 块:

$$A = \begin{pmatrix} \lambda & 1 & 0 \\ 0 & \lambda & 1 \\ 0 & 0 & \lambda \end{pmatrix}. \text{ 它可以分解为 } A = \lambda E + N, \text{ 其中 } N = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix} \text{ 是一个幂零矩阵,}$$

$$\text{满足 } N^3 = 0, \text{ 且 } N^2 = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}. \text{ 由于 } \lambda E \text{ 与 } N \text{ 可交换 (数量矩阵与任何矩阵可交)}$$

换), 故可用二项式定理展开 $\mathbf{A}^k = (\lambda \mathbf{E} + \mathbf{N})^k$:

$$\begin{aligned}\mathbf{A}^k &= \sum_{i=0}^k \binom{k}{i} (\lambda \mathbf{E})^{k-i} \mathbf{N}^i = \sum_{i=0}^k \binom{k}{i} \lambda^{k-i} \mathbf{N}^i \\&= \binom{k}{0} \lambda^k \mathbf{N}^0 + \binom{k}{1} \lambda^{k-1} \mathbf{N}^1 + \binom{k}{2} \lambda^{k-2} \mathbf{N}^2 \quad (\text{因为 } \mathbf{N}^3 = \mathbf{0}) \\&= \lambda^k \mathbf{E} + k \lambda^{k-1} \mathbf{N} + \frac{k(k-1)}{2} \lambda^{k-2} \mathbf{N}^2 \\&= \begin{pmatrix} \lambda^k & k \lambda^{k-1} & \frac{k(k-1)}{2} \lambda^{k-2} \\ 0 & \lambda^k & k \lambda^{k-1} \\ 0 & 0 & \lambda^k \end{pmatrix} \quad (k \geq 2).\end{aligned}$$

4.9.5 小结

- **线性运算**: 加法要求矩阵同型, 对应元素相加; 数乘是标量乘以每个元素。
- **矩阵乘法**: 核心运算是行与列的内积。需满足左列数等于右行数。牢记其结合律、分配律, 但不满足交换律、存在零因子、不满足消去律。
- **矩阵多项式**: 将多项式中的变量替换为方阵, 常数项替换为常数倍单位矩阵。同一矩阵上的多项式乘法可交换。
- **行列式乘法性**: $\det(\mathbf{AB}) = \det(\mathbf{A}) \det(\mathbf{B})$ 是方阵行列式的核心性质之一。
- **特殊矩阵的幂**: 对于像 Jordan 块或特殊结构矩阵 (如全 ± 1 矩阵), 可利用分解和二项式定理等方法求幂。

4.9.6 综合训练

4.9.6.0.1 判断题 (对/错)

1. $(\mathbf{A} + \mathbf{B})(\mathbf{A} - \mathbf{B}) = \mathbf{A}^2 - \mathbf{B}^2$ 。(错, 除非 $\mathbf{A}$ 与 $\mathbf{B}$ 可交换)
2. $(\mathbf{AB})^2 = \mathbf{A}^2 \mathbf{B}^2$ 。(错, 除非 $\mathbf{A}$ 与 $\mathbf{B}$ 可交换)
3. 设 $\mathbf{A} = \begin{pmatrix} a & 1 \\ 1 & a \end{pmatrix}$, 则对任意二阶 $\mathbf{B}$ 有 $\mathbf{AB} = \mathbf{BA}$ 。(错, 反例易寻)
4. 若 $\mathbf{A} = a\mathbf{E}$ (数量矩阵), 则对任意同阶方阵 $\mathbf{B}$ 有 $\mathbf{AB} = \mathbf{BA}$ 。(对, 数量矩阵与任何矩阵可交换)
5. 若 $\det \mathbf{A} = \det \mathbf{B}$, 则 $\mathbf{A} = \mathbf{B}$ 。(错, 行列式相等不能推出矩阵相等)
6. 若 $\mathbf{A}^2 = \mathbf{0}$, 则 $\mathbf{A} = \mathbf{0}$ 。(错, 存在非零幂零矩阵, 如 $\mathbf{N} = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$)

4.9.6.0.2 计算题 **练习 4.5** 根据例 4.9 中的矩阵 $\mathbf{A}$ 和 $\mathbf{B}$, 详细计算 $\mathbf{C} = \mathbf{AB}$ 的每个元素 c_{ij} , 明确写出其作为 $\mathbf{A}$ 的第 i 行与 $\mathbf{B}$ 的第 j 列的内积形式。

 **练习 4.6** 设 $\mathbf{A}$ 为 4×4 的全 -1 矩阵 (见例 4.12)。利用数学归纳法证明: $\mathbf{A}^k = (-4)^{k-1} \mathbf{A}$ 对任意正整数 k 成立。

 **练习 4.7** 设 $A = \frac{1}{2}(B + E)$, 其中 B 为 n 阶方阵, E 为 n 阶单位矩阵。证明: $A^2 = A$ 当且仅当 $B^2 = E$ 。

 **练习 4.8** 设 A 为 3×3 上 Jordan 块 $\begin{pmatrix} \lambda & 1 & 0 \\ 0 & \lambda & 1 \\ 0 & 0 & \lambda \end{pmatrix}$ 。使用数学归纳法证明例 4.13 中给出的幂公式对于所有 $k \geq 1$ 成立。

4.9.6.0.3 证明题  **练习 4.9** 详细证明定理 4.5 和定理 4.6。

代码视角

可选: NumPy 验证

```
import numpy as np

# 验证 Jordan 块幂公式
lambda_val = 2
k = 5
# 构造 Jordan 块 A 和幂零矩阵 N
N = np.array([[0,1,0],
               [0,0,1],
               [0,0,0]])
A = lambda_val * np.eye(3) + N

# 计算 A^k using NumPy
A_k = np.linalg.matrix_power(A, k)
print("A^5 (NumPy):\n", A_k)

# 根据公式计算
A_k_formula = (lambda_val**k) * np.eye(3) + \
               k*(lambda_val**(k-1)) * N + \
               (k*(k-1)//2)*(lambda_val**(k-2)) * (N @ N)
print("A^5 (Formula):\n", A_k_formula)

# 验证全 -1 矩阵的幂
n = 4
A_all_neg_one = -np.ones((n, n))
k = 3
# A^k 应该等于 (-4)^(k-1) * A
```



```
A_k_calculated = ((-4)**(k-1)) * A_all_neg_one
A_k_actual = np.linalg.matrix_power(A_all_neg_one, k)
print("A^3 (Formula):\n", A_k_calculated)
print("A^3 (Actual):\n", A_k_actual)
```

4.10 逆矩阵与矩阵的初等变换

要点提炼

本节核心要点

- **逆矩阵**：定义、唯一性、性质与计算方法（伴随矩阵法、初等变换法）
- **初等变换**：三类行/列变换、初等矩阵、矩阵等价关系
- **Gauss-Jordan 消元法**：求逆矩阵的高效算法与实现
- **应用连接**：逆矩阵与线性方程组求解（Cramer 法则）、矩阵分解

本节分为两部分：§ 4.10.1 系统介绍逆矩阵的概念、性质与求法；§ 4.10.2 深入讲解矩阵的初等变换、初等矩阵及其在求逆与化简中的关键作用。

4.10.1 逆矩阵

历史侧记

逆矩阵的概念在数学发展中逐步形成。19 世纪，英国数学家阿瑟·凯莱（Arthur Cayley）在创建矩阵理论时，首次系统提出了逆矩阵的定义和性质。伴随矩阵的概念则与行列式的展开密切相关，其公式为求解低阶矩阵的逆提供了显式方法。然而，随着数值计算的发展，伴随矩阵法因计算复杂度高（ $O(n!)$ ）而逐渐被更高效的初等变换法（Gauss-Jordan 消元法，复杂度 $O(n^3)$ ）所替代。

4.10.1.1 定义与唯一性

定义 4.10 (逆矩阵)

设 A 为 n 阶方阵。若存在 n 阶方阵 B 使得

$$AB = BA = E,$$

则称 A 可逆， B 称为 A 的逆矩阵，记作 $B = A^{-1}$ 。

定理 4.7 (唯一性)

若方阵 A 可逆，则其逆矩阵唯一。

证明 假设 B 和 C 都是 A 的逆矩阵，则

$$B = BE = B(AC) = (BA)C = EC = C.$$

故逆矩阵唯一。

几何视角

从几何角度看, 可逆矩阵 A 对应的线性变换是**可逆**的 (即一一对应且满射)。其逆矩阵 A^{-1} 对应了该线性变换的**逆变换**。行列式 $\det(A)$ 可理解为该变换对空间的缩放因子; $\det(A) \neq 0$ 意味着变换不降维, 从而可逆。

4.10.1.2 逆矩阵的性质

要点提炼

设 A, B 为 n 阶可逆矩阵, λ 为非零标量, 则:

1. $(A^{-1})^{-1} = A$
2. $(\lambda A)^{-1} = \frac{1}{\lambda} A^{-1}$
3. $(AB)^{-1} = B^{-1} A^{-1}$ (穿脱原则)
4. $(A^T)^{-1} = (A^{-1})^T$
5. $\det(A^{-1}) = \frac{1}{\det(A)}$

证明 [性质 (4) 证明] 由 $(A^{-1})^T A^T = (AA^{-1})^T = E^T = E$, 故 $(A^T)^{-1} = (A^{-1})^T$ 。

4.10.1.3 伴随矩阵法求逆

定义 4.11 (伴随矩阵)

设 $A = (a_{ij})_{n \times n}$, A_{ij} 为元素 a_{ij} 的代数余子式。定义 A 的伴随矩阵为

$$\text{adj}(A) = A^* = (A_{ij})^T = \begin{pmatrix} A_{11} & A_{21} & \cdots & A_{n1} \\ A_{12} & A_{22} & \cdots & A_{n2} \\ \vdots & \vdots & \ddots & \vdots \\ A_{1n} & A_{2n} & \cdots & A_{nn} \end{pmatrix}.$$



定理 4.8 (伴随矩阵公式)

设 A 为 n 阶方阵, 则

$$A \cdot \text{adj}(A) = \text{adj}(A) \cdot A = \det(A)E.$$

因此, 若 $\det(A) \neq 0$, 则 A 可逆, 且

$$A^{-1} = \frac{1}{\det(A)} \text{adj}(A).$$



例题 4.14 伴随矩阵法求逆 设

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 2 & 1 \\ 3 & 4 & 3 \end{pmatrix}.$$

1. 计算行列式: $\det(A) = 1 \cdot (2 \cdot 3 - 1 \cdot 4) - 2 \cdot (2 \cdot 3 - 1 \cdot 3) + 3 \cdot (2 \cdot 4 - 2 \cdot 3) = 1 \cdot 2 - 2 \cdot 3 + 3 \cdot 2 = 2 - 6 + 6 = 2 \neq 0$, 故 A 可逆。

2. 计算代数余子式:

$$\begin{aligned} A_{11} &= + \begin{vmatrix} 2 & 1 \\ 4 & 3 \end{vmatrix} = 2, & A_{12} &= - \begin{vmatrix} 2 & 1 \\ 3 & 3 \end{vmatrix} = -3, & A_{13} &= + \begin{vmatrix} 2 & 2 \\ 3 & 4 \end{vmatrix} = 2, \\ A_{21} &= - \begin{vmatrix} 2 & 3 \\ 4 & 3 \end{vmatrix} = 6, & A_{22} &= + \begin{vmatrix} 1 & 3 \\ 3 & 3 \end{vmatrix} = -6, & A_{23} &= - \begin{vmatrix} 1 & 2 \\ 3 & 4 \end{vmatrix} = 2, \\ A_{31} &= + \begin{vmatrix} 2 & 3 \\ 2 & 1 \end{vmatrix} = -4, & A_{32} &= - \begin{vmatrix} 1 & 3 \\ 2 & 1 \end{vmatrix} = 5, & A_{33} &= + \begin{vmatrix} 1 & 2 \\ 2 & 2 \end{vmatrix} = -2. \end{aligned}$$

3. 构造伴随矩阵:

$$\text{adj}(A) = \begin{pmatrix} A_{11} & A_{21} & A_{31} \\ A_{12} & A_{22} & A_{32} \\ A_{13} & A_{23} & A_{33} \end{pmatrix} = \begin{pmatrix} 2 & 6 & -4 \\ -3 & -6 & 5 \\ 2 & 2 & -2 \end{pmatrix}.$$

4. 求逆矩阵:

$$A^{-1} = \frac{1}{\det(A)} \text{adj}(A) = \frac{1}{2} \begin{pmatrix} 2 & 6 & -4 \\ -3 & -6 & 5 \\ 2 & 2 & -2 \end{pmatrix} = \begin{pmatrix} 1 & 3 & -2 \\ -\frac{3}{2} & -3 & \frac{5}{2} \\ 1 & 1 & -1 \end{pmatrix}.$$

注意

伴随矩阵法在理论推导中很优美, 但对于高阶矩阵 ($n \geq 4$), 其计算量巨大 (需计算 n^2 个代数余子式)。实际应用中, 多采用数值稳定的初等变换法 (Gauss-Jordan 消元法)。

4.10.1.4 逆矩阵与线性方程组

逆矩阵与线性方程组的求解密切相关。

定理 4.9 (矩阵方程的解)

设 A 为 n 阶可逆矩阵, B 为 $n \times m$ 矩阵, 则矩阵方程 $AX = B$ 有唯一解 $X = A^{-1}B$ 。♡

证明 由 A 可逆, 左乘 A^{-1} 得 $X = A^{-1}B$ 。代入验证: $A(A^{-1}B) = (AA^{-1})B = EB = B$ 。

定理 4.10 (Cramer 法则)

设 n 元线性方程组 $AX = b$, 若 $\det(A) \neq 0$, 则方程组有唯一解

$$x_j = \frac{D_j}{\det(A)} \quad (j = 1, 2, \dots, n),$$

其中 D_j 是将 A 的第 j 列替换为常数列 b 所得矩阵的行列式。♡

例题 4.15 Cramer 法则应用 解方程组

$$\begin{cases} x_1 + 2x_2 + 3x_3 = 1, \\ 2x_1 + 2x_2 + x_3 = 2, \\ 3x_1 + 4x_2 + 3x_3 = 3. \end{cases}$$

系数矩阵 A 同上例, $\det(A) = 2$ 。计算:

$$D_1 = \begin{vmatrix} 1 & 2 & 3 \\ 2 & 2 & 1 \\ 3 & 4 & 3 \end{vmatrix} = 2, \quad D_2 = \begin{vmatrix} 1 & 1 & 3 \\ 2 & 2 & 1 \\ 3 & 3 & 3 \end{vmatrix} = 0, \quad D_3 = \begin{vmatrix} 1 & 2 & 1 \\ 2 & 2 & 2 \\ 3 & 4 & 3 \end{vmatrix} = -2.$$

故解为 $x_1 = \frac{2}{2} = 1, x_2 = \frac{0}{2} = 0, x_3 = \frac{-2}{2} = -1$ 。

注 Cramer 法则虽形式简洁, 但计算量大 (需计算 $n+1$ 个 n 阶行列式), 实际解方程组多用高斯消元法。

4.10.1.5 特殊情形与反例

例题 4.16 待定系数法求逆 对于低阶或特殊矩阵, 有时可用待定系数法。设 $A = \begin{pmatrix} 1 & 0 \\ -2 & 1 \end{pmatrix}$, 求 A^{-1} 。

设 $A^{-1} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$, 由 $AA^{-1} = E$ 得:

$$\begin{pmatrix} 1 & 0 \\ -2 & 1 \end{pmatrix} \begin{pmatrix} a & b \\ c & d \end{pmatrix} = \begin{pmatrix} a & b \\ -2a+c & -2b+d \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

解方程组: $a = 1, b = 0, -2a + c = 0 \Rightarrow c = 2, -2b + d = 1 \Rightarrow d = 1$ 。故 $A^{-1} = \begin{pmatrix} 1 & 0 \\ 2 & 1 \end{pmatrix}$ 。

例题 4.17 多项式方程与可逆性 若矩阵 A 满足多项式方程 $A^2 - A - 2E = O$, 则

$$A(A - E) = 2E \Rightarrow A^{-1} = \frac{1}{2}(A - E).$$

但由 $(A - 2E)(A + E) = O$ 不能推出 $A + E$ 可逆。反例: 取 $A = -E$, 则 $A^2 - A - 2E = E + E - 2E = O$, 但 $A + E = O$ 不可逆。

实际上, $A + E$ 可逆当且仅当 -1 不是 A 的特征值。在满足 $A^2 - A - 2E = O$ 的前提下, A 的特征值只能是 2 或 -1 。若 $A + E$ 可逆, 则 -1 不是特征值, 故 A 的特征值全为 2 , 且由 Cayley-Hamilton 定理, $A = 2E$, 此时 $(A + E)^{-1} = \frac{1}{3}E$ 。

4.10.2 矩阵的初等变换

历史侧记

初等变换的思想源于高斯消元法。19 世纪，德国数学家卡尔·弗里德里希·高斯（Carl Friedrich Gauss）为简化线性方程组的求解，系统提出了消元操作。后经发展，形成现代的初等变换理论。初等矩阵的概念则完美连接了矩阵运算与变换，为矩阵分解和数值计算奠定了基础。

4.10.2.1 初等变换的定义与类型

定义 4.12 (初等行/列变换)

对矩阵的行（列）施行的以下三种变换称为初等行（列）变换：

1. 对换变换：交换两行（列），记作 $r_i \leftrightarrow r_j$ ($c_i \leftrightarrow c_j$)；
2. 倍乘变换：以非零常数 k 乘某一行（列），记作 $r_i \leftarrow kr_i$ ($c_i \leftarrow kc_i$)；
3. 倍加变换：将某一行（列）的 k 倍加到另一行（列），记作 $r_i \leftarrow r_i + kr_j$ ($c_i \leftarrow c_i + kc_j$)。

统称为初等变换。每种初等变换都是可逆的，且逆变换同类型。



注 初等变换是矩阵化简的核心操作。其几何意义对应着空间中的基本线性操作：对换（反射）、倍乘（缩放）、倍加（剪切）。

4.10.2.2 矩阵的等价与标准形

定义 4.13 (矩阵等价)

若矩阵 A 可经有限次初等变换化为 B ，则称 A 与 B 等价，记作 $A \cong B$ 。等价关系满足自反性、对称性和传递性。



定义 4.14 (行阶梯形与行最简形)

矩阵称为行阶梯形若：

1. 零行位于非零行下方；
2. 非零行的首非零元（主元）所在列随行增加严格右移。

进一步称为行最简形若：

1. 每个主元均为 1；
2. 主元所在列其他元素均为 0。



定理 4.11

任何矩阵均可通过初等行变换化为行阶梯形和行最简形。



例题 4.18 化行阶梯形与行最简形 将矩阵 $B_1 = \begin{pmatrix} 1 & 1 & 2 & 1 & 4 \\ 2 & 1 & 1 & 1 & 2 \\ 2 & 3 & 1 & 1 & 2 \\ 3 & 6 & 9 & 7 & 9 \end{pmatrix}$ 化为行阶梯形和行最

简形。

通过初等行变换：

$$\begin{aligned}
 & \xrightarrow[r_3-2r_1]{r_2-2r_1, r_4-3r_1} \begin{pmatrix} 1 & 1 & 2 & 1 & 4 \\ 0 & -1 & -3 & -1 & -6 \\ 0 & 1 & -3 & -1 & -6 \\ 0 & 3 & 3 & 4 & -3 \end{pmatrix} \\
 & \xrightarrow{r_3+r_2, r_4+3r_2} \begin{pmatrix} 1 & 1 & 2 & 1 & 4 \\ 0 & -1 & -3 & -1 & -6 \\ 0 & 0 & -6 & -2 & -12 \\ 0 & 0 & -6 & 1 & -21 \end{pmatrix} \\
 & \xrightarrow{r_4-r_3} \begin{pmatrix} 1 & 1 & 2 & 1 & 4 \\ 0 & -1 & -3 & -1 & -6 \\ 0 & 0 & -6 & -2 & -12 \\ 0 & 0 & 0 & 3 & -9 \end{pmatrix} \quad (\text{行阶梯形}) \\
 & \xrightarrow{r_2 \times (-1), r_3 \times (-\frac{1}{6}), r_4 \times \frac{1}{3}} \begin{pmatrix} 1 & 1 & 2 & 1 & 4 \\ 0 & 1 & 3 & 1 & 6 \\ 0 & 0 & 1 & \frac{1}{3} & 2 \\ 0 & 0 & 0 & 1 & -3 \end{pmatrix} \\
 & \xrightarrow{\text{消元}} \begin{pmatrix} 1 & 0 & 0 & 0 & * \\ 0 & 1 & 0 & 0 & * \\ 0 & 0 & 1 & 0 & * \\ 0 & 0 & 0 & 1 & -3 \end{pmatrix} \quad (\text{行最简形}).
 \end{aligned}$$

4.10.2.3 初等矩阵

定义 4.15 (初等矩阵)

由单位矩阵 E 经一次初等变换得到的矩阵称为初等矩阵。三类初等矩阵对应三类初等变换：

1. 对换初等矩阵： E_{ij} (交换 i, j 行/列)；
2. 倍乘初等矩阵： $E_i(k)$ (第 i 行/列乘 $k \neq 0$)；
3. 倍加初等矩阵： $E_{ij}(k)$ (第 j 行的 k 倍加到第 i 行，或第 i 列的 k 倍加到第 j 列)。



引理 4.1 (初等矩阵与变换的关系)

对矩阵 A 施行一次初等行变换，等价于左乘相应的初等矩阵；施行一次初等列变换，等价于右乘相应的初等矩阵。



例题 4.19 初等矩阵的作用 设 $E_{12}(3) = \begin{pmatrix} 1 & 3 \\ 0 & 1 \end{pmatrix}$, 则左乘 $A: E_{12}(3)A = \begin{pmatrix} 1 & 3 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} a & b \\ c & d \end{pmatrix} = \begin{pmatrix} a+3c & b+3d \\ c & d \end{pmatrix}$, 相当于将 A 的第 2 行的 3 倍加到第 1 行。

要点提炼

初等矩阵皆可逆，且其逆矩阵仍为同类型初等矩阵：

- $E_{ij}^{-1} = E_{ij}$
- $E_i(k)^{-1} = E_i(1/k)$
- $E_{ij}(k)^{-1} = E_{ij}(-k)$

定理 4.12 (可逆矩阵的初等矩阵分解)

n 阶方阵 A 可逆的充要条件是 A 可表示为若干初等矩阵的乘积。



证明 若 A 可逆，则 A 可经初等行变换化为 E ，即存在初等矩阵 $P_1, P_2, \dots, P_k$ 使 $P_k \cdots P_1 A = E$ ，故 $A = P_1^{-1} \cdots P_k^{-1}$ 。反之，初等矩阵乘积必可逆。

4.10.2.4 Gauss-Jordan 消元法求逆矩阵**代码视角**

Gauss-Jordan 求逆算法 求 n 阶可逆矩阵 A 的逆矩阵的步骤：

1. 构造增广矩阵 $(A | E)$ ；
2. 对 $(A | E)$ 施行初等行变换，将左块 A 化为 E ；
3. 此时右块即为 A^{-1} ，即 $(A | E) \xrightarrow{\text{行变换}} (E | A^{-1})$ 。

例题 4.20 Gauss-Jordan 法求逆 设 $A = \begin{pmatrix} 0 & 1 & 2 \\ 1 & 1 & 4 \\ -2 & -1 & 0 \end{pmatrix}$ ，求 A^{-1} 。

构造增广矩阵并施行行变换：

$$\begin{aligned}
 & \left(\begin{array}{ccc|ccc} 0 & 1 & 2 & 1 & 0 & 0 \\ 1 & 1 & 4 & 0 & 1 & 0 \\ -2 & -1 & 0 & 0 & 0 & 1 \end{array} \right) \xrightarrow{r_1 \leftrightarrow r_2} \left(\begin{array}{ccc|ccc} 1 & 1 & 4 & 0 & 1 & 0 \\ 0 & 1 & 2 & 1 & 0 & 0 \\ -2 & -1 & 0 & 0 & 0 & 1 \end{array} \right) \\
 & \xrightarrow{r_3+2r_1} \left(\begin{array}{ccc|ccc} 1 & 1 & 4 & 0 & 1 & 0 \\ 0 & 1 & 2 & 1 & 0 & 0 \\ 0 & 1 & 8 & 0 & 2 & 1 \end{array} \right) \xrightarrow{r_3-r_2} \left(\begin{array}{ccc|ccc} 1 & 1 & 4 & 0 & 1 & 0 \\ 0 & 1 & 2 & 1 & 0 & 0 \\ 0 & 0 & 6 & -1 & 2 & 1 \end{array} \right) \\
 & \xrightarrow{r_3 \times \frac{1}{6}} \left(\begin{array}{ccc|ccc} 1 & 1 & 4 & 0 & 1 & 0 \\ 0 & 1 & 2 & 1 & 0 & 0 \\ 0 & 0 & 1 & -\frac{1}{6} & \frac{1}{3} & \frac{1}{6} \end{array} \right) \xrightarrow[r_2-2r_3]{r_1-4r_3} \left(\begin{array}{ccc|ccc} 1 & 1 & 0 & \frac{2}{3} & -\frac{1}{3} & -\frac{2}{3} \\ 0 & 1 & 0 & \frac{4}{3} & -\frac{2}{3} & -\frac{1}{3} \\ 0 & 0 & 1 & -\frac{1}{6} & \frac{1}{3} & \frac{1}{6} \end{array} \right) \\
 & \xrightarrow{r_1-r_2} \left(\begin{array}{ccc|ccc} 1 & 0 & 0 & -\frac{2}{3} & \frac{1}{3} & -\frac{1}{3} \\ 0 & 1 & 0 & \frac{4}{3} & -\frac{2}{3} & -\frac{1}{3} \\ 0 & 0 & 1 & -\frac{1}{6} & \frac{1}{3} & \frac{1}{6} \end{array} \right). \\
 \text{故 } A^{-1} &= \begin{pmatrix} -\frac{2}{3} & \frac{1}{3} & -\frac{1}{3} \\ \frac{4}{3} & -\frac{2}{3} & -\frac{1}{3} \\ -\frac{1}{6} & \frac{1}{3} & \frac{1}{6} \end{pmatrix} = \frac{1}{6} \begin{pmatrix} -4 & 2 & -2 \\ 8 & -4 & -2 \\ -1 & 2 & 1 \end{pmatrix}.
 \end{aligned}$$

代码视角

NumPy 验证矩阵求逆

```
import numpy as np

# 定义矩阵 A
A = np.array([[0, 1, 2],
              [1, 1, 4],
              [-2, -1, 0]], dtype=float)

# 计算逆矩阵
A_inv = np.linalg.inv(A)
print("A^{-1} = \n", A_inv)

# 验证 A * A_inv 是否为单位阵
print("A * A^{-1} = \n", np.dot(A, A_inv))
```

title= 将矩阵表示为初等矩阵的乘积

若将可逆矩阵 A 通过初等行变换化为 E , 记录变换对应的初等矩阵 $P_1, P_2, \dots, P_k$ (即 $P_k \cdots P_1 A = E$), 则 $A = P_1^{-1} \cdots P_k^{-1}$ 。例如:

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \xrightarrow{r_2-3r_1} \begin{pmatrix} 1 & 2 \\ 0 & -2 \end{pmatrix} \xrightarrow{r_2 \times (-\frac{1}{2})} \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix} \xrightarrow{r_1-2r_2} \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

对应初等矩阵: $P_1 = E_{21}(-3)$, $P_2 = E_2(-1/2)$, $P_3 = E_{12}(-2)$, 故 $A = E_{21}(3)E_2(-2)E_{12}(2)$ 。

练习

练习 4.10 设 A 为 n 阶方阵。证明: A 可逆当且仅当 $\det(A) \neq 0$; 若 A 可逆, 则 $\det(A^{-1}) = \frac{1}{\det(A)}$ 。

练习 4.11 用伴随矩阵法和 Gauss-Jordan 法分别求 $A = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 2 & 1 \\ 3 & 4 & 3 \end{pmatrix}^{-1}$, 并比较计算复杂度。

练习 4.12 设 A, B 为同阶可逆矩阵。证明: $(AB)^{-1} = B^{-1}A^{-1}$ 。若 $AB = BA$, 证明 A 与任意多项式 $f(B)$ 可交换。

练习 4.13 若 $A^2 - A - 2E = O$, 证明 A 可逆且 $A^{-1} = \frac{1}{2}(A - E)$ 。讨论 $A + E$ 可逆的充要条件, 并求可逆时的 $(A + E)^{-1}$ 。

练习 4.14 将 $A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ 表示为初等矩阵的乘积。

参考答案提要:

• 练习 2: 见 §4.10.1 例题。伴随矩阵法需计算 9 个代数余子式和 1 个行列式; Gauss-Jordan 法约需 $O(n^3)$ 次运算。

• 练习 5: $A = E_{21}(3)E_2(-2)E_{12}(2)$, 其中 $E_{21}(3) = \begin{pmatrix} 1 & 0 \\ 3 & 1 \end{pmatrix}$, $E_2(-2) = \begin{pmatrix} 1 & 0 \\ 0 & -2 \end{pmatrix}$,

$$E_{12}(2) = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}.$$

4.11 转置矩阵与一些重要的方阵

矩阵的转置是矩阵运算中的基本操作，而对称矩阵、反对称矩阵、对角矩阵和正交矩阵等特殊方阵则在理论研究和实际应用中扮演着重要角色。本节将系统介绍这些概念及其性质，并简要探讨其在复数域上的推广——埃尔米特矩阵与酉矩阵。

4.11.1 转置矩阵

定义 4.16 (转置矩阵)

设矩阵 $\mathbf{A} = (a_{ij})_{m \times n}$ 。将其行与列互换所得的 $n \times m$ 矩阵称为 $\mathbf{A}$ 的转置矩阵，记作 $\mathbf{A}^\top$ ，其元素满足：

$$(\mathbf{A}^\top)_{ij} = a_{ji}, \quad \text{其中 } i = 1, 2, \dots, n; j = 1, 2, \dots, m.$$



例题 4.21

$$\mathbf{A} = \begin{pmatrix} 4 & 5 & 8 \\ 1 & 2 & 2 \end{pmatrix}, \quad \mathbf{A}^\top = \begin{pmatrix} 4 & 1 \\ 5 & 2 \\ 8 & 2 \end{pmatrix}; \quad \mathbf{B} = \begin{pmatrix} 18 \\ 6 \end{pmatrix}, \quad \mathbf{B}^\top = (18 \ 6).$$

历史侧记

矩阵转置的概念由英国数学家阿瑟·凯莱 (Arthur Cayley) 在 19 世纪发展矩阵理论时系统提出并形式化，成为线性代数中的基础运算。

要点提炼

转置运算的性质 设 $\mathbf{A}, \mathbf{B}$ 为适当维数的矩阵， λ 为标量，则：

1. $(\mathbf{A}^\top)^\top = \mathbf{A}$ (转置的转置等于自身)
2. $(\mathbf{A} + \mathbf{B})^\top = \mathbf{A}^\top + \mathbf{B}^\top$ (和的转置等于转置的和)
3. $(\lambda \mathbf{A})^\top = \lambda \mathbf{A}^\top$ (数乘的转置等于转置的数乘)
4. $(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top$ (乘积的转置等于转置的逆序乘积)
5. 若 $\mathbf{A}$ 可逆，则 $(\mathbf{A}^\top)^{-1} = (\mathbf{A}^{-1})^\top$ (逆的转置等于转置的逆)

例题 4.22 两种方法计算乘积的转置 设

$$\mathbf{A} = \begin{pmatrix} 1 & 3 & 2 \\ 2 & 0 & 1 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} 1 & -1 & 7 \\ 4 & 2 & 0 \\ 2 & 0 & 1 \end{pmatrix}.$$

1. 直接计算：先求乘积

$$\mathbf{AB} = \begin{pmatrix} 17 & 5 & 9 \\ 4 & -2 & 15 \end{pmatrix}, \quad \text{故 } (\mathbf{AB})^\top = \begin{pmatrix} 17 & 4 \\ 5 & -2 \\ 9 & 15 \end{pmatrix}.$$

2. 利用性质: $(AB)^T = B^T A^T$,

$$B^T = \begin{pmatrix} 1 & 4 & 2 \\ -1 & 2 & 0 \\ 7 & 0 & 1 \end{pmatrix}, \quad A^T = \begin{pmatrix} 1 & 2 \\ 3 & 0 \\ 2 & 1 \end{pmatrix}, \quad B^T A^T = \begin{pmatrix} 17 & 4 \\ 5 & -2 \\ 9 & 15 \end{pmatrix}.$$

两种方法结果一致。

注意

实矩阵与零矩阵的一个重要结论 若 A 为实矩阵且 $AA^T = 0$, 则 $A = 0$ 。

证明 考虑 AA^T 的第 i 个对角元:

$$(AA^T)_{ii} = \sum_{k=1}^n a_{ik}^2 = 0.$$

由于平方项非负, 故必有 $a_{ik} = 0$ 对所有 i, k 成立, 即 $A = 0$ 。

例题 4.23 正交矩阵与单位矩阵的和 若 A 为实正交矩阵 (即 $AA^T = E$) 且 $|A| = -1$, 则 $E + A$ 奇异 (不可逆)。

证明 由 $|E + A| = |AA^T + A| = |A(A^T + E)| = |A| \cdot |A^T + E|$ 。注意到 $|A^T + E| = |(A + E)^T| = |A + E|$, 故

$$|E + A| = -|E + A| \Rightarrow |E + A| = 0.$$

几何视角

从几何角度看, 矩阵转置对应着线性变换的伴随变换。在欧几里得空间中, 若矩阵 A 表示一个线性变换, 则其转置 A^T 表示该变换 (关于标准内积) 的伴随变换。

4.11.2 一些重要的方阵 (实数域)

对称矩阵与反对称矩阵

定义 4.17 (对称矩阵)

若实方阵 A 满足 $A^T = A$, 则称 A 为对称矩阵。



定义 4.18 (反对称矩阵)

若实方阵 A 满足 $A^T = -A$, 则称 A 为反对称矩阵 (或斜对称矩阵)。



注 反对称矩阵的主对角线元素必为零: 由 $a_{ii} = -a_{ii} \Rightarrow a_{ii} = 0$ 。

例题 4.24 矩阵 $A = \begin{pmatrix} 12 & 6 & 1 \\ 6 & 8 & 0 \\ 1 & 0 & 1 \end{pmatrix}$ 是对称矩阵; 对任意实矩阵 B , 乘积 BB^T 总是对称矩阵。

要点提炼

对称与反对称矩阵的性质

1. 奇数阶反对称矩阵的行列式为零：由 $|\mathbf{A}| = |\mathbf{A}^\top| = |-\mathbf{A}| = (-1)^n |\mathbf{A}|$ ，当 n 为奇数时，有 $|\mathbf{A}| = -|\mathbf{A}| \Rightarrow |\mathbf{A}| = 0$ 。
2. 任意方阵的对称-反对称分解：任何方阵 $\mathbf{A}$ 可唯一地分解为对称部分与反对称部分之和：

$$\mathbf{A} = \frac{1}{2}(\mathbf{A} + \mathbf{A}^\top) + \frac{1}{2}(\mathbf{A} - \mathbf{A}^\top).$$

例题 4.25 若实对称矩阵 $\mathbf{A}$ 满足 $\mathbf{A}^2 = \mathbf{0}$ ，则 $\mathbf{A} = \mathbf{0}$ 。

证明 由 $\mathbf{A}^2 = \mathbf{A}\mathbf{A} = \mathbf{A}\mathbf{A}^\top = \mathbf{0}$ ，根据前述结论立得 $\mathbf{A} = \mathbf{0}$ 。

Householder 反射（对称正交矩阵）

例题 4.26 Householder 矩阵 设向量 $\mathbf{x} \in \mathbb{R}^n$ 满足 $\mathbf{x}^\top \mathbf{x} = 1$ （单位向量），定义

$$\mathbf{H} = \mathbf{E} - 2\mathbf{x}\mathbf{x}^\top.$$

则：

- $\mathbf{H}^\top = \mathbf{H}$ （对称性）
- $\mathbf{H}^\top \mathbf{H} = \mathbf{H}^2 = (\mathbf{E} - 2\mathbf{x}\mathbf{x}^\top)^2 = \mathbf{E} - 4\mathbf{x}\mathbf{x}^\top + 4\mathbf{x}\mathbf{x}^\top \mathbf{x}\mathbf{x}^\top = \mathbf{E}$ （正交性）

故 $\mathbf{H}$ 既是对称矩阵又是正交矩阵。几何上， $\mathbf{H}$ 表示关于超平面 $\{\mathbf{y} \in \mathbb{R}^n : \mathbf{x}^\top \mathbf{y} = 0\}$ 的反射。

对角矩阵

定义 4.19 (对角矩阵)

若 n 阶方阵 $\mathbf{D}$ 的所有非主对角线元素均为零，即

$$\mathbf{D} = \text{diag}(d_1, d_2, \dots, d_n) = \begin{pmatrix} d_1 & 0 & \cdots & 0 \\ 0 & d_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & d_n \end{pmatrix},$$

则称 $\mathbf{D}$ 为对角矩阵。



要点提炼

对角矩阵的运算性质

1. 同阶对角矩阵的乘积仍为对角矩阵，且运算按对角元逐点进行：

$$\text{diag}(d_1, \dots, d_n) \cdot \text{diag}(e_1, \dots, e_n) = \text{diag}(d_1 e_1, \dots, d_n e_n).$$

2. 对角矩阵的幂运算： $\mathbf{D}^m = \text{diag}(d_1^m, \dots, d_n^m)$ 。

3. 对角矩阵可逆当且仅当所有对角元 $d_i \neq 0$ ，且逆矩阵为 $\mathbf{D}^{-1} = \text{diag}(d_1^{-1}, \dots, d_n^{-1})$ 。

title= 对角矩阵的“缩放”作用

对角矩阵左乘或右乘其他矩阵，相当于进行行缩放或列缩放：

- 右乘对角矩阵： $\mathbf{A} \text{diag}(d_1, \dots, d_n)$ 等价于将 $\mathbf{A}$ 的第 j 列缩放 d_j 倍。
- 左乘对角矩阵： $\text{diag}(d_1, \dots, d_m)\mathbf{A}$ 等价于将 $\mathbf{A}$ 的第 i 行缩放 d_i 倍。

正交矩阵

定义 4.20 (正交矩阵)

若实方阵 $\mathbf{Q}$ 满足 $\mathbf{Q}^\top \mathbf{Q} = \mathbf{E}$ ，则称 $\mathbf{Q}$ 为正交矩阵。



要点提炼

正交矩阵的性质

1. $\mathbf{Q}^{-1} = \mathbf{Q}^\top$ (逆等于转置)
2. $|\det(\mathbf{Q})| = 1$ (行列式的绝对值为 1)
3. 正交矩阵的乘积仍为正交矩阵：若 $\mathbf{Q}_1, \mathbf{Q}_2$ 正交，则 $(\mathbf{Q}_1 \mathbf{Q}_2)^\top (\mathbf{Q}_1 \mathbf{Q}_2) = \mathbf{E}$ 。
4. 列（或行）向量组是标准正交基：即 $\mathbf{q}_i^\top \mathbf{q}_j = \delta_{ij}$ (Kronecker delta)。

例题 4.27 伴随矩阵保持正交性 若 $\mathbf{Q}$ 正交，则其伴随矩阵 $\text{adj}(\mathbf{Q}) = \det(\mathbf{Q})\mathbf{Q}^{-1} = \det(\mathbf{Q})\mathbf{Q}^\top$ 也是正交矩阵。

证明 计算：

$$[\det(\mathbf{Q})\mathbf{Q}^\top]^\top [\det(\mathbf{Q})\mathbf{Q}^\top] = \det(\mathbf{Q})^2 \mathbf{Q} \mathbf{Q}^\top = \det(\mathbf{Q})^2 \mathbf{E} = \mathbf{E}.$$

例题 4.28 正交相似变换保持对称性 若 $\mathbf{A}$ 对称， $\mathbf{T}$ 正交，则 $\mathbf{T}^{-1}\mathbf{A}\mathbf{T} = \mathbf{T}^\top \mathbf{A} \mathbf{T}$ 仍为对称矩阵。

证明

$$(\mathbf{T}^\top \mathbf{A} \mathbf{T})^\top = \mathbf{T}^\top \mathbf{A}^\top \mathbf{T} = \mathbf{T}^\top \mathbf{A} \mathbf{T}.$$

代码视角

[Python 验证正交矩阵的性质]

```
import numpy as np
```

```
# 定义一个正交矩阵（旋转45度）
```

```
theta = np.pi/4
```

```
Q = np.array([[np.cos(theta), -np.sin(theta)],
```

```

[ np.sin(theta), np.cos(theta) ] ] )

print("Q = \n", Q)
print("Q^T Q = \n", Q.T @ Q)  # 应近似于单位阵
print("det(Q) = ", np.linalg.det(Q))  # 应近似于1

```

4.11.3 选学：复矩阵、埃尔米特矩阵与酉矩阵

定义 4.21 (共轭与共轭转置)

设复矩阵 $A = (a_{ij})$:

- 共轭矩阵: $\overline{A} = (\overline{a_{ij}})$ (每个元素取共轭复数)
- 共轭转置 (埃尔米特转置): $A^H = (\overline{A})^T$ (先共轭再转置)



要点提炼

共轭转置的性质

1. $(A^H)^H = A$
2. $(A + B)^H = A^H + B^H$
3. $(\lambda A)^H = \overline{\lambda} A^H$
4. $(AB)^H = B^H A^H$
5. 若 A 可逆, 则 $(A^H)^{-1} = (A^{-1})^H$

定义 4.22 (埃尔米特矩阵)

若复方阵 H 满足 $H^H = H$, 则称 H 为埃尔米特矩阵 (或自伴矩阵)。其主对角线元素必为实数。



定义 4.23 (酉矩阵)

若复方阵 U 满足 $U^H U = E$, 则称 U 为酉矩阵。这是正交矩阵在复数域上的推广。



要点提炼

酉矩阵的性质

1. $U^{-1} = U^H$
2. $|\det(U)| = 1$ (行列式的模为 1)
3. 酉矩阵的乘积仍为酉矩阵
4. 列 (或行) 向量组在复内积下构成标准正交基: $\langle u_i, u_j \rangle = u_j^H u_i = \delta_{ij}$

历史侧记

“酉”矩阵 (Unitary Matrix) 一词源自“么正”的日文翻译, 强调其保持内积不变 (“么”) 和模长为 1 (“正”) 的性质。其在量子力学中描述系统的么正演化时至关重要。

练习

- 练习 4.15 设 $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\mathbf{B} \in \mathbb{R}^{n \times p}$ 。详细证明 $(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top$, 并构造一个数值例子验证 (可参考例 4.22)。
- 练习 4.16 设 $\mathbf{A}$ 为实矩阵。证明: 若 $\mathbf{A}\mathbf{A}^\top = \mathbf{0}$, 则 $\mathbf{A} = \mathbf{0}$; 若 $\mathbf{A}$ 对称且 $\mathbf{A}^2 = \mathbf{0}$, 则亦有 $\mathbf{A} = \mathbf{0}$ 。
- 练习 4.17 设 $\mathbf{Q}$ 为 n 阶正交矩阵。证明:
- $\text{tr}(\mathbf{Q}^\top \mathbf{Q}) = n$
 - 对任意 $\mathbf{v} \in \mathbb{R}^n$, 有 $\|\mathbf{Q}\mathbf{v}\|_2 = \|\mathbf{v}\|_2$ (正交变换保持向量长度)
- 练习 4.18 证明: 任意方阵 $\mathbf{A} \in \mathbb{R}^{n \times n}$ 可唯一地分解为对称矩阵与反对称矩阵之和, 并写出该分解的显式表达式。
- 练习 4.19 选做 设 $\mathbf{U}$ 为 n 阶酉矩阵。证明:
- $|\det(\mathbf{U})| = 1$
 - 若 $\mathbf{H}$ 为埃尔米特矩阵, 则 $\mathbf{U}^H \mathbf{H} \mathbf{U}$ 也是埃尔米特矩阵。

部分练习参考答案提示:

- 练习 4: $\mathbf{A} = \frac{1}{2}(\mathbf{A} + \mathbf{A}^\top) + \frac{1}{2}(\mathbf{A} - \mathbf{A}^\top)$, 验证前者对称、后者反对称, 且唯一。
- 练习 5(2): 计算 $(\mathbf{U}^H \mathbf{H} \mathbf{U})^H = \mathbf{U}^H \mathbf{H}^H \mathbf{U} = \mathbf{U}^H \mathbf{H} \mathbf{U}$ 。

4.12 分块矩阵及其应用

要点提炼

本节核心要点

- 掌握分块矩阵的定义与表示方法
- 熟悉分块矩阵的基本运算规则（加法、数乘、乘法、转置）
- 理解特殊分块矩阵（对角分块、三角分块）的性质
- 掌握分块矩阵在求逆、行列式计算等方面的应用
- 了解分块矩阵初等变换的概念与操作方法

矩阵分块是处理高阶矩阵和特殊结构矩阵的重要技巧。通过将大矩阵划分为若干子块，可以使矩阵结构更加清晰，简化计算过程，并在理论证明和实际应用中发挥重要作用。本节将系统介绍分块矩阵的基本概念、运算规则及其典型应用。

4.12.1 分块矩阵的定义与表示

定义 4.24 (分块矩阵)

设 $\mathbf{A}$ 是一个 $m \times n$ 矩阵，用若干条横线和纵线将其划分成若干个小矩阵，每个小矩阵称为 $\mathbf{A}$ 的一个子块或子矩阵。以这些子块为元素构成的矩阵称为分块矩阵，记作：

$$\mathbf{A} = \begin{pmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} & \cdots & \mathbf{A}_{1t} \\ \mathbf{A}_{21} & \mathbf{A}_{22} & \cdots & \mathbf{A}_{2t} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{A}_{s1} & \mathbf{A}_{s2} & \cdots & \mathbf{A}_{st} \end{pmatrix}$$

其中 $\mathbf{A}_{ij}$ 表示第 i 行第 j 列的子块。



例题 4.29 分块矩阵示例 矩阵 $\mathbf{A} = \begin{pmatrix} a & 1 & 0 & 0 \\ 1 & a & 0 & 0 \\ 0 & 0 & b & 1 \\ 0 & 0 & 1 & b \end{pmatrix}$ 可分块为：

$$\mathbf{A} = \begin{pmatrix} \mathbf{A}_1 & \mathbf{O} \\ \mathbf{O} & \mathbf{B} \end{pmatrix}$$

其中 $\mathbf{A}_1 = \begin{pmatrix} a & 1 \\ 1 & a \end{pmatrix}$, $\mathbf{B} = \begin{pmatrix} b & 1 \\ 1 & b \end{pmatrix}$, $\mathbf{O} = \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$ 。

注 同一矩阵可以根据需要采用不同的分块方法，选择适当的分块方式能够显著简化计算。

历史侧记

矩阵分块的思想最早可追溯到 20 世纪初, 随着计算机科学的发展, 分块矩阵在高性能计算和科学工程计算中得到了广泛应用, 成为处理大规模矩阵问题的有效工具。

4.12.2 分块矩阵的运算规则

分块矩阵的运算规则与普通矩阵类似, 但需要确保分块方式满足运算的条件要求。

4.12.2.1 加法与数乘

命题 4.3 (分块矩阵的加法)

设 $\mathbf{A} = (\mathbf{A}_{ij})_{s \times t}$ 和 $\mathbf{B} = (\mathbf{B}_{ij})_{s \times t}$ 是同型矩阵且采用相同的分块方法, 则:

$$\mathbf{A} + \mathbf{B} = (\mathbf{A}_{ij} + \mathbf{B}_{ij})_{s \times t}$$

即对应子块相加。

命题 4.4 (分块矩阵的数乘)

设 $\mathbf{A} = (\mathbf{A}_{ij})_{s \times t}$, λ 为常数, 则:

$$\lambda \mathbf{A} = (\lambda \mathbf{A}_{ij})_{s \times t}$$

即每个子块乘以数 λ 。

4.12.2.2 乘法

命题 4.5 (分块矩阵的乘法)

设 $\mathbf{A}$ 是 $m \times l$ 矩阵, $\mathbf{B}$ 是 $l \times n$ 矩阵, 分块为:

$$\mathbf{A} = (\mathbf{A}_{ik})_{s \times t}, \quad \mathbf{B} = (\mathbf{B}_{kj})_{t \times r}$$

其中 $\mathbf{A}_{ik}$ 的列数等于 $\mathbf{B}_{kj}$ 的行数 (对一切 i, k, j), 则:

$$\mathbf{AB} = \mathbf{C} = (\mathbf{C}_{ij})_{s \times r}, \quad \mathbf{C}_{ij} = \sum_{k=1}^t \mathbf{A}_{ik} \mathbf{B}_{kj} \quad (i = 1, \dots, s; j = 1, \dots, r)$$

这与普通矩阵乘法规则形式相同, 但这里元素为子块, 乘法为子块乘法。

例题 4.30 分块矩阵乘法 设 $\mathbf{A} = \begin{pmatrix} 1 & 0 & 2 \\ 0 & 1 & 3 \\ 4 & 5 & 0 \end{pmatrix}, \mathbf{B} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix}$ 。分块为:

$$\mathbf{A} = \begin{pmatrix} \mathbf{E} & \mathbf{C} \\ \mathbf{D} & \mathbf{O} \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} \mathbf{U} \\ \mathbf{V} \end{pmatrix}$$

其中 $\mathbf{E} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ (单位矩阵), $\mathbf{C} = \begin{pmatrix} 2 \\ 3 \end{pmatrix}$, $\mathbf{D} = \begin{pmatrix} 4 & 5 \end{pmatrix}$, $\mathbf{O} = (0)$, $\mathbf{U} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$, $\mathbf{V} = \begin{pmatrix} 5 & 6 \end{pmatrix}$ 。则:

$$\mathbf{AB} = \begin{pmatrix} \mathbf{E} & \mathbf{C} \\ \mathbf{D} & \mathbf{O} \end{pmatrix} \begin{pmatrix} \mathbf{U} \\ \mathbf{V} \end{pmatrix} = \begin{pmatrix} \mathbf{EU} + \mathbf{CV} \\ \mathbf{DU} + \mathbf{OV} \end{pmatrix} = \begin{pmatrix} \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} + \begin{pmatrix} 10 & 12 \\ 15 & 18 \end{pmatrix} \\ \begin{pmatrix} 4 & 8 \\ 5 & 10 \end{pmatrix} \end{pmatrix} = \begin{pmatrix} 11 & 14 \\ 18 & 22 \\ 4 & 8 \end{pmatrix}$$

4.12.2.3 转置

命题 4.6 (分块矩阵的转置)

设 $\mathbf{A} = (\mathbf{A}_{ij})_{s \times t}$ 是一个分块矩阵, 则其转置矩阵为:

$$\mathbf{A}^T = (\mathbf{B}_{ji})_{t \times s}, \quad \text{其中 } \mathbf{B}_{ji} = \mathbf{A}_{ij}^T$$

即不仅子块位置互换, 每个子块自身也需转置。



例题 4.31 分块矩阵的转置 设 $\mathbf{A} = \begin{pmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{pmatrix}$, 则:

$$\mathbf{A}^T = \begin{pmatrix} \mathbf{A}_{11}^T & \mathbf{A}_{21}^T \\ \mathbf{A}_{12}^T & \mathbf{A}_{22}^T \end{pmatrix}$$

4.12.3 特殊分块矩阵

4.12.3.1 分块对角矩阵

定义 4.25 (分块对角矩阵)

若 n 阶方阵 $\mathbf{A}$ 的分块形式中, 非主对角线上的所有子块均为零矩阵, 且主对角线上的子块都是方阵, 即:

$$\mathbf{A} = \begin{pmatrix} \mathbf{A}_1 & \mathbf{O} & \cdots & \mathbf{O} \\ \mathbf{O} & \mathbf{A}_2 & \cdots & \mathbf{O} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{O} & \mathbf{O} & \cdots & \mathbf{A}_s \end{pmatrix}$$

则称 $\mathbf{A}$ 为分块对角矩阵 (或准对角矩阵), 记作 $\mathbf{A} = \text{diag}(\mathbf{A}_1, \mathbf{A}_2, \dots, \mathbf{A}_s)$ 。



要点提炼

分块对角矩阵具有以下性质：

1. 行列式性质： $|\mathbf{A}| = |\mathbf{A}_1| \cdot |\mathbf{A}_2| \cdots |\mathbf{A}_s|$
2. 可逆性： $\mathbf{A}$ 可逆当且仅当每个 $\mathbf{A}_i$ 可逆，且 $\mathbf{A}^{-1} = \text{diag}(\mathbf{A}_1^{-1}, \mathbf{A}_2^{-1}, \dots, \mathbf{A}_s^{-1})$
3. 幂运算： $\mathbf{A}^k = \text{diag}(\mathbf{A}_1^k, \mathbf{A}_2^k, \dots, \mathbf{A}_s^k)$ (k 为正整数)
4. 同结构矩阵运算：同结构的分块对角矩阵的和、差、积仍为分块对角矩阵，且运算表现为对应对角子块的运算

4.12.3.2 分块三角矩阵

定义 4.26 (分块三角矩阵)

若方阵 $\mathbf{A}$ 的分块形式中，主对角线上的子块都是方阵，且主对角线以下（或以上）的所有子块都是零矩阵，即：

$$\mathbf{A} = \begin{pmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} & \cdots & \mathbf{A}_{1s} \\ \mathbf{O} & \mathbf{A}_{22} & \cdots & \mathbf{A}_{2s} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{O} & \mathbf{O} & \cdots & \mathbf{A}_{ss} \end{pmatrix} \quad \text{或} \quad \mathbf{A} = \begin{pmatrix} \mathbf{A}_{11} & \mathbf{O} & \cdots & \mathbf{O} \\ \mathbf{A}_{21} & \mathbf{A}_{22} & \cdots & \mathbf{O} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{A}_{s1} & \mathbf{A}_{s2} & \cdots & \mathbf{A}_{ss} \end{pmatrix}$$

则称 $\mathbf{A}$ 为分块上三角矩阵或分块下三角矩阵。



要点提炼

分块三角矩阵的性质：

1. 行列式性质： $|\mathbf{A}| = |\mathbf{A}_{11}| \cdot |\mathbf{A}_{22}| \cdots |\mathbf{A}_{ss}|$
2. 可逆性： $\mathbf{A}$ 可逆当且仅当每个主对角线子块 $\mathbf{A}_{ii}$ 可逆
3. 同结构的分块三角矩阵的积仍是分块三角矩阵

4.12.4 分块矩阵的初等变换

分块矩阵也可以进行类似于普通矩阵的初等变换，这是处理分块矩阵问题的重要工具。

定义 4.27 (分块矩阵的初等变换)

分块矩阵的以下三种变换称为初等变换：

1. 交换两行（列）块
2. 用一个可逆矩阵左乘（右乘）某一行（列）块
3. 将某一行（列）块左乘（右乘）一个矩阵后加到另一行（列）块上



定义 4.28 (分块初等矩阵)

由分块单位矩阵经过一次分块初等变换得到的矩阵称为分块初等矩阵。分块单位矩阵是指形如 $\text{diag}(\mathbf{E}_1, \mathbf{E}_2, \dots, \mathbf{E}_s)$ 的矩阵, 其中 $\mathbf{E}_i$ 均为单位矩阵。



例题 4.32 分块初等矩阵示例 对于分块单位矩阵 $\begin{pmatrix} \mathbf{E}_m & \mathbf{O} \\ \mathbf{O} & \mathbf{E}_n \end{pmatrix}$, 相应的三类分块初等矩阵为:

1. 交换行块: $\begin{pmatrix} \mathbf{O} & \mathbf{E}_m \\ \mathbf{E}_n & \mathbf{O} \end{pmatrix}$
2. 乘以可逆矩阵: $\begin{pmatrix} \mathbf{P} & \mathbf{O} \\ \mathbf{O} & \mathbf{E}_n \end{pmatrix}$ ($\mathbf{P}$ 为 $m \times m$ 可逆矩阵)
3. 倍加变换: $\begin{pmatrix} \mathbf{E}_m & \mathbf{O} \\ \mathbf{Q} & \mathbf{E}_n \end{pmatrix}$ ($\mathbf{Q}$ 为 $n \times m$ 矩阵)

引理 4.2 (初等变换与初等矩阵的关系)

对分块矩阵作一次初等行(列)变换, 相当于左(右)乘相应的分块初等矩阵。

**4.12.5 分块矩阵的应用**

分块矩阵在矩阵运算、行列式计算、求逆矩阵等方面有广泛应用。

4.12.5.1 分块矩阵求逆

分块矩阵求逆是分块技巧的重要应用之一, 可以简化高阶矩阵求逆的计算。

例题 4.33 分块上三角矩阵的逆 设 $\mathbf{T} = \begin{pmatrix} \mathbf{A} & \mathbf{C} \\ \mathbf{O} & \mathbf{D} \end{pmatrix}$, 其中 $\mathbf{A}$ 、 $\mathbf{D}$ 可逆, 求 $\mathbf{T}^{-1}$ 。

解: 设 $\mathbf{T}^{-1} = \begin{pmatrix} \mathbf{X} & \mathbf{Y} \\ \mathbf{Z} & \mathbf{W} \end{pmatrix}$, 由 $\mathbf{T}\mathbf{T}^{-1} = \mathbf{E}$ 得:

$$\begin{pmatrix} \mathbf{A} & \mathbf{C} \\ \mathbf{O} & \mathbf{D} \end{pmatrix} \begin{pmatrix} \mathbf{X} & \mathbf{Y} \\ \mathbf{Z} & \mathbf{W} \end{pmatrix} = \begin{pmatrix} \mathbf{AX} + \mathbf{CZ} & \mathbf{AY} + \mathbf{CW} \\ \mathbf{DZ} & \mathbf{DW} \end{pmatrix} = \begin{pmatrix} \mathbf{E} & \mathbf{O} \\ \mathbf{O} & \mathbf{E} \end{pmatrix}$$

解方程组:

$$\mathbf{DZ} = \mathbf{O} \Rightarrow \mathbf{Z} = \mathbf{O} \quad (\text{因 } \mathbf{D} \text{ 可逆})$$

$$\mathbf{DW} = \mathbf{E} \Rightarrow \mathbf{W} = \mathbf{D}^{-1}$$

$$\mathbf{AX} + \mathbf{CZ} = \mathbf{E} \Rightarrow \mathbf{AX} = \mathbf{E} \Rightarrow \mathbf{X} = \mathbf{A}^{-1}$$

$$\mathbf{AY} + \mathbf{CW} = \mathbf{O} \Rightarrow \mathbf{AY} = -\mathbf{CD}^{-1} \Rightarrow \mathbf{Y} = -\mathbf{A}^{-1}\mathbf{CD}^{-1}$$

故:

$$\mathbf{T}^{-1} = \begin{pmatrix} \mathbf{A}^{-1} & -\mathbf{A}^{-1}\mathbf{CD}^{-1} \\ \mathbf{O} & \mathbf{D}^{-1} \end{pmatrix}$$

例题 4.34 反对角分块矩阵的逆 设 $A = \begin{pmatrix} O & B \\ D & O \end{pmatrix}$, 其中 B, D 可逆, 求 A^{-1} 。

解: 直接计算可得:

$$A^{-1} = \begin{pmatrix} O & D^{-1} \\ B^{-1} & O \end{pmatrix}$$

4.12.5.2 分块矩阵与行列式计算

分块矩阵的技巧可以简化某些特殊形式矩阵的行列式计算。

定理 4.13 (分块矩阵的行列式公式)

对于分块矩阵 $\begin{pmatrix} A & B \\ C & D \end{pmatrix}$, 有以下行列式计算公式:

1. 若 A 可逆, 则 $\begin{vmatrix} A & B \\ C & D \end{vmatrix} = |A| \cdot |D - CA^{-1}B|$
2. 若 D 可逆, 则 $\begin{vmatrix} A & B \\ C & D \end{vmatrix} = |D| \cdot |A - BD^{-1}C|$
3. 若 $B = O$ 或 $C = O$, 则 $\begin{vmatrix} A & B \\ C & D \end{vmatrix} = |A| \cdot |D|$



例题 4.35 分块矩阵行列式计算 计算行列式 $\begin{vmatrix} A & B \\ B & A \end{vmatrix}$, 其中 A, B 为同阶方阵。

解: 利用分块矩阵的行列式公式:

$$\begin{vmatrix} A & B \\ B & A \end{vmatrix} = |A - B| \cdot |A + B|$$

4.12.5.3 分块矩阵在线性代数中的应用

分块矩阵在线性代数中还有许多其他应用, 如矩阵秩的证明、线性方程组的求解等。

定理 4.14 (分块矩阵的秩)

设 A 为 $m \times n$ 矩阵, B 为 $m \times p$ 矩阵, 则:

$$\text{rank}(A \mid B) \leq \text{rank}(A) + \text{rank}(B)$$

等号成立当且仅当 $\text{Col}(A) \cap \text{Col}(B) = \{0\}$ 。



例题 4.36 分块矩阵与线性方程组 考虑线性方程组 $Ax = b$, 其中 A 为 $m \times n$ 矩阵。将 A 按列分块为 $A = (a_1, a_2, \dots, a_n)$, 则方程组可表示为:

$$x_1 a_1 + x_2 a_2 + \dots + x_n a_n = b$$

这表明 b 可由 A 的列向量线性表出, 方程组有解当且仅当 $b \in \text{Col}(A)$ 。

代码视角

Python/NumPy 中的分块矩阵操作 在 NumPy 中，可以使用 ‘np.block()’ 函数构造分块矩阵：

```
import numpy as np

# 定义子块
A11 = np.array([[1, 2], [3, 4]])
A12 = np.array([[5], [6]])
A21 = np.array([[7, 8]])
A22 = np.array([[9]])

# 构造分块矩阵
T = np.block([[A11, A12], [A21, A22]])
print("Block matrix T:\n", T)

# 计算分块矩阵的逆
T_inv = np.linalg.inv(T)
print("Inverse of T:\n", T_inv)
```

小结

分块矩阵是处理矩阵问题的重要技巧，通过将大矩阵划分为小矩阵，可以简化计算、揭示矩阵结构。掌握分块矩阵的运算规则、特殊分块矩阵的性质以及分块矩阵在求逆、行列式计算等方面的应用，对于深入理解矩阵理论和解决实际问题具有重要意义。

练习

 **练习 4.20** 设 $A = \begin{pmatrix} 1 & 2 & 0 & 0 \\ 3 & 4 & 0 & 0 \\ 0 & 0 & 5 & 6 \\ 0 & 0 & 7 & 8 \end{pmatrix}$ ，试求 A^{-1} 。

 **练习 4.21** 设 $A = \begin{pmatrix} B & O \\ C & D \end{pmatrix}$ ，其中 B 、 D 可逆，求 A^{-1} 。

 **练习 4.22** 证明： $\begin{vmatrix} A & B \\ C & D \end{vmatrix} = |A| \cdot |D - CA^{-1}B|$ （假设 A 可逆）。

 **练习 4.23** 设 A 、 B 为 n 阶方阵，证明：

$$\begin{vmatrix} A & B \\ B & A \end{vmatrix} = |A - B| \cdot |A + B|$$

代码视角

可选：NumPy 验证

```
import numpy as np

# 练习1验证
A = np.array([[1,2,0,0], [3,4,0,0], [0,0,5,6], [0,0,7,8]])
A_inv = np.linalg.inv(A)
print("A^{-1} = \n", A_inv)

# 练习4验证
n = 2
A = np.random.randn(n, n)
B = np.random.randn(n, n)
block_matrix = np.block([[A, B], [B, A]])
det_block = np.linalg.det(block_matrix)
det_prod = np.linalg.det(A-B) * np.linalg.det(A+B)
print("|Block matrix| = ", det_block)
print("|A-B| |A+B| = ", det_prod)
```

第5章 线性方程组

5.1 向量组与矩阵的秩

要点提炼

本节核心要点

- 理解向量组的线性组合、线性表示与等价的概念
- 掌握线性相关与线性无关的判定方法及几何意义
- 理解极大线性无关组与向量组秩的概念及其求法
- 掌握矩阵秩的定义、性质、计算方法及与向量组秩的关系
- 熟悉矩阵秩在判断线性方程组解的存在性与唯一性中的应用

在线性代数中，向量组与矩阵的秩是刻画线性相关性和结构特征的核心概念。本节将系统介绍向量组的线性相关性、极大线性无关组与秩的定义与性质，并深入探讨矩阵秩的概念、计算方法及其与向量组秩的内在联系。

5.1.1 向量组的秩

一、 n 维向量及其表示

定义 5.1 (n 维向量)

由数域 $\mathbb{F}$ (通常取实数域 $\mathbb{R}$ 或复数域 $\mathbb{C}$) 上的 n 个有次序的数 $a_1, a_2, \dots, a_n$ 组成的数组称为一个 n 维向量，记作 $\alpha = (a_1, a_2, \dots, a_n)$ 。其中 a_i 称为该向量的第 i 个分量。所有分量均为实数的向量称为实向量，所有分量均为复数的向量称为复向量。

例题 5.1 $(1, 2, 3)$ 是一个三维实向量； $(1 + i, 2 - 3i)$ 是一个二维复向量。

定义 5.2 (行向量与列向量)

n 维向量按行形式排列称为行向量，如 $\alpha = (a_1, a_2, \dots, a_n)$ ；按列形式排列称为列

向量，如 $\beta = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix}$ 。列向量常被视为 $n \times 1$ 矩阵。

注[向量与矩阵的联系] 矩阵与向量组可相互转化：

- 矩阵的列（行）向量组： $m \times n$ 矩阵 $\mathbf{A} = (a_{ij})$ 的 n 个列向量 $\alpha_j = \begin{pmatrix} a_{1j} \\ a_{2j} \\ \vdots \\ a_{mj} \end{pmatrix} (j = 1, 2, \dots, n)$ 构成其列向量组； m 个行向量 $\beta_i = (a_{i1}, a_{i2}, \dots, a_{in}) (i = 1, 2, \dots, m)$ 构成其行向量组。
- 向量组构成矩阵：给定一组同维向量，可按列（或行）排列构成一个矩阵。

二、线性组合、线性表示与向量组等价

定义 5.3 (线性组合与线性表示)

设 $\alpha_1, \alpha_2, \dots, \alpha_m$ 是一组 n 维向量， $k_1, k_2, \dots, k_m$ 是一组标量，则向量

$$k_1\alpha_1 + k_2\alpha_2 + \dots + k_m\alpha_m$$

称为向量组 $\alpha_1, \alpha_2, \dots, \alpha_m$ 的一个线性组合。若存在一组标量 $\lambda_1, \lambda_2, \dots, \lambda_m$ ，使得向量 β 满足

$$\beta = \lambda_1\alpha_1 + \lambda_2\alpha_2 + \dots + \lambda_m\alpha_m,$$

则称 β 可由向量组 $\alpha_1, \alpha_2, \dots, \alpha_m$ 线性表示。



定义 5.4 (向量组等价)

若向量组 $A: \alpha_1, \alpha_2, \dots, \alpha_s$ 中的每个向量都可由向量组 $B: \beta_1, \beta_2, \dots, \beta_t$ 线性表示，反之亦然，则称向量组 A 与 B 等价。等价关系具有自反性、对称性和传递性。



注[线性方程组的向量形式] 线性方程组 $\mathbf{Ax} = \mathbf{b}$ 可表示为向量形式：

$$x_1\alpha_1 + x_2\alpha_2 + \dots + x_n\alpha_n = \beta,$$

其中 α_j 是系数矩阵 $\mathbf{A}$ 的第 j 列向量， β 是常数项向量。方程组有解当且仅当 β 可由 $\alpha_1, \alpha_2, \dots, \alpha_n$ 线性表出。

三、线性相关与线性无关

定义 5.5 (线性相关与线性无关)

设 $\alpha_1, \alpha_2, \dots, \alpha_s (s \geq 1)$ 是一组 n 维向量。若存在一组不全为零的标量 $k_1, k_2, \dots, k_s$ ，使得

$$k_1\alpha_1 + k_2\alpha_2 + \dots + k_s\alpha_s = \mathbf{0},$$

则称向量组 $\alpha_1, \alpha_2, \dots, \alpha_s$ 线性相关；否则，称其线性无关。



定理 5.1 (线性相关的充要条件)

向量组 $\alpha_1, \alpha_2, \dots, \alpha_m$ 线性相关的充要条件是：存在一组不全为零的标量 $k_1, k_2, \dots, k_m$ ，使得 $\sum_{j=1}^m k_j \alpha_j = \mathbf{0}$ 。



证明 [证明概要] 必要性：若向量组线性相关，则至少有一个向量可由其余向量线性表示，例如 $\alpha_m = \sum_{j=1}^{m-1} \lambda_j \alpha_j$ 。移项得 $\sum_{j=1}^{m-1} \lambda_j \alpha_j - \alpha_m = \mathbf{0}$ ，系数不全为零。充分性：若存在不全为零的 $k_1, \dots, k_m$ 使得 $\sum k_j \alpha_j = \mathbf{0}$ ，不妨设 $k_i \neq 0$ ，则 $\alpha_i = -\sum_{j \neq i} \frac{k_j}{k_i} \alpha_j$ ，故向量组线性相关。

推论 5.1 (线性无关的判定)

向量组 $\alpha_1, \alpha_2, \dots, \alpha_s$ 线性无关的充要条件是：方程 $k_1 \alpha_1 + k_2 \alpha_2 + \dots + k_s \alpha_s = \mathbf{0}$ 仅有零解 $k_1 = k_2 = \dots = k_s = 0$ 。



例题 5.2 线性相关性的判定 判断向量组 $\alpha = (1, 0, 2, 1)^\top, \beta = (1, 2, 0, 1)^\top, \gamma = (2, 1, 3, 0)^\top$ 的线性相关性。解：设 $k_1 \alpha + k_2 \beta + k_3 \gamma = \mathbf{0}$ ，得到齐次线性方程组。通过高斯消元法（或系数矩阵行列式）可解得仅有零解，故向量组线性无关。

注 [几何直观] 线性相关性有直观的几何意义：

- 二维向量组 $\{\mathbf{u}, \mathbf{v}\}$ 线性相关 $\iff$ 两向量共线。
- 三维向量组 $\{\mathbf{u}, \mathbf{v}, \mathbf{w}\}$ 线性相关 $\iff$ 三向量共面。

四、线性表示与唯一性

定理 5.2 (线性表示的唯一性)

设向量组 $\alpha_1, \alpha_2, \dots, \alpha_s$ 线性无关，且向量组 $\{\alpha_1, \alpha_2, \dots, \alpha_s, \beta\}$ 线性相关，则 β 可由 $\alpha_1, \alpha_2, \dots, \alpha_s$ 唯一地线性表示。



证明 [证明思路] 由 $\{\alpha_1, \dots, \alpha_s, \beta\}$ 线性相关，知存在不全为零的 $k_1, \dots, k_s, k$ 使得 $\sum_{i=1}^s k_i \alpha_i + k \beta = \mathbf{0}$ 。若 $k = 0$ ，则 $\sum_{i=1}^s k_i \alpha_i = \mathbf{0}$ 且 k_i 不全为零，与 $\alpha_1, \dots, \alpha_s$ 线性无关矛盾。故 $k \neq 0$ ，有 $\beta = -\sum_{i=1}^s \frac{k_i}{k} \alpha_i$ 。唯一性可通过设两种表示相减并利用线性无关性得证。

重要推论

若一个向量组可由另一个线性无关的向量组线性表示，且表示式中向量个数更多，则该向量组必线性相关。特别地，任意 $n+1$ 个 n 维向量必线性相关。

五、向量组的运算与线性关系的保持

要点提炼

向量组的线性关系在以下运算下保持不变：

1. **加长 (添分量)**: 若原向量组线性无关, 则统一添加相同数量的分量后仍线性无关; 若原向量组线性相关, 则加长后仍相关。
2. **截短 (去分量)**: 若原向量组线性相关, 则统一去掉相同数量的分量后仍线性相关; 但线性无关的向量组截短后未必无关。

注 这可理解为: 增加方程 (分量) 不会引入新的自由度, 但可能消除原有的自由度; 减少方程可能保留原有的线性关系, 但也可能破坏独立性。

六、极大线性无关组与向量组的秩

定义 5.6 (极大线性无关组)

向量组 A 的一个子组若满足:

1. 该子组本身线性无关;
2. 从 A 中任取一个不在该子组中的向量添加到子组中, 新组都线性相关, 则称该子组为向量组 A 的一个极大线性无关组。



定义 5.7 (向量组的秩)

向量组 A 的极大线性无关组中所含向量的个数称为该向量组的秩, 记作 $r(A)$ 。规定仅含零向量的向量组的秩为 0。



定理 5.3 (极大无关组的性质)

向量组 A 的任意两个极大线性无关组:

1. 含有相同数量的向量 (即秩唯一);
2. 彼此等价 (即可以相互线性表示)。



秩的等价刻画

设向量组 A 的秩为 r , 则:

1. A 中任意 $r+1$ 个向量必线性相关;
2. A 中任意线性无关子组所含向量个数不超过 r ;
3. A 的任意一个含 r 个向量的线性无关子组都是极大线性无关组。

方法/算法

求向量组的极大无关组与秩（高斯消元法）给定向量组 $\alpha_1, \alpha_2, \dots, \alpha_s$ ，求其极大无关组和秩的步骤：

1. 将向量作为列向量构成矩阵 $A = (\alpha_1, \alpha_2, \dots, \alpha_s)$;
2. 对 A 施行初等行变换，将其化为行最简形矩阵 R ;
3. R 中主元（每行第一个非零元）所在的列对应的原向量即构成一个极大无关组；
4. 极大无关组所含向量的个数即为向量组的秩 r 。

例题 5.3 求极大无关组与表示关系 求向量组 $\alpha_1 = \begin{pmatrix} 1 \\ 0 \\ 2 \\ 1 \end{pmatrix}, \alpha_2 = \begin{pmatrix} 1 \\ 2 \\ 0 \\ 1 \end{pmatrix}, \alpha_3 = \begin{pmatrix} 2 \\ 1 \\ 3 \\ 0 \end{pmatrix}, \alpha_4 =$

$\begin{pmatrix} 2 \\ 5 \\ -1 \\ 4 \end{pmatrix}, \alpha_5 = \begin{pmatrix} 1 \\ -1 \\ 3 \\ -1 \end{pmatrix}$ 的极大无关组、秩，并将其他向量用极大无关组线性表示。解：构

造矩阵 $A = (\alpha_1, \alpha_2, \alpha_3, \alpha_4, \alpha_5)$ ，行化简得行最简形：

$$R = \begin{bmatrix} 1 & 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & 3 & -1 \\ 0 & 0 & 1 & -1 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

主元在第 1、2、3 列，故 $\{\alpha_1, \alpha_2, \alpha_3\}$ 是一个极大无关组，秩为 3。由 R 第 4、5 列易得：
 $\alpha_4 = -\alpha_1 + 3\alpha_2 - \alpha_3, \alpha_5 = -\alpha_2 + \alpha_3$ 。

5.1.2 矩阵的秩

一、矩阵秩的定义：行秩、列秩与子式秩

定义 5.8 (行秩与列秩)

矩阵 A 的行秩定义为其行向量组的秩；列秩定义为其列向量组的秩。

**定义 5.9 (k 阶子式)**

从 $m \times n$ 矩阵 A 中任取 k 行 ($k \leq m$) 和 k 列 ($k \leq n$)，位于这些行和列交叉处的 k^2 个元素按原顺序排列构成的 k 阶行列式，称为 A 的一个 k 阶子式。



定理 5.4 (行秩、列秩与子式秩的等价性)

对任意矩阵 $\mathbf{A}$ ，其行秩、列秩以及最高阶非零子式的阶数（称为子式秩）三者相等，统称为矩阵的秩，记作 $\text{rank}(\mathbf{A})$ 或 $r(\mathbf{A})$ 。



证明 [证明思路 (子式秩 = 行秩)]

1. 若 $\mathbf{A}$ 的行秩为 r ，则存在 r 个线性无关的行向量，其余行向量可由其线性表示。由此可证这 r 个行向量构成的子矩阵中存在一个 r 阶非零子式，且所有 $r+1$ 阶子式全为零。
2. 同理可证子式秩也等于列秩。

推论 5.2

矩阵的初等行（列）变换不改变其秩。



二、矩阵秩的性质

要点提炼

矩阵的秩具有以下基本性质（假设运算可行）：

1. $0 \leq \text{rank}(\mathbf{A}_{m \times n}) \leq \min(m, n)$
2. $\text{rank}(\mathbf{A}^\top) = \text{rank}(\mathbf{A})$
3. $\text{rank}(k\mathbf{A}) = \begin{cases} \text{rank}(\mathbf{A}), & k \neq 0 \\ 0, & k = 0 \end{cases}$
4. $\text{rank}(\mathbf{AB}) \leq \min(\text{rank}(\mathbf{A}), \text{rank}(\mathbf{B}))$ （乘积不等式）
5. 若 $\mathbf{P}$ 、 $\mathbf{Q}$ 可逆，则 $\text{rank}(\mathbf{PAQ}) = \text{rank}(\mathbf{A})$ （乘可逆矩阵秩不变）
6. $\text{rank}(\mathbf{A} + \mathbf{B}) \leq \text{rank}(\mathbf{A}) + \text{rank}(\mathbf{B})$ （和不等式）
7. $\text{rank}\left(\begin{bmatrix} \mathbf{A} & \mathbf{O} \\ \mathbf{O} & \mathbf{B} \end{bmatrix}\right) = \text{rank}(\mathbf{A}) + \text{rank}(\mathbf{B})$ （分块对角阵）

证明 [乘积不等式证明概要] $\mathbf{AB}$ 的列向量是 $\mathbf{A}$ 的列向量的线性组合，故 $\text{rank}(\mathbf{AB}) \leq \text{rank}(\mathbf{A})$ 。同理， $\mathbf{AB}$ 的行向量是 $\mathbf{B}$ 的行向量的线性组合，故 $\text{rank}(\mathbf{AB}) \leq \text{rank}(\mathbf{B})$ 。综上得证。

三、矩阵秩的计算方法

方法/算法

子式法（适用于低阶矩阵）

1. 从最高可能的阶数 $k = \min(m, n)$ 开始，依次检查所有 k 阶子式；
2. 若存在一个 k 阶子式不为零，则 $\text{rank}(\mathbf{A}) = k$ ；
3. 若所有 k 阶子式全为零，则令 $k = k - 1$ ，重复步骤 2，直至找到非零子式。

例题 5.4 子式法求秩 设 $\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 3 & 5 \\ 4 & 7 & -1 \end{bmatrix}$ 。计算 3 阶子式 $|\mathbf{A}| = 0$ ；但存在 2 阶子式

$$\begin{vmatrix} 1 & 2 \\ 2 & 3 \end{vmatrix} = -1 \neq 0, \text{ 故 } \text{rank}(\mathbf{A}) = 2.$$

方法/算法

高斯消元法（通用方法）

1. 对矩阵 $\mathbf{A}$ 施行初等行变换，将其化为行阶梯形矩阵；
2. 行阶梯形矩阵中非零行的行数即为矩阵 $\mathbf{A}$ 的秩。
3. (可选) 进一步化为行最简形，可同时得到列向量的极大无关组及线性关系。

例题 5.5 消元法求秩 求矩阵 $\mathbf{A} = \begin{bmatrix} 2 & 8 & -1 & -1 & -2 \\ 1 & 4 & 2 & 1 & 0 \\ 1 & 4 & -1 & 0 & 2 \\ 0 & 0 & 1 & 1 & 2 \end{bmatrix}$ 的秩。解：经行变换得行阶梯

$$\text{形: } \begin{bmatrix} 1 & 4 & 2 & 1 & 0 \\ 0 & 0 & 1 & 1 & 2 \\ 0 & 0 & 0 & 2 & 8 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}. \text{ 非零行有 3 行, 故 } \text{rank}(\mathbf{A}) = 3.$$

四、矩阵的标准形与线性关系

定理 5.5 (矩阵的等价标准形)

设 $\mathbf{A}$ 为 $m \times n$ 矩阵， $\text{rank}(\mathbf{A}) = r$ ，则存在可逆矩阵 $\mathbf{P}_{m \times m}$ 和 $\mathbf{Q}_{n \times n}$ ，使得

$$\mathbf{PAQ} = \begin{bmatrix} \mathbf{I}_r & \mathbf{O} \\ \mathbf{O} & \mathbf{O} \end{bmatrix}_{m \times n},$$

其中 $\mathbf{I}_r$ 为 r 阶单位矩阵。



推论 5.3 (初等变换保持线性关系)

1. 初等行变换不改变矩阵列向量组之间的线性关系;
2. 初等列变换不改变矩阵行向量组之间的线性关系。

**提示**

此性质表明, 可通过行变换研究列向量组的线性关系 (如求极大无关组), 通过列变换研究行向量组的线性关系。

五、矩阵的秩与线性方程组

矩阵的秩是判断线性方程组解的情况的重要工具。

定理 5.6 (Rouché–Capelli 定理)

n 元线性方程组 $\mathbf{Ax} = \mathbf{b}$ 有解的充要条件是系数矩阵 $\mathbf{A}$ 的秩等于增广矩阵 $(\mathbf{A} | \mathbf{b})$ 的秩, 即 $\text{rank}(\mathbf{A}) = \text{rank}(\mathbf{A} | \mathbf{b})$ 。进一步:

- 若 $\text{rank}(\mathbf{A}) = \text{rank}(\mathbf{A} | \mathbf{b}) = n$, 则方程组有唯一解;
- 若 $\text{rank}(\mathbf{A}) = \text{rank}(\mathbf{A} | \mathbf{b}) = r < n$, 则方程组有无穷多解, 其通解中含有 $n - r$ 个自由未知量。

**定理 5.7 (齐次线性方程组解的结构)**

n 元齐次线性方程组 $\mathbf{Ax} = \mathbf{0}$ 的解集合构成一个线性空间, 称为解空间。解空间的维数 (基础解系所含向量的个数) 为 $n - \text{rank}(\mathbf{A})$ 。

**综合练习**

练习 5.1 判断向量组 $\alpha_1 = (1, 2, 3)^\top$, $\alpha_2 = (2, 4, 6)^\top$, $\alpha_3 = (1, 0, 1)^\top$ 的线性相关性。若相关, 求出其全部线性关系。

解 以 $\alpha_1, \alpha_2, \alpha_3$ 为列向量构造矩阵 $\mathbf{A} = \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 0 \\ 3 & 6 & 1 \end{bmatrix}$ 。

行变换得行阶梯形: $\begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & -2 \\ 0 & 0 & 0 \end{bmatrix}$, 秩为 2, 故向量组线性相关。

设 $k_1\alpha_1 + k_2\alpha_2 + k_3\alpha_3 = \mathbf{0}$, 解得 $k_1 = -2t, k_2 = t, k_3 = 0$ (t 为任意常数)。

全部线性关系为 $(-2t)\alpha_1 + t\alpha_2 + 0 \cdot \alpha_3 = \mathbf{0}$, 即 $2\alpha_1 - \alpha_2 = \mathbf{0}$ 。

练习 5.2 设 $\mathbf{A}$ 为 n 阶方阵。证明: $\text{rank}(\mathbf{AB}) = \text{rank}(\mathbf{B})$ 对任意 $n \times s$ 矩阵 $\mathbf{B}$ 成立的充要条件是 $\mathbf{A}$ 可逆。

解 必要性: 取 $\mathbf{B} = \mathbf{I}_n$, 则 $\text{rank}(\mathbf{A}) = \text{rank}(\mathbf{I}_n) = n$, 故 $\mathbf{A}$ 可逆。

充分性：若 $\mathbf{A}$ 可逆，则 $\mathbf{B} = \mathbf{A}^{-1}(\mathbf{AB})$ ，故 $\text{rank}(\mathbf{B}) \leq \text{rank}(\mathbf{AB})$ 。又由乘积不等式 $\text{rank}(\mathbf{AB}) \leq \text{rank}(\mathbf{B})$ ，故 $\text{rank}(\mathbf{AB}) = \text{rank}(\mathbf{B})$ 。

 **练习 5.3** 构造一个 3×4 矩阵 $\mathbf{A}$ ，使其秩为 2。验证其所有 3 阶子式均为零，但存在非零的 2 阶子式。

常见误区分析

注意

- **误区一：**认为“向量维数越高越容易线性无关”。
正解：在 n 维空间中，任意 $n+1$ 个向量必线性相关。向量个数而非维数决定相关性。
- **误区二：**混淆行变换与列变换对线性关系的影响。
正解：行变换保持列向量间的线性关系；列变换保持行向量间的线性关系。用行变换研究行关系或列变换研究列关系会破坏原有关系。
- **误区三：**误以为有非零解意味着唯一解。
正解：齐次方程组总有零解。有非零解意味着有无穷多解（包括零解）。唯一解只有零解。
- **误区四：**认为矩阵的秩就是其非零行的个数。
正解：矩阵的秩是其行阶梯形中非零行的个数，但需通过化简得到，而非直接观察原矩阵。

历史注记与应用概览

历史侧记

“秩”(Rank) 的概念由德国数学家 Ferdinand Georg Frobenius 于 1879 年正式提出并系统研究。这一概念深刻揭示了矩阵和向量组的内部结构，是线性代数理论体系形成的关键环节之一。矩阵的秩与行列式、线性方程组解的理论紧密相连，成为现代数学和工程计算中不可或缺的工具。

例题 5.6 矩阵秩的应用概览 矩阵的秩在许多领域有重要应用：

- **控制理论：**系统能控性、能观性的判据。
- **图像处理：**图像压缩、降噪（基于低秩近似）。
- **机器学习：**推荐系统（矩阵补全）、降维（PCA）等。
- **网络分析：**判断网络的连通性。

小结

要点提炼

本节核心内容总结：

1. **向量组**：线性组合、表示、等价；线性相关/无关的判定与性质；极大无关组与秩的定义和求法。
2. **矩阵的秩**：行秩、列秩、子式秩的三位一体；基本性质（特别是乘积不等式）；计算方法（子式法、消元法）。
3. **内在联系**：矩阵的秩等于其行（列）向量组的秩；初等变换不改变秩；秩是矩阵的固有特征。
4. **核心应用**：判断线性方程组解的存在性与唯一性（Rouché–Capelli 定理）；刻画解空间的结构。

代码视角

NumPy 中计算矩阵秩

```
import numpy as np

# 定义矩阵
A = np.array([[1, 2, 3],
              [2, 3, 5],
              [4, 7, -1]])

# 计算矩阵的秩
rank_A = np.linalg.matrix_rank(A)
print("Rank of A:", rank_A)

# 定义向量组
vectors = np.array([[1, 0, 2, 1],
                    [1, 2, 0, 1],
                    [2, 1, 3, 0],
                    [2, 5, -1, 4],
                    [1, -1, 3, -1]]).T # 转置为列向量组

# 计算向量组构成的矩阵的秩
rank_vectors = np.linalg.matrix_rank(vectors)
print("Rank of the vector group:", rank_vectors)
```


5.2 线性方程组的解法

要点提炼

本节核心要点

- 掌握非齐次与齐次线性方程组的有解判定条件（秩准则）
- 熟悉解的三种类型（无解、唯一解、无穷多解）及其判定方法
- 熟练运用高斯消元法（行初等变换）求解方程组
- 理解解的结构：特解与通解的概念及关系
- 了解线性方程组在几何与工程中的基本应用

线性方程组的求解是线性代数的核心问题之一。本节系统介绍非齐次和齐次线性方程组的解法，包括有解的条件、解的分类、求解方法（高斯消元法）以及解的结构表达，并简要讨论其几何意义与实际应用。

5.2.1 非齐次线性方程组的解法

一、基本形式与矩阵表示

定义 5.10 (非齐次线性方程组)

形如

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2 \\ \vdots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n = b_m \end{cases}$$

的方程组称为非齐次线性方程组，其中 $b_1, b_2, \dots, b_m$ 不全为零。

定义 5.11 (矩阵表示)

引入：

- 系数矩阵： $\mathbf{A} = (a_{ij})_{m \times n}$
- 未知向量： $\mathbf{x} = (x_1, x_2, \dots, x_n)^\top$
- 常数项向量： $\mathbf{b} = (b_1, b_2, \dots, b_m)^\top$

$$\bullet \text{ 增广矩阵: } [\mathbf{A} \mid \mathbf{b}] = \left(\begin{array}{cccc|c} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ a_{21} & a_{22} & \cdots & a_{2n} & b_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} & b_m \end{array} \right)$$

则方程组可简洁表示为矩阵方程：

$$\mathbf{Ax} = \mathbf{b}.$$

注 当 $\mathbf{b} = \mathbf{0}$ 时, 方程组称为齐次线性方程组, 否则称为非齐次线性方程组。齐次方程组总有零解 $\mathbf{x} = \mathbf{0}$ 。

二、有解的条件 (Rouché–Capelli 定理)

定理 5.8 (非齐次方程组有解的充要条件)

非齐次线性方程组 $\mathbf{Ax} = \mathbf{b}$ 有解的充要条件是系数矩阵 $\mathbf{A}$ 的秩等于增广矩阵 $[\mathbf{A} | \mathbf{b}]$ 的秩, 即

$$\text{rank}(\mathbf{A}) = \text{rank}([\mathbf{A} | \mathbf{b}]).$$



证明 [证明思路] $\mathbf{Ax} = \mathbf{b}$ 有解, 意味着 $\mathbf{b}$ 可由 $\mathbf{A}$ 的列向量组线性表出。此时, 将 $\mathbf{b}$ 加入 $\mathbf{A}$ 的列向量组不会改变该向量组的秩, 故 $\text{rank}(\mathbf{A}) = \text{rank}([\mathbf{A} | \mathbf{b}])$ 。反之亦然。

三、解的类型与判定

根据系数矩阵的秩、增广矩阵的秩与未知数个数 n 的关系, 非齐次线性方程组的解可分为三种情况:

1. **无解**: 当 $\text{rank}(\mathbf{A}) \neq \text{rank}([\mathbf{A} | \mathbf{b}])$ 时, 方程组无解。
2. **唯一解**: 当 $\text{rank}(\mathbf{A}) = \text{rank}([\mathbf{A} | \mathbf{b}]) = n$ 时, 方程组有唯一解。
3. **无穷多解**: 当 $\text{rank}(\mathbf{A}) = \text{rank}([\mathbf{A} | \mathbf{b}]) < n$ 时, 方程组有无穷多解。解的自由度为 $n - \text{rank}(\mathbf{A})$, 即可取 $n - \text{rank}(\mathbf{A})$ 个自由未知量。

要点提炼

解的类型判定

$$\text{rank}(\mathbf{A}) \neq \text{rank}([\mathbf{A} | \mathbf{b}]) \iff \text{无解}$$

$$\text{rank}(\mathbf{A}) = \text{rank}([\mathbf{A} | \mathbf{b}]) = n \iff \text{唯一解}$$

$$\text{rank}(\mathbf{A}) = \text{rank}([\mathbf{A} | \mathbf{b}]) < n \iff \text{无穷多解}$$

例题 5.7 判断解的类型 判断方程组 $\begin{cases} x_1 + 2x_2 = 1 \\ 2x_1 + 4x_2 = 3 \end{cases}$ 的解的类型。

解: 系数矩阵 $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix}$, 增广矩阵 $[\mathbf{A} | \mathbf{b}] = \left(\begin{array}{cc|c} 1 & 2 & 1 \\ 2 & 4 & 3 \end{array} \right)$ 。易得 $\text{rank}(\mathbf{A}) = 1$, 而 $\text{rank}([\mathbf{A} | \mathbf{b}]) = 2$, 两者不等, 故方程组无解。

四、高斯消元法 (行初等变换法)

高斯消元法是求解线性方程组最通用和直接的方法, 其核心是通过初等行变换将增广矩阵化为行阶梯形或行最简形, 从而判断解的情况并求解。

方法/算法

高斯消元法步骤

1. **构造增广矩阵**: 将方程组写成增广矩阵 $[\mathbf{A} \mid \mathbf{b}]$ 。
2. **行初等变换**: 对增广矩阵施行初等行变换, 将其化为行阶梯形矩阵 (从上到下, 每行首个非零元所在列号严格递增)。
3. **判断解的情况**: 根据行阶梯形矩阵判断 $\text{rank}(\mathbf{A})$ 与 $\text{rank}([\mathbf{A} \mid \mathbf{b}])$ 是否相等, 以及是否等于 n 。
4. **求解**:
 - 若唯一解, 则继续变换为行最简形 (行阶梯形且每行首个非零元为 1, 且是该列唯一的非零元), 然后回代求解。
 - 若无穷多解, 同样化为行最简形, 将非主元对应的未知量设为自由变量, 用自由变量表示主元变量。

例题 5.8 高斯消元法求唯一解 求解方程组

$$\begin{cases} 2x_1 + x_2 - 2x_3 = -1 \\ x_1 - 2x_2 + x_3 = 5 \\ x_1 + x_2 - x_3 = 2 \end{cases}$$

解:

$$1. \text{ 增广矩阵: } [\mathbf{A} \mid \mathbf{b}] = \left(\begin{array}{ccc|c} 2 & 1 & -2 & -1 \\ 1 & -2 & 1 & 5 \\ 1 & 1 & -1 & 2 \end{array} \right)$$

2. 行变换至行阶梯形:

$$\begin{aligned} & \xrightarrow{r_1 \leftrightarrow r_2} \left(\begin{array}{ccc|c} 1 & -2 & 1 & 5 \\ 2 & 1 & -2 & -1 \\ 1 & 1 & -1 & 2 \end{array} \right) \xrightarrow[r_3 - r_1]{r_2 - 2r_1} \left(\begin{array}{ccc|c} 1 & -2 & 1 & 5 \\ 0 & 5 & -4 & -11 \\ 0 & 3 & -2 & -3 \end{array} \right) \xrightarrow{r_2 \leftrightarrow r_3} \left(\begin{array}{ccc|c} 1 & -2 & 1 & 5 \\ 0 & 3 & -2 & -3 \\ 0 & 5 & -4 & -11 \end{array} \right) \\ & \xrightarrow{r_3 - \frac{5}{3}r_2} \left(\begin{array}{ccc|c} 1 & -2 & 1 & 5 \\ 0 & 3 & -2 & -3 \\ 0 & 0 & -\frac{2}{3} & -6 \end{array} \right) \end{aligned}$$

 $\text{rank}(\mathbf{A}) = \text{rank}([\mathbf{A} \mid \mathbf{b}]) = 3 = n$, 故有唯一解。

3. 继续变换至行最简形并回代:

$$\begin{aligned} & \xrightarrow[r_3 \times (-\frac{3}{2})]{r_2 \times \frac{1}{3}} \left(\begin{array}{ccc|c} 1 & -2 & 1 & 5 \\ 0 & 1 & -\frac{2}{3} & -1 \\ 0 & 0 & 1 & 9 \end{array} \right) \xrightarrow[r_2 + \frac{2}{3}r_3]{r_1 - r_3} \left(\begin{array}{ccc|c} 1 & -2 & 0 & -4 \\ 0 & 1 & 0 & 5 \\ 0 & 0 & 1 & 9 \end{array} \right) \xrightarrow{r_1 + 2r_2} \left(\begin{array}{ccc|c} 1 & 0 & 0 & 6 \\ 0 & 1 & 0 & 5 \\ 0 & 0 & 1 & 9 \end{array} \right) \end{aligned}$$

得解: $x_1 = 6, x_2 = 5, x_3 = 9$ 。

例题 5.9 高斯消元法求无穷多解 求解方程组

$$\begin{cases} x_1 + 2x_2 + 2x_3 + x_4 = 0 \\ 2x_1 + x_2 - 2x_3 - 2x_4 = 0 \\ x_1 - x_2 - 4x_3 - 3x_4 = 0 \end{cases}$$

解:

1. 增广矩阵: $[\mathbf{A} \mid \mathbf{b}] = \left(\begin{array}{cccc|c} 1 & 2 & 2 & 1 & 0 \\ 2 & 1 & -2 & -2 & 0 \\ 1 & -1 & -4 & -3 & 0 \end{array} \right)$

2. 行变换至行最简形:

$$\begin{aligned} & \xrightarrow[r_3-r_1]{r_2-2r_1} \left(\begin{array}{cccc|c} 1 & 2 & 2 & 1 & 0 \\ 0 & -3 & -6 & -4 & 0 \\ 0 & -3 & -6 & -4 & 0 \end{array} \right) \xrightarrow{r_3-r_2} \left(\begin{array}{cccc|c} 1 & 2 & 2 & 1 & 0 \\ 0 & -3 & -6 & -4 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{array} \right) \xrightarrow{r_2 \times (-\frac{1}{3})} \left(\begin{array}{cccc|c} 1 & 2 & 2 & 1 & 0 \\ 0 & 1 & 2 & \frac{4}{3} & 0 \\ 0 & 0 & 0 & 0 & 0 \end{array} \right) \\ & \xrightarrow{r_1-2r_2} \left(\begin{array}{cccc|c} 1 & 0 & -2 & -\frac{5}{3} & 0 \\ 0 & 1 & 2 & \frac{4}{3} & 0 \\ 0 & 0 & 0 & 0 & 0 \end{array} \right) \end{aligned}$$

 $\text{rank}(\mathbf{A}) = 2 < 4 = n$, 故有无穷多解。

3. 求解: 主元变量为 x_1, x_2 , 自由变量为 x_3, x_4 。令 $x_3 = t, x_4 = s$ (t, s 为任意常数), 则:

$$\begin{aligned} x_2 &= -2t - \frac{4}{3}s \\ x_1 &= 2t + \frac{5}{3}s \end{aligned}$$

解向量为:

$$\mathbf{x} = \begin{pmatrix} 2t + \frac{5}{3}s \\ -2t - \frac{4}{3}s \\ t \\ s \end{pmatrix} = t \begin{pmatrix} 2 \\ -2 \\ 1 \\ 0 \end{pmatrix} + s \begin{pmatrix} \frac{5}{3} \\ -\frac{4}{3} \\ 0 \\ 1 \end{pmatrix}$$

五、解的结构: 特解与通解

当非齐次线性方程组 $\mathbf{Ax} = \mathbf{b}$ 有无穷多解时, 其解可表示为:

$$\mathbf{x} = \mathbf{x}_0 + \mathbf{x}_h$$

其中:

- $\mathbf{x}_0$ 是 $\mathbf{Ax} = \mathbf{b}$ 的一个特解 (一个具体的解)。
- $\mathbf{x}_h$ 是对应齐次方程组 $\mathbf{Ax} = \mathbf{0}$ 的通解, 包含 $n - \text{rank}(\mathbf{A})$ 个线性无关的解向量 (基础解系) 的线性组合。

例题 5.10 求特解与通解 对于上例中的方程组，其解 $\mathbf{x} = t \begin{pmatrix} 2 \\ -2 \\ 1 \\ 0 \end{pmatrix} + s \begin{pmatrix} \frac{5}{3} \\ -\frac{4}{3} \\ 0 \\ 1 \end{pmatrix}$ 中， $\mathbf{x}_0 = \mathbf{0}$

(零向量是一个特解)， $\mathbf{x}_h = t \begin{pmatrix} 2 \\ -2 \\ 1 \\ 0 \end{pmatrix} + s \begin{pmatrix} \frac{5}{3} \\ -\frac{4}{3} \\ 0 \\ 1 \end{pmatrix}$ 是对应齐次方程组的通解。

六、几何意义与应用

- **几何意义：**在 n 维空间中，
 - 唯一解对应一个点（所有超平面的唯一交点）。
 - 无穷多解对应一个线性流形（例如直线、平面等）。
 - 无解意味着所有超平面没有公共交点。
- **应用：**线性方程组广泛存在于科学与工程中，
 - 电路分析：根据基尔霍夫定律列写节点电压或回路电流方程。
 - 结构力学：求解桁架结构的受力平衡方程。
 - 经济学：列昂惕夫投入产出模型。
 - 计算机图形学：三维坐标变换与透视投影。

5.2.2 齐次线性方程组的解法

一、基本形式与有解条件

定义 5.12 (齐次线性方程组)

形如 $\mathbf{Ax} = \mathbf{0}$ 的方程组称为齐次线性方程组。



定理 5.9 (齐次方程组的解)

齐次方程组 $\mathbf{Ax} = \mathbf{0}$ 总有零解。它有非零解的充要条件是 $\text{rank}(\mathbf{A}) < n$ （未知量的个数）。



二、解的结构与基础解系

定理 5.10 (齐次方程组解的结构)

若 $\text{rank}(\mathbf{A}) = r < n$, 则齐次方程组 $\mathbf{Ax} = \mathbf{0}$ 的解集合构成一个 $n - r$ 维的线性空间, 称为解空间。解空间中任意一组极大线性无关组称为该方程组的基础解系, 其包含 $n - r$ 个线性无关的解向量。方程组的通解可表示为基础解系的线性组合:

$$\mathbf{x} = k_1 \xi_1 + k_2 \xi_2 + \cdots + k_{n-r} \xi_{n-r}$$

其中 $k_1, k_2, \dots, k_{n-r}$ 为任意常数。



方法/算法

基础解系的求法

1. 用行初等变换将系数矩阵 $\mathbf{A}$ 化为行最简形矩阵 $\mathbf{R}$ 。
2. 根据 $\mathbf{R}$, 确定主元列和自由元列。设自由元为 $x_{j_1}, x_{j_2}, \dots, x_{j_{n-r}}$ 。
3. 依次令某个自由元为 1, 其余自由元为 0, 回代解出主元变量, 得到一个解向量。
4. 重复步骤 3, 共得到 $n - r$ 个线性无关的解向量, 即为一个基础解系。

例题 5.11 求齐次方程组的基础解系 求方程组

$$\begin{cases} x_1 + 2x_2 + 2x_3 + x_4 = 0 \\ 2x_1 + x_2 - 2x_3 - 2x_4 = 0 \\ x_1 - x_2 - 4x_3 - 3x_4 = 0 \end{cases}$$

的基础解系和通解。

解: 系数矩阵 $\mathbf{A}$ 的行最简形同上例:

$$\mathbf{R} = \begin{pmatrix} 1 & 0 & -2 & -\frac{5}{3} \\ 0 & 1 & 2 & \frac{4}{3} \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

主元列为第 1、2 列, 自由元为 x_3, x_4 。令自由元取值:

- 令 $x_3 = 1, x_4 = 0$, 解得 $x_2 = -2, x_1 = 2$, 得解向量 $\xi_1 = \begin{pmatrix} 2 \\ -2 \\ 1 \\ 0 \end{pmatrix}$ 。
- 令 $x_3 = 0, x_4 = 1$, 解得 $x_2 = -\frac{4}{3}, x_1 = \frac{5}{3}$, 得解向量 $\xi_2 = \begin{pmatrix} \frac{5}{3} \\ -\frac{4}{3} \\ 0 \\ 1 \end{pmatrix}$ 。

故基础解系为 $\{\xi_1, \xi_2\}$, 通解为 $\mathbf{x} = k_1 \begin{pmatrix} 2 \\ -2 \\ 1 \\ 0 \end{pmatrix} + k_2 \begin{pmatrix} \frac{5}{3} \\ -\frac{4}{3} \\ 0 \\ 1 \end{pmatrix}$ (k_1, k_2 为任意常数)。

三、与非齐次方程组解的关系

定理 5.11 (非齐次方程组的解的结构)

设 $\mathbf{x}_0$ 是非齐次方程组 $\mathbf{Ax} = \mathbf{b}$ 的一个特解, $\xi_1, \xi_2, \dots, \xi_{n-r}$ 是对应齐次方程组 $\mathbf{Ax} = \mathbf{0}$ 的一个基础解系, 则 $\mathbf{Ax} = \mathbf{b}$ 的通解为:

$$\mathbf{x} = \mathbf{x}_0 + k_1 \xi_1 + k_2 \xi_2 + \cdots + k_{n-r} \xi_{n-r}$$

其中 $k_1, k_2, \dots, k_{n-r}$ 为任意常数。



例题 5.12 求非齐次方程组的通解 求方程组

$$\begin{cases} x_1 + 2x_2 + 2x_3 + x_4 = 1 \\ 2x_1 + x_2 - 2x_3 - 2x_4 = 2 \\ x_1 - x_2 - 4x_3 - 3x_4 = 3 \end{cases}$$

的通解。

解:

1. 先求对应齐次方程组的基础解系 (同上例): $\left\{ \begin{pmatrix} 2 \\ -2 \\ 1 \\ 0 \end{pmatrix}, \begin{pmatrix} \frac{5}{3} \\ -\frac{4}{3} \\ 0 \\ 1 \end{pmatrix} \right\}$ 。

2. 再求非齐次方程组的一个特解。令自由变量 $x_3 = 0, x_4 = 0$, 代入原方程组:

$$\begin{cases} x_1 + 2x_2 = 1 \\ 2x_1 + x_2 = 2 \\ x_1 - x_2 = 3 \end{cases}$$

解得 $x_1 = \frac{5}{3}, x_2 = -\frac{4}{3}$, 故特解 $\mathbf{x}_0 = \begin{pmatrix} \frac{5}{3} \\ -\frac{4}{3} \\ 0 \\ 0 \end{pmatrix}$ 。

3. 故非齐次方程组的通解为:

$$\mathbf{x} = \begin{pmatrix} \frac{5}{3} \\ -\frac{4}{3} \\ 0 \\ 0 \end{pmatrix} + k_1 \begin{pmatrix} 2 \\ -2 \\ 1 \\ 0 \end{pmatrix} + k_2 \begin{pmatrix} \frac{5}{3} \\ -\frac{4}{3} \\ 0 \\ 1 \end{pmatrix} \quad (k_1, k_2 \in \mathbb{R})$$

小结与说明

要点提炼

- **核心方法**：高斯消元法（行初等变换）是求解各类线性方程组的基础。
- **关键概念**：矩阵的秩决定了方程组解的情况（存在性、唯一性）。
- **解的结构**：非齐次方程组的通解 = 特解 + 对应齐次方程组通解。
- **数值计算**：对于大规模或病态方程组，需采用数值稳定算法（如 LU 分解、迭代法）。

历史侧记

高斯消元法以德国数学家卡尔·弗里德里希·高斯（Carl Friedrich Gauss）命名，但其思想早在古代中国数学著作《九章算术》中就已出现。矩阵的秩和现代线性代数理论则在 19 世纪末至 20 世纪初由众多数学家发展完善。

代码视角

Python/NumPy 求解线性方程组

```
import numpy as np

# 解非齐次方程组  $Ax = b$ 
A = np.array([[2, 1, -2],
               [1, -2, 1],
               [1, 1, -1]])
b = np.array([-1, 5, 2])
x = np.linalg.solve(A, b) # 求唯一解
print("Unique solution:", x)

# 解齐次方程组  $Ax = 0$ 
x_homogeneous = np.linalg.svd(A)[2][-1, :] # 最小奇异值对应的右奇异向量
print("Homogeneous solution (one basis):", x_homogeneous)

# 对于欠定或超定方程组，可使用最小二乘法
A_under = np.array([[1, 2, 2, 1],
                     [2, 1, -2, -2],
                     [1, -1, -4, -3]])
b_under = np.array([0, 0, 0])
x_under, residuals, rank, s = np.linalg.lstsq(A_under, b_under, rcond=None)
```

```
print("Underdetermined solution (min norm):", x_under)
```

练习

 **练习 5.4** 判断下列方程组是否有解，并求解（若有解）：

$$\begin{cases} x_1 + 2x_2 - x_3 = 1 \\ 2x_1 + 3x_2 + x_3 = 2 \\ 3x_1 + 5x_2 = 3 \end{cases}$$

 **练习 5.5** 求齐次线性方程组的基础解系：

$$\begin{cases} x_1 + 2x_2 + 3x_3 = 0 \\ 4x_1 + 5x_2 + 6x_3 = 0 \\ 7x_1 + 8x_2 + 9x_3 = 0 \end{cases}$$

 **练习 5.6** 设 $\mathbf{A}$ 为 4×3 矩阵， $\mathbf{b}$ 为 4 维向量。已知 $\text{rank}(\mathbf{A}) = 2$, $\text{rank}([\mathbf{A} \mid \mathbf{b}]) = 2$ ，问方程组 $\mathbf{Ax} = \mathbf{b}$ 的解有什么特点？并说明理由。

代码视角

练习参考答案

1. 增广矩阵行变换后最后一行为 $(0, 0, 0|1)$ ，故无解。

2. 系数矩阵行变换后为 $\begin{pmatrix} 1 & 2 & 3 \\ 0 & -3 & -6 \\ 0 & 0 & 0 \end{pmatrix}$ ，基础解系含一个向量，如 $(1, -2, 1)^T$ 。

3. 有无穷多解，解空间维数为 $3 - 2 = 1$ 。

5.3 线性方程组解的结构

要点提炼

本节核心要点

- 理解齐次线性方程组解空间的构成与基础解系的概念
- 掌握非齐次线性方程组解的结构：通解 = 特解 + 齐次通解
- 熟练运用高斯消元法求解基础解系与特解
- 理解解集的几何意义（线性子空间与仿射空间）
- 掌握与解结构相关的秩定理与矩阵恒等式

线性方程组的解的结构理论是线性代数的核心内容之一，它系统揭示了方程组的解集如何依赖于系数矩阵的秩，并提供了统一的求解框架。本节将深入探讨齐次与非齐次线性方程组的解集结构、求解方法及其几何解释，并给出若干重要的推论与应用。

5.3.1 齐次线性方程组的基础解系

一、解空间与零空间

定义 5.13 (解空间/零空间)

对于齐次线性方程组 $\mathbf{Ax} = \mathbf{0}$ ，其中 $\mathbf{A}$ 是 $m \times n$ 矩阵，其所有解向量构成的集合称为方程组的解空间，也称为矩阵 $\mathbf{A}$ 的零空间 (Null Space)，记作：

$$\mathcal{N}(\mathbf{A}) = \{\mathbf{x} \in \mathbb{R}^n \mid \mathbf{Ax} = \mathbf{0}\}.$$



定理 5.12 (解空间的线性性)

解空间 $\mathcal{N}(\mathbf{A})$ 是 $\mathbb{R}^n$ 的一个线性子空间。即：

1. 若 $\xi_1, \xi_2 \in \mathcal{N}(\mathbf{A})$ ，则 $\xi_1 + \xi_2 \in \mathcal{N}(\mathbf{A})$ ；
2. 若 $\xi \in \mathcal{N}(\mathbf{A})$ ， $k \in \mathbb{R}$ ，则 $k\xi \in \mathcal{N}(\mathbf{A})$ 。



二、基础解系的概念与存在性

定义 5.14 (基础解系)

设齐次线性方程组 $\mathbf{Ax} = \mathbf{0}$ 的解空间 $\mathcal{N}(\mathbf{A})$ 的维数为 d 。则该解空间的任意一组基 $\{\xi_1, \xi_2, \dots, \xi_d\}$ 称为方程组的一个基础解系。其通解可表示为：

$$\mathbf{x} = k_1\xi_1 + k_2\xi_2 + \dots + k_d\xi_d, \quad k_1, k_2, \dots, k_d \in \mathbb{R}.$$



定理 5.13 (秩—零化度定理)

设矩阵 $\mathbf{A}$ 的秩为 r ，则其零空间的维数为：

$$\dim \mathcal{N}(\mathbf{A}) = n - r.$$

因此，齐次方程组 $\mathbf{Ax} = \mathbf{0}$ 的基础解系包含 $n - r$ 个线性无关的解向量。



注 齐次方程组总有零解。当 $\text{rank}(\mathbf{A}) = n$ 时，方程组只有零解，解空间为 $\{\mathbf{0}\}$ ，维数为 0，无基础解系；当 $\text{rank}(\mathbf{A}) < n$ 时，方程组有非零解，存在含 $n - r$ 个向量的基础解系。

三、基础解系的求法 (高斯消元法)**方法/算法**

基础解系的构造步骤

1. **行化简**：对系数矩阵 $\mathbf{A}$ 施行初等行变换，化为行最简形矩阵 $\mathbf{R}$ 。
2. **确定自由未知量**：设 $\text{rank}(\mathbf{A}) = r$ ，则行最简形 $\mathbf{R}$ 中非主元列对应的 $n - r$ 个未知量为自由未知量。
3. **赋值求解**：依次令每个自由未知量为 1，其余自由未知量为 0，回代求解出主元未知量，得到一个解向量。
4. **重复构造**：重复步骤 3，共得到 $n - r$ 个线性无关的解向量 $\xi_1, \xi_2, \dots, \xi_{n-r}$ ，它们构成一个基础解系。

例题 5.13 求基础解系与通解 求齐次方程组

$$\begin{cases} x_1 + x_2 - x_3 - x_4 = 0 \\ 2x_1 - 5x_2 + 3x_3 + 2x_4 = 0 \\ 7x_1 - 7x_2 + 3x_3 + x_4 = 0 \end{cases}$$

的基础解系与通解。

解：系数矩阵 $\mathbf{A} = \begin{pmatrix} 1 & 1 & -1 & -1 \\ 2 & -5 & 3 & 2 \\ 7 & -7 & 3 & 1 \end{pmatrix}$ 经行变换化为行最简形：

$$\mathbf{R} = \begin{pmatrix} 1 & 0 & -\frac{2}{7} & -\frac{3}{7} \\ 0 & 1 & -\frac{5}{7} & -\frac{4}{7} \\ 0 & 0 & 0 & 0 \end{pmatrix}.$$

秩 $r = 2$ ，自由未知量个数为 $n - r = 2$ (选 x_3, x_4)。同解方程组：

$$\begin{cases} x_1 = \frac{2}{7}x_3 + \frac{3}{7}x_4 \\ x_2 = \frac{5}{7}x_3 + \frac{4}{7}x_4 \end{cases}$$

- 令 $x_3 = 1, x_4 = 0$ ，得 $\xi_1 = (\frac{2}{7}, \frac{5}{7}, 1, 0)^\top$ 。

- 令 $x_3 = 0, x_4 = 1$, 得 $\xi_2 = (\frac{3}{7}, \frac{4}{7}, 0, 1)^T$ 。

通解为 $\mathbf{x} = k_1\xi_1 + k_2\xi_2$ ($k_1, k_2 \in \mathbb{R}$)。

基础解系的特性与注意事项

- **不唯一性**: 基础解系不唯一, 自由未知量的选取不同, 所得基础解系也不同, 但所有基础解系都等价且包含相同个数的向量。
- **行变换保持列关系**: 初等行变换不改变列向量间的线性关系, 因此可用于求解基础解系; 但列变换会改变解空间, 应避免使用。
- **几何意义**: 当 $n - r = 1$ 时, 解空间是过原点的一条直线; 当 $n - r = 2$ 时, 解空间是过原点的一个平面; 以此类推。

5.3.2 非齐次线性方程组的解的结构

一、解的存在性与导出组

定义 5.15 (导出组)

对于非齐次线性方程组 $\mathbf{Ax} = \mathbf{b}$ ($\mathbf{b} \neq \mathbf{0}$), 其对应的齐次方程组 $\mathbf{Ax} = \mathbf{0}$ 称为它的导出组。

定理 5.14 (解的存在性)

非齐次方程组 $\mathbf{Ax} = \mathbf{b}$ 有解的充要条件是系数矩阵的秩等于增广矩阵的秩, 即 $\text{rank}(\mathbf{A}) = \text{rank}([\mathbf{A} | \mathbf{b}])$ 。进一步:

- 若 $\text{rank}(\mathbf{A}) = \text{rank}([\mathbf{A} | \mathbf{b}]) = n$, 则有唯一解;
- 若 $\text{rank}(\mathbf{A}) = \text{rank}([\mathbf{A} | \mathbf{b}]) < n$, 则有无穷多解。

二、解的结构: 特解与通解

定理 5.15 (非齐次方程组的解结构)

若 $\mathbf{Ax} = \mathbf{b}$ 有解, $\boldsymbol{\eta}^*$ 是它的一个特解 (即 $\mathbf{A}\boldsymbol{\eta}^* = \mathbf{b}$), $\xi_1, \xi_2, \dots, \xi_{n-r}$ 是其导出组 $\mathbf{Ax} = \mathbf{0}$ 的一个基础解系, 则 $\mathbf{Ax} = \mathbf{b}$ 的通解为

$$\mathbf{x} = \boldsymbol{\eta}^* + k_1\xi_1 + k_2\xi_2 + \dots + k_{n-r}\xi_{n-r}, \quad k_1, k_2, \dots, k_{n-r} \in \mathbb{R}.$$

证明

1. (特解 + 齐次解仍为解) 对于任意常数 $k_1, k_2, \dots, k_{n-r}$,

$$\mathbf{A}(\boldsymbol{\eta}^* + \sum_{i=1}^{n-r} k_i \xi_i) = \mathbf{A}\boldsymbol{\eta}^* + \sum_{i=1}^{n-r} k_i \mathbf{A}\xi_i = \mathbf{b} + \mathbf{0} = \mathbf{b}.$$

2. (任一解可表示为特解 + 齐次解) 设 $\mathbf{x}$ 是 $\mathbf{Ax} = \mathbf{b}$ 的任意一个解, 则

$$\mathbf{A}(\mathbf{x} - \boldsymbol{\eta}^*) = \mathbf{Ax} - \mathbf{A}\boldsymbol{\eta}^* = \mathbf{b} - \mathbf{b} = \mathbf{0},$$

故 $\mathbf{x} - \boldsymbol{\eta}^* \in \mathcal{N}(\mathbf{A})$, 即可由基础解系线性表出。

推论 5.4 (解的性质)

1. 非齐次方程组任意两个解的差是其导出组的解。
2. 非齐次方程组的一个解与其导出组的一个解之和仍是该非齐次方程组的解。♡

方法/算法

非齐次方程组通解的求解步骤

1. 判断解的存在性: 对增广矩阵 $[\mathbf{A} \mid \mathbf{b}]$ 施行行初等变换, 化为行阶梯形。若 $\text{rank}(\mathbf{A}) < \text{rank}([\mathbf{A} \mid \mathbf{b}])$, 则无解; 否则有解。
2. 求特解: 将有解的非齐次方程组继续化为行最简形。令所有自由未知量为 0, 解出主元未知量, 得到一个特解 $\boldsymbol{\eta}^*$ 。
3. 求齐次通解: 忽略常数项, 对系数矩阵 $\mathbf{A}$ 施行行初等变换, 求出导出组的基础解系 $\xi_1, \xi_2, \dots, \xi_{n-r}$ 。
4. 写出通解: $\mathbf{x} = \boldsymbol{\eta}^* + \sum_{i=1}^{n-r} k_i \xi_i$ 。

例题 5.14 求解非齐次方程组 求解方程组:

$$\begin{cases} x_1 - x_2 - x_3 + x_4 = 0 \\ x_1 - x_2 + x_3 - 3x_4 = 1 \\ x_1 - x_2 - 2x_3 + 3x_4 = -\frac{1}{2} \end{cases}$$

解: 增广矩阵为

$$[\mathbf{A} \mid \mathbf{b}] = \left(\begin{array}{cccc|c} 1 & -1 & -1 & 1 & 0 \\ 1 & -1 & 1 & -3 & 1 \\ 1 & -1 & -2 & 3 & -\frac{1}{2} \end{array} \right)$$

行变换得行最简形:

$$\sim \left(\begin{array}{cccc|c} 1 & -1 & 0 & -1 & \frac{1}{2} \\ 0 & 0 & 1 & -2 & \frac{1}{2} \\ 0 & 0 & 0 & 0 & 0 \end{array} \right)$$

$\text{rank}(\mathbf{A}) = \text{rank}([\mathbf{A} \mid \mathbf{b}]) = 2 < 4$, 故有无穷多解。同解方程组:

$$\begin{cases} x_1 = x_2 + x_4 + \frac{1}{2} \\ x_3 = 2x_4 + \frac{1}{2} \end{cases}$$

• **求特解:** 令自由未知量 $x_2 = 0, x_4 = 0$, 得特解 $\boldsymbol{\eta}^* = (\frac{1}{2}, 0, \frac{1}{2}, 0)^\top$ 。

• **求齐次通解:** 导出组为 $\begin{cases} x_1 = x_2 + x_4 \\ x_3 = 2x_4 \end{cases}$ 。令 $(x_2, x_4) = (1, 0)$ 和 $(0, 1)$, 得基础解系:

$$\xi_1 = (1, 1, 0, 0)^\top, \quad \xi_2 = (1, 0, 2, 1)^\top.$$

• 通解:

$$\mathbf{x} = \begin{pmatrix} \frac{1}{2} \\ 0 \\ \frac{1}{2} \\ 0 \end{pmatrix} + k_1 \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \end{pmatrix} + k_2 \begin{pmatrix} 1 \\ 0 \\ 2 \\ 1 \end{pmatrix}, \quad k_1, k_2 \in \mathbb{R}.$$

title= 非齐次解集的几何意义

非齐次方程组 $\mathbf{Ax} = \mathbf{b}$ 的解集是其导出组解空间 $\mathcal{N}(\mathbf{A})$ 的一个仿射平移:

$$\{\mathbf{x} \in \mathbb{R}^n \mid \mathbf{Ax} = \mathbf{b}\} = \boldsymbol{\eta}^* + \mathcal{N}(\mathbf{A}).$$

其几何形状与 $\mathcal{N}(\mathbf{A})$ 相同 (如直线、平面等), 但不再经过原点。

5.3.3 典型命题与推论

一、矩阵乘积与零空间

定理 5.16 ($\mathbf{AB} = \mathbf{O}$ 的刻画)

设 $\mathbf{A}_{m \times n}, \mathbf{B}_{n \times s}$, 则

$$\mathbf{AB} = \mathbf{O} \iff \mathbf{B} \text{ 的每一列都属于 } \mathcal{N}(\mathbf{A}).$$

证明 将 $\mathbf{B}$ 按列分块为 $\mathbf{B} = [\boldsymbol{\beta}_1, \boldsymbol{\beta}_2, \dots, \boldsymbol{\beta}_s]$, 则

$$\mathbf{AB} = [\mathbf{A}\boldsymbol{\beta}_1, \mathbf{A}\boldsymbol{\beta}_2, \dots, \mathbf{A}\boldsymbol{\beta}_s].$$

故 $\mathbf{AB} = \mathbf{O}$ 当且仅当对每一列 $\boldsymbol{\beta}_j$ 都有 $\mathbf{A}\boldsymbol{\beta}_j = \mathbf{0}$, 即 $\boldsymbol{\beta}_j \in \mathcal{N}(\mathbf{A})$ 。

二、秩的不等式

定理 5.17 (秩不等式)

若 $\mathbf{A}_{m \times n} \mathbf{B}_{n \times s} = \mathbf{O}$, 则

$$\text{rank}(\mathbf{A}) + \text{rank}(\mathbf{B}) \leq n.$$

证明 由 $\mathbf{AB} = \mathbf{O}$ 知 $\mathcal{C}(\mathbf{B}) \subseteq \mathcal{N}(\mathbf{A})$, 其中 $\mathcal{C}(\mathbf{B})$ 是 $\mathbf{B}$ 的列空间。故

$$\text{rank}(\mathbf{B}) = \dim \mathcal{C}(\mathbf{B}) \leq \dim \mathcal{N}(\mathbf{A}) = n - \text{rank}(\mathbf{A}),$$

移项即得结论。

三、重要恒等式 $\text{rank}(\mathbf{A}^\top \mathbf{A}) = \text{rank}(\mathbf{A})$

定理 5.18

对任意实矩阵 $\mathbf{A}_{m \times n}$, 有

$$\text{rank}(\mathbf{A}^\top \mathbf{A}) = \text{rank}(\mathbf{A}).$$



证明

1. 一方面, 若 $\mathbf{Ax} = \mathbf{0}$, 则 $\mathbf{A}^\top \mathbf{Ax} = \mathbf{0}$, 故 $\mathcal{N}(\mathbf{A}) \subseteq \mathcal{N}(\mathbf{A}^\top \mathbf{A})$ 。

2. 另一方面, 若 $\mathbf{A}^\top \mathbf{Ax} = \mathbf{0}$, 则

$$\mathbf{x}^\top \mathbf{A}^\top \mathbf{Ax} = (\mathbf{Ax})^\top (\mathbf{Ax}) = \|\mathbf{Ax}\|_2^2 = 0,$$

故 $\mathbf{Ax} = \mathbf{0}$, 即 $\mathcal{N}(\mathbf{A}^\top \mathbf{A}) \subseteq \mathcal{N}(\mathbf{A})$ 。

3. 综上, $\mathcal{N}(\mathbf{A}^\top \mathbf{A}) = \mathcal{N}(\mathbf{A})$, 故 $\text{nullity}(\mathbf{A}^\top \mathbf{A}) = \text{nullity}(\mathbf{A})$ 。由秩—零化度定理, $\text{rank}(\mathbf{A}^\top \mathbf{A}) = \text{rank}(\mathbf{A})$ 。

注 该定理表明, 矩阵 $\mathbf{A}^\top \mathbf{A}$ 与 $\mathbf{A}$ 有相同的零空间和列空间维数, 这在最小二乘法和数值线性代数中非常重要。

四、特解平移族的线性无关性

定理 5.19

设 $\mathbf{Ax} = \mathbf{b}$ ($\mathbf{b} \neq \mathbf{0}$) 有解, $\boldsymbol{\eta}^*$ 是它的一个特解, $\{\xi_1, \xi_2, \dots, \xi_{n-r}\}$ 是导出组 $\mathbf{Ax} = \mathbf{0}$ 的一个基础解系。则向量组

$$\boldsymbol{\eta}^*, \boldsymbol{\eta}^* + \xi_1, \boldsymbol{\eta}^* + \xi_2, \dots, \boldsymbol{\eta}^* + \xi_{n-r}$$

线性无关。



证明 设存在一组数 $k_0, k_1, \dots, k_{n-r}$, 使得

$$k_0 \boldsymbol{\eta}^* + \sum_{i=1}^{n-r} k_i (\boldsymbol{\eta}^* + \xi_i) = \mathbf{0}.$$

整理得:

$$\left(k_0 + \sum_{i=1}^{n-r} k_i \right) \boldsymbol{\eta}^* + \sum_{i=1}^{n-r} k_i \xi_i = \mathbf{0}.$$

左乘 $\mathbf{A}$, 并利用 $\mathbf{A}\boldsymbol{\eta}^* = \mathbf{b}$, $\mathbf{A}\xi_i = \mathbf{0}$, 得:

$$\left(k_0 + \sum_{i=1}^{n-r} k_i \right) \mathbf{b} = \mathbf{0}.$$

由于 $\mathbf{b} \neq \mathbf{0}$, 故 $k_0 + \sum_{i=1}^{n-r} k_i = 0$ 。代入原式得 $\sum_{i=1}^{n-r} k_i \xi_i = \mathbf{0}$ 。由基础解系的线性无关性, $k_1 = k_2 = \dots = k_{n-r} = 0$, 进而 $k_0 = 0$ 。证毕。

小结

要点提炼

本节核心内容总结：

1. **齐次方程组**：解空间是线性子空间，维数为 $n - \text{rank}(\mathbf{A})$ ；基础解系是解空间的基，通解为基础解系的线性组合。
2. **非齐次方程组**：有解 $\iff \text{rank}(\mathbf{A}) = \text{rank}([\mathbf{A} \mid \mathbf{b}])$ ；通解 = 特解 + 齐次通解；解集是仿射空间。
3. **求解方法**：高斯消元法是求基础解系和特解的通用方法。
4. **重要结论**： $\mathbf{AB} = \mathbf{O} \Rightarrow \text{rank}(\mathbf{A}) + \text{rank}(\mathbf{B}) \leq n$ ； $\text{rank}(\mathbf{A}^\top \mathbf{A}) = \text{rank}(\mathbf{A})$ 。

历史侧记

解空间与基础解系的概念由德国数学家理查德·戴德金（Richard Dedekind）和大卫·希尔伯特（David Hilbert）等人在 19 世纪末发展完善，成为线性代数理论体系的重要基石。秩-零化度定理则完美连接了矩阵的像空间与零空间，体现了线性代数中深刻的对偶思想。

代码视角

NumPy 求解基础解系与特解

```
import numpy as np
from scipy.linalg import null_space

# 求解齐次方程组 Ax=0 的基础解系
A = np.array([...]) # 输入系数矩阵
null_vecs = null_space(A) # 返回的列向量构成基础解系（近似）
print("基础解系（列向量形式）：\n", null_vecs)

# 求解非齐次方程组 Ax=b 的特解（最小二乘意义下的特解）
b = np.array([...])
eta, residuals, rank, s = np.linalg.lstsq(A, b, rcond=None)
print("一个特解（最小范数解）：", eta)
```

第 6 章 伟大的抽象：线性空间

本章导览

- 从具体解到解空间：视角的跃迁
- 线性空间的公理体系：源于几何，归于抽象
- 抽象的统一性：向量空间的泛化实例
- 空间的度量：基、坐标与维数
- 高维空间的特性、挑战与应用

6.1 引言：从具体解到解空间

历史侧记

线性方程组的求解历史源远流长，其雏形可追溯至古代文明。然而，真正开启代数思维大门的，是中国汉代的数学巨著《九章算术》（约公元前 2 世纪），其中“方程”一章系统地阐述了使用算筹（一种计算工具）求解线性方程组的方法，这在本质上就是我们今天所说的高斯消元法。在欧洲，直到 17、18 世纪，随着笛卡尔创立解析几何，以及莱布尼茨和克拉默对行列式理论的探索，线性方程组的研究才开始系统化。然而，在长达两千年的时间里，数学家们的焦点始终是“求解”——找到那一个或一组满足所有方程的具体数值。

思想的革命性飞跃发生在 19 世纪。数学家们开始将目光从孤立的解，转向由所有解构成的“集合”。这一转变意义非凡，它标志着数学思维从“计算答案”向“理解结构”的深刻跃迁。当人们发现齐次方程组的解集（包括零解）在加法和数乘运算下是封闭的，而有解的非齐次方程组的解集（不包含零解）则是这个封闭集合的“平移”时，一个全新的、更宏大的图景徐徐展开。这个解的集合不再是一盘散沙，它拥有自己的内在规律和几何形态。正是对这种“解的结构”的探寻，最终催生了线性代数中最核心、最抽象也最强大的概念——线性空间。本章的旅程，正是要追随先贤的足迹，完成这次从“解”到“空间”的伟大思想升维。

在上一章中，我们掌握了求解线性方程组的有效方法——高斯消元法。无论是处理非齐次方程组 $Ax = b$ 还是齐次方程组 $Ax = 0$ ，我们的目标都聚焦于寻找具体的解向量。当解唯一时，它对应几何空间中的一个点；当解无穷时，解集则表现为一条直线或一个平面。在这些计算中，解向量是探究的终点。

然而，数学的深刻发展往往源于视角的转变。现在，我们将进行一次至关重要的思想升维：不再将目光局限于孤立的解，而是转向审视所有可能解构成的集合，并探究该集合内蕴的代数结构。让我们从一个根本性问题开始：齐次线性方程组 $Ax = 0$ 的所有

解，构成了一个怎样的集合？

这个问题的答案揭示了一种深刻的结构性差异。齐次方程组的解集必然包含**零向量** ($\vec{0}$)，因为 $A\vec{0} = \vec{0}$ 恒成立。相比之下，一个有解的非齐次方程组 $Ax = b$ ($b \neq \vec{0}$) 的解集，虽然也可能呈现为直线或平面，但它**永不包含零向量**。从几何上看，非齐次解集可视作其对应的齐次解集（即 $Ax = 0$ 的解空间）的整体平移。

这个看似细微的差别——是否包含原点——是区分两种集合代数本质的关键。包含原点的齐次解集拥有一种自洽的、封闭的代数属性，而平移后的非齐次解集则不具备。这种特殊的代数结构不仅存在于方程组的解集中，它在众多科学领域中反复涌现：

- 在物理学中，所有可能作用于同一质点上的力的集合，其合成与分解遵循何种法则？
- 在计算机图形学中，所有由红、绿、蓝三原色混合而成的颜色集合，构成了怎样的色彩空间？
- 在自然语言处理中，词语的“语义”应如何表示与组织，才能通过计算揭示“国王”与“女王”之间的逻辑关系？

这些看似无关的问题，其背后指向了同一个数学模型。这个模型，就是**线性空间** (Linear Space)，或称**向量空间** (Vector Space)。本章的核心任务，是学习并掌握描述这种普适性结构的通用**语言**。我们将完成一次思维范式的转变：从“**计算具体答案**”的**算术式思维**，跃迁至“**理解抽象结构**”的**体系化思维**。

这一过程并非一次性的计算任务，而是一种能力的构建。我们正在学习一种用以描述和操控“结构”本身的语言。在本章，我们将掌握这门语言的“名词”——线性空间本身，以及描述其规模的“形容词”——基与维数。在下一章，我们将学习其“动词”——**线性变换**，即作用于空间之上、建立空间之间联系的映射。这套语言一经掌握，将成为我们理解从物理世界到人工智能等众多复杂系统的理论基石。

6.2 线性空间的公理体系：源于几何，归于抽象

历史侧记

线性空间的公理化定义并非一蹴而就，而是数学家们历经近一个世纪，努力从具体的几何直观中提炼纯粹代数本质的成果。19 世纪以前，数学家们处理的“向量”大多是二维或三维空间中的有向线段，其运算法则（如平行四边形法则）与几何直觉紧密绑定。然而，随着代数学的发展，数学家们渴望一种不依赖于图形、能够推广到任意维度的通用语言。这一历史进程的高潮，便是线性空间公理体系的诞生。本节将追溯这一抽象过程，从格拉斯曼超越时代的宏大构想，到皮亚诺精确简洁的公理化定义，揭示这八条公理是如何成为连接几何与现代数学的桥梁。

掌握一门新语言，始于理解其基本规则——即公理。线性空间的公理体系初看可能

显得抽象，但它们并非凭空捏造，而是对二维、三维空间中向量运算法则的提炼与升华，是数学家们为了用最精确的语言捕捉几何直观而长期努力的结晶。

6.2.1 一种新数学的诞生：格拉斯曼的远见

历史侧记

要理解线性空间这一概念的本质，我们必须回到 19 世纪的德国，认识一位其思想超越了整个时代的天才——赫尔曼·格拉斯曼（Hermann Grassmann, 1809–1877）[4, 5]。

历史背景：几何与代数的张力

在 19 世纪的数学中，几何学与代数学之间存在一种深刻的张力。一方面，以欧几里得几何为代表的传统几何学，植根于人类对物理空间的直观感知，被视为数学严谨与确定性的典范 [9]。另一方面，是日渐强大的代数学，它凭借符号演算高效地解决问题，但其纯粹的符号推演有时被认为缺乏坚实的几何基础，甚至引起了部分数学家（如柯西）的警惕，他们担心这会导致没有几何意义的谬误 [6]。当时，爱尔兰数学家哈密顿（William Rowan Hamilton）在 1843 年发明的四元数，虽然成功地将复数推广到了四维，但其乘法不满足交换律，这让许多人感到困惑，也凸显了从几何直观向高维代数推广的困难。

格拉斯曼的革命性远见

格拉斯曼的非凡之处在于，他提出了一个颠覆性的哲学观点。在他看来，我们所处的物理空间及其几何学，并非数学的基础，而仅仅是数学理论的一个**应用实例** [7]。他坚信，在几何背后，存在着一种更纯粹、更普适的理论，他称之为“形式的科学”（science of forms）或“扩张论”（Ausdehnungslehre）[4, 7]。此理论不依赖于任何关于空间的直观预设，而是纯粹经由抽象逻辑构建。其研究对象不再是点、线、面，而是由某些基本“单位”（units）通过一套普适的运算法则生成的一切“扩张量”（extensive magnitudes）[4]。

在他的开创性著作《线性扩张论》（Die lineale Ausdehnungslehre, 1844）中，格拉斯曼阐述了其思想的起源。他从几何中有向线段 AB 与 BA 互为相反量得到启发，并将加法法则 $AB + BC = AC$ 从共线情形推广至空间中的任意点，从而将向量加法的概念从一维的长度测量中解放出来，赋予其普适的代数意义 [5, 7]。至关重要的是，他意识到，一旦几何被代数化，空间的维数便不再受限于直观的三维，而是可以进行任意的扩展 [7]。他还定义了多种乘积，其中“外积”（outer product，即楔积）至今仍是微分几何和物理学中的核心工具，其重要性远超当

时人们的理解。

超越时代的悲剧

然而，格拉斯曼的思想对于他所处的时代而言，实在太过超前。其著作充满了哲学思辨和高度的抽象性，令同时代的数学家们困惑不解，难以接受 [4, 16]。当时的数学权威，如莫比乌斯 (Möbius)，批评他的著作在引入抽象概念时未能给读者提供任何直观的引导 [4]。另一位著名数学家库默尔 (Kummer) 在评审他的著作时，虽承认其中蕴含着新颖的思想，但认为其表述晦涩，最终不建议授予他大学教职 [4]。其结果是，这部巨著销量惨淡，甚至在二十年后被出版商作为废纸处理 [4]。即便他在 1862 年出版了大幅修订和简化的第二版，其命运也未有改观。

事实上，格拉斯曼的理论已经包含了线性空间、子空间、线性无关、维数、基、坐标变换等几乎所有核心概念，其阐述方式与现代线性代数教科书惊人地相似 [4, 5]。但他生不逢时，当时的数学界尚未准备好迎接这样一场彻底的抽象革命。历史的转折点出现于 19 世纪末。非欧几何的发现与集合论中悖论的出现，动摇了数学家们对几何直观和朴素常识的绝对信赖，建立严格化、公理化的数学基础成为一项迫切的需求 [10]。正是在这样的背景下，意大利数学家朱塞佩·皮亚诺 (Giuseppe Peano) 于 1888 年，在深刻理解格拉斯曼工作的基础上，于其著作《几何演算》(Calcolo Geometrico) 中首次给出了线性空间的公理化定义，他称之为“线性系统” (linear systems) [5, 14]。皮亚诺的成功在于，他剥离了格拉斯曼理论中深奥的哲学外衣，提炼出其数学的精髓——一组清晰、简洁的公理。最终，在赫尔曼·外尔 (Hermann Weyl) 等后继者的推广下，线性空间的概念于 20 世纪初被数学界广泛接受，成为现代数学的通用语言 [17]。

注 这段历史揭示了科学进步的一种深刻模式：革命性的洞见往往需要一位“翻译者”或“体系建构者”，才能使其被更广泛的学术共同体所接纳。格拉斯曼是那位提出了颠覆性世界观的先知，而皮亚诺则是那位用严谨、普适的语言将其记录下来，使其成为数学共同财富的立法者。

6.2.2 几何蓝图：从直观到封闭性

历史侧记

在我们今天看来理所当然的几何向量及其运算法则，其清晰的代数形式是在 19 世纪逐渐形成的。在此之前，物理学家和数学家们处理力、速度等多是依赖几何图形，缺乏统一的代数工具。19 世纪 30 年代，意大利数学家朱斯托·贝拉维蒂斯 (Giusto Bellavitis) 提出了“等价线段” (equipollence) 理论，被认为是向量演

算的先驱之一。他定义了有向线段的相等、相加，实际上已经触及了向量的核心概念。然而，是哈密顿的四元数和格拉斯曼的扩张论，最终将向量从二维和三维几何的束缚中解放出来，使其成为一种普适的代数对象。本小节所探讨的“封闭性”，正是从这些早期几何直观中提炼出的、最核心的代数属性，它构成了从具体几何向量通往抽象线性空间的第一级台阶。

为了搭建一座由几何直观通往抽象公理的桥梁，让我们回到熟悉的二维平面与三维空间。在这里，我们用带箭头的线段来表示向量。这个由几何向量构成的世界，其最根本的代数特征是什么？

答案是，这个世界容许两种基本运算，并且对这两种运算是**封闭**的。

6.2.2.0.1 向量加法 (Vector Addition) 在几何上，我们通过“首尾相接”或“平行四边形”法则来定义两个向量 $\vec{v}$ 和 $\vec{w}$ 的和 $\vec{v} + \vec{w}$ 。从物理位移的角度审视，这一法则是极其自然的：先完成位移 $\vec{v}$ ，再从其终点开始完成位移 $\vec{w}$ ，其净效应等同于一次性完成的位移 $\vec{v} + \vec{w}$ 。

6.2.2.0.2 标量乘法 (Scalar Multiplication) 一个标量（在此为实数） c 与一个向量 $\vec{v}$ 的乘积记为 $c\vec{v}$ 。在几何上，此运算表现为对向量 $\vec{v}$ 的长度进行缩放，缩放因子为 $|c|$ 。若 $c > 0$ ，方向保持不变；若 $c < 0$ ，方向反转；若 $c = 0$ ，则结果为零向量 $\vec{0}$ 。

这两种运算最核心、最关键的性质，便是**封闭性 (Closure)**。所谓封闭性，是指在一个集合中对任意元素执行某种运算后，其结果依然是该集合的成员。运算无法使我们“逃离”这个集合。

以我们熟悉的二维欧氏平面 $\mathbb{R}^2$ 为例：

- **加法封闭性**：任意两个平面向量之和，其结果仍然是该平面上的一个向量。
- **标量乘法封闭性**：任意一个平面向量与任意实数之积，其结果仍在同一直线上，故也必然位于该平面之内。

这个看似平凡的“封闭性”，正是构筑线性空间这座宏伟大厦的基石。一个线性空间，在本质上，就是一个集合连同其内部的加法与标量乘法运算，共同构成了一个自洽且封闭的代数系统。

6.2.3 八条公理：线性结构的代数基石

历史侧记

19 世纪末至 20 世纪初是数学的“公理化运动”时期。此前，数学的许多分支（如几何学）都建立在直观的、不言自明的基础上。然而，非欧几何的发现和集合论悖论的出现，迫使数学家们重新审视数学的基础。大卫·希尔伯特 (David Hilbert) 在 1899 年出版的《几何基础》中，为欧几里得几何建立了一套严格的公理体系，

成为这场运动的典范。皮亚诺在 1888 年为线性空间提出的公理化定义，正是这一时代精神的体现。这八条公理的诞生，标志着线性代数研究的成熟：它不再仅仅是关于解方程或几何向量的技巧集合，而是一门研究抽象结构的、逻辑严谨的数学分支。这些公理的目的，就是用最少的假设，捕捉到所有“线性”现象的共同本质，从而赋予理论以最大的普适性。

至此，我们便能理解数学家们定义八条公理的动机。这些公理的唯一目的，就是用精确、无歧义的代数语言，来刻画我们在几何直观中建立的那个“运算封闭且性质良好”的向量世界。它们是格拉斯曼抽象思想的最终结晶，是数学家们对线性结构本质的深刻提炼。

这些公理的精妙之处，不仅在于它们规定了什么，更在于它们遗漏了什么。在公理体系中，我们找不到任何关于“长度”、“角度”或“距离”的定义。这是一种“结构性的抽象”：数学家们有意忽略了欧几里得空间中特有的度量属性，仅仅保留了最核心的、关于加法和数乘的代数结构。正是这种极简主义，赋予了线性空间概念惊人的普适性，使其能够被应用于几何实体之外的广阔领域。

这八条公理可以清晰地划分为两组 [1, 14]。

第一组：关于加法的运算法则 (构成阿贝尔群)

前四条公理仅涉及向量加法，确保了其运算性质与我们熟悉的实数加法类似。在抽象代数中，一个集合及其上的二元运算若满足这四条规则，则构成一个交换群或阿贝尔群 (Abelian Group)。对于向量空间 V 中的任意向量 $\vec{u}, \vec{v}, \vec{w}$ ：

1. 加法结合律 (Associativity of addition): $(\vec{u} + \vec{v}) + \vec{w} = \vec{u} + (\vec{v} + \vec{w})$
2. 加法交换律 (Commutativity of addition): $\vec{u} + \vec{v} = \vec{v} + \vec{u}$
3. 存在零元 (Identity element of addition): 存在一个零向量 $\vec{0} \in V$ ，使得对于任意 $\vec{v} \in V$ ，都有 $\vec{v} + \vec{0} = \vec{v}$ 。
4. 存在逆元 (Inverse element of addition): 对于任意 $\vec{v} \in V$ ，都存在一个逆向量 $-\vec{v} \in V$ ，使得 $\vec{v} + (-\vec{v}) = \vec{0}$ 。

第二组：关于标量乘法及其与加法交互的法则

后四条公理描述了标量与向量的相互作用，共同定义了标量乘法与向量加法这两种运算之间的协调关系。对于向量空间 V 中的任意向量 $\vec{u}, \vec{v}$ 和任意标量 a, b ：

5. 标量乘法与域乘法的相容性 (Compatibility): $a(b\vec{v}) = (ab)\vec{v}$
6. 标量乘法的单位元 (Identity element of scalar multiplication): $1\vec{v} = \vec{v}$
7. 标量乘法对向量加法的分配律 (Distributivity): $a(\vec{u} + \vec{v}) = a\vec{u} + a\vec{v}$
8. 标量乘法对标量加法的分配律 (Distributivity): $(a + b)\vec{v} = a\vec{v} + b\vec{v}$

定义 6.1 (线性空间)

一个线性空间 (或向量空间) 是一个集合 V (其元素称为向量), 连同域 F (其元素称为标量), 以及定义在它们上的两种运算 (向量加法和标量乘法), 满足上述八条公理 [1, 14]。



6.3 抽象的统一性：向量空间的泛化实例

历史侧记

20 世纪上半叶, 以法国的布尔巴基学派 (Bourbaki) 为代表的数学家们, 大力倡导“结构主义”的数学观。他们认为, 数学的核心任务就是研究各种抽象的“结构” (如代数结构、序结构、拓扑结构), 而线性空间正是其中最基本、最重要的代数结构之一。布尔巴基的目标是重写整个数学, 将其建立在少数几个核心结构之上, 从而揭示不同数学分支之间深刻的内在统一性。本节所展示的, 将多项式、矩阵、函数等不同对象均视为“向量”, 正是这种结构主义思想的直接体现。我们不再问“这个东西是什么”, 而是问“它遵循什么运算规则? 它属于哪种结构? ”。这种视角的转变, 是现代数学思维方式的精髓, 它极大地推动了数学的统一与发展。

在掌握了线性空间的基本公理之后, 我们迎来了本章最具决定性的一步: 实现概念的泛化。我们必须深刻认识到, 任何对象的集合, 只要我们能为其定义“加法”和“标量乘法”两种运算, 并验证这两种运算满足全部八条公理, 那么该集合就是一个线性空间, 其每一个成员在结构意义上都是一个向量。

6.3.1 结构主义视角：究竟什么是“向量”？

这个问题的答案, 是理解整个线性代数抽象思想的关键。一个“向量”的本质并非一个带箭头的线段, 也不是一行或一列数字。这些只是向量在特定情境下的具体表现形式。一个对象的“向量性”并非其固有属性, 而是由它所处的代数结构以及它在该结构中所扮演的角色所决定的。

注 任何满足向量空间公理的数学对象, 就是一个向量。

这一思想与哲学家保罗·贝纳塞拉夫 (Paul Benacerraf) 在 1965 年提出的著名“同一性问题”异曲同工 [3]。贝纳塞拉夫诘问: 数字“3”究竟是什么? 在策梅洛-弗兰克尔集合论中, 它可以被构造成集合 $\{\{\{\emptyset\}\}\}$, 也可以被构造成 $\{\emptyset, \{\emptyset\}, \{\emptyset, \{\emptyset\}\}\}$ 。这两种构造方式都能完美承载自然数的所有算术性质。他的结论是, 数字“3”没有一个唯一的、内在的“集合”身份。它的本质在于它在整个数系结构中所扮演的关系角色——即它位于 0 之后的第三个位置, 是 2 的后继, 是 4 的前驱。

同理, 一个“向量”也没有内在的“箭头”或“数组”身份。当我们断言一个多项式是

向量时，我们是在特定视角下，暂时忽略了它可以被求值、被求导、拥有根等其他属性，而仅仅关注它能与其他多项式相加、能被常数缩放，并且这些操作满足八条公理。这种“选择性关注”正是数学抽象的力量所在：我们忽略对象的具体形态，而聚焦于它们在运算规则下所扮演的结构性角色。这便是数学结构主义思想的核心，也是我们学习线性代数的终极目标：理解并运用这种强大的抽象思维范式。

6.3.2 向量空间的泛化实例

现在，让我们考察几个非传统的、由抽象思维定义的线性空间。要验证一个集合是否构成线性空间，我们只需明确其元素（向量）、加法和标量乘法的定义，然后检验封闭性与八条公理即可。

实例一：多项式空间 P_n

考虑所有次数不超过 n 的实系数多项式构成的集合，记作 P_n 。

- **元素（“向量”）**：一个形如 $p(x) = a_n x^n + \cdots + a_1 x + a_0$ 的多项式。
- **加法定义**：两个多项式相加，定义为对应项系数的相加。
- **标量乘法定义**：一个实数 c 与一个多项式相乘，定义为 c 与多项式的每一项系数相乘。
- **零元（“零向量”）**：所有系数均为零的零多项式 $0(x) = 0$ 。
- **公理检验**：不难验证，在上述定义下， P_n 满足所有八条公理，是一个线性空间 [1]。

实例二：矩阵空间 $M_{m \times n}$

考虑所有 $m \times n$ 的实数矩阵构成的集合，记作 $M_{m \times n}$ 。

- **元素（“向量”）**：一个 $m \times n$ 的矩阵本身。
- **加法与标量乘法**：采用标准的矩阵加法和标量乘法。
- **零元（“零向量”）**：所有元素均为零的 $m \times n$ 零矩阵。
- **公理检验**： $M_{m \times n}$ 同样是一个线性空间 [1]。

实例三（核心桥梁）：齐次线性方程组的解空间

这是连接理论与应用的最具启发性的例子。考虑一个齐次线性方程组 $A\vec{x} = \vec{0}$ 。所有满足此方程的解向量 $\vec{x}$ 所构成的集合，是一个线性空间。

证明 由于解向量本身是 $\mathbb{R}^n$ 中的元素，而 $\mathbb{R}^n$ 已知是一个线性空间，因此八条公理中的多数是自动满足的。我们只需验证该解集对于加法和标量乘法是封闭的即可。

1. 包含零向量：零向量 $\vec{x} = \vec{0}$ 显然是一个解，因为 $A\vec{0} = \vec{0}$ 。

2. 对加法封闭：设 $\vec{x}_1$ 和 $\vec{x}_2$ 为任意两个解。这意味着 $A\vec{x}_1 = \vec{0}$ 且 $A\vec{x}_2 = \vec{0}$ 。检验它们的和 $\vec{x}_1 + \vec{x}_2$ ：

$$A(\vec{x}_1 + \vec{x}_2) = A\vec{x}_1 + A\vec{x}_2 = \vec{0} + \vec{0} = \vec{0}$$

因此，和向量 $\vec{x}_1 + \vec{x}_2$ 仍然是方程组的解。

3. 对标量乘法封闭：设 $\vec{x}$ 是一个解 ($A\vec{x} = \vec{0}$)， c 是任意一个标量。检验 $c\vec{x}$ ：

$$A(c\vec{x}) = c(A\vec{x}) = c(\vec{0}) = \vec{0}$$

因此，标量乘积 $c\vec{x}$ 仍然是方程组的解。

封闭性得证。

结论

我们在前一章中努力计算的齐次方程组的解，并非一盘散沙，而是构成了一个具有完备代数结构的“空间”。这个空间，是其所在的全空间 $\mathbb{R}^n$ 的一个子空间 (subspace)。

6.3.3 代码视角：用 Python 定义向量的抽象行为

为了使抽象的公理更加具体，我们可以利用 Python 的面向对象编程，创建一个 Vector 类来模拟向量的代数行为。通过运算符重载 (operator overloading)，我们可以使代码的表达形式无限接近数学符号的自然书写。

代码视角

[title= 一个基础的 Python Vector 类实现]

```
import math

class Vector:
    def __init__(self, coordinates):
        if not coordinates:
            raise ValueError("Coordinates cannot be empty")
        self.coordinates = tuple(coordinates)
        self.dimension = len(coordinates)

    def __str__(self):
        return f"Vector: {self.coordinates}"

    def __repr__(self):
        return f"Vector({self.coordinates})"
```

```

def __add__(self, other):
    if not isinstance(other, Vector) or self.dimension != other.dimension:
        raise ValueError("Vectors must be of the same type and dimension to add")
    new_coords = [x + y for x, y in zip(self.coordinates, other.coordinates)]
    return Vector(new_coords)

def __mul__(self, scalar):
    if not isinstance(scalar, (int, float)):
        raise TypeError("Can only multiply Vector by a scalar (int or float)")
    new_coords = [c * scalar for c in self.coordinates]
    return Vector(new_coords)

def __rmul__(self, scalar):
    # Handle scalar * vector
    return self.__mul__(scalar)

# --- 使用示例 ---
v1 = Vector([1, 2, 3])
v2 = Vector([4, 5, 6])
v_sum = v1 + v2
print(f"{v1} + {v2} = {v_sum}")
v_scaled = 3 * v1
print(f"3 * {v1} = {v_scaled}")

```

6.3.4 应用前沿：为“意义”构建向量空间

线性空间的概念是当前人工智能领域最核心的基石之一，尤其体现在自然语言处理中。我们可以为“词语的意义”本身构建一个向量空间。这一革命性的想法在 **Word2Vec** 等词嵌入（word embedding）模型中得到体现 [13]。其核心思想是，一个词的意义由其上下文语境决定。模型通过分析海量文本，将每个词映射到高维空间中的一个特定向量。

这个由所有词向量构成的巨大高维空间，就是一个（近似的）线性空间。其真正令人惊叹之处在于，我们可以利用这个空间的代数结构，进行“语义运算”。最著名的例子是 [12, 13]：

$$\vec{v}_{\text{国王}} - \vec{v}_{\text{男人}} + \vec{v}_{\text{女人}} \approx \vec{v}_{\text{女王}}$$

这个等式并非文字游戏，而是一次真实的高维向量运算。从向量 $\vec{v}_{\text{国王}}$ 中减去向量 $\vec{v}_{\text{男人}}$ ，在几何上可以理解为剥离“男性”的语义分量，保留如“皇权”、“统治”等核心概念。随

后，再给结果向量加上“女性”的语义分量，最终得到的向量在空间中与 $\vec{v}_{女王}$ 的位置极为接近。

注 值得注意的是，这个著名的例子虽然极具启发性，但也存在被过度简化的风险。在实际的词向量模型中，要稳定地得到“女王”这个结果，通常需要在搜索最近邻向量时排除原始输入词（国王、男人、女人）作为候选答案 [11]。尽管如此，这种通过空间结构来捕捉和计算语义关系的思想，其威力是毋庸置疑的。

6.4 n 维线性空间

历史侧记

将几何学从直观的三维推广到任意 n 维，是 19 世纪数学的一大突破。英国数学家阿瑟·凯莱 (Arthur Cayley) 在 1843 年发表的论文中，首次明确地探讨了“ n 维几何”，他指出，尽管我们无法直观想象四维以上的空间，但这并不妨碍我们通过代数（坐标）来研究其性质。与他同时代的格拉斯曼，则从更根本的代数结构出发，构建了他的“扩张论”，其中维数 n 可以是任意正整数。起初，这种高维空间的概念被许多人视为纯粹的数学游戏，缺乏现实意义。然而，历史证明了这一抽象的巨大威力：20 世纪初，爱因斯坦的狭义相对论将时间和空间统一为四维的“闵可夫斯基时空”，高维空间从此成为现代物理学的标准语言。本节将要定义的基、维数等概念，正是为探索这些我们无法“看见”的抽象空间提供了精确的测量工具。

我们已经建立了线性空间的抽象概念。现在，需要为这些抽象空间引入一个至关重要的几何属性：**维数 (dimension)**。维数是对空间“自由度”或“规模”的量度。这个直观的概念如何被精确地推广到任意线性空间？答案在于找到构建空间的“基本元素”——即**基 (basis)**。

6.4.1 空间的构造基石：张成、线性无关与基

历史侧记

“张成”、“线性无关”与“基”这三个概念，是线性代数理论的“三位一体”，它们的清晰化是 19 世纪末数学严谨化运动的重要成果。在此之前，数学家们（例如在解方程组时）实际上已经在不自觉地使用这些思想，但缺乏精确的、通用的定义。德国数学家理查德·戴德金 (Richard Dedekind) 在研究代数数论时，为了描述数域的结构，于 1871 年清晰地定义了域上的“线性无关”概念，这被认为是该概念在抽象代数背景下的首次明确表述。将这些概念系统化，并确立“基”作为沟通“线性无关”与“张成”的桥梁，最终形成我们今天所学的理论体系，是格拉

斯曼、戴德金、皮亚诺等人共同努力的结果。它标志着数学家们不再满足于找到一组“生成元”，而是开始精确地探问：生成一个空间，最“经济”的方式是什么？

为“维数”给出一个严格的定义，需要引入三个线性代数中最核心的概念：张成 (span)、线性无关 (linear independence) 和基 (basis)。

张成 (Span)

给定一组向量 $\{\vec{v}_1, \dots, \vec{v}_k\}$ ，它们所有可能的**线性组合** $c_1\vec{v}_1 + \dots + c_k\vec{v}_k$ (其中 c_i 为标量) 所构成的集合，被称为这组向量的**张成空间**，记作 $\text{span}\{\vec{v}_1, \dots, \vec{v}_k\}$ [1]。这个张成空间本身也是一个线性空间。

线性无关 (Linear Independence)

一组向量 $\{\vec{v}_1, \dots, \vec{v}_k\}$ 被称为是**线性无关**的，当且仅当方程 $c_1\vec{v}_1 + \dots + c_k\vec{v}_k = \vec{0}$ 只有唯一的**平凡解**，即 $c_1 = c_2 = \dots = c_k = 0$ [1]。这意味着该组向量中没有任何一个向量可以被其他向量的线性组合所表示。

基 (Basis)

一个线性空间 V 的一组**基**，是 V 中的一个向量集合 $B = \{\vec{b}_1, \dots, \vec{b}_n\}$ ，它同时满足以下两个条件 [1]：

1. B 中的向量是**线性无关**的 (无冗余)。
2. B **张成**整个空间 V ，即 $\text{span}(B) = V$ (足够多)。

基是生成整个空间的最小向量集合。

6.4.2 空间的度量：维数

历史侧记

我们直观地认为一条线的维数是 1，一个平面的维数是 2。但这个直觉能否被严格证明？一个平面能否找到三个线性无关的向量？或者能否用一个向量就张成？这些问题的答案在今天看来是显然的，但在数学史上，将“维数”定义为“基的元素个数”，并证明这个数对于一个给定的空间是唯一的 (即“维数不变性定理”)，是一个深刻且非平凡的成就。这个定理确保了“维数”是一个空间的内禀属性，不因我们选择的具体基而改变。这一关键性的证明，通常归功于德国数学家恩斯特·施泰尼茨 (Ernst Steinitz)，他在 1910 年发表的关于抽象域论的奠基性论文中系统地发展了这些思想。施泰尼茨的工作，为线性空间的维度理论打下了坚实的逻辑基础，使得我们可以放心地在任意抽象空间中谈论其维数。

有了“基”的定义，我们终于可以给“维数”一个严格的定义。这源于线性代数中一个 foundational 的定理：

定理 6.1 (基的不变性定理)

对于一个给定的有限维线性空间（除零空间外），它的任何一组基都包含相同数量的向量 [1]。



这个基所包含的向量数量，是该空间内禀的、不变的属性。我们称这个唯一的数量为该线性空间的**维数 (dimension)**。

例题 6.1 空间的维数

- **多项式空间 P_2** ：所有次数不超过 2 的多项式空间。一组基是 $\{1, x, x^2\}$ ，包含 3 个向量，因此 $\dim(P_2) = 3$ 。
- **矩阵空间 $M_{2 \times 2}$** ：所有 2×2 矩阵的空间。一组标准基由四个只有一个元素为 1、其余为 0 的矩阵构成，因此 $\dim(M_{2 \times 2}) = 4$ 。

6.4.3 代码视角：用 NumPy 高效实现高维运算

在实际的科学计算与数据分析中，对性能的要求是极致的。此时，我们需要借助 Python 生态系统中的核心库——**NumPy**。NumPy 的基石是其 ndarray 对象，这是一个高效的、同构的多维数组，是实现坐标向量运算的理想工具。

代码视角

[title= 使用 NumPy 进行向量运算及性能对比]

```
import numpy as np
import timeit

# 创建 NumPy 数组（向量）
v1 = np.array([1.0, 2.0, 3.0])
v2 = np.array([4.0, 5.0, 6.0])

# NumPy 利用重载的运算符，语法简洁
v_sum = v1 + v2
v_scaled = 3 * v1

# --- 性能对比 ---
dim = 1_000_000
# 使用原生 Python 列表
py_list1 = [1.0] * dim
```

```

py_list2 = [2.0] * dim
# 使用 NumPy 数组
np_arr1 = np.array(py_list1)
np_arr2 = np.array(py_list2)

# 测量 Python 列表加法用时
py_time = timeit.timeit('[x + y for x, y in zip(py_list1, py_list2)]',
                        globals=globals(), number=10)
print(f"Pure Python list addition time: {py_time:.4f} seconds")

# 测量 NumPy 数组加法用时
np_time = timeit.timeit('np_arr1 + np_arr2', globals=globals(), number=10)
print(f"NumPy array addition time: {np_time:.4f} seconds")

```

NumPy 的运算速度通常比纯 Python 实现快数个数量级，因为它将计算任务委托给了底层的高度优化的、由 C 或 Fortran 语言编写的线性代数库，例如 **BLAS** (Basic Linear Algebra Subprograms) 和 **LAPACK** (Linear Algebra PACKage)。

6.4.4 应用前沿：应对“维数灾难”

历史侧记

“维数灾难”这一术语由美国应用数学家理查德·贝尔曼 (Richard Bellman) 在 1957 年其关于动态规划的研究中首次提出。贝尔曼发现，当他试图解决多阶段决策问题时，计算所需的状态空间会随着变量（即维度）数量的增加而呈指数级增长，很快就会超出任何计算机的处理能力。他用这个生动的词汇来描述高维空间中体积的“空洞性”和数据点的“稀疏性”所带来的计算和统计上的巨大挑战。这一概念的提出，深刻地影响了后来的机器学习、数据挖掘和统计学领域。它促使研究者们认识到，直接在高维空间中进行操作是不可行的，必须寻找数据的内在低维结构。主成分分析 (PCA) 虽然其数学基础（特征值分解）早已存在，但正是在应对“维数灾难”的背景下，它作为一种核心的降维技术，其重要性被重新认识并得到了前所未有的广泛应用。

高维线性空间是现代机器学习和人工智能的“主战场”。然而，高维空间的行为与我们的三维直觉相去甚远，它充满了被称为“**维数灾难**” (Curse of Dimensionality) 的现象 [2]。幸运的是，许多现实世界中的高维数据集并非均匀地散布在整个空间，而是集中在一个低维的流形或子空间上。主成分分析 (Principal Component Analysis, PCA) 正是揭示并利用这种低维结构的经典线性方法 [8, 15]。

PCA 的核心思想是一次坐标系变换（基变换）。它为给定的数据集寻找一套新的正

交基。这套新基的基向量（称为主成分）的方向，恰好是数据协方差矩阵的特征向量所指的方向；而每个主成分的重要性（即数据在该方向上的方差大小）则由对应的特征值来衡量。通过仅保留特征值最大的前 k 个主成分，我们便可以将数据投影到一个 k 维子空间中，从而在最大化保留原始数据变异性的前提下实现降维。

动手实践：使用 scikit-learn 对鸢尾花数据进行 PCA 降维

让我们使用经典的鸢尾花（Iris）数据集来直观地演示 PCA 的过程和效果。

代码视角

```
[title= 使用 scikit-learn 进行 PCA 降维]

import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_iris

# 1. 加载并准备数据
iris = load_iris()
X = iris.data # 4维特征数据
y = iris.target # 类别标签
target_names = iris.target_names

# 2. 数据标准化（PCA对数据尺度敏感，此步至关重要）
X_scaled = StandardScaler().fit_transform(X)

# 3. 初始化并应用 PCA，从4维降至2维
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# 4. 可视化降维后的结果
plt.figure(figsize=(8, 6))
colors = ['navy', 'turquoise', 'darkorange']
for color, i, target_name in zip(colors, [0, 1, 2], target_names):
    plt.scatter(X_pca[y == i, 0], X_pca[y == i, 1],
                color=color, alpha=.8, lw=2, label=target_name)
plt.legend(loc='best', shadow=False, scatterpoints=1)
plt.title('PCA of IRIS dataset')
```

```
plt.xlabel(f'Principal Component 1 ({pca.explained_variance_ratio_[0]:.1%} variance_ratio)')
plt.ylabel(f'Principal Component 2 ({pca.explained_variance_ratio_[1]:.1%} variance_ratio)')
plt.grid(True)
# plt.savefig('pca_iris_plot.png') % 取消注释以保存图像
plt.show()

print(f"Explained variance ratio of the two components: {pca.explained_variance_ratio_[:2]:.2%}")
print(f"Total variance explained: {sum(pca.explained_variance_ratio_):.2%}")
```

这个例子完美地体现了本章的核心思想：从抽象的线性空间和维数理论出发，通过基变换这一核心操作，我们最终获得了一个强大的、能够从高维数据中提取洞见并解决实际问题的分析工具。

第 7 章 伟大的动词：线性变换

title= 本章导览

- 线性变换的定义：一种描述“运动”的规则
- 矩阵：线性变换的代数表示
- 特征值与特征向量：变换的不变本质

7.1 线性变换的文法：一种描述“运动”的规则

历史侧记

数学的发展史，在很大程度上是一部对“变化”与“关系”的认知不断深化的历史。在古代，数学家们关注静态的量与形。到了 17 世纪，微积分的诞生标志着人类首次拥有了系统研究“变化率”这一动态过程的工具。几乎同时，笛卡尔的解析几何用代数方程赋予了几何曲线以生命，使静态的图形可以通过参数的变化而“运动”起来。18 世纪的数学巨匠欧拉（Leonhard Euler）首次将“函数”定义为核心研究对象，即一个变量如何依赖于另一个变量的规则。然而，这些早期的“变换”或“函数”概念，大多局限于数与数之间的关系。本章的主角——线性变换，代表了思想上的又一次飞跃。它所描述的不再是数的变化，而是整个“空间”的结构性变化。我们将看到，数学家们是如何从纷繁的几何运动（如旋转、投影）和分析运算（如求导、积分）中，提炼出两条最根本的“线性”法则，从而为描述一类最重要、最普适的结构性变化，奠定了坚实的理论基石。

在第四章，我们已经构建了各种线性空间，它们如同一个个静态的国度，居住着多项式、矩阵、函数等形形色色的“向量”公民。然而，一个只有名词和形容词的语言是贫乏的，一个只有静态物体的世界是死寂的。数学的真正威力在于描述关系与变化。因此，本节我们将为这门语言引入“动词”——那些描述从一个向量到另一个向量的“运动”或“映射”的规则，我们称之为**变换**（Transformation）。在所有可能的变换中，有一类最为重要，因为它们完美地保持了线性空间的内在结构，这一类变换就是**线性变换**。

7.1.1 历史的脉络：从几何运动到代数结构

历史侧记

“变换”的思想源远流长。古希腊的几何学家们早已研究过平移、旋转、反射等几何操作。然而，在很长一段时间里，这些操作被看作是独立的几何技巧，而非一种可以被统一研究的数学对象。这种状况直到 17 世纪解析几何的诞生才得以改变，代数被系统地引入几何研究，从而为描述变换提供了新的语言。例如，一个二维平面上的旋转，可以通过由三角函数构成的代数公式来精确表达。18 世纪的数学家们，如欧拉和达朗贝尔，在研究物理问题（如振动弦）时，更是将函数本身作为变换的对象，这为后来的泛函分析埋下了伏笔。

尽管如此，将变换本身视为一个可研究的“代数实体”的决定性一步，要到 19 世纪才真正成熟。当时，抽象代数结构（如群论）的兴起，促使数学家们开始关注变换的集合及其复合运算。在这场思想变革中，英国数学家**阿瑟·凯莱**（Arthur Cayley）扮演了关键角色。凯莱在 1855 年至 1858 年间发表的一系列论文中，不仅仅是发明了矩阵的表示法，更重要的是，他深刻地洞察到，**矩阵的本质是线性变换的代数表示**。

凯莱的工作，与格拉斯曼的抽象思想可谓异曲同工。如果说格拉斯曼关注的是空间的内在结构，那么凯莱则关注的是保持这种结构的操作。正是这两股思潮的交汇，最终催生了线性变换的现代定义。这个定义的核心，不再关注变换的具体形式（是旋转还是求导），而是聚焦于它是否遵守两条最基本的“文法规则”。

7.1.2 线性变换的定义：保持结构的两个核心条件

历史侧记

“保持结构”是贯穿整个现代数学的核心思想。在抽象代数中，研究不同代数系统（如群、环、域）之间关系的映射，如果能够保持其核心运算，就被称为“同态”（Homomorphism）。线性变换的定义，正是“向量空间同态”的精确表述。这两条看似简单的规则——保持加法与保持标量乘法——其深刻之处在于，它们抓住了所有“线性”现象的共性。这个定义的确立，使得数学家们可以将在一个领域（如几何旋转）中得到的结论，通过抽象的阶梯，应用到另一个看似无关的领域（如微分方程求解）。这种通过“结构保持映射”来建立不同数学分支联系的方法，是 20 世纪数学，特别是以布尔巴基学派为代表的结构主义数学思想的标志性特征。

一个从线性空间 V 到自身的映射 $T: V \rightarrow V$ ，如果被称为“线性”的，它就必须尊重空间的两种基本运算：向量加法和标量乘法。换言之，一个变换要想获得“线性”这一称号，就必须保持线性空间最根本的代数结构。

定义 7.1 (线性变换)

设 T 是线性空间 V 上的一个变换。如果对于 V 中任意的向量 α, β 和任意标量 k ，都满足以下两个条件，则称 T 是一个线性变换：

1. 保持加法 (Additivity): $T(\alpha + \beta) = T(\alpha) + T(\beta)$
2. 保持标量乘法 (Homogeneity): $T(k\alpha) = kT(\alpha)$



这两个条件是线性变换的精髓所在。第一个条件“保持加法”意味着，先将两个向量相加，再对结果进行变换，其效果与先将两个向量分别变换，再将变换后的结果相加是完全一样的。第二个条件“保持标量乘法”则意味着，先将一个向量拉伸 k 倍再进行变换，其效果与先对向量进行变换再将结果拉伸 k 倍是完全一样的。

因此，这两个条件可以合并为一个等价的、更紧凑的表述：**变换一个线性组合，等于先分别变换再进行同样的线性组合**。即对于任意标量 a, b 和任意向量 α, β ：

$$T(a\alpha + b\beta) = aT(\alpha) + bT(\beta) \quad (7.1)$$

一个变换如果满足这两个条件，就意味着它不会“扭曲”或“破坏”线性空间固有的网格结构。它可能进行整体性的拉伸、旋转或投影，但绝不会把直线变成曲线，也不会破坏平行四边形法则。

7.1.3 变换的检验：哪些是线性的，哪些不是？**历史侧记**

在数学史上，每当一个新概念被定义出来，接踵而至的便是一场“大分类运动”。数学家们会拿起这个新“标签”，去审视已有的数学世界，看看哪些旧事物符合新标准，哪些不符合。本节的检验过程，正是这种数学活动的缩影。将旋转、投影等几何操作纳入线性变换的范畴，使得几何学与代数学的联系更加紧密。而发现求导和积分这两种微积分的核心运算也同样是线性的，则是一次更为深刻的洞见，它直接催生了泛函分析 (Functional Analysis) 这一重要数学分支，后者将线性代数方法推广到研究函数构成的无穷维空间。反之，识别出平移等操作的“非线性”，也同样重要，因为它界定了线性方法的适用边界，并激发了对更复杂的非线性现象的研究。

现在，我们可以用这两个核心条件作为“试金石”，来检验一些我们熟悉的变换。

- **几何旋转**：在二维平面 $\mathbb{R}^2$ 中，将所有向量绕原点逆时针旋转一个角度 φ 的变换 T 是线性的。从几何直观上看，将两个向量相加（平行四边形法则）再旋转，与先将两个向量分别旋转再相加，得到的最终向量是相同的。从代数上看，这个变换可以写成矩阵乘法 $T(\vec{x}) = A\vec{x}$ ，其中 $A = \begin{pmatrix} \cos \varphi & -\sin \varphi \\ \sin \varphi & \cos \varphi \end{pmatrix}$ 。由于矩阵乘法本身满足分配律，即 $A(p_1 + p_2) = Ap_1 + Ap_2$ 和 $A(kp_1) = k(Ap_1)$ ，所以任何由矩阵乘法定义的变换都必然是线性的。

- 微积分中的变换：

- **求导运算**：在多项式空间 $P_n[x]$ 中，求导运算 $D(p(x)) = p'(x)$ 是一个线性变换。这直接源于微积分的基本性质：和的导数等于导数的和，即 $(p(x) + q(x))' = p'(x) + q'(x)$ ；并且常数可以被提出，即 $(kp(x))' = kp'(x)$ 。

- **积分运算**：在连续函数空间 $C[a, b]$ 中，定积分运算 $T(f(x)) = \int_a^x f(t)dt$ 也是一个线性变换，其原因同样是积分算子具有线性性质。

- 非线性变换的例子：

- **平移变换**：在 $\mathbb{R}^2$ 中，将每个向量 $\vec{v}$ 变为 $\vec{v} + \vec{\alpha}$ （其中 $\vec{\alpha}$ 是一个固定的非零向量）的变换 T **不是**线性的。因为它不满足加法保持性： $T(\vec{v} + \vec{w}) = (\vec{v} + \vec{w}) + \vec{\alpha}$ ，而 $T(\vec{v}) + T(\vec{w}) = (\vec{v} + \vec{\alpha}) + (\vec{w} + \vec{\alpha}) = (\vec{v} + \vec{w}) + 2\vec{\alpha}$ 。两者显然不同。一个更根本的判断方法是，任何线性变换都必须将零向量映射到零向量（因为 $T(\vec{0}) = T(0 \cdot \vec{v}) = 0 \cdot T(\vec{v}) = \vec{0}$ ），而这个平移变换将 $\vec{0}$ 映射到了非零的 $\vec{\alpha}$ 。

- **平方变换**：在 $\mathbb{R}^3$ 中，变换 $T(x_1, x_2, x_3) = (x_1^2, x_2 + x_3, 0)$ **不是**线性的，因为它破坏了加法结构，因为一般情况下 $(a_1 + b_1)^2 \neq a_1^2 + b_1^2$ 。

7.2 矩阵：线性变换的代数表示

历史侧记

在 19 世纪中叶之前，数学家们处理线性方程组和线性替换时，通常会完整地写出所有变量和系数，形式繁琐且难以洞察其内在结构。矩阵的引入，是数学符号和思想的一次重大革命。凯莱和西尔维斯特的开创性工作，使得一个庞杂的线性变换可以被“打包”成一个单一、紧凑的代数对象——矩阵。这不仅仅是记法上的便利，更是一次深刻的观念转变：它使得变换本身成为了可以被加、减、乘、逆的数学实体。这就如同代数学从用文字描述方程，进步到使用 x, y, z 等符号一样，矩阵为线性变换提供了专属的、强大的符号语言。本节的核心，正是要揭示这种“编码”过程：如何将一个抽象的变换规则，精确地翻译成一个具体的矩阵。

我们已经确立了线性变换是保持线性空间结构的一种特殊映射。但是，这个定义是抽象的。在实际计算中，我们如何具体地描述和操作一个线性变换呢？答案是：通过矩阵。在第四章，我们学会了用坐标向量来给抽象空间中的向量一个具体的“身份地址”。本节，我们将遵循同样的逻辑，为线性变换也配发一个具体的“身份证”——**矩阵**。

7.2.1 思想的飞跃：由基的变换确定整个变换

历史侧记

这一思想是有限维线性代数的基石，它体现了“由部分决定整体”的强大威力。这个思想的根源可以追溯到高斯等人对线性方程组的研究。他们早已发现，一个线性系统的行为，完全由其少数几个系数决定。然而，将其明确提升为“只需确定基的像，即可确定整个线性变换”这一普适原理，是 19 世纪末线性空间理论成熟后的产物。这一原理的深刻之处在于它大大降低了问题的复杂性：我们不再需要追踪空间中无穷多个向量的去向，而只需关注有限个基向量的命运。这使得对抽象变换的分析，可以简化为对一个有限数组（即矩阵）的分析，是实现从无限到有限、从抽象到具体的关键一步。

一个线性变换 T 的行为，看似需要描述它如何作用于空间中的无穷多个向量。然而，线性变换的优美之处在于，我们只需要知道它如何作用于空间的一组**基向量**，就可以完全确定它对任何其他向量的作用。

这个结论的推导过程是严谨且直观的。首先，我们知道空间中的任何向量 α 都可以唯一地表示为一组基 $\{\epsilon_1, \dots, \epsilon_n\}$ 的线性组合，即 $\alpha = x_1\epsilon_1 + \dots + x_n\epsilon_n$ 。然后，由于 T 是一个线性变换，它保持线性组合的结构，所以我们可以进行如下推导：

$$T(\alpha) = T(x_1\epsilon_1 + \dots + x_n\epsilon_n) = x_1T(\epsilon_1) + \dots + x_nT(\epsilon_n) \quad (7.2)$$

这个公式揭示了一个深刻的真理：**只要我们知道了基向量的“像” $T(\epsilon_1), \dots, T(\epsilon_n)$ ，我们就能通过同样的线性组合系数 $(x_1, \dots, x_n)$ 计算出任何向量 α 的像。**因此，基向量的像 $\{T(\epsilon_1), \dots, T(\epsilon_n)\}$ 包含了这个线性变换的全部信息，抓住了变换的本质。

7.2.2 构造变换矩阵：为变换建立档案

历史侧记

将变换信息“编码”进矩阵的列向量中，这一看似自然的操作，是凯莱的核心洞见之一。在凯莱之前，人们看待线性替换时，眼光是“横向”的——关注每个新坐标是如何由旧坐标线性组合而成的（即矩阵的行）。而凯莱的视角是“纵向”的，他意识到，矩阵的每一“列”都具有独立的意义：它代表了一个基本坐标轴（基向量）在变换之后的新坐标。这种视角的转换至关重要，因为它将矩阵从一个单纯的系数表，提升为一个描述几何“动作”的结构化对象。矩阵的每一列，都是对一个基本自由度的变换结果的“快照”。这个构造过程，正是将变换的几何行为翻译为代数语言的核心步骤。

基于上述思想，我们现在可以为变换 T 制作它的“身份证”了。这个过程分为三个逻辑步骤：

1. **确定坐标系**：首先，在 n 维线性空间 V 中选定一组基 $B = \{\epsilon_1, \dots, \epsilon_n\}$ 。这是我们进行测量的“参考框架”。
2. **记录基的像的坐标**：接着，我们计算出每一个基向量的像 $T(\epsilon_j)$ 。由于 $T(\epsilon_j)$ 本身也是 V 中的一个向量，所以它也可以用我们选定的基 B 来唯一地表示：

$$T(\epsilon_j) = a_{1j}\epsilon_1 + a_{2j}\epsilon_2 + \dots + a_{nj}\epsilon_n$$

这组唯一的系数 $(a_{1j}, a_{2j}, \dots, a_{nj})^T$ 就是 $T(\epsilon_j)$ 在基 B 下的坐标向量。

3. **列队成阵**：最后，我们将这 n 个坐标向量作为列，依次排列，就构成了一个 $n \times n$ 的矩阵 A 。

$$A = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{pmatrix}$$

这个矩阵 A 就是线性变换 T 在基 B 下的**矩阵表示**。它的第 j 列，精确地记录了第 j 个基向量的像 $T(\epsilon_j)$ 在该基下的坐标。

这个构造过程建立了一座至关重要的桥梁：在给定的基底下，**每一个线性变换都唯一对应一个方阵，反之，每一个方阵也唯一地定义了一个线性变换**。从此，对抽象变换的研究，可以完全转化为对具体矩阵的计算和分析。

7.2.3 坐标的变换： $Y = AX$ 的推导与意义

历史侧记

公式 $Y = AX$ 的优雅简洁，掩盖了其背后漫长的演化。在矩阵符号出现之前，描述线性替换需要写下一长串的方程组，如 $y_1 = a_{11}x_1 + \dots, y_2 = a_{21}x_1 + \dots$ 。这种写法使得思考变换的复合（即连续应用两次替换）变得极为 **cumbersome**。凯莱引入的矩阵乘法，其定义方式（行乘列）正是为了让两次替换的复合，在代数上精确对应于两个系数矩阵的乘积。因此， $Y = AX$ 不仅仅是一个计算公式，它是整个线性变换理论能够代数化的基石。它将一个复杂的、作用于整个空间的操作，压缩成了一个优雅的、可以进行符号运算的代数表达式，这是数学抽象力量的完美体现。

有了变换矩阵 A ，计算就变得异常简单。如果向量 α 在基 B 下的坐标是 $X = (x_1, \dots, x_n)^T$ ，那么它的像 $T(\alpha)$ 在同一基下的坐标 $Y = (y_1, \dots, y_n)^T$ 是什么呢？我们可以通过之前建立的逻辑链条推导出它们的关系。我们有 $T(\alpha) = \sum_{j=1}^n x_j T(\epsilon_j)$ 。将 $T(\epsilon_j) = \sum_{i=1}^n a_{ij}\epsilon_i$ 代入，得到：

$$T(\alpha) = \sum_{j=1}^n x_j \left(\sum_{i=1}^n a_{ij}\epsilon_i \right) = \sum_{i=1}^n \left(\sum_{j=1}^n a_{ij}x_j \right) \epsilon_i$$

另一方面，我们知道 $T(\alpha)$ 的坐标是 Y ，即 $T(\alpha) = \sum_{i=1}^n y_i \epsilon_i$ 。由于向量在给定基下的坐

标表示是唯一的，因此比较两边 ϵ_i 的系数，我们得到：

$$y_i = \sum_{j=1}^n a_{ij} x_j$$

这正是矩阵乘法 $Y = AX$ 的定义。这个简洁的公式，是凯莱思想的结晶，也是整个线性代数计算的核心。它将一个抽象的函数作用 $T(\alpha)$ ，转化成了一个具体的代数运算 AX 。

7.2.4 相似矩阵：同一变换的不同代数面孔

历史侧记

“在变换下保持不变的量”，即“不变量”（Invariant），是 19 世纪代数学和几何学的核心主题。凯莱和西尔维斯特开创了“不变量理论”，旨在寻找当坐标系变换时，代数表达式中哪些组合量保持不变。而“相似矩阵”的概念，正是这一宏大思想在线性变换理论中的具体体现。公式 $B = M^{-1}AM$ 描述了当坐标系（基）通过矩阵 M 变换时，线性变换的表示矩阵是如何变化的。那些在相似变换下保持不变的矩阵属性（如行列式、迹、特征值），正是变换本身固有的、不依赖于观察者（坐标系）的“客观”属性。这一思想后来被德国数学家菲利克斯·克莱因（Felix Klein）在其著名的“爱尔兰根纲领”（1872 年）中发扬光大，他提出“一种几何学就是研究其特定变换群下的不变量”，从而深刻地统一了当时的各种几何学。

一个关键的问题随之而来：线性变换的矩阵表示，是依赖于我们所选择的基的。如果我们换一组基，变换的“身份证”（矩阵）也会随之改变。那么，同一个线性变换在不同基下的矩阵之间，存在什么关系呢？

假设 T 在旧基 $B = \{\epsilon_1, \dots, \epsilon_n\}$ 下的矩阵是 A ，在新基 $B' = \{\eta_1, \dots, \eta_n\}$ 下的矩阵是 B 。再假设从旧基到新基的**过渡矩阵**是 M （即新基用旧基表示时，系数矩阵的转置，或者说 $[\eta_1, \dots, \eta_n] = [\epsilon_1, \dots, \epsilon_n]M$ ）。那么，新旧两个矩阵之间的关系可以通过逻辑推导出：

$$B = M^{-1}AM \quad (7.3)$$

这个关系被称为**矩阵的相似**（Similarity）。我们称矩阵 A 和 B 是**相似的**。

注 相似关系有着深刻的几何内涵：**相似的矩阵，描述的是同一个线性变换在不同坐标系下的样子**。它们是同一个“客观实体”（变换）在不同“主观视角”（基）下的不同“照片”。因此，它们必然共享某些不依赖于坐标系的内在属性，例如它们的行列式、迹，以及我们即将在下一节探讨的——**特征值**。

7.3 特征值与特征向量：变换的不变本质

历史侧记

在数学和物理学中，寻找“不变量”始终是一个核心的驱动力。从古希腊人寻找完美的几何形状，到伽利略和牛顿探寻普适的物理定律，其本质都是在纷繁复杂的变化中寻找那些永恒不变的规律。线性变换描绘了空间中向量的“运动”，而特征值和特征向量所回答的，正是在这场运动中，“什么东西没有发生本质改变？”。这个问题并非纯粹的数学游戏，它的根源深植于物理世界：一个旋转的陀螺，其旋转轴的方向就是不变的；一个振动的琴弦，其基频和泛音的振动模式（驻波）在时间演化中也保持其空间形态不变。本节将要探讨的，正是如何将这种对“不变性”的物理和几何直观，转化为一套精确、强大的代数分析工具。

我们已经学会了如何用矩阵来描述一个线性变换。现在，我们要提出一个更深刻的问题：一个线性变换的“本质”是什么？在纷繁复杂的拉伸、旋转、剪切之中，是否存在某些“不变”的元素？更精确地说，是否存在某些特殊的向量，它们在变换的作用下，方向保持不变，仅仅被拉伸或压缩？

这些特殊的、能够揭示变换内在属性的向量，就是**特征向量** (Eigenvectors)，而它们被拉伸或压缩的比例，就是**特征值** (Eigenvalues)。

7.3.1 思想的演进：从天体运动到“本征”值

历史侧记

特征值与特征向量的概念，并非诞生于抽象的矩阵理论，而是源于 18 世纪对物理世界的探索，特别是对刚体旋转和天体运动的研究

- **欧拉与主轴**：18 世纪，数学家**莱昂哈德·欧拉** (Leonhard Euler) 在研究刚体旋转时发现，任何刚体的旋转运动，都存在一个瞬时旋转轴。这个轴上的点在旋转中保持位置不变（或只沿轴线移动）。这实际上就是“不变方向”思想的物理雏形，这些方向被称为**主轴** (principal axes)。
- **拉格朗日、拉普拉斯与天体摄动**：在分析行星轨道长期稳定性时，拉格朗日和拉普拉斯遇到了求解线性微分方程组的问题。他们发现，解的稳定性和周期性取决于一个特定代数方程的根，这个方程后来被称为“特征方程”。这些“根”决定了整个太阳系运动的基本模式，是系统内在的振动频率。
- **柯西与“特征根”**：到了 19 世纪初，**奥古斯丁-路易·柯西** (Augustin-Louis Cauchy) 在 1829 年的一篇论文中，系统地推广了这一思想，研究二次曲面（如椭球体）的分类。为了找到曲面的主轴方向（即对称轴），他求解了一个方程，并称其解为“**特征根**” (racine caractéristique)，这正是“特征值”一

词的来源

- **斯图姆-刘维尔理论**：几乎同时，在数学物理领域，对热传导和振动问题的研究（如傅里叶分析）引出了斯图姆-刘维尔理论。该理论表明，许多重要的微分算子（它们是无穷维空间中的线性变换）也拥有离散的“特征值”谱和对应的“特征函数”，这些构成了所有可能振动模式的“基”。
- **希尔伯特与“Eigen”**：我们今天使用的术语“eigenvalue”和“eigenvector”中的“eigen”是一个德语词，意为“自身的”、“固有的”、“特征的”。这个词是由 20 世纪初的数学巨匠**大卫·希尔伯特**（David Hilbert）在研究积分方程和无穷维空间（后被称为希尔伯特空间）时，系统性地使用的

7.3.2 定义与求解：寻找不变方向的代数方法

历史侧记

特征方程 $\det(\lambda I - A) = 0$ 的建立，是 19 世纪代数学的一次伟大综合。它巧妙地将三个看似独立的领域融为一体：一是凯莱等人发展的线性变换与矩阵理论（体现在矩阵 A ）；二是由高斯、伽罗瓦等人深入研究的 polynomial 方程求根理论（特征值 λ 正是特征多项式的根）；三是自 17 世纪起由莱布尼茨、克拉默、柯西等人发展的行列式理论（ $\det(\cdot) = 0$ 是齐次方程组存在非零解的关键判据）。这个单一的方程，成为了连接几何（不变方向）、代数（多项式根）和分析（微分方程稳定性）的枢纽。它的求解过程，即先求特征值再求特征向量，也因此成为线性代数中最核心、最经典的计算流程之一。

从代数上看，寻找特征向量和特征值的过程，就是求解如下的方程：

$$A\vec{x} = \lambda\vec{x} \quad (7.4)$$

其中 A 是一个 $n \times n$ 矩阵， $\vec{x}$ 是一个非零的 n 维向量， λ 是一个标量。

定义 7.2 (特征值与特征向量)

对于一个给定的 $n \times n$ 矩阵 A ，如果存在一个标量 λ 和一个非零向量 $\vec{x}$ 满足方程 $A\vec{x} = \lambda\vec{x}$ ，那么我们称 $\vec{x}$ 是矩阵 A 的一个特征向量， λ 是与之对应的特征值。



为了求解这个方程，我们将其进行代数变形：

$$A\vec{x} - \lambda\vec{x} = \vec{0}$$

$$A\vec{x} - \lambda I\vec{x} = \vec{0}$$

$$(\lambda I - A)\vec{x} = \vec{0}$$

这是一个齐次线性方程组。根据定义，我们寻找的是它的**非零解** $\vec{x}$ 。根据我们在前面章节的知识，一个齐次线性方程组有非零解的充分必要条件是，它的系数矩阵的行列式为

零。因此，我们必须有：

$$\det(\lambda I - A) = 0 \quad (7.5)$$

这个方程被称为矩阵 A 的**特征方程**。它是一个关于 λ 的 n 次多项式，其根就是矩阵 A 的所有特征值。对于每一个求出的特征值 λ_i ，我们再将其带回到齐次方程组 $(\lambda_i I - A)\vec{x} = \vec{0}$ 中，求解其基础解系，就得到了对应于 λ_i 的所有线性无关的特征向量。

7.3.3 对角化：寻找变换的“自然”坐标系

历史侧记

“对角化”的思想，本质上是数学中“寻找理想坐标系以简化问题”这一普适策略的体现。在几何上，拉格朗日和柯西通过旋转坐标轴来消去二次型中的交叉项，使其化为“标准形”，这正是对角化思想的早期形式。在代数领域，法国数学家卡米尔·若尔当（Camille Jordan）在 1870 年证明了任何复数域上的线性变换（矩阵）都可以在某个基下表示为一个近乎对角的形式，即今天所称的“若尔当标准型”。这个深刻的结果表明，即使一个变换无法被完全“对角化”（因为可能没有足够的线性无关的特征向量），它仍然可以被分解为若干个在低维子空间上的、结构相对简单的“若尔当块”的组合。因此，对角化可以被看作是最理想、最简单的一种标准型，而特征向量构成的基，正是能够实现这种理想简化的“自然”坐标系。

特征向量最重要的应用之一，就是**对角化**（Diagonalization）。

我们在 5.2.4 节看到，同一个线性变换在不同基下的矩阵是相似的。这就提出了一个终极问题：我们能否找到一个“最好”的基，使得线性变换在这个基下的矩阵形式最简单？

答案是肯定的，前提是这个变换足够“好”。如果一个 $n \times n$ 矩阵 A 拥有 n 个线性无关的特征向量 $\{\vec{p}_1, \dots, \vec{p}_n\}$ ，那么我们就可以用这些特征向量作为新的基。

在这个由特征向量组成的“自然”坐标系中，线性变换 A 的作用变得异常清晰。对于每一个基向量 $\vec{p}_i$ ，我们有 $A\vec{p}_i = \lambda_i \vec{p}_i$ 。这意味着，在这个基下，变换 A 的矩阵是一个**对角矩阵**，其对角线上的元素正是对应的特征值：

$$D = \begin{pmatrix} \lambda_1 & 0 & \dots & 0 \\ 0 & \lambda_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \lambda_n \end{pmatrix}$$

从旧的标准基到这个新的特征基的过渡矩阵 M ，其列向量就是这些特征向量 $M = [\vec{p}_1, \dots, \vec{p}_n]$ 。而对角化过程，正是我们之前学过的相似变换：

$$D = M^{-1}AM \quad (7.6)$$

一个矩阵 A 如果能通过这种方式变成一个对角矩阵，我们就说 A 是**可对角化的**。一个 $n \times n$ 矩阵可对角化的充分必要条件是，它拥有 n 个线性无关的特征向量。一个更易于判断的充分条件是：如果矩阵的 n 个特征值互不相同，那么它一定可对角化，因为不同特征值对应的特征向量必然线性无关。

title= 对角化的威力

计算一个矩阵的高次幂 A^{100} 通常非常复杂。但如果 A 可对角化，即 $A = MDM^{-1}$ ，那么计算将大大简化。

解

$$A^{100} = (MDM^{-1})^{100} = MDM^{-1}MDM^{-1} \dots MDM^{-1} = MD^{100}M^{-1}$$

计算对角矩阵的 100 次幂非常简单，只需将对角元各自 100 次方即可。这样，一个复杂的计算就被大大简化了。

更重要的是，对角化在思想上完成了本章的目标：通过找到一个变换内在的不变方向（特征向量），我们为这个变换找到了最能体现其本质的坐标系。在这个坐标系里，复杂的变换被分解为沿着各个“主轴”方向的简单伸缩。这正是我们在第四章结尾处，通过 PCA 寻找数据“主成分”时所追求的境界。PCA 中的主成分，正是数据协方差矩阵的特征向量；而将数据投影到主成分上，正是在利用对角化的思想，来揭示数据最主要的结构。

7.3.4 结论：从静态结构到动态变化

本章，我们为线性代数这门语言引入了至关重要的“动词”——线性变换。我们追溯了它的历史，看到它如何从具体的几何运动中抽象出保持加法和数乘这两个核心条件，从而成为描述结构化变化的通用工具。

在此基础上，我们建立了从抽象变换到具体矩阵的桥梁，明白了矩阵不仅是解方程的工具，更是线性变换的代数表示。进而，相似矩阵的概念让我们理解了同一个变换在不同视角（基）下的不同呈现。

最终，我们通过特征值和特征向量，探寻了变换最深层的“本性”——那些在变化中保持方向不变的“主轴”。这个源于物理学和天文学的古老思想，最终引领我们找到了对角化这一强大工具。对角化，不仅是计算上的简化，更是一种深刻的哲学思想：将复杂的运动分解为在“自然”坐标系下的一系列简单伸缩。

至此，我们已经掌握了描述静态空间（线性空间）和动态变化（线性变换）的语言。我们不仅能构建世界，还能描述和理解这个世界中的运动规律。这套语言和思想，将成为我们深入学习更高级数学，以及理解从量子力学到数据科学等众多应用领域的坚实基础。

第 8 章 欧氏空间

要点提炼

- 内积：长度与角度的代数基石
- 欧氏空间：装备了“度量尺”的线性空间
- 正交变换：保持几何形态的“刚性”运动

8.1 欧氏空间：从抽象到可度量

历史侧记

在前几章中，我们构建的线性空间是一个“仿射”的世界，而非“度量”的世界。我们能讨论平行、共线、共面，但无法衡量长短、远近、角度。这种纯粹的代数结构虽然强大，却脱离了我们赖以生存的物理空间的几何直观。本章的核心任务，就是为这个抽象的代数世界重新引入几何的度量衡。这一思想的回归，在历史上至关重要。19 世纪末 20 世纪初，当数学家们（如希尔伯特）试图将线性代数的思想推广到研究函数的无穷维空间时，他们发现，若要分析函数的收敛、近似等问题，就必须有一种方法来衡量两个函数之间的“距离”或一个函数的“大小”。正是这种源于分析学的迫切需求，推动了数学家们将欧几里得几何中最核心的度量概念——点积——进行提炼和公理化，最终孕育出“内积”这一普适的数学工具。

在此之前，我们所构建的线性空间是一个纯粹的代数世界。我们可以在其中谈论向量的加法与数乘，探讨基底与维数，分析线性变换的结构。然而，这个世界缺少了我们最直观的几何概念：我们无法测量一个向量的**长度**，也无法定义两个向量之间的**夹角**。我们所熟悉的“垂直”、“距离”等概念，在这个抽象的代数王国中都无从谈起。

为了弥合抽象代数与直观几何之间的鸿沟，本章将为线性空间引入一个新的核心结构——**内积** (Inner Product)。正是这个结构，如同为空间配备了一把万能的“度量尺”，使得长度、角度和正交等几何概念得以在任意线性空间中获得精确的代数定义。一个装备了内积的线性空间，我们称之为**欧氏空间** (Euclidean Space)，它标志着线性代数从纯粹的结构研究，迈向了对几何形态与度量关系的深刻探索。

8.1.1 历史的脉络：从几何测量到内积公理

历史侧记

几何概念的量化，其源头可以追溯到古希腊的**欧几里得** (Euclid)。在他的《几何原本》中，长度与角度是几何学的基本要素，但它们是作为公理而存在的，是一种不证自明的直观概念。这种状态持续了近两千年。

决定性的转折点发生在 17 世纪，**勒内·笛卡尔** (René Descartes) 创立了解析几何，将代数引入几何学。通过坐标系，几何图形被代数方程所描述，几何量也获得了计算公式。例如，平面上两点间的距离，可以通过著名的**勾股定理** (Pythagorean theorem) 来计算： $d^2 = (\Delta x)^2 + (\Delta y)^2$ 。这正是我们今天所熟知的二维向量**点积** (Dot Product) 的雏形： $\vec{v} \cdot \vec{v} = x^2 + y^2 = \|\vec{v}\|^2$ 。点积运算，作为计算长度和角度（通过余弦定理）的工具，在 19 世纪被物理学家和工程师（特别是通过哈密顿的四元数理论，以及后来吉布斯和亥维赛发展的现代向量分析）广泛应用于力学和电磁学研究。

然而，将点积的“性质”本身提炼出来，并将其推广到任意抽象空间的思想，直到 19 世纪末至 20 世纪初才完全成熟。这是数学公理化运动浪潮的一部分，数学家们开始寻求用最少的、最核心的公理来定义各种数学结构。其中，德国数学家**赫尔曼·格拉斯曼** (Hermann Grassmann) 在他超前的著作中，已经孕育了内积的抽象思想。最终，经过**大卫·希尔伯特** (David Hilbert) 等人在研究积分方程和无穷维函数空间（即后来的希尔伯特空间）时的奠基性工作，点积所具有的三个核心性质——**线性、对称、正定**——被提炼为公理，用以定义抽象的**内积**。这一步的伟大之处在于，它将欧几里得几何中的基本测量工具，成功地移植到了函数空间、多项式空间等更广阔的数学领域，使得我们可以在这些看似与几何无关的空间里，谈论函数的“长度”或多项式的“夹角”。

8.1.2 内积的定义：赋予空间度量结构

历史侧记

公理化方法的精髓在于“抓本质，舍表象”。当数学家们试图定义“内积”时，他们问了这样一个问题：我们日常使用的点积运算，其背后最核心、不可或缺的代数规则是什么？他们发现，无论是在二维平面还是三维空间，点积都天然满足交换律（对称性）、分配律（线性性）以及一个向量与自身的点积非负（正定性）。于是，他们大胆地断言：任何一个满足这几条规则的二元运算，无论其具体形式如何，都应该被“尊称”为内积。这四条公理，构成了进入“可度量”的线性世界的“门票”。任何一个通过了这四项检验的线性空间，都将自动继承所有由长度和角度衍生出的丰富几何性质，这正是公理化方法以简驭繁的威力所在。

我们如何将 $\mathbb{R}^n$ 空间中那个熟悉的点积运算 $\alpha \cdot \beta = x_1y_1 + x_2y_2 + \cdots + x_ny_n$ 推广到任意线性空间 V 呢？关键在于抓住其本质的代数属性，并将其公理化。

定义 8.1 (内积)

设 V 是一个实数域 $\mathbb{R}$ 上的线性空间。在 V 上定义的一个内积，是一个函数，它将 V 中的任意两个向量 α, β 映射为一个实数，记作 (α, β) ，并且满足以下四个公理：

1. 对称性 (Symmetry): $(\alpha, \beta) = (\beta, \alpha)$
2. 线性性 (Linearity): $(\alpha + \beta, \gamma) = (\alpha, \gamma) + (\beta, \gamma)$
3. 齐次性 (Homogeneity): $(k\alpha, \beta) = k(\alpha, \beta)$ ，其中 $k \in \mathbb{R}$
4. 正定性 (Positive-definiteness): $(\alpha, \alpha) \geq 0$ ，且 $(\alpha, \alpha) = 0$ 当且仅当 $\alpha = \vec{0}$

一个定义了内积的线性空间，就被称为欧氏空间。



注 线性性和齐次性通常被合并为“第一变元的线性性”。结合对称性，可以轻易推出内积对第二个变元同样是线性的。因此，内积是一个**双线性**的运算。而正定性是整个结构的核心，它确保了任何一个非零向量，其自身的“内积”都是一个正数，这就为“长度”的定义提供了可能。

8.1.3 长度、角度与正交：几何概念的代数重生

历史侧记

柯西-施瓦茨不等式是数学中“曝光率”最高的不等式之一，其历史也反映了数学思想的统一进程。1821 年，法国数学家柯西 (Cauchy) 在他的分析学教程中，以代数形式证明了关于有限个实数求和的离散版本。半个多世纪后，德国数学家赫尔曼·施瓦茨 (Hermann Schwarz) 在 1888 年研究极小曲面时，独立地证明了该不等式对于函数积分的连续版本。此前，俄国数学家维克托·布尼亚科夫斯基 (Viktor Bunyakovsky) 已于 1859 年发表了积分形式的不等式，因此在俄语文献中它常被称为“柯西-布尼亚科夫斯基-施瓦茨不等式”。这些在代数和分析领域中的独立发现，最终被内积空间的公理化框架所统一。本节中的巧妙证明，正是这一统一观点的体现：它揭示了该不等式并非代数或分析的特有技巧，而是任何满足内积公理的空间所固有的基本几何事实。

一旦欧氏空间被定义，所有我们熟悉的几何概念便可由内积这块基石推导出来。

首先，我们可以定义向量的**长度**，也称为**范数** (Norm)。

定义 8.2 (向量的长度/范数)

在欧氏空间 V 中，向量 α 的长度 (范数) 定义为：

$$\|\alpha\| = \sqrt{(\alpha, \alpha)}$$



正定性公理保证了根号下的值非负，并且只有零向量的长度为零。

接下来，为了定义角度，我们需要一个至关重要的不等式，它构成了欧氏空间几何理论的基石。

定理 8.1 (柯西-施瓦茨不等式 (Cauchy-Schwarz Inequality))

对于欧氏空间中任意两个向量 α, β ，恒有：

$$(\alpha, \beta)^2 \leq (\alpha, \alpha)(\beta, \beta)$$

或者写成范数形式：

$$|(\alpha, \beta)| \leq \|\alpha\| \|\beta\|$$



推导

这个不等式的推导十分巧妙。考虑向量 $\alpha + t\beta$ ，其中 t 是任意实数。根据内积的正定性，我们知道 $(\alpha + t\beta, \alpha + t\beta) \geq 0$ 。利用内积的双线性性质展开它：

$$(\alpha, \alpha) + 2t(\alpha, \beta) + t^2(\beta, \beta) \geq 0$$

这是一个关于变量 t 的二次三项式，它恒大于等于零。这意味着对应的二次方程 $(\beta, \beta)t^2 + 2(\alpha, \beta)t + (\alpha, \alpha) = 0$ 最多只有一个实根。因此，它的判别式必须小于或等于零：

$$\Delta = (2(\alpha, \beta))^2 - 4(\beta, \beta)(\alpha, \alpha) \leq 0$$

移项并除以 4，即可得到 $(\alpha, \beta)^2 \leq (\alpha, \alpha)(\beta, \beta)$ 。

柯西-施瓦茨不等式保证了 $\frac{(\alpha, \beta)}{\|\alpha\| \|\beta\|}$ 的值总是在 $[-1, 1]$ 区间内，这使得我们可以用它来定义向量间的夹角。

定义 8.3 (向量的夹角)

对于欧氏空间中任意两个非零向量 α, β ，它们之间的夹角 θ 由下式定义：

$$\cos \theta = \frac{(\alpha, \beta)}{\|\alpha\| \|\beta\|}$$



至此，最核心的几何概念——长度和角度——都在抽象的线性空间中找到了它们坚实的代数基础。由此，一个关键概念应运而生：**正交** (Orthogonality)，即我们对“垂直”的推广。当 $(\alpha, \beta) = 0$ 时， $\cos \theta = 0$ ，夹角为 90° 。

定义 8.4 (正交)

如果 $(\alpha, \beta) = 0$ ，则称向量 α 和 β 正交。



8.1.4 标准正交基与 Gram-Schmidt 过程

历史侧记

“正交化”的思想，即将一组倾斜的坐标轴“摆正”，在历史上是与二次型化简和主轴问题紧密相连的。然而，将其发展成一个明确的、构造性的算法，则要归功于丹麦数学家约尔根·佩德森·格拉姆（Jørgen Pedersen Gram）和德国数学家埃哈德·施密特（Erhard Schmidt）。格拉姆在 1883 年研究最小二乘法时，发表了这一正交化过程。二十多年后，希尔伯特的学生施密特在 1907 年研究积分方程的博士论文中，独立地（或推广了）这一方法，并将其应用到无穷维的函数空间中，证明了希尔伯特空间中标准正交基的存在性。因此，这个算法的名字将两位数学家联系在一起，它不仅是有限维欧氏空间中的一个实用计算工具，更是现代泛函分析理论的一块重要基石。

在熟悉的二维或三维空间中，我们最喜欢使用的坐标系是由单位长度、两两垂直的基向量构成的。这种基底让坐标计算变得异常简洁。我们自然希望在任何抽象的欧氏空间中都能找到这样“好”的基。

定义 8.5 (标准正交基)

欧氏空间的一组基 $\{\eta_1, \dots, \eta_n\}$ 如果满足条件：

$$(\eta_i, \eta_j) = \delta_{ij} = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases}$$

则称之为**一组标准正交基 (Orthonormal Basis)**。这个条件意味着基向量两两正交，且每个向量的长度都为 1。



那么，这样的基一定存在吗？答案是肯定的。我们可以通过一个构造性的算法，将任何一组普通的基转化为标准正交基。这个算法就是著名的 **Gram-Schmidt 正交化过程**。

这个过程的思想是循序渐进、逐步“矫正”的。假设我们有一组线性无关的基 $\{\alpha_1, \alpha_2, \dots, \alpha_n\}$ 。

- 第一步：确立方向。**取第一个向量 $\beta_1 = \alpha_1$ 作为新基的第一个方向。
- 第二步：修正第二个向量。**取 α_2 ，它很可能与 β_1 不正交。为了得到与 β_1 正交的新向量 β_2 ，我们需要从 α_2 中减去它在 β_1 方向上的**投影分量**。 α_2 在 β_1 上的投影向量为 $\frac{(\alpha_2, \beta_1)}{(\beta_1, \beta_1)}\beta_1$ 。因此，我们构造：

$$\beta_2 = \alpha_2 - \frac{(\alpha_2, \beta_1)}{(\beta_1, \beta_1)}\beta_1$$

通过简单的内积计算可以验证 $(\beta_2, \beta_1) = 0$ 。

- 第三步：修正第三个向量。**取 α_3 ，并减去它在已经正交的 β_1 和 β_2 方向上的所有投影分量：

$$\beta_3 = \alpha_3 - \frac{(\alpha_3, \beta_1)}{(\beta_1, \beta_1)}\beta_1 - \frac{(\alpha_3, \beta_2)}{(\beta_2, \beta_2)}\beta_2$$

4. **重复该过程。**对于第 k 个向量，我们有：

$$\beta_k = \alpha_k - \sum_{j=1}^{k-1} \frac{(\alpha_k, \beta_j)}{(\beta_j, \beta_j)} \beta_j$$

这样，我们就得到了一组两两正交的基 $\{\beta_1, \dots, \beta_n\}$ 。

5. **最后一步：单位化。**将每个正交基向量除以它自身的长度，得到单位向量：

$$\eta_k = \frac{\beta_k}{\|\beta_k\|}$$

最终得到的 $\{\eta_1, \dots, \eta_n\}$ 就是一组标准正交基。这个算法从理论上保证了任何有限维欧氏空间都存在标准正交基，这是欧氏空间理论中一个极其深刻且有用的结论。

8.2 正交变换：空间中的刚性运动

历史侧记

对“刚性运动”的研究，是几何学最古老的主题之一。然而，用统一的代数语言来描述它们，则是 18、19 世纪的产物。欧拉对三维旋转的深入研究（欧拉角、欧拉旋转定理）是这一领域的里程碑。进入 19 世纪，随着群论的诞生，数学家们开始从更抽象的视角看待这些变换。菲利克斯·克莱因（Felix Klein）在他 1872 年著名的“爱尔兰根纲领”中，提出了一个革命性的观点：一种“几何学”可以被定义为研究在某个特定“变换群”作用下保持不变的性质。据此，欧氏几何的核心，正是研究在所有刚性运动（平移、旋转、反射）构成的变换群下，长度、角度、面积等不变量的科学。本节所要研究的“正交变换”，正是这个刚性运动群中的核心组成部分（除去平移），它们构成了所谓的“正交群” $O(n)$ ，是现代数学和物理学（如李群理论、规范场论）中最重要的研究对象之一。

在第五章，我们研究了线性变换，它们是保持线性空间代数结构（加法和数乘）的映射。现在，我们进入了装备有度量结构的欧氏空间，一个自然而然的问题浮出水面：什么样的线性变换能够同时保持空间的**几何结构**？也就是说，在变换之后，所有向量的长度和它们之间的夹角都保持不变。

这些特殊的变换，对应于我们现实世界中的**刚体运动**，如旋转和反射。它们移动和翻转物体，但不会拉伸或扭曲物体的形状。在数学上，我们称之为**正交变换**（Orthogonal Transformation）。

8.2.1 正交变换的定义与等价刻画

历史侧记

在数学中，一个深刻的性质往往有多种等价的表述，从不同侧面揭示其本质。正交变换的“保内积”、“保长度”、“保标准正交基”这三个条件的等价性，就是一个经典的例子。从历史上看，数学家们最初可能是从最直观的“保长度”（即保持刚性）出发来思考这类变换的。然而，“保内积”的定义在代数上更为根本和优雅。而“保标准正交基”则提供了具体的判定和构造方法。证明这三者等价的关键，即“极化恒等式”，在历史上是与二次型和双线性型理论的发展紧密相连的。这一系列等价刻画的建立，使得我们可以在几何直观、代数定义和具体计算之间自由切换，是理论成熟的标志。

我们的目标是找到那些保持长度和角度的变换。一个惊人的事实是，我们只需要一个条件就足够了。

定义 8.6 (正交变换)

设 T 是欧氏空间 V 上的一个线性变换。如果它保持内积不变，即对于 V 中任意向量 α, β ，都有：

$$(T(\alpha), T(\beta)) = (\alpha, \beta)$$

则称 T 是一个正交变换。



这个定义直接保证了角度的不变性，因为角度是由内积和长度共同定义的。但更有趣的是，它也同时保证了长度的不变性。在定义中令 $\beta = \alpha$ ，我们得到 $(T(\alpha), T(\alpha)) = (\alpha, \alpha)$ ，两边开方即得 $\|T(\alpha)\| = \|\alpha\|$ 。

反过来，一个保持长度的线性变换，是否也一定保持内积呢？答案是肯定的。这可以通过一个叫做**极化恒等式** (Polarization Identity) 的公式来证明，它揭示了内积可以完全由范数来表达：

$$(\alpha, \beta) = \frac{1}{4} (\|\alpha + \beta\|^2 - \|\alpha - \beta\|^2)$$

如果一个线性变换 T 保持长度，即 $\|T(\vec{v})\| = \|\vec{v}\|$ 对所有向量 $\vec{v}$ 成立，那么：

$$(T(\alpha), T(\beta)) = \frac{1}{4} (\|T(\alpha) + T(\beta)\|^2 - \|T(\alpha) - T(\beta)\|^2)$$

利用 T 的线性，上式变为：

$$= \frac{1}{4} (\|T(\alpha + \beta)\|^2 - \|T(\alpha - \beta)\|^2)$$

由于 T 保持长度，这等于：

$$= \frac{1}{4} (\|\alpha + \beta\|^2 - \|\alpha - \beta\|^2) = (\alpha, \beta)$$

这证明了保持长度和保持内积是等价的。因此，我们有了一系列判断正交变换的等价条件。

定理 8.2

对于欧氏空间 V 上的线性变换 T ，以下命题等价：

1. T 是一个正交变换（即保持内积）。
2. T 保持向量的长度（即 $\|T(\alpha)\| = \|\alpha\|$ 对所有 α 成立）。
3. T 将任意一组标准正交基映射为另一组标准正交基。



第三个条件在实际应用中非常有用。它意味着，要判断一个变换是否为正交变换，我们无需检验空间中无穷多的向量，只需检验它对一组标准正交基的作用即可。

8.2.2 正交矩阵：正交变换的代数身份证

历史侧记

“正交”（Orthogonal）一词由德国数学家弗里德里希·弗洛贝尼乌斯（Friedrich Frobenius）于 1878 年引入矩阵理论。他发现，那些在坐标变换下保持二次型 $x_1^2 + \cdots + x_n^2$ 不变的矩阵，具有 $A^T A = I$ 这一优美的代数性质。这个代数条件，完美地将保持长度这一几何概念，翻译成了简洁的矩阵语言。正交矩阵的出现，使得对旋转和反射等刚性运动的研究，可以完全转化为对一类特殊矩阵的代数分析。例如，两个旋转的复合，对应于两个旋转矩阵的乘积；而一个旋转的“撤销”，则对应于其逆矩阵——而这个逆矩阵竟然简单到只需取其转置即可。这种几何与代数之间的深刻对应，是线性代数魅力的集中体现。

现在，我们将这个抽象的几何概念与具体的矩阵表示联系起来。一个正交变换，在标准正交基下，会对应一个什么样的矩阵呢？

设 A 是变换 T 在一组**标准正交基** $\{\epsilon_1, \dots, \epsilon_n\}$ 下的矩阵。设任意两个向量 α 和 β 在这组基下的坐标分别为列向量 X 和 Y 。由于基是标准正交的，向量的内积可以直接通过坐标的转置乘法计算：

$$(\alpha, \beta) = X^T Y$$

经过变换后，向量 $T(\alpha)$ 和 $T(\beta)$ 的坐标分别为 AX 和 AY 。它们的内积为：

$$(T(\alpha), T(\beta)) = (AX)^T (AY) = X^T A^T A Y$$

因为 T 是一个正交变换，所以必须有 $(T(\alpha), T(\beta)) = (\alpha, \beta)$ 对任意的 α, β 成立。这意味着：

$$X^T Y = X^T (A^T A) Y$$

由于这个等式对任意坐标向量 X, Y 都成立，唯一的可能性就是中间的矩阵为单位矩阵，即：

$$A^T A = I$$

定义 8.7 (正交矩阵)

一个 $n \times n$ 的实数方阵 A 如果满足 $A^T A = I$ ，则称之为正交矩阵 (Orthogonal Matrix)。



这个简洁的代数关系 $A^T A = I$ 蕴含了丰富的几何信息：

- **逆矩阵即转置**：由 $A^T A = I$ 可知，正交矩阵 A 必然可逆，且其逆矩阵就是它的转置矩阵，即 $A^{-1} = A^T$ 。
- **行列向量的几何特性**： $A^T A = I$ 意味着 A 的**列向量**构成了一组标准正交基。同样地，由 $AA^T = I$ 可知， A 的**行向量**也构成了一组标准正交基。
- **行列式**：对 $A^T A = I$ 两边取行列式，得到 $\det(A^T) \det(A) = 1$ ，即 $(\det(A))^2 = 1$ 。因此，正交矩阵的行列式必定为 $+1$ 或 -1 。

行列式的值揭示了正交变换的两种基本几何类型：

- 当 $\det(A) = 1$ 时，变换被称为**旋转** (Rotation)，它保持了空间的“定向”或“手性”（例如，保持坐标系的右手准则）。
- 当 $\det(A) = -1$ 时，变换被称为**瑕旋转或反射** (Improper Rotation / Reflection)，它会反转空间的定向（例如，将右手坐标系变为左手坐标系）。一个纯粹的镜面反射就是典型的例子。

因此，正交矩阵 $A^T A = I$ 这个简单的代数条件，完美地捕捉了所有保持几何形态不变的刚性运动的本质。

8.2.3 正交变换的几何本质：旋转与反射

历史侧记

欧拉旋转定理 (1775 年) 是运动学中的一个基本而深刻的结论。欧拉证明，一个球面上任意复杂的、保持球心不动的刚性位移，最终的效果都等同于绕某一个穿过球心的固定轴旋转一个角度。这个发现在当时对于描述陀螺等刚体的复杂运动至关重要。然而，在欧拉的时代，还没有特征值和特征向量的概念。直到 150 多年后，随着线性代数理论的成熟，数学家们才恍然大悟：欧拉发现的那个“固定轴”，在代数上，正是在旋转变换下方向保持不变的向量——也就是旋转矩阵对应于特征值 1 的特征向量！这个历史案例完美地展示了数学的统一性：一个源于 18 世纪刚体动力学的几何定理，其最深刻的代数解释，却要等到 19 世纪末的特征值理论来提供。

让我们回到我们最熟悉的二维和三维空间，看看正交变换的矩阵究竟是什么样子。

在二维平面 $\mathbb{R}^2$ 中，任何一个正交矩阵 A 都可以写成以下两种形式之一：

1. **旋转矩阵** ($\det(A) = 1$):

$$A = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$$

这代表了将整个平面绕原点逆时针旋转角度 θ 。

2. 反射矩阵 ($\det(A) = -1$):

$$A = \begin{pmatrix} \cos \theta & \sin \theta \\ \sin \theta & -\cos \theta \end{pmatrix}$$

这代表了关于过原点且与 x 轴夹角为 $\theta/2$ 的直线的镜面反射。

在三维空间 $\mathbb{R}^3$ 中，情况更为深刻。一个著名的定理，即**欧拉旋转定理** (Euler's Rotation Theorem)，指出：

定理 8.3

三维空间中任何一个保持原点不变的刚体运动（即行列式为 1 的正交变换），都等价于围绕某一个固定轴的旋转。



这个“固定轴”在代数上有什么意义呢？它正是在旋转中保持不变的方向。换言之，这个轴就是该正交变换对应于**特征值为 1 的特征向量**！一个三维旋转矩阵，必然有一个特征值为 1。这个结论将本章的正交变换与上一章的特征值理论完美地联系在了一起，它告诉我们，要理解一个三维旋转，关键就是找到它的旋转轴，也就是找到那个不变的特征向量。

这再次印证了线性代数的核心思想：通过寻找变换下的“不变性”（如特征向量），我们可以洞察和理解变换最根本的结构与几何本质。从抽象的内积公理出发，我们最终回到了对空间旋转这一直观运动的深刻代数理解。

第9章 二次型

要点提炼

- **问题的起源**：从笛卡尔的坐标系到恼人的“交叉项”，理解二次型的几何本质。
- **思想的第一次飞跃**：如何用对称矩阵这一优雅的代数工具，来“捕获”一个二次型？
- **代数的蛮力与智慧**：拉格朗日的配方法为何能成功？它揭示了什么，又隐藏了什么？
- **不变量的探索**：在千变万化的形式背后，西尔维斯特如何找到二次型永恒的“灵魂”——惯性指数？
- **正定性的价值**：为何“处处为正”的性质在优化与几何中如此关键？如何高效地判别它？
- **终极的解决方案**：正交变换如何像一位几何大师，通过一次完美的“旋转”，让一切回归简单与和谐？主轴定理的深刻内涵是什么？

9.1 引言：超越线性，直面旋转的世界

历史侧记

二次曲线的研究可以追溯到古希腊的数学家，特别是阿波罗尼奥斯（Apollonius of Perga），他撰写的《圆锥曲线论》是古代几何学的巅峰之作。然而，在长达1800年的时间里，对椭圆、双曲线和抛物线的研究都是纯粹几何的。17世纪，笛卡尔和费马创立的解析几何带来了第一次革命，将这些曲线与代数方程联系起来。数学家们很快注意到，所有这些曲线都可以由一个统一的二元二次方程描述。其中， xy 交叉项的存在，总是对应着图形的一次旋转，这是一个棘手的几何问题。早期的处理方法依赖于复杂的、针对特定问题的三角换元法，缺乏普适性和理论深度。本章所要讲述的，正是数学家们在19世纪如何最终征服了这个“交叉项”问题。他们所用的武器，不再是零敲碎打的几何技巧，而是系统化的、威力强大的线性代数理论。

自笛卡尔创立解析几何以来，代数与几何的联姻便深刻地改变了数学的面貌。直线由一次方程 $ax + by + c = 0$ 描述，平面由 $ax + by + cz + d = 0$ 描述。它们的共性是“线性”，这种简洁性使得我们可以运用线性方程组的理论来分析它们的相交与位置关系，这也是我们前几章学习的核心。

然而，当我们把目光投向更广阔的几何世界——椭圆、双曲线、抛物线——线性便

不够用了。这些美丽的圆锥曲线，无一例外，都由**二次方程**所定义。一个一般的二维二次方程形如：

$$Ax^2 + Bxy + Cy^2 + Dx + Ey + F = 0$$

通过坐标系的平移，我们可以消去一次项 Dx 和 Ey 。此时，方程的核心便落在了二次部分—— $Ax^2 + Bxy + Cy^2$ ——我们称之为**二次型**。

这个二次型的结构，尤其是**交叉项** Bxy 的存在与否，直接决定了图形的“姿态”。

- 当 $B = 0$ 时，方程形如 $Ax^2 + Cy^2 = k$ 。这是我们最熟悉的形式，它定义的椭圆或双曲线，其对称轴（主轴）与坐标轴 x, y 完全对齐。图形的性质一目了然。
- 当 $B \neq 0$ 时，交叉项的出现如同一个“捣乱者”。它使得图形发生了**旋转**，主轴偏离了坐标轴的方向。图形的几何性质，如主轴长度、焦点位置等，都被隐藏在了复杂的系数之中。

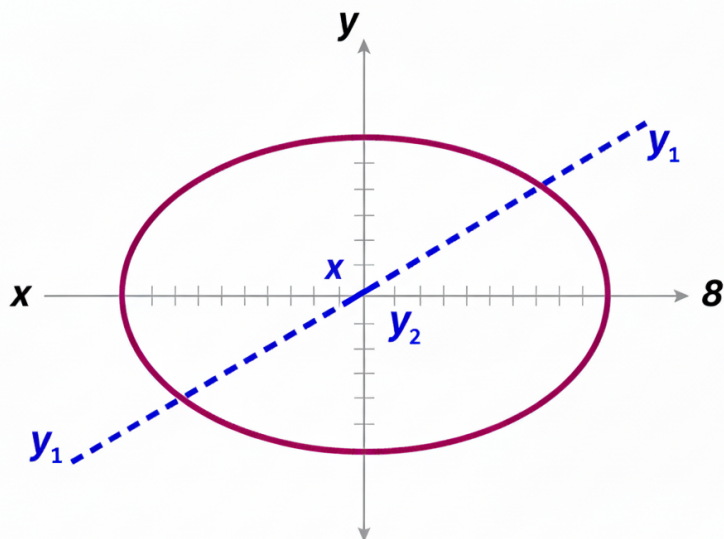


图 9.1: 一个包含交叉项 xy 的二次方程 $5x^2 - 6xy + 5y^2 = 8$ 所对应的椭圆。其主轴并未与 x, y 轴对齐，几何性质不够直观。本章的核心目标就是找到一个旋转后的新坐标系 (y_1, y_2) ，使得在该坐标系下方程不含交叉项。

这个问题，从二维推广到三维乃至 n 维，构成了本章的核心动机。一个 n 元二次型定义了 n 维空间中的一个超二次曲面。交叉项的存在，意味着这个曲面相对于我们观察的坐标系是“歪斜”的。因此，我们的根本任务可以表述为：

> **核心问题：**我们能否找到一个新的坐标系，使得在这个新坐标系下观察同一个几何体时，它的方程中所有的交叉项都消失？

这本质上是一个“拨乱反正”的过程。我们希望通过变换视角（改变坐标系），来消除表象的复杂性（交叉项），从而洞悉事物的内在本质（图形的标准形状和主轴）。这个问题的解决，将不再依赖于解析几何的技巧，而是要诉诸于我们在前面章节中建立起来的、更强大更抽象的武器库——矩阵、线性变换、特征值与特征向量。我们将见证，一

个纯粹的几何问题，如何被转化为一个代数问题，并最终通过线性代数的理论获得完美解答。

9.2 n 元实二次型和标准型

历史侧记

二次型的研究，从 18 世纪起就成为数学分析和物理学的中心议题之一。拉格朗日在其巨著《分析力学》中，用广义坐标下的二次型来表达系统的动能。高斯在他的大地测量学和数论研究中，深入地探讨了二元和三元二次型。到了 19 世纪，柯西在研究弹性力学中的应力与应变时，发现它们的关系由二次型（应变能函数）所刻画。这些来自不同领域的需求，共同推动数学家们去寻找一种统一的语言来描述和简化这些多变量的二次齐次多项式。本节将要介绍的矩阵表示法和标准型，正是这场长达一个世纪的探索所结出的硕果。

为了系统地解决上述问题，我们首先需要发展一套能够精确描述和操作二次型的代数语言。

9.2.1 思想的第一次飞跃：用矩阵“捕获”二次型

历史侧记

在凯莱于 19 世纪 50 年代系统地引入矩阵运算之前，数学家们处理二次型时，只能依赖于冗长的多项式表达式。凯莱的革命性贡献在于，他提供了一种方法，能将一个看似无序的系数集合“组织”成一个结构化的对象——矩阵。而 $\vec{x}^T A \vec{x}$ 这一符号表示，更是一次伟大的智力飞跃，它将一个多项式求值过程，变成了一次优雅的矩阵乘法。其中，强制要求矩阵 A 对称，并非仅仅为了表示的唯一性。以柯西为代表的数学家们早已凭直觉和计算发现，二次型的几何属性（如主轴的存在性）与系数的对称性密切相关。因此，选择对称矩阵作为二次型的“标准身份证”，是深刻洞察了其代数形式与几何本质之间联系的结果。

让我们审视一个三元二次型：

$$f(x_1, x_2, x_3) = a_{11}x_1^2 + a_{22}x_2^2 + a_{33}x_3^2 + 2a_{12}x_1x_2 + 2a_{13}x_1x_3 + 2a_{23}x_2x_3$$

(此处将交叉项系数写为 $2a_{ij}$ 是为了后续矩阵形式的优雅，这是一个有意的选择。)

这个表达式冗长而缺乏结构性。我们如何才能系统地组织这些系数？这里的思想飞跃是：**尝试用矩阵乘法来生成这个多项式**。我们注意到，每一项都是两个变量的乘积。这启发我们构造一个行向量 $\vec{x}^T = [x_1, x_2, x_3]$ 和一个列向量 $\vec{x} = [x_1, x_2, x_3]^T$ 。那么，什么东西应该放在它们中间，才能在相乘后得到我们的二次型呢？答案是一个 3×3 的系数矩阵 A 。

$$\vec{x}^T A \vec{x} = \begin{pmatrix} x_1 & x_2 & x_3 \end{pmatrix} \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}$$

展开这个矩阵乘法，我们得到 $\sum_{i=1}^3 \sum_{j=1}^3 a_{ij} x_i x_j$ 。为了使它与原始二次型完全对应，我们发现对角元 a_{ii} 就是 x_i^2 的系数。对于交叉项，例如 $x_1 x_2$ ，它在展开式中出现了两次： $a_{12} x_1 x_2$ 和 $a_{21} x_2 x_1$ 。它们的和是 $(a_{12} + a_{21}) x_1 x_2$ 。

9.2.1.0.1 一个关键的“设计选择”：对称性 我们希望二次型的矩阵表示是唯一的。为了达到此目的，我们做出一个至关重要的规定：将交叉项 $x_i x_j$ 的系数在 a_{ij} 和 a_{ji} 之间平均分配。即，我们强制要求 $a_{ij} = a_{ji}$ 。

• 这么做的好处是什么？

1. **唯一性**：一旦规定了矩阵必须对称，那么对于任意一个二次型，其对应的矩阵就唯一确定了。 x_i^2 的系数是 a_{ii} ， $x_i x_j$ ($i \neq j$) 的系数是 $2a_{ij}$ ，反之亦然。
2. **理论的宝库**：更重要的是，我们在线性代数中已经知道，**实对称矩阵** 拥有一系列极其优美的性质（例如，必可对角化，特征值为实数，不同特征值的特征向量正交等）。通过将我们的研究对象限定在对称矩阵上，我们等于为自己打开了一个蕴含着强大理论工具的宝库。

至此，我们完成了第一次思想飞跃：任何 n 元实二次型，都可以唯一、紧凑地表示为 $f(\vec{x}) = \vec{x}^T A \vec{x}$ ，其中 A 是一个实对称矩阵。这个表示法将一个看似杂乱的多项式，转化为了一个结构清晰的代数对象。

9.2.2 代数的力量：变量代换与合同变换

历史侧记

变量代换是数学中最古老也最强大的思想之一。而在 19 世纪，随着不变量理论的兴起，数学家们开始系统地研究：在变量代换下，一个代数表达式的“形式”会如何改变，而它的“本质”又有哪些是不变的？线性代换 $\vec{x} = C\vec{y}$ 是最基本的一类代换。数学家们发现，当变量经历一次线性“整形”（由 C 描述）时，二次型的系数矩阵 A 会经历一次不同的、被称为“合同”的“整形”（由 $C^T A C$ 描述）。“合同”和“相似”这两个概念的区分，是 19 世纪末矩阵理论成熟过程中的一个重要里程碑。它揭示了同一个代数对象（矩阵）在扮演不同角色（描述线性变换 vs. 描述二次型）时，其在坐标变换下的行为法则是不同的。

现在，我们用新的矩阵语言来重新表述我们的目标：消除交叉项。一个二次型没有交叉项，当且仅当它的矩阵 A 是一个**对角矩阵**。因此，我们的问题转化为：**> 代数核心问题**：给定一个对称矩阵 A ，我们能否找到一个**坐标变换**，使得在新坐标下，二次型的矩阵变成一个对角矩阵 D ？

一个坐标变换，最普遍的形式是**可逆线性变换**，用矩阵表示为 $\vec{x} = C\vec{y}$ ，其中 C 是一个可逆矩阵。 $\vec{y}$ 是新坐标， $\vec{x}$ 是旧坐标。让我们看看这个代换如何影响二次型的矩阵。

9.2.2.0.1 推导： 将 $\vec{x} = C\vec{y}$ 代入二次型的表达式：

$$\begin{aligned} f(\vec{x}) &= \vec{x}^T A \vec{x} \\ &= (C\vec{y})^T A (C\vec{y}) \end{aligned}$$

利用转置的性质 $(AB)^T = B^T A^T$ ，我们得到：

$$= (\vec{y}^T C^T) A (C\vec{y})$$

利用矩阵乘法的结合律：

$$= \vec{y}^T (C^T A C) \vec{y}$$

9.2.2.0.2 结论： 在新的 $\vec{y}$ 坐标系下，二次型的矩阵不再是 A ，而是一个新的矩阵 $B = C^T A C$ 。我们的目标就是找到一个可逆矩阵 C ，使得 $B = C^T A C$ 是一个对角矩阵。

定义 9.1 (矩阵的合同)

如果存在一个可逆矩阵 C ，使得 $B = C^T A C$ ，则称矩阵 A 与 B 是合同的 (Congruent)。



注 必须将“合同”与我们之前学过的“相似”区分开。

- **相似** ($B = C^{-1} A C$): 描述的是**同一个线性变换在不同基下**的矩阵表示。它保留了所有内在的变换属性，如特征值、行列式、迹。
- **合同** ($B = C^T A C$): 描述的是**同一个二次型在不同基**（由坐标变换引起）下的矩阵表示。它不一定保留特征值，但我们将看到，它保留了另一些关键属性。

9.2.3 历史的回响：拉格朗日的配方法

历史侧记

约瑟夫-路易斯·拉格朗日 (Joseph-Louis Lagrange) 是 18 世纪末法国最伟大的数学家之一。他发展出“配方法”这一纯代数技巧，并非主要为了解决几何问题，而是为了处理他在天体力学和分析力学研究中遇到的多变量函数。例如，在分析一个物理系统的振动时，系统的动能和势能通常可以近似表示为广义坐标和广义速度的二次型。通过配方法将这些二次型化为平方和的形式，可以极大地简化运动方程的求解，并从中分离出系统独立的振动模式。拉格朗日的成功，展示了纯粹的代数操作如何能够深刻地揭示物理系统的内在结构。他的方法是一种“算法”上的胜利：它提供了一个必定能成功的、一步步消除交叉项的机械流程。

在 19 世纪矩阵理论完善之前，拉格朗日就已经用纯粹的代数技巧解决了将二次型化为标准型的问题。他的方法——配方法——虽然看似“笨拙”，却蕴含着深刻的洞察，并且其每一步操作都与我们刚刚推导的矩阵变换一一对应。

9.2.3.0.1 思路的演进：拉格朗日的核心思想是：**一次只解决一个变量**。让我们以 $f(x_1, x_2) = x_1^2 - 4x_1x_2 + 5x_2^2$ 为例，看看他是如何思考的。

1. **聚焦 x_1 ：**将所有包含 x_1 的项集中起来： $(x_1^2 - 4x_1x_2)$ 。

2. **配成完全平方：**把 x_2 暂时看作常数，对 x_1 进行配方。为了得到 $(x_1 - k)^2$ 的形式，我们凑出：

$$(x_1 - 2x_2)^2 = x_1^2 - 4x_1x_2 + 4x_2^2$$

3. **替换与整理：**将此式代回原式：

$$f = (x_1 - 2x_2)^2 - 4x_2^2 + 5x_2^2 = (x_1 - 2x_2)^2 + x_2^2$$

4. **引入新变量：**令 $y_1 = x_1 - 2x_2$, $y_2 = x_2$ 。这是一个变量代换。

最终，我们得到了标准型 $f = y_1^2 + y_2^2$ 。

9.2.3.0.2 与矩阵变换的联系：这个看似简单的配方，背后隐藏着一个矩阵变换。我们的变量代换是：

$$\begin{cases} y_1 = x_1 - 2x_2 \\ y_2 = x_2 \end{cases} \quad \text{或者} \quad \begin{pmatrix} y_1 \\ y_2 \end{pmatrix} = \begin{pmatrix} 1 & -2 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

这是 $\vec{y} = C^{-1}\vec{x}$ 的形式。我们需要的是 $\vec{x} = C\vec{y}$ ，所以求逆得到：

$$\vec{x} = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix} \vec{y}, \quad \text{即} \quad C = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}$$

原始矩阵是 $A = \begin{pmatrix} 1 & -2 \\ -2 & 5 \end{pmatrix}$ 。让我们来验证一下合同变换：

$$C^T A C = \begin{pmatrix} 1 & 0 \\ 2 & 1 \end{pmatrix} \begin{pmatrix} 1 & -2 \\ -2 & 5 \end{pmatrix} \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 2 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ -2 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

这正是标准型 $f = 1 \cdot y_1^2 + 1 \cdot y_2^2$ 所对应的对角矩阵！

历史侧记

拉格朗日的配方法，本质上是在进行一系列“凑”出来的、形式为下三角矩阵的基变换。它是一种**算法上的成功**，保证能将任何二次型化为平方和。这种算法的确定性和有效性，完美体现了 18 世纪分析学的精神。但它的**局限性**在于，这个过程缺乏几何直观，是一种“代数蛮力”。我们得到的变换矩阵 C 通常不是正交的，这意味着它不仅旋转了坐标系，还进行了剪切和拉伸。我们虽然得到了一个代数上简洁的形式，但却破坏了空间的原始度量（长度和角度），使得新旧

坐标系之间的几何关系变得复杂。这种为了代数简洁而牺牲几何“保真度”的做法，激发了后来的数学家们（如柯西）去寻找一种“更好”的变换——即只旋转、不形变的变换——这为正交变换的登场埋下了伏笔。

9.2.4 不变量的荣耀：西尔维斯特惯性定理

历史侧记

詹姆斯·约瑟夫·西尔维斯特（James Joseph Sylvester）是 19 世纪英国最富创造力的数学家之一，他与凯莱共同开创了不变量理论。在 1852 年，他发表了关于二次型“惯性”的这一定理。他之所以选择“惯性”这个物理学词汇，是因为他敏锐地察觉到，标准型中正负项的个数，如同物体的质量一样，是二次型内在的、不随观察者（坐标系）改变的属性。这个定理的发现，是 19 世纪代数学从“计算”转向“结构”研究的典范。它首次揭示了，在看似无穷无尽的化简可能性背后，隐藏着一个简单而深刻的数字不变量。这一定理不仅为二次型的分类提供了坚实基础，也启发了后来的数学家在更广泛的领域（如拓扑学中的同调群）中寻找不变量，成为现代数学的核心思想之一。

我们已经看到，将一个二次型化为标准型的方法不唯一。拉格朗日配方法的顺序不同，得到的标准型系数也可能不同。那么，在这些变化的表象之下，是否有某种东西是永恒不变的？

这个问题驱动了 19 世纪伟大的代数学家 J.J. 西尔维斯特。他所处的时代，是物理学大发展的时代，守恒律和不变量的思想深入人心。他相信，在代数变换中，也必然存在着类似的“守恒量”。

定理 9.1 (西尔维斯特惯性定理 (Sylvester's Law of Inertia))

对于任意一个实二次型，无论通过何种可逆线性变换将其化为标准型 $f = d_1 y_1^2 + \cdots + d_n y_n^2$ ，其标准型中正系数的个数 (p)、负系数的个数 (q) 都是唯一确定的不变量 [sylvester1852]。



(零系数的个数 $n - p - q$ 也因此是不变的，它等于 $n - r$ ，其中 r 是矩阵 A 的秩。)

9.2.4.0.1 这个定理的深刻意义是什么？ 它告诉我们，一个二次型的“本性”——它在多少个维度上是“扩张”的（正系数），在多少个维度上是“收缩”的（负系数）——是其固有的代数 DNA，不会因为我们选择不同的坐标系（即可逆变换）而改变。这就像我们给一个人拍照，无论从哪个角度拍（变换），照片里的人总是一个头、两只手，这些基本特征是不变的。

- **正惯性指数 (p) 与 负惯性指数 (q)** 是二次型的核心不变量。
- **秩 ($r = p + q$)** 也是一个不变量，它等于二次型矩阵的秩。

- **符号差** ($s = p - q$) 经常被用来给二次型进行唯一的“签名”。

西尔维斯特惯性定理是二次型理论的第一个深刻结果。它将我们的关注点从具体的、依赖于计算过程的系数值，提升到了抽象的、内蕴于二次型结构的指数上。这为我们下一步根据符号对二次型进行分类，尤其是定义“正定性”，提供了坚实的理论基石。

9.3 正定二次型

历史侧记

“正定性”这一概念的明确化，与 19 世纪数学分析的严谨化进程息息相关。在多元微积分的发展中，数学家们（如拉格朗日和柯西）试图将单变量函数通过二阶导数判断极值的方法，推广到多变量。他们发现，判断一个多元函数在临界点是极大值、极小值还是鞍点，其关键在于其泰勒展开式中二次项（即一个二次型）的符号。然而，早期的方法依赖于复杂的代数变形。直到 19 世纪下半叶，随着矩阵理论和行列式技巧的成熟，数学家们才发展出系统的判别法。特别是西尔维斯特提出的顺序主子式判别法，为这个问题提供了一个优雅且可计算的解决方案。正定性的研究，是纯粹代数理论（二次型）与分析学应用（优化问题）完美结合的典范。

在所有类型的二次型中，有一类扮演着无可替代的角色，它就是正定二次型。它与物理学中的能量、统计学中的方差、优化理论中的极小值点以及几何中的“距离”概念紧密相连。

9.3.1 动机：为何“恒为正”如此重要？

历史侧记

将一个物理系统的稳定性与一个数学函数的极小值联系起来，是分析力学的核心思想之一，其代表人物是拉格朗日和后来的哈密顿。19 世纪中叶，德国数学家狄利克雷（Peter Gustav Lejeune Dirichlet）将这一思想提炼为“狄利克雷稳定性判据”：如果一个保守系统在某个平衡点附近的势能函数是一个正定二次型，那么该平衡点就是稳定的。这个判据的物理直观非常清晰：系统如同一个处在碗底的小球，无论向哪个方向轻推，它的势能都会增加，因此它会倾向于滚回最低点。这一深刻的物理洞察，为正定二次型的研究提供了强大的现实动机，使其不仅仅是一个代数概念，更是描述自然界稳定性的数学语言。

9.3.1.0.1 1. 优化理论：寻找山谷的最低点 考虑一个二元函数 $F(x, y)$ 。根据泰勒展开，在临界点 $(0, 0)$ 附近，它的行为主要由二阶项决定：

$$F(x, y) \approx F(0, 0) + \frac{1}{2} \left(\frac{\partial^2 F}{\partial x^2} x^2 + 2 \frac{\partial^2 F}{\partial x \partial y} xy + \frac{\partial^2 F}{\partial y^2} y^2 \right)$$

括号里的正是一个二次型，其矩阵是著名的**海森矩阵 (Hessian Matrix)** H 。

- 如果这个二次型对于任何非零的 (x, y) 位移，其值都**恒为正**，这意味着无论朝哪个方向偏离临界点，函数值都会增加。这正是**局部极小值点**的定义！
- 反之，如果恒为负，则是局部极大值点。
- 如果时正时负，则是一个无法稳定停留的**鞍点**，像马鞍的中心。

因此，“恒为正”这个性质，是判断一个系统是否处于稳定平衡（能量极小）的关键。

9.3.1.0.2 2. 几何与度量：定义一种“长度” 在欧氏空间中，一个向量 $\vec{x}$ 的长度的平方是 $\|\vec{x}\|^2 = x_1^2 + \cdots + x_n^2 = \vec{x}^T I \vec{x}$ 。这本身是一个二次型，其矩阵是单位矩阵 I 。它满足我们对“长度平方”的一切想象：非负，且只在向量为零时才为零。

一个正定二次型 $f(\vec{x}) = \vec{x}^T A \vec{x}$ 可以被看作是这种长度概念的推广。它定义了一种新的“长度的平方”，只不过空间在不同方向上被进行了不同的“加权”或“拉伸”。在这样的空间中，两个向量的“内积”也可以被推广为 $\langle \vec{u}, \vec{v} \rangle_A = \vec{u}^T A \vec{v}$ 。正定性保证了这个内积具有所有我们期望的良好性质。这在广义相对论等领域中是核心思想，其中时空的度量本身就由一个（不一定是正定的）二次型（度规张量）定义。

9.3.2 正定性的判别：从定义到实用准则

历史侧记

判别正定性的历史，是一部不断追求更高效、更直接方法的历史。最初的“惯性指数法”虽然理论上完美，但依赖于计算量较大的配方法。后来出现的“特征值法”，将问题与更深刻的谱理论联系起来，但求解特征方程在实践中可能很困难。西尔维斯特在 1852 年提出的“顺序主子式判别法”，在当时是一个巨大的进步，因为它将问题转化为了计算一系列行列式——这在 19 世纪是一个相对成熟和程序化的代数运算。这三大准则，分别从变换、内禀谱性质和行列式计算三个不同层面刻画了正定性，体现了数学理论发展的典型路径：从一个直观但难于操作的定义出发，逐步发展出理论上深刻且计算上可行的等价条件。

我们已经有了明确的动机，现在需要的是一套行之有效的判别方法。

定义 9.2 (正定性)

设 $f(\vec{x}) = \vec{x}^T A \vec{x}$ 是一个 n 元实二次型。

- 若对 $\forall \vec{x} \neq \vec{0}$ ，恒有 $f(\vec{x}) > 0$ ，则称 f 是正定的。
- 若对 $\forall \vec{x} \neq \vec{0}$ ，恒有 $f(\vec{x}) < 0$ ，则称 f 是负定的。... (其他定义如半正定、不

定等)



直接用定义去检验是极其困难的，因为它要求我们验证无穷多个向量。因此，数学家们发展了几个等价的、可操作的判别准则。

9.3.2.0.1 准则一 (基于不变量): 惯性指数判别法 这是最符合逻辑的出发点。一个二次型 f 化为标准型后为 $f = d_1 y_1^2 + \cdots + d_n y_n^2$ 。由于变换 $\vec{x} = C\vec{y}$ 是可逆的， $\vec{x} \neq \vec{0}$ 等价于 $\vec{y} \neq \vec{0}$ 。

- **思路:** 要使 f 对于任意非零的 $\vec{y}$ 都恒为正，一个显而易见的充分必要条件是，所有系数 d_i 都必须为正。如果有一个 $d_i \leq 0$ ，我们只需令对应的 $y_i = 1$ 而其他 $y_j = 0$ ，就能使 $f \leq 0$ ，从而与正定性矛盾。

- **结论:** 二次型正定 $\iff$ 它的所有标准型系数均为正 $\iff$ 它的正惯性指数 $p = n$ 。这个准则在理论上是完美的，但实际操作中，我们仍需先通过配方法等手段求出标准型。我们渴望一个更直接的方法。

9.3.2.0.2 准则二 (基于内禀属性): 特征值判别法 **思路的进步:** 我们不想通过一个复杂的变换过程来得到答案。有没有什么矩阵 A 自身的、不依赖于坐标变换的内禀属性，可以直接决定正定性？**特征值**正是这样的属性！

- **推导与直观:** 我们将在下一节证明，任何对称矩阵 A 都可以找到一组标准正交的特征向量 $\{\vec{p}_1, \dots, \vec{p}_n\}$ ，它们构成 $\mathbb{R}^n$ 的一组基。任何向量 $\vec{x}$ 都可以表示为它们的线性组合 $\vec{x} = c_1 \vec{p}_1 + \cdots + c_n \vec{p}_n$ 。那么：

$$f(\vec{x}) = \vec{x}^T A \vec{x} = (c_1 \vec{p}_1 + \cdots + c_n \vec{p}_n)^T A (c_1 \vec{p}_1 + \cdots + c_n \vec{p}_n)$$

利用 $A\vec{p}_i = \lambda_i \vec{p}_i$ 和特征向量间的正交性 $\vec{p}_i^T \vec{p}_j = \delta_{ij}$ ，上式可以被完美地化简为：

$$f(\vec{x}) = \lambda_1 c_1^2 + \lambda_2 c_2^2 + \cdots + \lambda_n c_n^2$$

这本质上就是通过特征向量基得到的标准型！因此，要使 $f(\vec{x}) > 0$ 对所有非零 $\vec{x}$ (等价于不全为零的 c_i) 成立，其充要条件就是**所有的特征值 λ_i 都为正**。

- **结论:** 二次型正定 $\iff$ 其矩阵 A 的所有特征值均为正。

这个准则极其重要，它将一个关于二次型取值的分析问题，转化为了一个关于矩阵谱 (特征值集合) 的代数问题。

9.3.2.0.3 准则三 (基于计算捷径): 西尔维斯特判别法 **思路的再进步:** 求特征值需要解高次方程，在没有计算机的时代，这仍然是一项艰巨的任务。西尔维斯特等人追问：我们能否只通过计算行列式——一种更成熟、更程序化的运算——来判断正定性？

- **思路:** 这个准则的背后思想与矩阵的 LU 分解或 LDL^T 分解有关。一个对称矩阵 A 是正定的，可以被证明等价于它可以被分解为 $A = LL^T$ (Cholesky 分解)，其中 L 是一个对角元为正的下三角矩阵。而这个分解能够顺利进行，其条件恰好与顺

序主子式的符号有关。

- **结论 (西尔维斯特判别法):** 二次型正定 $\iff$ 其矩阵 A 的所有顺序主子式都为正。

$$\Delta_1 = a_{11} > 0, \quad \Delta_2 = \det \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} > 0, \quad \dots, \quad \Delta_n = \det(A) > 0$$

这个准则在 19 世纪是巨大的计算福音。在今天，它仍然是理论推导和中小规模问题判定的有力工具。

9.4 几何的守护者：正交变换

历史侧记

当拉格朗日用纯代数配方法“肢解”二次型时，他并未关心坐标系是否被扭曲。然而，对于几何学家和物理学家来说，保持空间的度量结构（长度和角度）是至关重要的。他们需要的是一种“刚性”的变换。对这类变换的系统研究，始于欧拉对三维空间旋转的研究。到了 19 世纪，随着群论思想的渗透，数学家们开始研究由所有保距变换构成的集合，即“正交群”。“正交”一词本身就暗示了其与“垂直”这一几何概念的深刻联系。将“保距”这一几何要求，翻译为 $P^T P = I$ 这一简洁的代数条件，是 19 世纪矩阵理论的一大成就，它使得对刚性运动的复杂几何分析，可以被优雅的矩阵运算所取代。

回顾拉格朗日配方法，我们意识到它所对应的变换 $\vec{x} = C\vec{y}$ 像一个“野蛮的改革者”，为了达到代数上的简洁（对角化），不惜扭曲空间的几何结构（长度和角度）。这在许多物理和几何问题中是不可接受的。

我们的新追求：我们能否找到一位“温和的守护者”，一种只进行**刚体运动**（旋转或反射）的变换，来达到同样的目标？这种变换必须保持空间的度量结构不变。

9.4.1 如何用代数语言刻画“保距”变换？

历史侧记

将几何性质翻译成代数语言，是解析几何的核心精神。这里的核心问题是：如何翻译“保距”？数学家们发现，直接从长度相等 $\|P\vec{x}\| = \|\vec{x}\|$ 出发推导，虽然可行，但不如从更根本的“保内积”出发来得优雅和深刻。内积是比长度更基本的结构。 $P^T P = I$ 这一代数条件的推导过程，正是这一思想的体现。它揭示了，一个变换要想成为几何的“守护者”，其矩阵必须满足一个极其严苛的代数“誓言”。这个誓言一旦许下，就自动保证了长度、角度、体积（由行列式为 ± 1 保证）等所有欧氏几何的核心要素都得到保全。

核心思想：一个变换保持了几何结构，当且仅当它保持了向量间的**内积**。因为长度

是向量与自身的内积的平方根 ($\|\vec{x}\| = \sqrt{\vec{x} \cdot \vec{x}}$), 角度由内积定义 ($\cos \theta = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \|\vec{v}\|}$)。只要保住了内积, 就保住了一切。

设变换为 $\vec{y} = P\vec{x}$ 。我们要求变换后的内积等于变换前的内积:

$$\begin{aligned}\vec{y}_1 \cdot \vec{y}_2 &= \vec{x}_1 \cdot \vec{x}_2 \\ (P\vec{x}_1) \cdot (P\vec{x}_2) &= \vec{x}_1^T \vec{x}_2\end{aligned}$$

用矩阵转置来表达内积:

$$\begin{aligned}(P\vec{x}_1)^T (P\vec{x}_2) &= \vec{x}_1^T \vec{x}_2 \\ (\vec{x}_1^T P^T) (P\vec{x}_2) &= \vec{x}_1^T \vec{x}_2 \\ \vec{x}_1^T (P^T P) \vec{x}_2 &= \vec{x}_1^T I \vec{x}_2\end{aligned}$$

由于这个等式必须对任意的 $\vec{x}_1, \vec{x}_2$ 都成立, 我们必然得出结论:

$$P^T P = I$$

这个简洁的矩阵方程, 就是对“几何守护者”的完美代数刻画。

定义 9.3 (正交矩阵)

一个 n 阶实方阵 P 被称为正交矩阵, 如果它满足 $P^T P = I$ 。由它定义的线性变换 $\vec{y} = P\vec{x}$ 被称为正交变换。



9.4.1.0.1 一个方程, 三重含义: $P^T P = I$ 这个定义极其凝练, 但它背后是几何、代数与向量组三种视角的完美统一。

- 代数视角:** P 的逆矩阵就是它的转置, $P^{-1} = P^T$ 。这是一个巨大的计算优势。
- 几何视角:** 该变换保持内积、长度、角度和距离不变。它对应于空间的**旋转** (若 $\det(P) = 1$) 或**旋转加反射** (若 $\det(P) = -1$)。
- 向量组视角:** 让我们考察 $P^T P = I$ 的矩阵乘法细节。令 $P = [\vec{p}_1, \dots, \vec{p}_n]$, 其中 $\vec{p}_i$ 是 P 的列向量。那么 P^T 的行向量就是 $\vec{p}_i^T$ 。 $P^T P$ 的 (i, j) 元是 (P^T 的第 i 行) $\times$ (P 的第 j 列) $= \vec{p}_i^T \vec{p}_j = \vec{p}_i \cdot \vec{p}_j$ 。由于 $P^T P = I$, 我们得到:

$$\vec{p}_i \cdot \vec{p}_j = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases}$$

这正是**标准正交基**的定义!

结论

一个矩阵是正交的, 当且仅当它的列向量 (或行向量) 构成一组标准正交基。这是线性代数中又一个深刻而优美的等价关系。它告诉我们, 要构造一个保持几何性质的变换, 我们只需找到一个新的标准正交坐标系, 并将新基向量作为列来构建变换矩阵即可。

9.5 用正交变换化二次型为标准型

历史侧记

本节内容是 19 世纪线性代数发展的顶峰。它将两个看似源头不同的研究领域汇聚到了一起。一条线索是几何学和力学，从欧拉、拉格朗日到柯西，他们一直在寻找二次曲面和惯量椭球的“主轴”，这是一个寻找理想坐标系以简化几何描述的问题。另一条线索是纯代数学，从高斯、雅可比到魏尔斯特拉斯，他们致力于研究对称矩阵的性质，特别是其特征值问题。主轴定理的诞生，标志着数学家们终于认识到，这两条线索实际上是同一个问题的两个侧面：二次型的主轴方向，恰好就是其对称矩阵的特征向量方向；而实现主轴变换的几何“旋转”，正对应于由这些特征向量构成的正交矩阵。这一发现是线性代数核心思想——“谱理论”——的伟大胜利。

现在，我们迎来了本章的最高潮。我们拥有了待解决的问题（化简 $\vec{x}^T A \vec{x}$ ）和理想的工具（正交变换 $\vec{x} = P\vec{y}$ ）。我们能否用最优雅的工具，来解决这个最核心的问题？

9.5.1 核心定理：主轴定理的诞生

历史侧记

“谱定理”（Spectral Theorem）是泛函分析中的一个核心定理，其有限维版本（即关于实对称矩阵的结论）在 19 世纪已经由柯西、雅可比、魏尔斯特拉斯等人基本完成。这个定理的威力在于，它保证了对于实对称矩阵这一类重要的对象，我们总能找到一个由“本征”向量（特征向量）构成的完美坐标系（标准正交基）。“谱”这个词来源于物理学，因为原子光谱的谱线位置，正对应于量子力学中某个算符（无限维的对称“矩阵”）的特征值。主轴定理可以被看作是谱定理在二次型几何问题上的直接应用。它将一个抽象的谱分解理论，翻译成了具体的几何结论：任何一个二次曲面，都存在一组正交的对称轴。

我们的目标是寻找一个**正交矩阵** P ，使得在变换 $\vec{x} = P\vec{y}$ 之后，新的二次型矩阵 $P^T A P$ 是一个对角矩阵 D 。由于 P 是正交矩阵， $P^T = P^{-1}$ ，所以这个条件等价于 $P^{-1} A P = D$ 。

这正是我们在第六章研究的**矩阵对角化**问题！但是，并非所有矩阵都可以被对角化。更不用说被**正交**对角化了。

幸运的是，我们研究的二次型，其矩阵 A 是一个**实对称矩阵**。这就激活了我们之前学过的、关于实对称矩阵的强大理论——**谱定理 (Spectral Theorem)**。

定理 9.2 (谱定理, 回顾)

任何一个实对称矩阵 A :

1. 必然可以被对角化。
2. 它的所有特征值都是实数。
3. 对应不同特征值的特征向量是相互正交的。
4. 它总能找到 n 个线性无关的特征向量, 甚至可以找到一组标准正交的特征向量基。
5. 结论: 它必然可以被正交对角化。



谱定理为我们扫清了所有理论障碍。它像一个神谕, 告诉我们: 对于任何实对称矩阵 A , 我们苦苦追寻的那个正交矩阵 P **必然存在!**

这便引出了二次型理论的基石——主轴定理。

定理 9.3 (主轴定理 (Principal Axis Theorem))

对任意一个 n 元实二次型 $f(\vec{x}) = \vec{x}^T A \vec{x}$, 必然存在一个正交变换 $\vec{x} = P\vec{y}$, 能够将其化为标准型:

$$f = \lambda_1 y_1^2 + \lambda_2 y_2^2 + \cdots + \lambda_n y_n^2$$

其中:

- 标准型的系数 $\lambda_1, \dots, \lambda_n$ 恰好是矩阵 A 的全部特征值。
- 正交矩阵 P 的列向量 $\vec{p}_1, \dots, \vec{p}_n$ 恰好是与这些特征值对应的标准正交特征向量。

**9.5.2 几何的诠释: 在自然的坐标系中看世界****历史侧记**

主轴定理的几何图像是如此清晰和令人满意, 它为 19 世纪的几何学家们提供了一个处理复杂二次曲面的普适“路线图”。在此之前, 研究一个倾斜的椭球体需要复杂的几何构图和坐标运算。而主轴定理告诉我们: 别在原来的坐标系里苦苦挣扎了! 宇宙为这个物体“内定”了一套最自然的坐标系, 这套坐标系的方向就是它对称矩阵的特征向量方向。你所要做的, 就是通过一次旋转, 让你的观察视角与这套自然坐标系对齐。一旦对齐, 所有的复杂性(交叉项)都将烟消云散, 物体的真实形态(由特征值决定的标准方程)便会清晰地呈现在你面前。这一思想——寻找系统“内在”的、最自然的描述框架——已经成为整个理论物理和现代工程学的基本方法论。

主轴定理的代数证明简洁明了, 但其几何内涵更为深刻。它完美地回答了我们在引言中提出的问题。

- **主轴的存在与方向:** 二次曲面 $f(\vec{x}) = c$ 的那些几何上直观的“对称轴”(主轴), 在

代数上正对应于矩阵 A 的**特征向量的方向**。这些方向是空间中特殊的方向，当线性变换 A 作用于其上时，仅仅进行伸缩而不改变其方向。

- **坐标系的旋转**：正交矩阵 P 的作用，就是将我们原始的、随意的标准坐标系 $(\vec{e}_1, \dots, \vec{e}_n)$ ，通过一次纯粹的**旋转**，精确地对准到由特征向量构成的这组新的、内在的、“自然”的坐标系 $(\vec{p}_1, \dots, \vec{p}_n)$ 上。
- **标准型的意义**：在新坐标系 $(y_1, \dots, y_n)$ 中，二次型的表达式 $f = \sum \lambda_i y_i^2$ 变得纯粹。这意味着，沿第 i 个主轴方向（即 $\vec{p}_i$ 方向）前进，二次型的值仅仅与该方向的坐标分量 y_i 的平方成正比，比例系数就是特征值 λ_i 。特征值 λ_i 的大小，直接决定了二次曲面在该主轴方向上的“胖瘦”或“曲率”。

9.5.3 一个完整的范例：让代数与几何共舞

历史侧记

本例所解决的问题——分析一个旋转的椭圆——是解析几何的经典问题，18 世纪的数学家（如欧拉）就已经能够通过专门的三角代换技巧来解决。然而，我们在此处运用的方法，则是一套完全不同且更为强大的“现代化”流程。我们没有使用任何三角函数，而是将问题翻译成矩阵语言，通过求解特征值这一纯代数操作，直接找到了“旋转角度”和“主轴长度”等所有几何信息。这个范例的意义在于，它展示了 19 世纪发展起来的抽象线性代数理论，是如何作为一个“万能钥匙”，系统性地、优雅地解决了那些曾经需要特殊技巧才能处理的经典几何问题。它是一次完美的“古为今用”，是用抽象理论之“牛刀”来解经典几何之“鸡”的精彩演示。

让我们用一个例子走完这趟旅程，将所有理论付诸实践。**问题**：分析二次曲面 $5x_1^2 + 5x_2^2 - 6x_1x_2 = 8$ 。

9.5.3.0.1 1. 代数建模：写出矩阵 A

$$f(x_1, x_2) = 5x_1^2 + 5x_2^2 - 6x_1x_2 = \begin{pmatrix} x_1 & x_2 \end{pmatrix} \begin{pmatrix} 5 & -3 \\ -3 & 5 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

我们的核心研究对象是 $A = \begin{pmatrix} 5 & -3 \\ -3 & 5 \end{pmatrix}$ 。

9.5.3.0.2 2. 寻找主轴方向：计算特征向量

- **特征值**：解特征方程 $\det(A - \lambda I) = (5 - \lambda)^2 - 9 = 0$ ，解得 $\lambda_1 = 2, \lambda_2 = 8$ 。
- **特征向量**：

- 对 $\lambda_1 = 2$ ，解 $(A - 2I)\vec{v} = 0$ ，即 $\begin{pmatrix} 3 & -3 \\ -3 & 3 \end{pmatrix} \vec{v} = 0$ ，得特征向量 $\vec{v}_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$ 。

• 对 $\lambda_2 = 8$, 解 $(A - 8I)\vec{v} = 0$, 即 $\begin{pmatrix} -3 & -3 \\ -3 & -3 \end{pmatrix} \vec{v} = 0$, 得特征向量 $\vec{v}_2 = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$ 。

几何洞察: 我们找到了两个相互正交的方向 $\begin{pmatrix} 1 \\ 1 \end{pmatrix}$ 和 $\begin{pmatrix} 1 \\ -1 \end{pmatrix}$, 这就是椭圆的长短轴方向。

9.5.3.0.3 3. 构建旋转矩阵: 单位化与构造 P 将特征向量单位化, 得到一组标准正交基:

$$\vec{p}_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad \vec{p}_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

将它们作为列, 构造正交矩阵:

$$P = [\vec{p}_1, \vec{p}_2] = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

这个矩阵代表了一个旋转加反射 (因为 $\det(P) = -1$)。如果我们希望是纯旋转, 可以交换两列或将第二列反号, 这不影响结果。例如, 取 $P' = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}$, 这是一个逆时针旋转 45 度的矩阵。

9.5.3.0.4 4. 变换与新方程 我们进行正交变换 $\vec{x} = P'\vec{y}$ 。主轴定理告诉我们, 无需计算 $P'^T A P'$, 结果必然是特征值构成的对角矩阵。原二次型在新坐标下变为:

$$f = \lambda_1 y_1^2 + \lambda_2 y_2^2 = 2y_1^2 + 8y_2^2$$

原方程 $f = 8$ 变为:

$$2y_1^2 + 8y_2^2 = 8 \implies \frac{y_1^2}{4} + \frac{y_2^2}{1} = 1$$

9.5.3.0.5 5. 最终的几何图像 这个结果告诉我们:

- 原始的、倾斜的图形, 是一个椭圆。
- 在旋转了 45 度的新坐标系 (y_1, y_2) 中, 这个椭圆的方程是标准形式。
- 它的长半轴在 y_1 轴上 (对应特征值 $\lambda = 2$ 较小), 长度为 $\sqrt{4} = 2$ 。
- 它的短半轴在 y_2 轴上 (对应特征值 $\lambda = 8$ 较大), 长度为 $\sqrt{1} = 1$ 。

代数计算的每一步, 都完美地翻译为一次几何操作, 最终为我们呈现了一幅清晰、和谐的图像。

结论

从消除恼人的交叉项出发, 我们踏上了一段从具体到抽象, 再回归具体的旅程。二次型, 这个看似纯粹的代数对象, 被证明是二次几何体的灵魂。而对称矩阵的谱理论, 尤其是主轴定理, 则为我们提供了沟通代数与几何的完美桥梁。它深刻地体现了线性代数的核心思想范式: **为一个复杂的系统找到它内在的、“自**

然”的基（由特征向量构成），并在此基下，系统将展现出其最简洁、最深刻的结构。这一思想的力量，远远超出了二次型的范畴，成为了现代科学与工程中分析复杂系统的基本世界观。

附录 A Book 修订记录

V0.0.3 (2025-10-14)

- **[第四章]** 重新编排了第四章的逻辑讲述顺序，每一小节采取历史-> 定理-> 性质-> 代码的编排顺序。
- **[第四章]** 优化了性质定理的证明部分，添加了更多的证明步骤细节以及讲解，同时辅以简单例子加以理解。
- **[第四章]** 为每一小节添加 PyTorch 代码辅以读者进行代码验证。

V0.0.2 (2025-10-09)

- **[第三章]** 重新编排了第三章的逻辑讲述顺序，每一小节采取历史-> 定理-> 性质-> 代码的编排顺序。
- **[第三章]** 优化了性质定理的证明部分，添加了更多的证明步骤细节以及讲解，同时辅以简单例子加以理解。
- **[第三章]** 为每一小节添加 PyTorch 代码辅以读者进行代码验证。

V0.0.1 (2025-10-07)

- 修改了历史侧记、代码视角、问题板块的格式样式
- **[第一章]** 修正了丢番图 (Diophantus) 的生平介绍。
- **[第二章]** 对“线性代数简史”的整体结构和叙事脉络进行重构，并为莱布尼茨、凯莱、希尔伯特等核心人物增补了详细的贡献说明。
- **[第二章]** 新增了图文并茂的双线发展时间轴，用以宏观展示线性代数的思想变迁。
- **[第二章]** 增补了与深度学习融合的前沿应用，详细介绍了 LoRA、ADMM-Net、压缩感知及逆问题等概念。
- **[第三章]** 扩充了早期数学家的介绍。
- **[第三章]** 为关键代数概念增加了辅助理解的几何可视化图示
- **[第三章]** 对核心知识点内容进行了系统性的补充、修正和完善。

V0.0.0 (2025-09-27)

- 线代第一节课，发布初始版本。

附录 B Slides 修订记录

V0.0.1 (2025-10-13)

- [第一章] 为 PPT 内容添加历史过渡部分的知识。
- [第一章] 调整了习题的呈现播放顺序，提升课题范例习题部分的教学效果。
- [第一章] 修改了习题中行列式答题格式的呈现，更方便学生去理解每一步的变形。
- [第一章] 添加了更多的解释，为一些较为难懂的概念和性质加上了解释。
- [第一章] 更正了部分习题的答案和过程。